

Tudor Sorin

Vlad Tudor

BAZELE PROGRAMĂRII ÎN



Java

Infobits



Academy

Cuprins

Capitolul 1. Java, primele noțiuni.....	11
1.1. Instalarea mediului de programare	12
1.2. Primul program	14
1.3. Conceptul de Mașină Virtuală Java (JVM).....	15
1.4. Tipuri de aplicații Java	16
1.5. Comentarii	17
1.6. Afișarea șirurilor de caractere	17
1.7. Programe care se găsesc în alte folder-e	17
Capitolul 2. Tipuri și operații de date	19
2.1. Tipuri de date	20
2.1.1. Tipurile de date primitive	20
2.1.2. Tipul referință	22
2.1.3. Observații importante	22
2.2. Operatori.....	23
2.2.1. Operatori aritmetici.....	23
2.2.2. Operatori relaționali.....	25
2.2.3. Operatori de egalitate	25
2.2.4. Operatori de incrementare și decrementare	25
2.2.5. Operatori logici	26
2.2.6. Operatori logici pe biți	27
2.2.7. Operatori de atribuire	28
2.2.8. Operatori condițional	30
2.2.9. Operatori de conversie explicită.....	31
2.2.10. Prioritatea (precedența) operatorilor	31
2.3. Instrucțiuni	32
2.4. Masive	38
2.5. Citirea datelor de la tastatură	43
2.6. Probleme rezolvate	45
Probleme propuse	49
Răspunsuri	55

Capitolul 3. Metode	57
3.1. Generalități.....	58
3.2. Structura unei metode și apelul ei.....	59
3.3. Transmiterea parametrilor	61
3.4. Despre variabile.....	63
3.5. Exemple de utilizare ale metodelor	64
3.5.1. Exemple elementare.....	64
3.5.2. Un exemplu de backtracking în Java	67
3.6. Metode recursive	68
3.7. Exemple de utilizare a metodelor recursive.....	70
3.7.1. Exemple elementare.....	70
3.7.2. Backtracking recursiv	71
3.8 Supraîncărcarea metodelor.....	73
Probleme propuse	74
Răspunsuri	79
 Capitolul 4. Clase – primele noțiuni, exemple	80
4.1. Ce este o clasă ?	81
4.2. Constructori	83
4.3. Date membru statice și metode statice	84
4.4. Cuvântul cheie this	85
4.5. Referințe către obiecte.....	85
4.6. Masive de obiecte	87
4.7. Aplicații ale noțiunilor prezentate.....	88
4.7.1. Lucrul cu numere raționale.....	89
4.7.2. Lucrul cu mulțimi de numere naturale	92
4.8. Garbage Collector	96
4.9. Cum sunt memorate clasele ?	96
4.10. Pachete.....	98
4.11. Clase interioare	101
4.12. O problemă de terminologie	102
Probleme propuse	103
Răspunsuri	106

Capitolul 5. Studiul unor clase ale limbajului Java	110
5.1. Clasa Math	111
5.2. Clasa String	112
5.2.1. Constructorii și lungimea unui șir de caractere	112
5.2.2. Compararea șirurilor de caractere	113
5.2.3. Subșiruri	114
5.2.4. Concatenarea șirurilor de caractere.....	115
5.2.5. Parametrii metodei main()	116
5.2.6. Aplicații.....	117
5.3. Clasa StringTokenizer	119
5.4. Clase înfășurătoare.....	120
5.5. Lucrul cu numere mari.....	123
Probleme propuse	125
Indicații / rezolvări	128
 Capitolul 6. Extinderea claselor	 131
6.1. Noțiuni generale	132
6.2. Un exemplu de extindere al unei clase.....	134
6.3. O modalitate de simulare a extinderii multiple	138
6.4. Referințe către obiectele superclaselor și referințe către obiectele subclaselor	139
Probleme propuse	142
Indicații / rezolvări	144
 Capitolul 7. Noțiuni avansate de programare orientată pe obiecte	 147
7.1. Polimorfism.....	148
7.2. Un exemplu de polimorfism.....	150
7.3. Clase abstracte.....	152
7.4. Un exemplu de clasă abstractă	153
7.5. Interfețe	155
7.6. Aplicații ale interfetelor	158
7.7. Modificatori	161

Probleme propuse	165
Răspunsuri	167
Capitolul 8. Excepții	169
8.1. Noțiunea de excepție	170
8.2. Mecanismul de tratare a excepțiilor	170
8.3. Metodele care pot returna, în caz de excepții, obiecte ale unor alte clase	176
8.4. Exemple de tratare a excepțiilor	177
Probleme propuse	178
Răspunsuri	179
Capitolul 9. Alte clase esențiale în Java	181
9.1. Clasa Vector	182
9.2. Fișiere	184
9.2.1. Noțiunea de flux	184
9.2.2. Fluxuri de octeți	184
9.2.3. Fluxuri de caractere	186
9.2.4. Intrări de la tastatură	188
9.3. Fire de execuție	188
Probleme propuse	190
Răspunsuri	191
Capitolul 10. Inițiere în programarea vizuală	193
10.1. Prima fereastră	194
10.2. Mecanismul prin care se atașează componente ferestrei. Clasa Container	195
10.3. Un mecanism prin care butoanele răspund evenimentului de "apăsare"	197
10.4. Clasa JComponent	198
10.5. Clasa Toolkit	200
10.6. Poziționarea componentelor	200
10.6.1. Poziționarea absolută	201

10.6.2. Gestionarul de poziționare FlowLayout	201
10.6.3. Gestionarul de poziționare GridLayout	203
10.6.4. Gestionarul de poziționare BorderLayout	204
10.6.5. Gestionarul GridBagLayout	205
10.6.6. Gestionarul CardLayout	207
10.7. Studiul principalelor componente	208
10.7.1. Componente de tip JButton	208
10.7.2. Componente de tip JLabel	209
10.7.3. Componente de tip JPanel	210
10.7.4. Componente de tip TextField	211
10.7.5. Componente de tip ComboBox	214
10.7.6. Componente de tip JCheckBox și JRadioButton și gruparea lor	215
10.7.7. Meniuri	218
10.7.8. Cutii (casete) de dialog predefinite.....	221
10.7.8.1. Cutii de dialog de intrare.....	222
10.7.8.2. Cutii de dialog pentru afișarea mesajelor	222
10.7.8.3. Cutii de dialog de confirmare	223
10.7.8.4. Cutii de dialog cu opțiuni	223
10.7.9. Componente de tip TextArea	225
10.7.10. Componente de tip ProgressBar	227
10.7.11. Componente de tip JSpinner	228
10.7.12. Cutii de tip Open / Save	231
10.7.13. Componente de tip JColorChooser	235
10.7.14. Componente de tip JToolBar	237
10.7.15. Componente de tip JTable	238
10.8. Mai mult despre clasa Container	242
10.9. Cutii de dialog	246
10.10. Clasa Graphics	250
10.10.1. Afișarea imaginilor.....	250
10.10.2. Cum scriem ?	251
10.10.3. Cum desenăm ?.....	252
10.11. Clasa Graphics2D	254

10.11.1. Cum desenăm ?.....	254
10.11.2. Stabilirea modului de trasare a liniilor.....	256
10.11.3. Gradient.....	257
10.11.4. Clasa BufferedImage	259
10.11.5. Texturi.....	260
10.11.6. Dreptunghiuri 3D.....	262
10.11.7. Spațiul de coordonate al utilizatorului	263
10.12. Evenimente.....	265
10.12.1. Generalități	265
10.12.2. Evenimente de tip Action	265
10.12.3. Evenimente de tip MouseEvent	267
10.12.4. Focus , evenimente de tip FocusEvent	271
10.12.6. Evenimente de selectare a item-ilor	273
10.12.7. Evenimente pentru fereastră	274
10.13. Animarea imaginilor	276
Probleme propuse	278
Indicații	281
Capitolul 11. Applet-uri	285
11.1. Generalități.....	286
11.2. Crearea și executarea applet-urilor	286
11.3. Arhive Java și atributele elementului APPLET.....	287
11.4. Redefinirea unor metode ale applet-ului	289
11.5. Afișarea imaginilor. Clasa URL.....	290
11.6. Redarea sunetelor	291
11.7. Exemple de applet-uri.....	292

CAPITOLUL 1

Java, primele noțiuni



Din cuprins:

Instalarea mediului de programare

Primul program

Mașina Virtuală Java

Tipuri de aplicații

Comentarii

Afișarea șirurilor de caractere

Programe care se găsesc în alte folder-e

1.1. Instalarea mediului de programare

Pentru a lucra în **Java**, utilizăm soft-ul pus la dispoziție de firma **Sun**, creatoarea limbajului. Mai precis, utilizăm **Java 2 Platform, Standard Edition (J2SE), v1.4.2, Software Development Kit (SDK)**.



În cele ce urmează, considerăm că lucrăm sub sistemul de operare **Windows 2000** și calculatorul are acces la **Internet**.

A. Începem prin a download-a softul. Adresa este: <http://java.sun.com>. Evident, vom alege o versiune recentă pentru Windows. În continuare, instalăm soft-ul download-at. De exemplu, eu l-am instalat în `c:\j2sdk1.4.2_04`.

↳ Poate până la apariția acestei cărți veți găsi versiuni mai noi.

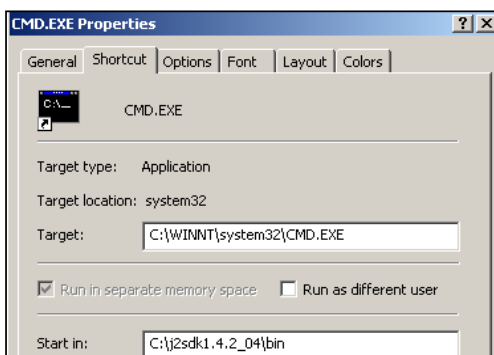
B. Pentru că soft-ul pus la dispoziție de firma **Sun** lucrează prin comenzi date de la tastatură (ca în **MS-DOS**, un sistem de operare mai vechi, creat de **Microsoft**) va trebui să folosim un program, numit **CMD.exe**, care simulează sistemul de operare amintit. Programul se găsește în folder-ul **WINNT**. Identificați-l cu **Search**. El creează o fereastră prin care se pot da comenzi **MS-DOS**.



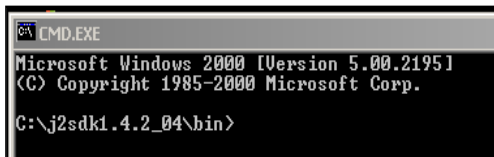
C. Pentru a utiliza cu ușurință acest **WINNT**, vom crea pe **Desktop** un shortcut către el (vedeți alăturat).

D. Va trebui ca fereastra **CMD** să fie deschisă în subfolder-ul **bin**, al folder-ului care conține softul download-at și instalat. Executați click cu butonul drept al mouse-ului și selectați **Properties**.

La apariția cutiei de dialog alăturate, introduceți calea folder-ului amintit în edit-ul **Start in**.



Dacă shortcut-ul a fost creat corect, atunci când executați dublu click pe el va apărea fereastra alăturată, de unde vom da comenzi de la tastatură.

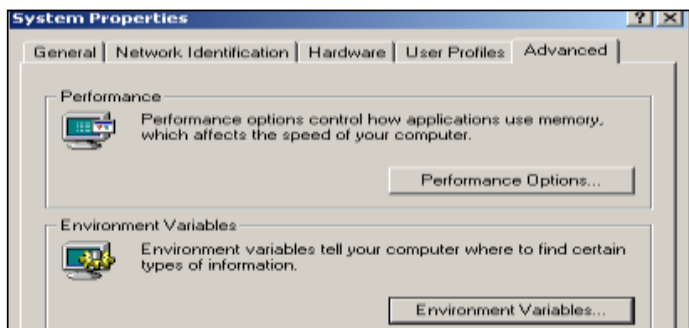


E. Întrucât în perioada de inițiere programele noastre se vor găsi în folder-ul **bin**, pentru a lucra mai ușor, este recomandabil să creăm pe **Desktop** și un shortcut către acest folder.

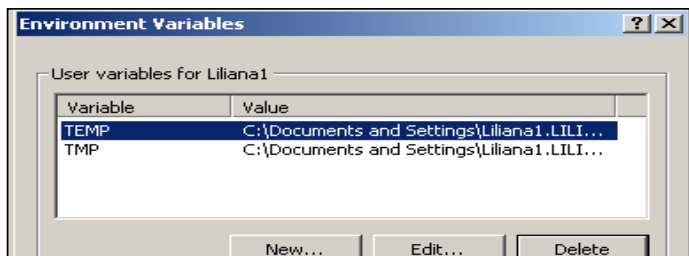


F. Deși de la instalare, mediul Java ar trebui, implicit, să lucreze în folder-ul **bin**, este bine ca dvs. să-l setați corespunzător. În acest scop, vom folosi variabila de sistem **CLASSPATH** (calea către clase) care este inițializată cu respectiva cale. Pentru aceasta, vom respecta următorii pași:

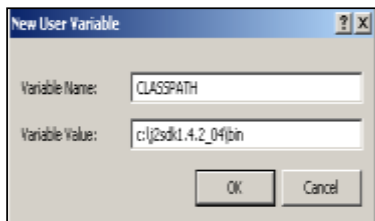
1. Din meniul **Start**, apăsați **Settings**, **Control Panel** și **System**.
2. Va apărea o cutie de dialog în care selectați **Advanced** și **Environment Variables...**



3. În cutia de dialog de mai jos, apăsați **New**.



4. În cutia de dialog alăturată, introduceți numele variabilei de sistem (**CLASSPATH**) apoi calea către folder-ul **bin**. Atenție! Calea trebuie să corespundă locului unde aveți instalat mediul **Java** pe calculatorul dv. Apoi, apăsați **OK**!



1.2. Primul program

De acum, putem scrie primul program. Trebuie să știm că mediul **Java**, pus la dispoziție de firma **Sun**, nu ne oferă un editor de text. Din acest motiv, textul program va fi introdus cu ajutorul **NOTEPAD**-ului, dar va trebui să fim atenți ca extensia fișierului text astfel obținut să fie **.java**.



Nu uitați: pentru început, fișierul text care conține programul trebuie să se găsească în folder-ul **bin**.

Pentru a introduce și rula un program, vom respecta pașii următori:

A. Vom deschide shortcut-ul către **bin** din meniul flotant, vom alege **New** și **Text Document**, apoi schimbăm numele fișierului în **prima.java**. Fișierul cu acest nume va fi deschis din meniul flotant asociat, cu **Open with...** și **Notepad**. Apoi, se introduce programul de mai jos, care afișează mesajul **Acesta este primul program Java** și salvăm programul:

```
class prima {  
public static void main(String[] args) {  
    System.out.println("Acesta este primul program Java");  
}
```

- Este obligatoriu ca numele clasei să coincidă cu numele fișierului. În exemplu, clasa se numește **prima**, iar fișierul este **prima.java**.
- Este obligatoriu ca extensia fișierului text să fie **.java** !

B. Din fereastra **CMD**, apelăm compilatorul **Java** prin comanda:

```
javac prima.java
```

În urma acestei comenzi, se va crea fișierul **prima.class**. Acest fișier conține programul compilat. Verificați existența lui în **bin**!

C. Pentru a executa programul astfel compilat, vom da comanda:

```
java prima
```

Iată cum arată acum fereastra **CMD**:

```
CMD.EXE  
Microsoft Windows 2000 [Version 5.00.2195]  
(C) Copyright 1985-2000 Microsoft Corp.  
C:\j2sdk1.4.2_04\bin>javac prima.java  
C:\j2sdk1.4.2_04\bin>java prima  
Acesta este primul program Java  
C:\j2sdk1.4.2_04\bin>_
```



Dacă ați reușit să rulați acest prim program, ați făcut un pas important. Înseamnă că ați instalat corect mediul și de acum puteți lucra liniștit. Sunt mulți cei care se împotmolesc în acest punct și, din acest motiv, renunță...

Să aruncăm o privire asupra programului:

- Orice program este inclus într-o clasă. Noțiunea de clasă aparține programării orientate pe obiecte și va fi explicată pe larg în această carte. În exemplu, clasa se numește **prima**.
- Așa cum am precizat, numele clasei trebuie să coincidă cu numele fișierului text sursă (cel creat cu **NOTEPAD**), numai că acesta din urmă trebuie să aibă extensia **.java**.
- Orice program are o metodă (funcție **main()**) și executarea sa începe cu instrucțiunile acestei metode.
- În exemplu, programul afișează un șir de caractere. Instrucțiunea (apelul de metodă) care afișează acest șir de caractere este **system.out.println()**. După afișare, cursorul sare pe rândul următor.
- În Java, în locul termenului consacrat "**funcție**", vom folosi pe acela de **metodă**.
- Este obligatoriu ca metoda **main()** să conțină anumiți parametri de intrare: **String[] args**. Asupra semnificației acestor parametri revenim în capitolele următoare.
- Elementele **public** și **static** sunt modificatori. Prezența lor în antetul metodei **main()** este obligatorie. Și asupra lor vom reveni.
- **void** este tipul metodei **main()**. Semnificația este cea din C++, adică metoda nu returnează nici o valoare.
- În Java, se face distincție între literele mari și cele mici. De exemplu, **Mama** și **mama** sunt identificatori diferiți.

1.3. Conceptul de Mașină Virtuală Java (JVM)

Una dintre cerințele care au stat la baza creării limbajului Java a fost aceea de a obține un limbaj portabil. Aceasta înseamnă că un program scris pe un anumit calculator, pe care rulează un anumit sistem de operare, să poată rula fără modificări și pe un alt calculator, pe care rulează alt sistem de operare. De exemplu, să se poate scrie un program care să fie rulat pe **Windows** și pe **Linux**.



Pentru a rezolva această problemă, cei de la Sun au recurs la următorul procedeu:

- ❖ Fiecare utilizator al unui sistem de operare poate download-a mediul specific acelui sistem de operare. De aceasta v-ați convins atunci când ați download-at mediul pentru **Windows**.
- ❖ Mediul specific pentru un anumit sistem de operare conține un compilator special creat pentru el. El preia programul din fișierul sursă și "îl traduce" într-un cod anume, numit **byte-code**. Compilatorul este programul **javac**, iar fișierul compilat are extensia **.class**.
- ❖ Programele în **byte-code** sunt rulate pe așa numita **Mașină Virtuală Java (JVM)**. Mașina virtuală Java este un suport soft care trebuie să fie instalat pe calculatoarele care rulează astfel de fișiere. Când ați instalat mediul **Java**, implicit ați instalat și mașina virtuală java. De altfel, mașina virtuală Java poate fi instalată și în absența mediului de dezvoltare Java.
- ❖ Un fișier în **byte-code** (extensia **.class**) poate fi rulat pe orice calculator, indiferent de sistemul de operare pe care-l are, dacă are instalată mașina virtuală Java. Practic, se ia prima instrucțiune din fișierul **byte-code**, este tradusă în instrucțiuni ale calculatorului pe care rulează aplicația și acestea din urmă sunt executate; se ia o a doua instrucțiune din **byte-code** care se traduce și se rulează, ș.a.m.d.
- ❖ Se poate observa că Java are și un compilator care traduce sursa în **byte-code (javac.exe)** dar și un interpret care traduce și execută fiecare instrucțiune din **byte-code** în instrucțiuni ale calculatorului pe care se execută aplicația.



Datorită faptului că Java este înzestrat cu interpret (ia o instrucțiune, o traduce și o execută, apoi altă instrucțiune, o traduce și o execută...), programele se execută mai lent. Acest fapt este compensat de procesoarele din ce în ce mai performante, dar și de performanțele interpretului.

1.4. Tipuri de aplicații Java

În Java există trei tipuri de aplicații:

1. Aplicații de sine stătătoare (**stand-alone**). Acestea sunt precum aplicațiile studiate în Pascal sau C++: se rulează pe un anumit calculator.
2. Aplicații care se rulează pe partea de client (applet-uri). Un **applet** este alcătuit dintr-unul sau mai multe fișiere cu extensia **.class** și este utilizat în cadrul paginilor **web**. Practic, el este atașat textului **HTML**, la fel

cum se atașează o imagine, iar fișierele care-l alcătuiesc se transferă pe **site**. El este adus pe calculatorul clientului atunci când se apelează pagina respectivă și este executat prin intermediul browser-ului.

3. Aplicații care se execută pe partea de server (**servlet-uri**).

! În această carte sunt tratate doar aplicațiile din primele două tipuri.

1.5. Comentarii

În Java, se pot pune comentarii oriunde dorim, la fel ca și în C++. Ele sunt de două feluri:

1. Comentariile la nivel de linie: `//` tot ce urmează după ele, pe linia celor două caractere, este considerat comentariu.
2. Comentarii pe mai multe linii se găsesc între `/*` și `*/`.

```
class prima { // un comentariu
public static void main(String[] args) {
/* Acesta
este alt comentariu */
System.out.println("Acesta este primul program Java");}
}
```

1.6. Afișarea șirurilor de caractere

Pentru afișarea datelor vom utiliza două metode:

1. `System.out.print( sir_de_caractere );`
2. `System.out.println( sir_de_caractere );`

Ambele metode afișează șirul de caractere căutat. Diferența dintre ele este dată de faptul că după afișare, în cazul `System.out.print()`, cursorul rămâne pe rândul curent (după ultimul caracter scris), iar în cazul `System.out.println()`, după ce s-a scris șirul de caractere, cursorul trece pe linia următoare.

1.7. Programe care se găsesc în alte folder-e

Nu este obligatoriu ca programele să se găsească în folder-ul `bin`. Există posibilitatea ca programele `Java` să se rețină și în alte folder-e. Să vedem cum!



Creați un folder pe `C:\`, numit `TestJava`. Mutați acolo fișierul `prima.java`, prezentat în primul paragraf.

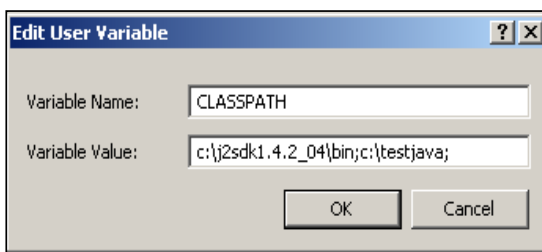
Pentru a-l compila, dați comanda următoare, unde ați scris și calea către fișier:

```
javac c:\testjava\prima.java
```

Pentru a-l rula, dați comanda următoare, unde parametrul `-classpath` spune programului `java.exe` unde găsește fișierul cu extensia `.class`.

```
java -classpath c:\testjava prima
```

De altfel, parametrul `classpath` poate fi pus și în variabila de mediu `CLASSPATH`, așa cum am arătat. Mai mult, calea se poate scrie în continuarea celei existente, așa cum vedeți mai jos:



În acest caz, la apelul programului **Java** nu mai este necesar să dăm parametrul `-classpath`.



Atenție: dacă setăm calea ca mai sus, cu ajutorul variabilei de mediu `CLASSPATH`, fișierul cu extensia `.class` se caută în fiecare cale trecută, în ordinea în care au fost trecute căile. De exemplu, mai întâi fișierul cu extensia `.class` este căutat în `bin` și apoi este căutat în `TestJava`. Aceasta înseamnă că dacă îl avem în ambele locuri, este executat cel aflat în `bin`.

CAPITOLUL 2

Manipularea datelor



Din cuprins:

Tipuri de date

Operatori

Instrucțiuni

Masive

Citirea datelor de la tastatură

Probleme rezolvate

Probleme propuse

2.1. Tipuri de date

În teoria clasică a limbajelor de programare, prin *tip de date* se înțelege:

- ❖ o mulțime de valori;
- ❖ o regulă prin care aceste valori se codifică;
- ❖ o mulțime de operații definite pe mulțimea valorilor.

Limbajul **Java** extinde noțiunea de tip de date, la aceea de *tip*. Aici se lucrează cu clase, care descriu într-o unică structură atât date cât și metode (funcții). Obiectele sunt instanțieri ale claselor, deci fiecare obiect conține atât date cât și metode. Asupra acestor probleme vom reveni, dar rețineți faptul că, în Java, prin *tip* vom înțelege o clasă, iar un obiect are un anumit *tip* dacă rezultă ca instanțiere a unei clase.

Totuși, limbajul Java utilizează și *tipuri* în sens clasic, vezi prima definiție. Pentru a nu se crea confuzii, aceste tipuri se numesc *tipuri primitive*. Astfel, avem tipuri de date primitive și tipuri referință.

2.1.1. Tipurile de date primitive

a) Tipuri întregi

Tip	Octeți ocupați	Numere întregi din intervalul
byte	1	[-128, 127]
short	2	[-32768, 32768]
int	4	[-2147483648, +2147483647]
long	8	[-9223372036854775808, 9223372036854775807]

Exemple:

Declarăm și afișăm mai multe variabile de tip întreg:

```
short t=32767;
byte z=-128, x=15;
int a=100000;
System.out.println(t);
System.out.println(z);
System.out.println(x);
System.out.println(a);
```

b) Tipuri reale

Tip	Octeți ocupați	Modulul valorilor între
float	4	$[3.4 \times 10^{-38}, 3.4 \times 10^{38}]$
double	8	$[1.7 \times 10^{-308}, 1.7 \times 10^{308}]$

Pentru inițializarea tipurilor reale se folosesc literalii.



Pentru tipul real `float`, literalii trebuie urmați în mod obligatoriu de litera `f`. Excepție fac literalii care reprezintă valori întregi.

Exemple:

1. `float a=3,b=2.5f,c=-7,d=3E2f,g=-2.4E-2f`; Valorile cu care am inițializat variabilele sunt: 3, 2, 2.5, -7, 3×10^2 , -2.4×10^{-2} .

Aceleași valori se folosesc pentru a inițializa variabile de tip `double`:

2. `double a=3,b=2.5,c=-7,d=3E2,g=-2.4E-2`;

c) Tipul `char`

O variabilă de tipul `char` reține un caracter. În `Pascal`, `C/C++`, caracterele sunt reținute prin utilizarea codului `ASCII`. În `Java`, codul utilizat pentru memorarea caracterelor este `Unicode` și, în acest cod, un caracter ocupă 2 octeți.

Exemplu: mai jos puteți observa declarația a trei variabile de tip `char`:

```
char a='m', b='\u006d', c=109;
System.out.println(a); System.out.println(b); System.out.println(c);
```

Toate variabilele rețin caracterul `m`. Prima dată caracterul este declarat în mod clasic, a doua oară este declarat prin codul său în hexa și a treia oară este declarat în zecimal. Cele afirmate sunt probate prin afisare, pentru că se afișează de trei ori `m`.

Caracterele se pot declara sub formă de coduri escape:

Caracter	Semnificație
\\	backslash
\n	newline
\'	apostrof
\"	ghilimele
\r	carriage return
\t	tab
\f	salt la pagină nouă

Exemple:

1. `char a='\'`; am declarat caracterul apostrof. Dacă am încerca să declarăm acest caracter prin `char a='''`; primim eroare de sintaxă.
2. `char b='\n'`; am declarat caracterul **newline**. Dacă se afișează acest caracter, cursorul sare pe linia următoare.
3. `char c='\\'`; am declarat caracterul **backslash**. Dacă am încerca să declarăm acest caracter prin `char a='\'`; primim eroare de sintaxă.

d) Tipul boolean

O variabilă de acest tip poate reține la un moment dat, una din cele două valori: **false** (fals) sau **true** (adevărat).

Exemplu: se declară, se inițializează și se afișează conținutul unei variabile de tip **boolean**:

```
boolean c=true;
System.out.println(c);
```



Metodele `System.out.print()` și `System.out.println()` permit afișarea tuturor tipurilor primitive.

2.1.2. Tipul referință

O variabilă de **tip referință** reține o referință (adresă) către un obiect. Cu ajutorul referinței, putem accesa datele membru și metodele obiectului. Numele tipului unei astfel de variabile coincide cu numele clasei. Cum obiectele sunt instanțieri ale claselor și cum putem proiecta o infinitate de clase, înseamnă că *există o infinitate de de tipuri referință*. Prin abuz de limbaj, vom spune despre o variabilă *care poate reține o referință către un anumit obiect. că este de tip referință*. Orice variabilă de tip referință poate reține valoarea **null**. Ea are semnificația de “nici o referință”.

2.1.3. Observații importante

Așa cum am arătat, cu ajutorul celor două metode,

```
System.out.print() și System.out.println(),
```

putem afișa conținuturile variabilelor de un tip primitiv. Mai avem de precizat două lucruri esențiale:

- a. Dacă în corpul unei metode, în exemplu `main()`, se găsește o variabilă neinițializată și se încearcă afișarea conținutului pe

care îl are, tentativa este sancționată prin eroare de sintaxă. Vedeti mai jos:

```
int i;  
System.out.println(i);
```

- b. Avem posibilitatea de a afișa mai multe variabile pe aceeași linie. Aceasta se obține prin convertirea conținutului variabilelor în șiruri de caractere și concatenarea șirurilor cu ajutorul operatorului +. Vedeti exemplul de mai jos care afișează pe o linie `a=2 b=4.5`:

```
int a=2; float b=4.5f;  
System.out.println("a="+a+" b="+b);
```

2.2. Operatori

În acest paragraf vor fi prezentați operatorii utilizați în Java.

2.2.1. Operatori aritmetici

Există următorii operatori aritmetici:

- - minus (unar, adică acționează asupra unui singur operand);
- + plus (unar);
- + (binar), pentru adunare;
- - (binar), pentru scădere;
- * (binar), are semnificația de înmulțire;
- / (binar), pentru împărțire;
- % (binar) restul împărțirii întregi.

Observații:

1. Operatorul '/' (împărțire) acționează în mod diferit în funcție de operanzi:
 - a) dacă ambii sunt de tip întreg, rezultatul este întreg și are semnificația de împărțire întreagă. Cu toate acestea, rezultatul este corect (din punct de vedere matematic) numai dacă valorile care se împart sunt pozitive.
 - b) dacă cel puțin un operand este de unul din tipurile reale rezultatul este real (se efectuează împărțirea obișnuită).
2. Operatorul '%' acționează numai asupra operanzilor de tip întreg. Rezultatul obținut este corect din punct de vedere matematic numai dacă ambii operanzi sunt numere naturale.

3. În cazul în care se împart două valori întregi se procedează astfel:

- se face împărțirea întreagă a celor două valori care sunt considerate în modul;
- semnul câtului se stabilește după regula semnelor (+ cu + rezultat +, + cu -, rezultat -), etc.
- În cazul operatorului '%', se face împărțirea ca anterior (se obține C), iar restul se obține după formula $R = D - I \times C$.
- Se pot utiliza oricâte perechi de paranteze rotunde, pentru a impune ca anumite operații să se facă prioritar.

Exemple:

1. Fie declarația: `int a=10;`. Atunci expresia `4*a/3` este de tip `int` și la evaluare se obține 13. S-a înmulțit 4 cu conținutul variabilei `a` (10) și s-a obținut 40. Apoi, $40/3=13$ (a fost efectuată împărțirea întreagă). În aceeași condiții, expresia `4*(a/3)` are ca rezultat numărul 12. La început s-a efectuat $a/3$ și s-a obținut 3, apoi $4*3=12$. Iată un exemplu în care se observă faptul că una este o expresie la matematică, și alta în **Java**.

2. Fie declarația: `float a=10;`. Expresia `4*(a/3)` are rezultatul 13.3333, la fel ca expresia `4*a/3`. De această dată, s-a efectuat împărțirea a două numere reale.

3. Fie declarația: `int a= -10;`. Atunci expresia `a%3` are ca rezultat numărul -1.

4. Fie declarația: `int a=-10;`. Atunci expresia `a*-3` are ca rezultat 30.

5. Fie declarațiile: `int a=10;` `char b=2;` `float c=5;`. Atunci expresia: `a+b+c` are rezultatul 17.0.

În exemple, apare o problemă foarte importantă: *care este tipul rezultatului, în cazul în care o expresie aritmetică are operanzi de mai multe tipuri?* Problema de mai sus se numește **problema conversiilor aritmetice implicite** (după cum vom vedea, conversiile se pot face și explicit).

Regula generală este: **se convertește unul din operanzi către tipul celuilalt (mai cuprinzător)**. De exemplu, dacă unul din operanzi este de tip `int`, iar celălalt de tip `float`, rezultatul va fi de tip `float`.



Fără o conversie prealabilă, operatorii aritmetici nu acționează asupra variabilelor de tip `char`. De exemplu, secvența `char x='1'; x=x+1;` este eronată din punct de vedere sintactic.

2.2.2. Operatori relaționali

În **Java** există următorii operatori relaționali:

- **<** (mai mic);
- **<=** (mai mic sau egal);
- **>** (mai mare);
- **>=** (mai mare sau egal).

Întrucât în **Java** există valorile logice **true** și **false**, rezultatul unei operații logice este *una dintre aceste valori*.

Exemple:

1. **3>5** - expresia ia valoarea **false**;
2. **3<5** - expresia ia valoarea **true**;
3. **3+7>=11-1** - expresia ia valoarea **true** (este posibil să scriem astfel de expresii, întrucât operatorii relaționali au o prioritate mai mică decât operatorul aditiv).

2.2.3. Operatori de egalitate

Aceștia sunt:

- **==** pentru egalitate;
- **!=** pentru inegalitate.

Ca și operatorii relaționali, expresiile de acest tip returnează **0** sau **1** - în funcție de modul în care este respectată sau nu egalitatea (inegalitatea).

Exemple:

1. **3==3**, rezultat **true**;
2. **3!=3** rezultat **false**;

2.2.4. Operatorii de incrementare și decrementare

Acești operatori sunt unari și au rolul de a incrementa (adună 1) sau decrementa (scad 1) conținutul unei variabile. Operatorii sunt:

- **++** pentru incrementare;
- **--** pentru decrementare.

Operatorii pot fi **prefixați** (aplicați în fața operandului) sau **postfixați** (aplicați după operand). Exemplu: fie **a** o variabilă de tip **int**.

Operatorul de incrementare prefixat aplicat variabilei **a** este **++a**;
Operatorul de incrementare postfixat aplicat variabilei **a** este **a++**;
Operatorul de decrementare prefixat aplicat variabilei **a** este **--a**;
Operatorul de decrementare postfixat aplicat variabilei **a** este **a--**.

Care este diferența între efectul unui operator aplicat prefixat sau postfixat? Să ne gândim la faptul că pot fi expresii care conțin și alți operatori în afara celor de incrementare (decrementare).

⇒ *Dacă operatorul este prefixat, variabila este incrementată (decrementată) înainte ca valoarea reținută de ea să intre în calcul.*

⇒ *Dacă operatorul este postfixat, variabila este incrementată (decrementată) după ce valoarea reținută de ea intră în calcul.*

Exemple:

1. Fie **a** o variabilă de tip **int** care reține valoarea 1. În urma evaluării expresiei **1+a++** se obține valoarea 2, dar după evaluare, **a** va reține valoarea 2 (variabila a fost incrementată după ce valoarea reținută de ea a intrat în calcul).

2. La fel ca în cazul anterior, expresia **1-++a** produce valoarea -1 (variabila a fost întâi incrementată, s-a obținut 2, apoi s-a efectuat 1-2).

3. Expresia **1+++a** este eronată sintactic. S-a considerat că operatorul de incrementare este aplicat postfixat pentru 1 și nu prefixat pentru **a**. *Aplicarea unui operator de acest tip se poate face pentru o variabilă.*

4. Dacă **a** și **b** sunt variabile de tip **int** care rețin valorile 1 și 3, atunci expresia **a+++b++** produce valoarea 3, dar după evaluare cele două variabile rețin 2 și 4.

5. În aceleași condiții, expresia **++a*++b** produce valoarea 8, iar după evaluare **a** și **b** rețin 2 și 4 (ca în cazul anterior).

! Acțiunea operatorului de decrementare este similară celei de incrementare, cu diferența că se scade 1 din conținutul variabilei.

2.2.5. Operatori logici

Există trei tipuri de operatori logici care acționează asupra datelor de tip **boolean**:

- **!** negare logică;
- **&&** și logic;
- **||** sau logic.

Acești operatori se aplică oricărei variabile și constante de tip logic.

Operatorul *negare logică* acționează astfel: dacă operandul este o valoare **true**, rezultatul este **false**, altfel rezultatul este **true**.

Operatorul *și logic* (binar) acționează astfel: dacă ambii operanzi sunt **true** rezultatul este **true**, altfel el este **false**.

Operatorul *sau logic* (binar) acționează astfel: dacă cel puțin unul din operandi este **true**, rezultatul este **true**, altfel rezultatul este **false**.

Example: în condițiile: **boolean a=true, b=false;**

1. **!a** returnează **false**;
2. **a||b** returnează **true**;
3. **a&&b** returnează **false**;

Operatorii logici binari garantează modul în care se tratează operandii - întâi cel din stânga, apoi (dacă este cazul) cei din dreapta. Astfel, dacă operandul din stânga este **true**, operatorul *sau logic* nu mai acționează asupra operandului din dreapta (este clar că rezultatul este **true**). Tot așa, dacă operandul din stânga este **false**, operatorul *și logic* nu mai evaluează operandul din dreapta (oricum rezultatul este **false**). Observația este foarte importantă pentru cazul în care al doilea argument este funcție (metodă) de tip **boolean**.

2.2.6. Operatori logici pe biți

Limbajul **Java** este dotat cu un set de operatori care permit accesul la bit. Aceștia sunt:

- **<<, >>, >>>** operatori de deplasare;
- **&** și pe biți;
- **|** sau pe biți;
- **^** sau exclusiv pe biți;
- **~** negare pe biți (operator unar).

Acești operatori acționează numai asupra operandilor de tip întreg.

Operatorul << este binar. El are rolul de a deplasa către stânga conținutul tuturor biților operandului din stânga sa, cu un număr de poziții egal cu valoarea reținută de al doilea operand. Pozițiile rămase libere (în dreapta) vor reține valoarea 0. Dacă al doilea operand reține valoarea **m**, o astfel de deplasare este echivalentă cu înmulțirea cu 2^m (evident, dacă **m** este mai mic decât numărul de biți rezervat primului operand).

Operatorul >> este binar. El are rolul de a deplasa către dreapta conținutul tuturor biților operandului din stânga cu un număr de poziții egal cu valoarea reținută de al doilea operand. **Atenție:** în pozițiile din stânga, rămase libere după deplasare, se copiază conținutul bitului de semn (acesta reține 1 în cazul numerelor negative și 0, în cazul numerelor pozitive). Dacă operandul din stânga este un întreg pozitiv, pozițiile rămase libere (în stânga) vor reține valoarea 0. În cazul întregilor pozitivi, dacă al doilea

operand reține valoarea m , o astfel de deplasare este echivalentă cu împărțirea întregă cu 2^m .

Operatorul >>> acționează la fel ca și operatorul >>, dar diferența este dată de faptul că pozițiile rămase libere din dreapta vor reține 0.



Atenție: operatorii de deplasare sunt utilizați doar pentru evaluarea expresiilor, operandii, chiar dacă sunt variabile, rămân nemodificați! Desigur, valorile astfel obținute pot fi atribuite. Aceasta înseamnă, de fapt, acces la bit.

În cazul operatorilor binari $\&$, $\wedge$, $|$, rezultatul se obține aplicând pentru fiecare pereche de biți aflați pe aceeași poziție regulile din tabelul următor:

OP_1	OP_2	$OP_1 \& OP_2$	$OP_1 \wedge OP_2$	$OP_1 OP_2$
0	0	0	0	0
1	0	0	1	1
0	1	0	1	1
1	1	1	0	1

Operatorul $\sim$ (negare pe biți) are rolul de a inversa conținutul biților (dacă un bit conține 0 va conține 1 și invers).

Exemple: dacă a este de tip `int` și reține 3, iar b este de același tip și reține 1 atunci:

1. $a \& b$ returnează 1;
2. $a | b$ returnează 3;
3. $a \wedge b$ returnează 2;
4. $a \ll 2$ returnează 6;
5. $a \gg 1$ returnează 1;
6. $\sim a$ returnează -4.

2.2.7. Operatori de atribuire

În **Java** atribuirea este operator, dar după cum vom vedea, este și instrucțiune. În plus, avem mai mulți operatori de atribuire. Operatorul '=' se folosește într-o expresie de forma:

v=expresie

Aici, v este o variabilă.

Principiul de executare este următorul:

- se evaluează expresia;
- variabilei v i se atribuie valoarea obținută (dacă este cazul se efectuează conversia respectivă).

Se pot efectua și atribuiri multiple de forma:

$$v=v_1=v_2=\dots=v_n=\text{expresie},$$

unde $v, v_1, \dots, v_n$ sunt variabile.

În acest caz, principiul de executare este următorul:

- se evaluează expresia;
- valoarea obținută este atribuită variabilei v_n (eventual convertită - dacă este cazul);
- conținutul variabilei v_n este atribuit variabilei v_{n-1} (eventual se efectuează conversia necesară);
- . . .
- conținutul variabilei v_1 este atribuit variabilei v (eventual se efectuează conversia necesară).

Acest operator se asociază de la dreapta la stânga. Ce am prezentat anterior evidențiază acest lucru.

Pentru atribuiri se mai pot utiliza și operatorii: $\ast=$, $/=$, $\%=$, $+=$, $-=$, $<<=$, $>>=$, $\&>$, $\wedge=$, $|=$.

O atribuire de forma: $v \text{ op}=\text{expresie}$, are același rezultat ca

$$v=v \text{ op expresie}$$

(diferența este că în primul caz se generează un cod mașină eficient).



Observație! Am văzut faptul că atribuirea este operator care figurează într-o expresie. Ca orice expresie, aceasta are o valoare rezultată în urma evaluării. În cazul atribuirilor, valoarea rezultată este valoarea atribuită variabilei care este prima din stânga, (într-un șir de atribuiri, eventual după conversie).

Exemple:

1. Fie declarațiile: `int a=5, b=3; float c;`. În urma atribuirii `c=a/b`, `c` va conține valoarea reală 1.0. Este normal să fie așa întrucât expresia `a/b` este de tip `int` (se efectuează împărțirea întreagă și se obține 1), apoi această valoare este atribuită variabilei `c` (după ce este convertită).

2. Fie declarațiile: `int a,b; float c;` și expresia `c=a=b=1`. După evaluare, **a** reține 1, **b** reține 1, **c** reține 1.0, iar valoarea rezultată în urma evaluării este 1.0 (reală).

3. Fie declarația: `int a=3;` și expresia `a*=2`. În urma evaluării, **a** reține valoarea 6, iar expresia va produce valoarea de tip `int` 6.



Observație foarte importantă! Sunt permise doar atribuiri în care tipul variabilei căreia i se atribuie conținutul unei variabile de alt tip, conține, printre valorile permise, cele ale tipului variabilei care este atribuită.

Exemple:

1. Fie declarațiile: `long a=2; int b=3;`
`a=b; //corect`
`b=a; //incorect`

2) Fie declarațiile: `char a=2; int b=3;`
`b=a; //corect`
`a=b; //incorect`

3) Fie declarațiile: `float a=2; int b=3;`
`a=b; //corect`
`b=a; //incorect`

4) Fie declarațiile: `float a=2; double b=3;`
`b=a; //corect`
`a=b; //incorect`



După cum observați, aici apare o diferență față de `C/C++`. **Java**, ca și **Pascal**-ul, încearcă să te ferească de erori ascunse, rezultate în urma conversiilor implicite.

2.2.8. Operatorul condițional

Se folosește în expresii de genul:

`exp1?exp2:exp3`

Cerința este ca **`exp1`** să fie de tipul `boolean`. Principiul de executare este următorul:

- se evaluează **`exp1`**;
- dacă aceasta produce `true`, se evaluează **`exp2`** și **`exp3`** este ignorată (nu se evaluează);
- altfel, se evaluează **`exp3`** și **`exp2`** este ignorată.

În ansamblu, expresia este de tipul lui **exp₂** sau **exp₃** și produce valoarea **exp₂** sau **exp₃** (în funcție de cea care se evaluează).

Exemple:

1. Variabila **c** ia valoarea 5 (valoarea maximă).

```
int a=1, b=5, c;
c=a>b?a:b;
```

2. Se afișează valoarea 5.

```
int a=1, b=5, c;
System.out.println(a>b?a:b);
```

3. Variabila **h** ia valoarea 9. Aici se observă că nu este obligatoriu ca **exp₂** și **exp₃** să aibă același tip:

```
int a=1, b=5; float d=9,h;
h=a>b?a:d;
```

2.2.9. Operatorul de conversie explicită

De multe ori, vrem ca unul sau mai mulți operanzi să intre în calcul convertiți așa cum dorim (nu implicit). Pentru aceasta, înaintea operandului se trece între paranteze tipul său.

Exemple:

1) Variabila **a** ia valoarea 9. Iată că, deși implicit conversia nu poate fi realizată, în acest mod s-a efectuat fără probleme.

```
int a=1; double d=9.6;
a=(int)d;
```

2) Se afișează 0.5. Iată că, deși, operanzii sunt întregi și rezultatul ar trebui să fie 0, prin utilizarea operatorului de conversie explicită se obține rezultatul corect.

```
int a=3, b=6;
System.out.println((float) a/b);
```

2.2.10. Prioritatea (precedența) operatorilor

Operatorii au o anumită prioritate (precedență). Mai jos, sunt prezentați operatorii în ordinea descrescătoare a priorității.

- 1) **() , . , []**
- 2) **++, --, +(unary), -(unary), !, new, operatorul de conversie explicită**
- 3) ***, /; %**

- 4) `+`, `-` (priviți ca operatori binari)
- 5) `<<`, `>>`, `>>>`
- 6) `<`, `<=`, `>`, `>=`
- 7) `==`, `!=`
- 8) `&`
- 9) `^`
- 10) `|`
- 11) `&&`
- 12) `||`
- 13) `?:`
- 14) `=`, `+=`, etc (operatori de atribuire)

2.3. Instrucțiuni

1. Instrucțiunea vidă. Instrucțiunea nu face nimic, dar are sens în programul `Java`, după cum va rezulta din studiul altor instrucțiuni. Instrucțiunea vidă se marchează prin `;`.

2. Instrucțiunea compusă. Se utilizează atunci când se dorește ca mai multe instrucțiuni să fie considerate ca o singură instrucțiune. Mai jos, puteți observa o instrucțiune compusă care subordonează mai multe instrucțiuni vide.

```
{
;
;
;
}
```

3. Instrucțiunile de incrementare și decrementare. Se utilizează pentru a incrementa sau decrementa o anumită variabilă.

Exemple:

```
a++; ++a; a--; --a;
```



Există o diferență între operatorii de incrementare (decrementare) și instrucțiunea de incrementare (decrementare). Instrucțiunea de incrementare (decrementare) conține numai un operator de incrementare (decrementare), dar de exemplu, `b=++a+1` este o instrucțiune (atribuire, vezi mai jos) care conține și un operator de incrementare.

4. Instrucțiunea de atribuire. Conține o expresie prin care unei variabile i se atribuie o altă expresie (aflată în dreapta operatorului de atribuire).

Exemple:

- a) `int a, b=6;`
`a=b+5; // după atribuire a reține 11.`
- b) `int a, b=6;`
`a=++b+5; // după atribuire a reține 12.`
- c) `int a=1, b=6;`
`a+=b; // după atribuire a reține 7.`
- d) `a=b=c=1; // a, b, c sunt de tip int.`

Întrebare: Ce valoare are `a` după următoarea instrucțiune:
`a=b=c=1;`

Lată o atribuire multiplă. Inițial, `c` ia valoarea 1, apoi `b=c`, atunci `b=1`, apoi `a=b`, deci `a=1`. După atribuire `a`, `b`, și `c` rețin 1.

Observații:

- ✓ Nu sunt permise atribuiri de genul: `++a=b+5;`
- ✓ Există o diferență între operatorul de atribuire și instrucțiunea de atribuire. De exemplu, `a=b=c=1` este o instrucțiune de atribuire, dar conține 3 operatori (de atribuire). Practic, o instrucțiune de atribuire conține cel puțin un operator de atribuire.

5. Instrucțiunea if. Are două forme:**Forma 1.**

`if (expresie de tip boolean) instrucțiune1 else instrucțiune2`

Principiul de executare este următorul:

- ⇒ se evaluează expresia;
- ⇒ dacă valoarea produsă de aceasta este `true`, se execută **instrucțiune₁**;
- ⇒ dacă valoarea produsă este `false` se execută **instrucțiune₂**.

Forma 2.

`if (expresie de tip boolean) instrucțiune`

Principiul de executare este următorul:

- ⇒ se evaluează expresia;
- ⇒ dacă valoarea produsă de aceasta este `true`, se execută instrucțiunea subordonată, altfel se trece la cea următoare.

Exemple: a) Se afișează cea mai mare valoare dintre valorile reținute de variabilele `a` și `b`:

```
if (a>b) System.out.println(a);
```

```
else System.out.println(b);
```

b) Se rezolvă ecuația $ax+b=0$, unde pentru valorile lui a și b sunt reținute variabilele cu același nume.

```
if (a!=0)
{
    x=-b/a;
    System.out.println(x);
}
else
    if (b==0) System.out.println("infinitate de solutii");
    else System.out.println("nu are solutie");
```

6. Instrucțiunea while. Forma generală este:

while (expresie de tip boolean) instrucțiune

Principiul de executare este următorul:

- ⇒ Se evaluează expresia;
- ⇒ Dacă valoarea produsă de aceasta este **true**, se execută instrucțiunea subordonată, apoi se revine la evaluarea expresiei, altfel se trece la instrucțiunea următoare.

Exemple:

a) Pentru $n>0$, număr natural, să se calculeze suma cifrelor sale (pentru $n=213$, se va afișa 6):

```
int n=213,s=0;
while (n!=0)
{
    s+=n%10;
    n/=10;
}
System.out.println(s);
```

b) Pentru n , număr natural, să se afișeze numărul obținut prin inversarea cifrelor sale (pentru $n=213$, se va afișa 312):

```
while (n!=0)
{
    ninv=ninv*10+n%10;
    n=n/10;
}
System.out.println(ninv);
```

7. Instrucțiunea do while. Forma generală a acestei instrucțiuni este următoarea:

do
instrucțiune
while(expresie);

Principiul de executare este următorul:

- ⇒ Se execută instrucțiunea subordonată;
- ⇒ Se evaluează expresia. În cazul în care valoarea produsă la evaluare este **false**, execuția instrucțiunii **do while** se termină, altfel se trece la pasul anterior..

Observație: secvența se execută cel puțin o dată, după care se pune problema dacă să se repete sau nu (prin evaluarea expresiei logice).

Exemple:

a) Se dă un număr natural **n**, mai mare sau egal cu 1. Să se calculeze suma primelor **n** numere naturale nenule.

```
int n=10,s=0,i=1;
do
{
    s=s+i;
    i=i+1;
} while (i<=n);
System.out.println(s);
```

b) Se dă un număr natural **n**, mai mare sau egal cu 1. Să se descompună **n** în factori primi.

```
int n=115,i=2,fm;
do
{
    fm=0;
    while (n%i==0)
    {
        fm++;
        n/=i;
    }
    if (fm!=0) System.out.println(i+" la puterea "+fm);
    i++;
}while (n!=1);
```

8. Instrucțiunea **for** are forma generală:

for (**expresie**_{inițializare}; **expresie**_{test}; **expresie**_{incrementare}) **instrucțiune**

După cum se vede, între paranteze se găsesc 3 expresii:

- **Expresie**_{inițializare} se folosește de regulă pentru inițializarea variabilei de ciclare. Este de remarcat faptul că în cadrul acestei expresii (cu rol special) este posibil chiar să declarăm variabila de ciclare (cu valoare inițială).
- **Expresie**_{test} este de tip **boolean** și se folosește pentru a testa dacă se execută instrucțiunea subordonată - dacă expresia produce la

evaluare **true**, instrucțiunea subordonată **for** se execută.

- **Expresie**_{incrementare} se folosește pentru incremenatarea variabilei de ciclare.

Principiul de executare:

P1. Se evaluează **expresie**_{initalizare} (un caz special este când aceasta conține și declarația variabilei de ciclare);

P2. Se evaluează **expresie**_{test}. În cazul în care aceasta produce **true**, se execută instrucțiunea subordonată **for**; apoi se trece la P3, altfel se trece la instrucțiunea următoare (se termină execuția instrucțiunii **for**).

P3. Se evaluează expresia de incrementare și se revine la P2.



Toate expresiile pot fi vide. În concluzie, expresiile de mai sus au rolul precizat în mod normal - dar nu obligatoriu și nici restrictiv. De exemplu, dacă **expresie**_{test} este vidă, se execută un ciclu infinit. **for(;;) instrucțiune.**

Exemple:

a) Se afișează numerele, 10, 9, ..., 1, fiecare pe câte un rând. Observați faptul că variabila **i** a fost declarată în cadrul ciclului **for**.

```
for (int i=10;i>=1;i--) System.out.println(i);
```

b) Se afișează alfabetul, de la 'a' la 'z' și pentru fiecare caracter se afișează și codul său.

```
for (char c='a';c<='z';c++)
    System.out.println(c+" "+(int)c);
```

c) Se calculează suma $S = 1 + 1 \cdot 2 + 1 \cdot 2 \cdot 3 + \dots + 1 \cdot 2 \cdot \dots \cdot n$

```
int i,s=0,p=1,n=3;
for(i=1;i<=n;i++)
{
    p*=i;//p=p*i;
    s+=p; //s=s+p;
}
System.out.println(s);
```

9. Instrucțiunea **break**.

Are rolul de a forța ieșirea dintr-un ciclu. În cazul în care există cicluri imbricate (un ciclu în interiorul altui ciclu), forțează doar ieșirea din primul ciclu (vezi secvența următoare, unde **for**-ul după **j** se execută doar pentru **j=1**). Se folosește pentru instrucțiunile **for**, **while**, **do while** și, în

particular, pentru **switch** (cazul va fi studiat separat). Analizați secvența de mai jos:

<code>for (int i=1; i<=4; i++)</code>	Se afișează
<code>for (int j=1; j<=4; j++)</code>	
<code>if (j==2) break;</code>	1 1
<code>else System.out.println(i+" "+j);</code>	2 1
	3 1
	4 1

10. **Instrucțiunea `continue`**; este folosită în cazul ciclurilor (**for**, **while**, **do while**) și are rolul de a determina ca, după executarea ei, să se sară direct la testul condiției de continuare. Astfel, se ignoră instrucțiunile care îi urmează în secvența care se repetă.

Exemplu: secvența următoare afișează. 1. 3. 4. Dacă **i** ia valoarea 2, atunci, se execută **continue** și se sare peste apelul metodei de afișare.

```
for (int i=1; i<=4; i++)
{ if (i==2) continue;
  System.out.println (i); }
```

11. **Instrucțiunea `switch`** are forma particulară:

```
switch (expresie)
{
    case exp1: secvență instrucțiuni1; break;
    case exp2: secvență instrucțiuni2; break;
    .....
    case expn: secvență instrucțiunin; break;
    [default: secvență instrucțiunin+1];
}
```

unde:

- *expresie* are semnificația: expresie de tip întreg (sau **char**);
- *exp_i*: sunt expresii constante de tip întreg (sau **char**);
- *instrucțiuni_i*; reprezintă o secvență oarecare de instrucțiuni.

Principiul de executare:

⇒ se evaluează expresia;

⇒ dacă aceasta produce o valoare egală cu cea produsă de **exp_i**, se execută, în ordine, **instrucțiuni_i**, și se trece la instrucțiunea următoare, altfel se execută numai secvența **instrucțiuni_{n+1}**.

❶ **Observație.** Alternativa **default** este facultativă. În absență, în cazul în care nu există coincidență de valori, se trece la instrucțiunea următoare.

Exemplu: Secvența care urmează probează instrucțiunea **switch**. În exemplu se afișează 1:

```
int i =1;
switch(i)
{
    case 1: System.out.println(1);break;
    case 2: System.out.println(2);break;
    default: System.out.println("altceva");
}
```

- ❷ **Observație.** Am prezentat o formă simplificată a instrucțiunii **switch**, mai precis, cea corespunzătoare programării structurate. În absența instrucțiunii **break**, în cazul în care există egalitate între **expresie** și **exp_i** se execută secvența corespunzătoare și, pe rând, toate secvențele care urmează, ș.a.m.d.

Exemplu: secvența care urmează afișează: 2, **altceva**.

```
int i =2;
switch(i)
{
    case 1: System.out.println(1);
    case 2: System.out.println(2);
    default: System.out.println("altceva");
}
```



În forma generală, instrucțiunea **switch** are rolul de a stabili mai multe puncte de intrare într-o secvență de instrucțiuni, în funcție de valoarea pe care o reține **expresie**.

12. Instrucțiunea apel metodă (funcție) va fi tratată la momentul potrivit. Pentru moment, putem da ca exemplu de astfel de instrucțiune doar **System.out.print()** sau **System.out.println()**.

13. Instrucțiunea de declarare a variabilelor. Până în prezent, am întâlnit-o de multe ori. Ea poate fi plasată oriunde în corpul unei metode (funcții).

Exemplu: `int x=2,y.`

2.4. Masive

În **Java** există un mecanism excepțional de lucru cu masive (vectori, matrice, etc).

Din motive didactice, pentru început, vom prezenta declararea masivelor în doi pași:

Pasul 1. Se declară o variabilă care poate reține o referință (adresă) către vector, matrice, etc.

Exemple:

a) `int[] v;` am declarat o variabilă referință către un vector cu componente de tip `int`. O declarație echivalentă poate fi făcută prin: `int v[];`

b) `int[][] Mat;` am declarat o variabilă referință către o matrice cu componente de tip `int`.

c) `float[] v, v1;` am declarat două variabile referință către vectori, numite `v`, respectiv `v1`. Ambii vectori au componente de tip `float`.

d) `float Mat[][] , Mat1[][];` am declarat două variabile de tip referință către matrice. Cele două variabile se numesc `Mat` și `Mat1`.

Pasul 2. Alocăm spațiu în memorie pentru masiv (vector matrice). Adresa vectorului va fi reținută în variabila `v`.

Exemple:

a) `v=new int[4];`. Am alocat spațiu în memorie pentru un vector cu 4 componente de tip `int` (practic am creat vectorul). Referința către el va fi reținută în `v`, și, pornind de la această variabilă putem accesa `v`.

b) `Mat=new int [4][3];`. Am alocat spațiu în memorie pentru o matrice cu 4 linii și 3 coloane cu componente de tip `int`.

⇒ Prima componentă a unui vector are indicele 0, a doua indicele 1 ș.a.m.d. Tot așa, liniile unei matrice se numerează începând cu 0, coloanele unei matrice se numerează începând cu 0.

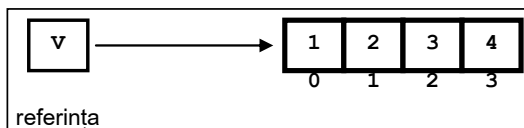
⇒ O componentă a unui vector se adresează pornind de la referință și scriind indicele între paranteze drepte (exemplu `v[3]`) o componentă a unei matrice se adresează pornind de la referință și scriind între paranteze pătrate fiecare din cei doi indici (exemplu `Mat [2][1]`).

Exemple:

a) În secvența următoare se creează un vector, se initializează fiecare componentă a lui și se afișează.

```
int[] v; // sau int v[];
v=new int[4]; int i;
for (i=0;i<4;i++) v[i]=i+1;
for (i=0;i<4;i++) System.out.println(v[i]);
```

Schema alăturată reflectă modul în care arată vectorul, după ce i-am inițializat componentele.



- ✓ Vectorii se pot declara și așa cum se vede mai jos, gata inițializați. În urma unei astfel de declarații se poate lucra direct cu ei, exact ca anterior.

```
int[] V={1,2,3,4}; int i;
for (i=0;i<4;i++) V[i]=i+1;
for (i=0;i<4;i++) System.out.println(V[i]);
```

- b) În secvența următoare se creează o matrice, se inițializează fiecare componentă a ei și se afișează.

```
int[][] Mat;
int i,j;
Mat=new int [4][3];
for (i=0;i<4;i++)
    for (j=0;j<3;j++) Mat[i][j]=i+j;
for (i=0;i<4;i++)
    { for (j=0;j<3;j++)
        System.out.print(Mat[i][j]+" ");
      System.out.println();
    }
```

- ✓ Ca și vectorii, matricele se pot declara și așa cum se vede mai jos, gata inițializate. În urma unei astfel de declarații se poate lucra direct cu ele, exact ca anterior.

```
int[][] Mat={{1,2,3,4},{3,4,5,6}};
int i,j;
for (i=0;i<2;i++)
    { for (j=0;j<3;j++)
        System.out.print(Mat[i][j]+" ");
      System.out.println();
    }
```

- ✓ Declararea unui masiv se poate face și într-un singur pas, ca mai jos:

```
float[] V=new float[4];
```

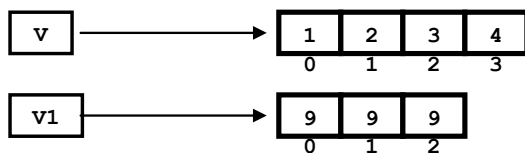
- ⇒ De vreme ce numele masivului (**V**, **Mat**, în exemplele date) reține o referință către masiv, înseamnă că unei astfel de variabile *i* se poate atribui o referință către alt masiv, ceea ce este echivalent cu faptul că atribuirea "funcționează" în cazul masivelor ca în cazul variabilelor de unul din tipurile de date primitive. Totuși, există o diferență esențială: *se atribuie doar referința, nu conținutul masivului*, care rămâne nemodificat. Masivul care nu mai este referit va fi șters, automat (fără ca, în acest scop, să fie scrisă o secvență specială).

Exemplu: Avem doi vectori, **v** și **v1**. Cei doi vectori sunt creați și inițializați. După aceasta, se interschimbă referințele către cei doi vectori și, în vederea probării operațiilor efectuate se afișează.

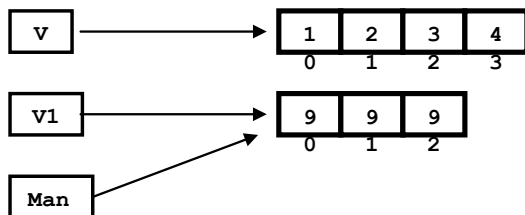
```
float[] V=new float[4], V1=new float[3], Man; int i;
for (i=0;i<4;i++) V[i]=i+1;
for (i=0;i<3;i++) V1[i]=9;
Man=V1; V1=V; V=Man;
for (i=0;i<4;i++) System.out.println(V1[i]);
for (i=0;i<3;i++) System.out.println(V[i]);
```

Grafic, interschimbarea de mai sus se poate prezenta astfel:

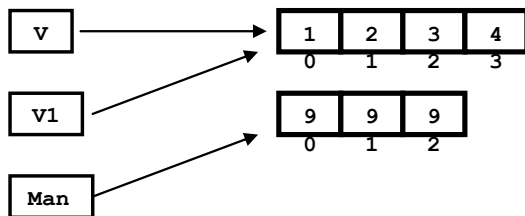
- După crearea și inițializarea celor doi vectori:



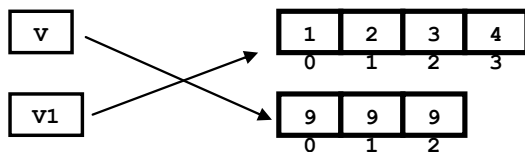
- **Man=V1;**



- **V1=V;**



- **V=Man;**



- ⇒ O altă utilizare de excepție a acestui mecanism este aceea prin care se declară un masiv exact cu numărul de componente dorit și nu un număr de componente mare, care se presupune că indiferent de intrare, nu poate fi depășit (ca în **Pascal**, **C/C++**). Aceasta conduce la economie de memorie.

```
int n;
// se citește n;
V=new int [n];
```

- ⇒ Deși din punct de vedere teoretic nu se poate explica în acest moment, numărul de componente ale unui vector (masiv unidimensional) este returnat de data membru (vom învăța despre date membru) **length**, apelată ca mai jos. În urma executării acestei secvențe se afișează 10.

```
int[] vector=new int[10];
System.out.println(vector.length);
```

- ⇒ O matrice poate fi privită (și, de fapt asta și este) ca un vector de vectori. Fie declarația **int Mat[][]=new int [4][5]**. Putem spune că avem un vector cu 4 componente. Fiecare componentă este la rândul ei, un vector cu 5 componente. Atunci, **length**, aplicată matricei, reține 4, și aplicată, de exemplu, componentei de indice 2 returnează 5. La executarea secvenței de mai jos se afișează 4 și 5.

```
int Mat[][]=new int [4][5];
System.out.println(Mat.length);
System.out.println(Mat[2].length);
```

- ⇒ În cazul matricelor (sau, în general, masivelor de dimensiune mai mare sau egală cu 2) există posibilitatea ca liniile să aibă lungime diferită. Cu alte cuvinte, o componentă a vectorului de vectori (matricea) poate avea un număr de componente, altă componentă, alt număr de componente. În secvența următoare am declarat o matrice cu 2 linii. Prima linie are 3 componente, a doua linie are două componente. În final, se afișează conținutul unui element al matricei.

```
int Mat[][]=new int [2][];
Mat[0]=new int[3]; Mat[1]=new int[2];
Mat[0][0]=1; Mat[0][1]=2;Mat[1][0]=3;
System.out.println(Mat[0][1]);
```

- ✓ Vă dați seama câtă economie de memorie se face?

Aplicație: secvența următoare creează, inițializează și afișează o matrice triunghiulară. Alături puteți observa rezultatul rulării. Este ceva faptul că pentru a reține o matrice triunghiulară se reține exact atât cât este necesar!

```
int[] vector=new int[10];
int Mat[][]=new int [4][]; int i, j, k=1;
for (i=0;i<4;i++)
{ Mat[i]=new int[i+1]; // declararea este instructiune
```

1
23
456
78910

```
        for (j=0;j<=i;j++)Mat[i][j]=k++; }  
for (i=0;i<4;i++)  
{ for (j=0;j<Mat[i].length;j++)  
    System.out.print(Mat[i][j]);  
  System.out.println();  
}
```

2.5. Citirea datelor de la tastatură

Limbajul **Java** conține o mulțime de facilități, dar pentru începători există o reală dificultate în a citi date de la tastatură. Pentru această operație este necesar să cunoaștem o serie de clase. Ca să le înțelegem, sunt necesare cunoștințe avansate de **Java**. Și nu putem aștepta până atunci, pentru că este relativ dificil să prezinti materia, mai ales la nivel de liceu, fără a putea citi ceva de la tastatură. Pentru a rezolva această problemă, am recurs, ca de altfel și în alte cursuri de Java, la prezentarea unei clase prin care să putem realiza citirea de la tastatură.

Clasa respectivă, o prezentăm acum și în cele ce urmează, nu ne vom referi decât la utilizarea ei. Ulterior, vom prezenta modul în care a fost realizată clasa precum și multe alte cunoștințe privind operațiile de intrare/ieșire. Iată clasa pe care o vom folosi (se numește **cin**):

```
import java.io.*;  
import java.util.*;  
class cin {  
    static InputStreamReader f = new InputStreamReader (System.in);  
    static BufferedReader tamp=new  BufferedReader(f);  
    static String linie() {  
        try { return tamp.readLine();}  
        catch (IOException e) {return ("Eroare");}  
    }  
    static String Token(){  
        StringTokenizer v=new StringTokenizer(linie());  
        if (v.hasMoreTokens())return v.nextToken();  
        else return"";}  
}
```

Textul de mai sus îl vom introduce în fișierul **cin.java**. Apoi, îl compilăm cu **javac cin.java**. În urma compilării rezultă fișierul **cin.class**. Acest fișier va fi utilizat *de orice program* care utilizează o metodă din clasa **cin**. Dacă fișierele **cin.java** și **cin.class** se vor găsi în folder-ul **bin**, atunci, fără nici o problemă, utilizăm aceste metode ca și cum ar fi ale limbajului Java (practic, orice program care apelează clasele respective le găsește în **bin**).

⇒ Metoda **linie()** are rolul de a citi întreaga linie pe care utilizatorul o introduce de la tastatură. Rezultatul este de tip referință la **String**, adică referință către șirul de caractere citit.

Exemplu: programul următor citește și afișează o linie introdusă de la tastatură:

```
class prima {
    public static void main(String[] args) {
        String l=cin.linie();
        System.out.println(l);}
}
```

Putem considera că un șir de caractere conține unul sau mai multe token-uri (unități lexicale). De exemplu, șirul "Ionel are 100 de lei noi" conține 6 token-uri ("Ionel", "are", "100", "de", "lei", "noi"). După cum observați, token-urile sunt separate printr-unul sau mai multe spații albe.

⇒ Metoda **Token()** citește primul **token** de pe linia introdusă de la tastatură și-l returnează.

Exemplu: în programul următor se citește și se afișează un **token**. Dacă se tastează șirul "Un token", se afișează token-ul "Un". **Rețineți:** metoda returnează numai primul token !

```
class prima {
    public static void main(String[] args) {
        String t=cin.Token();
        System.out.println(t); }
}
```

⇒ Dacă dorim să citim numai un caracter, vom utiliza metoda **charAt(nr)**, care returnează caracterul aflat pe poziția **nr** a șirului de caractere. **Atenție:** primul caracter al șirului are indicele 0.

Exemplu: în programul următor se citește și se afișează un caracter. Observați modul de utilizare al metodei **charAt()**.

```
class prima {
    public static void main(String[] args) {
        String l=cin.linie();
        char c=l.charAt(0);
        System.out.println(c);
    }
}
```

⇒ Dacă dorim să citim numere, putem folosi tot metoda **Token()**. În acest caz este sarcina noastră să convertim rezultatul către tipul dorit. În acest caz, vom utiliza metode ale unor clase, numite *înfășurătoare*, clase care vor fi studiate la momentul potrivit.

⇒ Metoda **Integer.parseInt(sir)** returnează șirul convertit către **int**.

Exemplu: Se citesc două numere naturale **a** și **b** și se afișează suma lor:

```
class prima {
    public static void main(String[] args) {
```

```
System.out.print("a=");  
int a=Integer.parseInt(cin.Token());  
System.out.print("b=");  
int b=Integer.parseInt(cin.Token());  
System.out.println(a+b);  
}
```

⇒ Metoda `Long.parseLong(sir)` returnează șirul convertit către `long`.

Exemplu: în secvența următoare, se citesc două numere naturale mai mari `a` și `b` și se afișează suma lor:

```
long a=Long.parseLong(cin.Token());  
long b=Long.parseLong(cin.Token());  
System.out.println(a+b);
```

⇒ Metoda `Float.parseFloat(sir)` returnează șirul convertit către `float`.

Exemplu: în secvența următoare, se citesc două numere reale `a` și `b` și se afișează suma lor:

```
float a=Float.parseFloat(cin.Token());  
float b=Float.parseFloat(cin.Token());  
System.out.println(a+b);
```

⇒ Metoda `Double.parseDouble(sir)` returnează șirul convertit către `float`.

Exemplu: în secvența următoare, se citesc două numere reale `a` și `b` și se afișează suma lor:

```
double a=Double.parseDouble(cin.Token());  
double b=Double.parseDouble(cin.Token());  
System.out.println(a+b);
```



Nu trebuie să vă îngrijorați că nu ați înțeles toate noțiunile prezentate. Pentru moment, luați-le ca atare. Toate vor fi studiate în amănunt. Pentru moment, să ne mulțumim cu faptul că putem citi și afișa un șir de caractere.

2.6. Probleme rezolvate

Întrucât cunoaștem deja operatorii și instrucțiunile și putem citi date cu ajutorul clasei `cin`, de acum putem să scriem mici programe. Toate instrucțiunile se vor găsi în `main()`.

1. Se citesc două numere reale. Se cere să se afișeze numărul cel mai mare dintre ele (calculul maximului).

```
class prima {
```

```

public static void main(String[] args) {
    System.out.print("a=");
    double a=Double.parseDouble(cin.Token());
    System.out.print("b=");
    double b=Double.parseDouble(cin.Token());
    if (a>b) System.out.println(a);
    else System.out.println(b);
}
}

```

2. Să se rezolve ecuația $ax+b=0$ ($a, b \in \mathbb{R}$). Numerele a și b se citesc de la tastatură.

```

class prima {
    public static void main(String[] args) {
        System.out.print("a=");
        double a=Double.parseDouble(cin.Token());
        System.out.print("b=");
        double b=Double.parseDouble(cin.Token());
        if (a==0)
            if (b==0) System.out.println("infinitate de solutii");
            else System.out.println("nu are solutie");
        else System.out.println(-b/a);
    }
}

```

3. Se citește n , număr natural mai mare decât 0. Să se calculeze $1+2+\dots+n$.

```

class prima {
    public static void main(String[] args) {
        System.out.print("n=");
        int i, s=0, n=Integer.parseInt(cin.Token());
        for (i=1; i<=n; i++) s+=i;
        System.out.println("Suma este " +s);
    }
}

```

- ✓ Observați modul în care se afișează, mai întâi, șirul "n=", și apoi se așteaptă introducerea lui n .
- ✓ Prin expresia: "Suma este " +s se convertește mai întâi tipul `int` către `String` (obiect care reține un șir de caractere), apoi cele două se concatenează. În final, se afișează șirul astfel obținut.
- ✓ Aceasta înseamnă că pentru a converti o valoare numerică, a , către șir, se poate face șirul sumă între șirul vid și a : ""+a. De reținut!

4. Se citește n , număr natural. Se cere să se calculeze suma cifrelor lui. De exemplu, dacă se citește 124 se va afișa 7. De acum, vom scrie numai metoda `main()`.

```

public static void main(String[] args) {

```

```
System.out.print("n=");
int s=0,n=Integer.parseInt(cin.Token());
while (n!=0)
{ s+=n%10;
  n=n/10;}
System.out.println("Suma cifrelor este " +s);
}
```

5. Se citește n , număr natural. Se cere să se afișeze numărul obținut prin inversarea cifrelor sale. De exemplu, dacă se citește 123, se va afișa 321.

```
public static void main(String[] args) {
    System.out.print("n=");
    int n=Integer.parseInt(cin.Token()), ninv=0;
    while (n!=0)
    { ninv=ninv*10+n%10;
      n=n/10;
    }
    System.out.println("Numarul este " +ninv);
}
```

6. Se citește n număr natural. Să se afișeze descompunerea sa în factori primi.

```
public static void main(String[] args) {
    System.out.print("n=");
    int n=Integer.parseInt(cin.Token()), i=2, fm;
    do
    { fm=0;
      while (n%i==0)
      { fm++;
        n/=i;
      }
      if (fm!=0) System.out.println(i+" la puterea "+fm);
      i++;
    } while (n!=1);
}
```

7. Se citește n și un vector cu n componente numere întregi. Să se afișeze conținutul componentelor vectorului.

```
public static void main(String[] args) {
    System.out.print("n=");
    int n=Integer.parseInt(cin.Token()), i;
    int V[]=new int[n];
    for (i=0;i<n;i++)
    { System.out.print("V["+i+"]=");
      V[i]=Integer.parseInt(cin.Token()); }
    for (i=0;i<n;i++) System.out.println(V[i]);
}
```

- ✓ Observați faptul că vectorul are exact n componente și nu a fost necesar ca acesta să fie dimensionat de la început cu un număr de componente presupus maxim.

8. Se citește n și un vector cu n componente numere reale. Să se afișeze valoarea maximă conținută de vector.

```
public static void main(String[] args) {
    System.out.print("n=");
    int n=Integer.parseInt(cin.Token()),i;
    double V[]=new double[n];
    for (i=0;i<n;i++)
    { System.out.print("V["+i+"]=");
      V[i]=Double.parseDouble(cin.Token());
    }
    double max=V[0];
    for (i=1;i<n;i++)
        if (max<V[i]) max=V[i];
    System.out.println("Valoarea maxima gasita="+max);
}
```

9. Se citește n și un vector cu n componente numere reale. Să se afișeze conținutul componentelor vectorului sortate crescător. Se va utiliza metoda buclelor.

```
public static void main(String[] args) {
    System.out.print("n=");
    int n=Integer.parseInt(cin.Token()),i;
    double man; boolean gasit;
    double V[]=new double[n];
    for (i=0;i<n;i++)
    { System.out.print("V["+i+"]=");
      V[i]=Double.parseDouble(cin.Token());
    }
    do
    { gasit=false;
      for (i=0;i<n-1;i++)
          if (V[i]>V[i+1])
          { man=V[i];
            V[i]=V[i+1];
            V[i+1]=man;
            gasit=true;}
    } while (gasit==true);
    for (i=0;i<n;i++) System.out.println(V[i]);
}
```

10. **Interclasare.** Se citesc m numere reale sortate crescător. De asemenea, se citesc n numere reale sortate crescător. Se cere să se afișeze cele $m+n$ valori în ordine crescătoare.

```
public static void main(String[] args) {
    System.out.print("m=");
    int m=Integer.parseInt(cin.Token()),i,j;
    double A[]=new double[m];
    for (i=0;i<m;i++)
    { System.out.print("A["+i+"]=");
      A[i]=Double.parseDouble(cin.Token());
    }
    System.out.print("n=");
}
```

```

int n=Integer.parseInt(cin.Token());
double B[]=new double[n];
for (j=0;j<n;j++)
{ System.out.print("B["+j+"]=");
  B[j]=Double.parseDouble(cin.Token());
}
int k=0;
double C[]=new double[m+n];
i=j=0;
while (i<m && j<n)
  if (A[i]<B[j]) C[k++]=A[i++];
  else C[k++]=B[j++];
if (i<m) for (j=i;j<m;j++) C[k++]=A[j];
else for (i=j;i<n;i++) C[k++]=B[i];
for (i=0;i<k;i++) System.out.println(C[i]);
}

```

11. Să se citească și să se afișeze o matrice cu m linii și n coloane, unde fiecare element reține un număr întreg.

```

public static void main(String[] args) {
  System.out.print("m=");
  int m=Integer.parseInt(cin.Token()),i,j;
  System.out.print("n=");
  int n=Integer.parseInt(cin.Token());
  int A[][]=new int[m][n];
  for (i=0;i<m;i++)
    for (j=0;j<n;j++)
    { System.out.print("A["+i+", "+j+"]=");
      A[i][j]=Integer.parseInt(cin.Token());
    }
  for (i=0;i<m;i++)
    { for (j=0;j<n;j++) System.out.print(A[i][j]+" ");
      System.out.println();
    }
}

```

12. Mai jos puteți observa o secvență simplă prin care se pot inversa liniile i și j ale unei matrice cu n linii și m coloane ($i, j < n$).

```

int Man[];
Man=A[i]; A[i]=A[j]; A[j]=Man;

```

Probleme propuse

1. Ce se va afișa în urma executării secvențelor următoare?

1.1)

<code>int x=3;</code>	a) 3;
<code>x=~x;</code>	b) 4;
<code>System.out.println(x);</code>	c) -3;
	d) -4.

1.2)

<code>int x=5;</code>	a) 0;
<code>x=x<<3;</code>	b) 2;
<code>System.out.println(x);</code>	c) 40;
	d) 5.

1.3)

<code>int x=5;</code>	a) 0;
<code>x=x>>1;</code>	b) 2;
<code>System.out.println(x);</code>	c) 5;
	d) 1.

1.4) `System.out.println(7>10);` a) eroare de execuție;
b) 7>10;
c) false;
e) true.

1.5) `float x=1/2;` a) 0;
`System.out.println(x);` b) 0.5;
c) Eroare;
d) 1/2.

1.6) `System.out.print(-15/4);` a) -3 -3;
`System.out.print(" ");` b) -4 2;
`System.out.print(-15%4);` c) -4 -2;
d) eroare.

2. Fie **x** și **y** două variabile logice. Să se demonstreze relațiile de mai jos (de Morgan):

- a) `!(x || y) == !x && !y.`
b) `!(x && y) == !x || !y.`

3. Evaluați expresiile următoare, precizând rezultatul:

- a) `x+1.5`, unde **x** este o variabilă întreagă ce conține numărul **7**;
b) `x<=4`, unde **x** este o variabilă întreagă ce conține numărul **4**;
c) `a+1>3`, unde **a** este o variabilă reală care conține numărul **3**;

- d) $6/2$;
- e) $x/y+y/x$ unde x și y sunt variabile întregi care conțin, 1 respectiv 6;
- f) $x<3 \ || \ x>6$ (unde x este o variabilă reală care reține 7);
- g) $x<3 \ \&\& \ x>6$ (la fel ca mai sus);

4. Fie x o variabilă de tip `int`. Pentru ce valori ale lui x expresia: $x+7==8*x$ ia valoarea `true`?

5. La fel ca mai sus, numai că expresia este: $2*x+3==6$.

6. Considerăm `long a=3; char b=2;` Care este valoarea produsă de expresia $a+b$, și care este tipul rezultatului?

- a) Nu se poate aduna conținutul unei variabile de tip `char` cu acela al unei variabile numerice. Compilatorul furnizează eroare de sintaxă la compilare;
- b) Rezultatul este 5 și este de tip `char`;
- c) Rezultatul este 5 și este de tip `long`.

7. Considerăm `int a=-10,b=3;` Care sunt valorile produse de expresiile a/b și $a\%b$?

- a) $-3,33333\dots3$ și 2;
- b) -3 și -1 ;
- c) -4 și 2.

8. Considerăm `int a=2;` Care este valoarea rezultată în urma evaluării expresiei $1/(2*a)$?

- a) 0.25;
- b) Operatorul `"/"` acționează numai asupra variabilelor de un tip real. Avem eroare de sintaxă.
- c) 0.

9. Considerăm `int a=2,b=5;` Care este valoarea rezultată în urma evaluării expresiei $a+3<=b+2$?

- a) `true`;
- b) `false`;
- c) Este obligatoriu să folosim paranteze, pentru că altfel nu se poate face evaluarea.

10. Considerăm `int a=5,b=5;` Care este valoarea rezultată în urma evaluării expresiei $a++/++b$?

- a) 1; b) 0,833333; c) 0.

11. Avem `int a=5,b=3`; Care este valoarea rezultată în urma evaluării expresiei `a&b+a|b+a^b`?

- a) 11; b) nedecidabil; c) 21.

12. Considerăm `int a=0,b=3`; Care este valoarea rezultată în urma evaluării expresiei `a+=b+1`?

- a) În partea stângă a unei atribuirii trebuie să figureze o singură variabilă și nu o expresie;
b) 4;
c) c) 3.

13. Considerăm `int a=0,b=3`; Care este valoarea rezultată în urma evaluării expresiei `a=1=b`?

- a) Eroare de sintaxă;
b) 3;
c) `true`.

14. Dacă `x` și `y` sunt de tip `int`, ce valori trebuie să rețină pentru ca expresia `(x | y) == (x ^ y)` să producă valoarea `true`?

- a) 3 3; b) 3 5; c) 4 7; d) 4 2.

15. Dacă `x` este de tip `int`, care dintre expresiile de mai jos produce valoarea 0, indiferent de conținutul variabilei `x`?

- a) `x & x`; b) `x | x`; c) `x ^ x`; d) `x | !x`.

16. Dacă `x` este de tip `int`, rezolvați ecuația: `(x & x) == x`.

17. Dacă `x` este de tip `int`, rezolvați ecuația: `(x ^ x) == x`.

18. Dacă `x` și `y` sunt de tip `int`, rezolvați ecuația:

$$(x \wedge x \wedge y) == y.$$

19. Dacă `x` este de tip `int`, rezolvați ecuația: `(x | ~x) == x`.

20. Care este funcția evaluată de expresia de mai jos (`x` este real)?

$$x < 2 ? x - 1 : x * x$$

21. Care este valoarea pe care o reține variabila `rez` după executarea secvenței de mai jos?

```
int k, rez;
k=4; rez=3+k/5;
```

- a) 3; b) secvența este gresită; c) 4.

22. Care dintre secvențele de mai jos este corectă din punct de vedere sintactic?

- | | | |
|----------------------------|---------------------------|--------------------------|
| a) <code>k=4;</code> | b) <code>k=4;</code> | c) <code>k=4;</code> |
| <code>if (k>=0);</code> | <code>if (k>=0)</code> | <code>if k>=0;</code> |
| <code>else;</code> | <code>then;</code> | <code>else;</code> |
| | <code>else;</code> | |

23. Care va fi valoarea variabilei `s` după executarea secvenței?

```
int i,s=0;
for (i=1;i<=10;i++)
{ i=i/10;
  s+=i;}
```

- a) 0; b) 10; c) Secvența ciclează; d) 1.

24. Ce se întâmplă dacă se rulează secvența următoare?

```
int a=1,b=6;
while (a!=b)
{ a++;
  b--;}
System.out.println(a);
```

- a) secvența ciclează; b) se afișează 3;
c) secvența conține erori de sintaxă;
d) nici una din variantele de mai sus nu este adevărată.
25. Ce se întâmplă dacă se rulează secvența următoare?

```
int x=3;
while (x!=x*x) x=x/2;
System.out.println(x);
```

- a) secvența ciclează; b) se afișează 1; c) se afișează 0;
d) nici una dintre variantele de mai sus nu este adevărată.

Testele de la 26 la 29 se referă la programul următor:

```
int x=3;
do
{ x--; x*=2;}
while (x!=10);
```

26. De câte ori se execută secvența subordonată instrucțiunii `do`?

- a) o dată b) de două ori c) de trei ori d) secvența ciclează

27. Înlocuiți cele două atribuiri subordonate instrucțiunii `do` cu una singură, astfel încât programele obținute să fie echivalente.

28. Dacă valoarea inițială a lui x este 5 ($x=5$), care trebuie să fie condiția de continuare (`while (x=?)`) astfel încât atribuiriile subordonate lui `do` să se execute exact de 4 ori?

- a) 10 b) 8 c) 50 d) 28

29. Pentru care dintre valorile inițiale ale lui x (de mai jos) programul nu ciclează?

- a) 0 b) 1 c) 2 d) 4

30. Care dintre atribuiriile de mai jos înlocuiește secvența celor trei atribuiri:

$x:=x-1; \quad x:=2*x; \quad x:=x+1;$

- a) $x:=2*x-1$ b) $x:=2*x-3$ c) $x:=2*x+1$ d) $x:=2*x$

31. Să se scrie un program care rezolvă ecuația $ax^2+bx+c=0$, unde: $(a,b,c \in \mathbb{R})$.

32. Să se calculeze valoarea funcției F , definită pe mulțimea numerelor reale, pentru un x citit:

$$F = \begin{cases} x*x-3, & x < 5 \\ x+1, & 5 \leq x \leq 25 \\ x*x-5*x+6, & x > 25 \end{cases}$$

33. Se citesc n numere reale. Să se afișeze valoarea minimă citită.

34. Se citește un șir de numere întregi până la întâlnirea numărului 0. Să se calculeze media aritmetică a numerelor din șir.

35. Citim un șir de numere întregi, la fel ca la problema precedentă. Să se calculeze suma dintre primul număr, al treilea, al cincilea,... și produsul dintre al doilea, al patrulea,... etc.

36. Se citește n , număr natural. Să se calculeze expresiile de mai jos (programe separate):

$$E_1 = 1 + \frac{1}{2^2} + \frac{1}{3^2} + \dots + \frac{1}{n^2};$$

$$E_2 = \frac{1}{2 \cdot 3} + \frac{2}{3 \cdot 4} + \dots + \frac{n}{(n+1)(n+2)};$$

$$E_3 = 1 + \frac{1}{1!} + \frac{1}{2!} + \dots + \frac{1}{n!}, \text{ unde } n! = 1 \cdot 2 \cdot \dots \cdot n.$$

37. Se citește o succesiune de cifre 1 și 0, prima fiind 1. Aceasta are semnificația unui număr în baza 2. Să se afișeze numărul în baza 10.

38. Se consideră șirul definit astfel (**FIBONACCI**).

$$u(n) = \begin{cases} 1, & \text{dacă } n = 1 \text{ sau } n = 2 \\ u(n-1) + u(n-2), & \text{altfel} \end{cases}$$

Se citește o valoare naturală $a > 0$. Să se calculeze $u(a)$.

39. Se consideră șirul definit la problema anterioară. Să se afișeze numărul de termeni care sunt strict mai mici decât o valoare citită.

40. Se citește un număr natural. Câte cifre conține?

41. Se citește un număr natural cu 5 cifre. Afișați numărul format după eliminarea cifrei din mijloc.

42. Un număr se numește perfect dacă este egal cu suma divizorilor săi (exclusiv el). **Exemplu:** $6=1+2+3$. Afișați toate numerele perfecte mai mici decât o valoare dată.

43. Afișați toate numerele naturale mai mici decât o valoare dată, care sunt egale cu suma pătratelor cifrelor lor.

44. Afișați primele n perechi de numere prime care sunt consecutive în mulțimea numerelor impare. **Exemplu:** $(3, 5)$, $(5, 7)$.

45. Se citește un număr natural par. Să se descompună în sumă de numere prime (conjectura **GOLDBACH**).

46. Se citește un vector cu n componente, numere naturale. Să se afișeze cel mai mare număr rațional subunitar în care numărătorul și numitorul fac parte din mulțimea valorilor citite. **Exemplu:** dacă am citit valorile 1 2 3 se afișează $2/3$.

47. Același enunț ca la problema anterioară, numai că se cere să se afișeze cel mai mic număr rațional supraunitar.

48. Se citește un vector cu n componente numere naturale. Se cere să se obțină toate permutările circulare la dreapta.

Exemplu: dacă $n=4$, și vectorul este 1 2 3 4, permutările circulare sunt:

1 2 3 4, 4 1 2 3, 3 4 1 2, 2 3 4 1.

Observați faptul că, de fiecare dată, ultimul element trece pe prima poziție, iar restul elementelor sunt deplasate la dreapta cu o poziție.

49. Se dau n numere raționale, reținute în doi vectori a și b . Astfel, numărătorul primului număr este $a[1]$, numitorul său este $b[1]$, etc. Programul va decide dacă numerele date sunt distincte. În cazul în care răspunsul este afirmativ, programul va afișa **DA**, altfel va afișa **NU** și o pereche de numere egale.

50. Se dau n numere raționale *distincte*, reținute în doi vectori a și b și m numere raționale distincte reținute în vectorii c și d . Se consideră că cele n numere alcătuiesc o mulțime X și cele m numere alcătuiesc o mulțime Y . Se cere să se afișeze:

a) $X \cup Y$;

b) $X \cap Y$;

c) $X - Y$.

Observație: pentru fiecare cerință se va face un program.

Răspunsuri:

1.1 d), 1.2 c), 1.3 b) 1.4 c) 1.5 a) 1.6 a) 4. $x=1$. 5. $x \in \emptyset$
6. c) 7. b) 8. c) 9. a) 10. c) 11. a) 12. b) 13. a) 14. d)
15. c) 16. Orice număr care poate fi reținut de tipul `int`; 17. $x=0$;
18. Orice x și y care pot fi reținute de tipul `int`; 19. $x=-1$;
20. $f(x)$ $x-1$ dacă $x < 2$, $x*x$ altfel; 21. a) 22. a) 23. c) 24. a)
25. b) 26. c) 27. $x=2*(x-1)$; 28. c); 29. d); 30. a).

CAPITOLUL 3

Metode



Din cuprins:

Generalități

Structura unei metode și apelul ei

Transmiterea parametrilor

Despre variabile

Exemple de utilizare ale metodelor

Metode recursive – exemple

Supraîncărcarea metodelor

Probleme propuse

3.1. Generalități

În **Java** se lucrează orientat spre obiect (prin programare orientată pe obiecte). Din rațiuni didactice, vom încerca să prezentăm lucrurile pornind de la ceea ce știm deja, dacă am lucrat, cât de cât, într-un alt limbaj de programare. Mai precis, vom învăța să lucrăm cu variabile (*date membru*) și funcții (*metode*) atașate clasei care conține `main()`. Aceasta ne permite să exersăm utilizarea subprogramelor în **Java**, fără a ne complica de la început cu mecanismele specifice programării pe obiecte. Acest fapt este important pentru că nu are sens ca absolut orice problemă să fie rezolvată prin mecanismele programării orientate pe obiecte.

- ⇒ În **Java**, există posibilitatea de a declara variabile vizibile la nivel de clasă, așa cum se vede în programul de mai jos, unde se citesc conținuturile numerice a două astfel de variabile și se afișează sumele lor. Ca să respectăm terminologia consacrată, le vom numi *date membru* (ale clasei care conține `main()`). Pentru moment, pentru a putea lucra ca acest mod este obligatoriu ca datele membru să fie precedate de modificatorul `static`. Justificările celor prezentate acum vor fi date ulterior.

```
public class Clasa1 {  
    static int a,b;  
    public static void main(String[] args) {  
        a=Integer.parseInt(cin.Token());  
        b=Integer.parseInt(cin.Token());  
        System.out.println(a+b);  
    }  
}
```

- ⇒ Orice dată membru (variabilă) a clasei este inițializată automat, la pornirea programului. Astfel, datele numerice sunt inițializate cu 0, datele de tip boolean cu `false`, datele de tip caracter cu caracterul de cod 0, datele de tip referință cu `null` (nu conțin nici o adresă). Cele afirmate pot fi verificate cu programul următor, dar caracterul de cod 0 nu este afișabil.

```
class prima {  
    static int a; static float b;  
    static char c; static boolean d;  
    public static void main(String[] args) {  
        System.out.println(a+" "+b+" "+c+" "+d);  
    }  
}
```

- ⇒ În **Java**, nu vom mai folosi termenul de funcție, ci un altul, specific programării orientate pe obiecte, numit *metodă*. Aceasta înseamnă că *atunci când spunem metodă, vom înțelege că vorbim despre o funcție (subprogram)*. Justificarea termenului va fi făcută în capitolul următor. Orice metodă atașată clasei de bază va fi precedată de modificatorul

static. Deocamdată, reținem acest lucru în mod mecanic. Motivația va fi dată, ca și în cazul precedent, în capitolul următor.

Exemplu de metodă:

1. Metoda `test()` are rolul de a afișa un mesaj. Observați modul în care am apelat-o din `main()`:

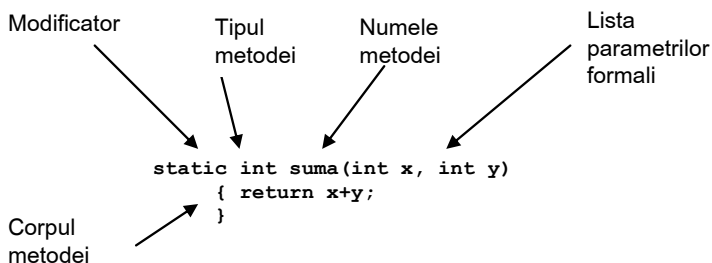
```
public class Clasa1 {  
    static void test()  
    { System.out.println("eu sunt o metoda");}  
  
    public static void main(String[] args) {  
        test();  
    }  
}
```

3.2. Structura unei metode și apelul ei

În esență, o metodă este alcătuită din:

- **Antet** - acesta conține mai multe informații importante necesare compilatorului, *numele metodei, lista parametrilor formali, tipul metodei și, eventual, modificatori.*
- **Corpul metodei** - acesta cuprinde instrucțiunile metodei.

Exemplu: mai jos, puteți observa metoda `suma()` ce are rolul de a calcula suma a două numere naturale. În schema atașată, puteți identifica elementele constitutive ale ei:



- **Numele metodei** – se utilizează pentru a o identifica atunci când metoda este apelată.
- **Lista parametrilor formali** - parametri formali sunt valorile pe care metoda le primește la apel. În exemplu, lista este formată din doi parametri formali, **x** și **y**, ambii de tipul **int**.

- **Tipul metodei** – sau altfel spus, precizează natura rezultatului. În exemplu, metoda returnează o valoare întreagă (de tip `int`), dar pot fi metode de alt tip, cum ar fi `float` `double`, etc. Există și metode care nu returnează nimic, caz în care tipul metodei este `void`.
- **Corpul metodei** – este sub forma unei instrucțiuni compuse. În exemplu, acesta conține o singură instrucțiune, `return`, prin care se calculează și se returnează suma parametrilor formali.
- **Modificator** – `static`

Mai jos puteți observa modul în care se apelează metoda `suma()`:

```
public class Clasa1 {  
    static int suma(int x, int y)  
    {return x+y;}  
  
    public static void main(String[] args) {  
        System.out.println(suma(2,3));  
        int x=2, y=5;  
        System.out.println(suma(x,y));  
        System.out.println(suma(3,y));  
    }  
}
```

Prin `suma(2,3)` se apelează metoda `suma()`, unde `x=2`, și `y=3`. Metoda returnează suma parametrilor (5) și valoarea este afișată. Faptul că apelul metodei este scris direct în metoda `System.out.println()`, înseamnă că metoda este, în expresia din `System.out.println()`, evaluată ca o valoare (în exemplu, de tip `int`). Prin `suma(x,y)`, se apelează metoda `suma()`, unde parametrii sunt valorile variabilelor `x` și `y` (2 și 5).

⇒ Metodele care sunt de orice alt tip în afara tipului `void`, trebuie să returneze o valoare din tipul respectiv. Returnarea valorii se face prin utilizarea instrucțiunii `return`. În exemplu, metoda `suma()` prin `return x+y;` returnează suma celor doi parametrii primiți.

Mai jos puteți observa modul în care se apelează o metodă de tip `void`:

```
public class Clasa1 {  
    static void met()  
    { System.out.println("Eu sunt o metoda de tip void"); }  
  
    public static void main(String[] args) {  
        met(); }  
}
```

⇒ O metodă se apelează printr-o instrucțiune de apel. Un exemplu de astfel de instrucțiune de apel este apelul de mai sus: `met()`. Sunt permise și apeluri de genul `suma(x,y)`, deși nu întotdeauna au sens. De asemenea, se pot utiliza expresii prin care se apelează o metodă: de exemplu, `System.out.println(1+suma(2,3))`, unde expresia este: `1+suma(2,3)`.

- ⇒ Parametrii care figurază în apelul unei metode se numesc **parametri efectivi**. De exemplu, în apelul `suma(2,3)`, parametrii efectivi sunt valorile 2 și 3. În apelul `suma(x,y)` parametrii efectivi sunt conținuturile variabilelor `x` și `y`.
- ⇒ Metodele se pot apela una pe alta, indiferent de poziția pe care o ocupă definiția lor în cadrul clasei. În exemplul de mai jos, metoda `met1()` apelează metoda `met()`, deși aceasta din urmă este definită, în cadrul clasei, după ea.

```
class prima {  
    static void met1()  
    { System.out.println("Met1");  
      met();}  
  
    static void met()  
    { System.out.println("Met");}  
  
    public static void main(String[] args) {  
        met1();  
    }  
}
```

3.3. Transmiterea parametrilor

Așa cum am arătat, parametrii care se găsesc în antetul metodei se numesc *parametri formali*, iar cei care se găsesc în instrucțiunea de apel se numesc *parametri efectivi*.

Între parametri formali și cei efectivi trebuie să existe o anumită concordanță, care este descrisă prin regulile următoare:

- ⇒ **Numărul** parametrilor formali trebuie să coincidă cu numărul parametrilor efectivi.
- ⇒ **Tipul** parametrilor formali trebuie să coincidă cu **tipul** parametrilor efectivi sau **tipul** parametrilor efectivi să poată fi convertit implicit (ca în cazul atribuirii) către **tipul** parametrilor formali la fel ca în cazul atribuirii.
- ⇒ Nu este obligatoriu ca **numele** parametrilor formali să coincidă cu **numele** parametrilor efectivi.

Exemple pentru metoda `static int suma(int x, int y)`:

a) prin apelul `suma(1,2)` se apelează metoda `suma()` cu două valori de tip `int`, 2 și 3. În acest caz, numărul parametrilor formali este 2 și numărul parametrilor efectivi este tot 2.

b) prin apelul `suma('1',3)` am apelat metoda `suma()` cu doi parametri: unul de tip `char`, altul de tip `int`. Apelul este corect pentru că o variabilă, constantă, de tip `char` poate fi convertită către tipul `int` (la fel ca la atribuire).

c) dacă **x**, de exemplu, este tipul **long**, atunci apelul **suma(x,y)** nu este un apel corect pentru că nu sunt permise conversiile de la **long** la **int**. La fel, nu este permis apelul de **float** la **int**, etc.

d) parametrii efectivi pot fi expresii, ca în apelul **suma(2-3,3+4)**, unde se returnează 6. În exemplu, se apelează **suma(-1,7)**.

e) parametrii efectivi pot conține și operatorul de conversie explicită, ca în apelul: **suma(2, (int)3.7)**, unde se returnează 5.

⇒ În **Java**, parametrii se transmit numai prin valoare. Aceasta înseamnă că parametrul efectiv, chiar dacă este variabilă, rămâne nemodificat. Priviți exemplul următor:

```
public class Clasa1 {  
    static void exemplu(double x)  
    { x=3.7; }  
  
    public static void main(String[] args) {  
        double x=5; exemplu(x);  
        System.out.println(x);  
    }  
}
```

În **main()** avem o variabilă **x**, de tip **double**, inițializată cu 5. Apelăm metoda de tip **void exemplu()**. În metodă, lui **x** i se atribuie altă valoare (3.7). După executarea metodei, se revine în **main()** și se afișează **x**. Valoarea afișată este cea inițială, 5. Conținutul variabilei **x** din **main()** a rămas nemodificat. Cum este posibil acest lucru?

⇒ Pentru fiecare parametru formal, metoda apelată reține în *stivă* (o memorie specială, asupra căreia vom reveni) un spațiu. În mod practic, *se creează o variabilă cu numele parametrului formal*.

Prin urmare, metoda **exemplu()** a creat în *stivă* o variabilă cu numele **x**. Acestei variabile i s-a atribuit valoarea 3.7. Variabila **x** din **main()** este cu totul alta. Din acest motiv, conținutul lui **x** din **main()** rămâne nemodificat.

⇒ Am arătat faptul că numele unui masiv este o referință către masiv ce se găsește în memorie. În acest caz, transmiterea unei referințe către un masiv ca parametru al unei metode are ca efect posibilitatea modificării conținuturilor componentelor masivului.



Întrucât masivele conțin și data membru **length**, *dacă o metodă primește ca parametru o referință către masiv*, atunci nu mai este necesar ca, în alte limbaje de programare, să transmitem și numărul de componente ale masivului.

Exemplu:

Metoda `exemplu()` calculează suma numerelor reținute de un vector și face ca fiecare componentă a vectorului să rețină valoarea 1. În programul principal se inițializează vectorul, apoi se probează efectul metodei.

```
public class Clasa1 {
    static int exemplu(int[] t )
    { int s=0,i;
      for (i=0;i<t.length;i++) s+=t[i];
      for ( i=0;i<t.length;i++) t[i]=1;
      return s;
    }
    public static void main(String[] args) {
        int[] v=new int [5];
        int i;
        for (i=0;i<5;i++) v[i]=Integer.parseInt(cin.Token());
        System.out.println(exemplu(v));
        for (i=0;i<v.length;i++) System.out.print(v[i]+" ");
    }
}
```

3.4. Despre variabile

Până în prezent am utilizat trei feluri de variabile:

1. **În corpul metodelor.** Aceste variabile se numesc *variabile locale*.
2. **În antetul metodelor.** Si acestea sunt, după cum vom vedea, tot variabile și se numesc *parametri*.
3. **La nivelul clasei.** Acestea se numesc, așa cum am arătat *date membru*.

⇒ În cadrul metodelor, parametrii sunt tratați precum variabilele locale și pot fi accesați din orice punct al metodei. De exemplu, dacă avem metoda `static int met(int a)` și o apelăm prin `met(2)`, metoda va avea o variabilă locală `a` care va fi inițializată cu 2.



Dacă avem un parametru cu un anumit nume, atunci nu mai este permis să declarăm variabile locale cu acest nume. De exemplu, *secvența următoare dă eroare de compilare*, pentru că numele variabilei locale coincide cu numele parametrului.

```
static void met(int a)
{ int a=7;
  System.out.println(a);
}
```

- ⇒ Variabilele locale pot fi accesate din blocul în care au fost declarate și, imediat ce au fost executate instrucțiunile din acel bloc, nu vor mai exista. De exemplu, în metoda de mai jos am declarat două variabile locale, ambele cu numele `i`. Prima este declarată în `for` și este recunoscută de blocul (în exemplu instrucțiunea) subordonat. După încheierea execuției `for`-ului, această variabilă nu mai există, prin urmare *poate fi declarată din nou*.

```
static void met()
{ for (int i=1;i<=3;i++) System.out.println(i);
  int i=5;
  System.out.println(i); }
```

- ⇒ În cazul în care avem o *data membru* cu un anumit nume și dacă într-o metodă declarată cu modificatorul `static` (și până acum am dat numai astfel de exemple) avem o variabilă locală (parametru) cu același nume, atunci din metodă se poate adresa doar variabila locală (parametrul). De exemplu, programul de mai jos afișează 2, adică valoarea variabilei locale `a` și nu 3, valoarea reținută de data membru.

```
public class Clasa1 {
  static int a=3;
  public static void main(String[] args) {
    int a=2;
    System.out.println(a);
  }
}
```

- ⇒ Parametrii și variabilele locale sunt memorate într-o structură specială de date, numită **stivă**. Stiva funcționează pe principiul **LIFO (Last In First Out)** - "*ultimul intrat, primul iese*".

Pentru a înțelege modul de lucru cu stiva, ne imaginăm un număr `n` de farfurii identice, așezate una peste alta (o "stivă" de farfurii). Adăugarea sau scoaterea unei farfurii se face cu ușurință, numai în vârful stivei. După ce am scos toate farfuriile din stivă, spunem că aceasta este vidă. Oricât ar părea de simplu principiul stivei, el are consecințe uriașe în programare. Vedeti recursivitatea!

3.5. Exemple de utilizare ale metodelor



3.5.1. Exemple elementare

1. Se citesc două numere întregi `m` și `n`. Se cere să se tipărească cel mai mare divizor comun și cel mai mic multiplu comun al lor.

Rezolvare. Presupunem cunoscut algoritmul care determină cel mai mare divizor comun pentru două numere. Cel mai mic multiplu comun al două

numere se poate determina împărțind produsul lor la cel mai mare divizor comun al lor. Prin urmare, scriem o metodă `cmmdc` cu doi parametri formali `m` și `n`, care întoarce o valoare întreagă - cel mai mare divizor comun al lor.

```
public class Clasa1 {
    static int cmmdc(int m, int n)
    { while (m!=n)
      { if (m>n) m-=n;
        else    n-=m;
        return m; }
    public static void main(String[] args) {
        int m=Integer.parseInt(cin.Token());
        int n=Integer.parseInt(cin.Token());
        int cm=cmmdc(m,n);
        System.out.println(cm); }
}
```

2. Se citesc două numere naturale $m < n$. Se cere să se tipărească toate numerele palindrom aflate între `m` și `n`. Un număr este palindrom dacă citit direct și invers rezultatul este același. Exemplu: 121.

Rezolvare. Să analizăm problema. Este necesar să fie testate toate numerele între `m` și `n`, pentru ca apoi să fie tipărite numai cele care îndeplinesc condiția cerută. Pentru aceasta, vom scrie o metodă numită `palin()` care are rolul de a testa dacă numărul este sau nu palindrom. Evident, metoda trebuie să fie de tipul `boolean`.

Care este *avantajul* scrierii metodei?

Acesta este dat de faptul că, atunci când scriem metoda, ne concentrăm exclusiv asupra ei, deci posibilitatea de a greși este minimă. Dacă dispunem de metodă, algoritmul este extrem de simplu: `for` de la `m` la `n`. În plus, este posibil ca această metodă să poată fi folosită și în alte cazuri.

```
public class Clasa1 {
    static boolean palin(int i)
    { int isalv=i, iinv=0;
      while (i!=0)
      { iinv=iinv*10+i%10;
        i=i/10; }
      return isalv==iinv;
    }
    public static void main(String[] args) {
        int m=Integer.parseInt(cin.Token());
        int n=Integer.parseInt(cin.Token());
        for(int i=m; i<n; i++)
            if (palin(i)) System.out.println(i);
    }
}
```

3. Superpalindrom. Un număr natural este palindrom dacă citit de la stânga la dreapta și de la dreapta la stânga rămâne nemodificat. De exemplu, numărul 12321 este palindrom. Un număr natural este superpalindrom dacă este palindrom atât el cât și pătratul său. Scrieți un program care listează toate numerele cu această proprietate aflate între doi întregi a și b .

Rezolvare. Modul în care testăm dacă un număr este palindrom este acum lămurit. Mai mult, avem și metoda. Nu trebuie decât să o folosim:

```
public class Clasa1 {
    static boolean palin(int i)
    { int isalv=i,iinv=0;
      while (i!=0)
      { iinv=iinv*10+i%10; i=i/10;}
      return isalv==iinv;
    }
    public static void main(String[] args) {
        int m=Integer.parseInt(cin.Token());
        int n=Integer.parseInt(cin.Token());
        for(int i=m;i<n;i++)
            if (palin(i))
                if (palin(i*i)) System.out.println(i+" "+i*i);
    }
}
```

4. Numere rotunde. Se citește $n < 10000$. Se cere să se afișeze numărul de numere mai mici sau egale cu n care, în binar, au același număr de cifre 0 și 1. De exemplu, 9 este un astfel de număr $9_{10}=1001_2$.

Rezolvare. Trebuie testate numerele de la 1 la n . Prin urmare folosim instrucțiunea `for` cu i de la 1 la n . Acum trebuie testat dacă, în binar, fiecare i are tot atâtea cifre 0 câte cifre 1. Pentru asta folosim o metodă booleană, numită `valabil`.

```
public class Clasa1 {
    static boolean valabil(int nr)
    { int zero=0,unu=0;
      while (nr!=0)
      { if (nr%2!=0) unu++;
        else zero++;
        nr=nr / 2; }
      return zero==unu;
    }
    public static void main(String[] args) {
        int n=Integer.parseInt(cin.Token()),s=0;
        for(int i=1;i<=n;i++)
            if (valabil(i)) s++;
        System.out.println(s);
    }
}
```

3.5.2. Un exemplu de backtracking în Java

Nu prezentăm aici tehnica **backtracking**. Este totuși, o carte dedicată limbajului Java și spațiul nu ar permite prezentarea integrală a tehnicilor de programare. Oricum, majoritatea dintre dvs. cunosc deja tehnica, iar dacă nu o cunosteți, recomand să o citiți din manualele de informatică pentru liceu.

- **Generarea permutărilor.** Se citește n , se cere să se afișeze toate permutările multimii $\{1, 2, \dots, n\}$. Observați modul de construcție și utilizare a metodelor.

```
public class Clasa1 {

    static void Init(int k,int[] st)
    { st[k]=0; }

    static boolean Am_Succesor (int n, int k,int[] st)
    { boolean rasp=false;
      if (st[k]<n) {
        st[k]++;
        rasp=true;}
      return rasp;}

    static boolean E_Valid(int k,int[] st)
    { boolean rasp=true;
      for (int i=1;i<k;i++)
        if (st[i]==st[k]) rasp=false;
      return rasp; }

    static boolean Solutie(int k, int n)
    { return k==n;}

    static void Tipar(int n, int[] st)
    { for (int i=1;i<=n;i++) System.out.print(st[i]);
      System.out.println();}

    static void back(int n)
    { boolean AS;
      int k=1;
      int[] st=new int[n+1];
      Init(k,st);
      while (k>0)
      { do { } while ((AS=Am_Succesor(n,k,st)) && !E_Valid(k,st));
        if (AS)
          if (Solutie(k,n)) Tipar(n,st);
          else {k++;
                Init(k,st);}
        else k--; }
    }

    public static void main(String[] args) {
      int n=Integer.parseInt(cin.Token());
      back(n);
    }
}
```




Se puteau folosi date membru pentru **n** și **st**. În acest caz, am fi avut mai puțini parametri...

3.6. Metode recursive

În **Java**, metodele pot fi recursive. Aceasta înseamnă că o metodă se poate autoapela.

Exemplul 1. Să se calculeze recursiv **n!**. Pentru a scrie o metodă recursivă care efectuează același calcul, vom porni de la o definiție recursivă a lui **n!**. Aceasta este:

$$n! = \text{fact}(n) = \begin{cases} 1, & n = 0 \\ n \cdot \text{fact}(n-1), & \text{altfel} \end{cases} \quad n \in \mathbb{N}$$

De exemplu, pentru a calcula **3!**, procedăm astfel:

$$3! = \text{fact}(3) = 3 \times \text{fact}(2) = 3 \times 2 \times \text{fact}(1) = 3 \times 2 \times 1 \times \text{fact}(0) = 3 \times 2 \times 1 \times 1 = 6.$$

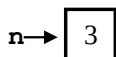
Metoda recursivă **fact()** nu face altceva decât să transcrie definiția recursivă prezentată anterior:

```
public class Clasa1 {
    static int fact(int n)
    { if (n==0) return 1;
      else return n*fact(n-1);
    }
    public static void main(String[] args) {
        System.out.println(fact(3));
    }
}
```

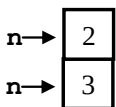
Care este mecanismul prin care subprogramele se pot autoapela? Să ne amintim modul în care subprogramele memorează parametrii transmiși:

- Pentru memorarea parametrilor, subprogramele folosesc o zonă de memorie numită **stivă**.
- Stiva este o structură de date de tip special. Astfel, ultima dată introdusă în stivă este prima care se poate accesa (scoate din stivă). Apoi, se accesează iar ultima, dar care în cazul inițial era prima ș.a.m.d.
- Memorarea parametrilor transmiși se face în ordinea în care aceștia figurează în antet: de la stânga la dreapta.

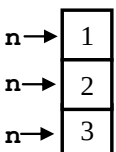
În continuare, prezentăm grafic modul în care funcționează recursivitatea în cazul metodei `fact()` a programului anterior. Această metodă nu are variabile locale, deci în stivă se depune doar parametrul `n`.



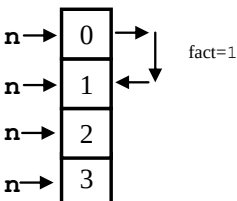
La primul apel al metodei `fact()` se creează în stivă o variabilă numită `n`, care reține **3** - valoarea parametrului formal.



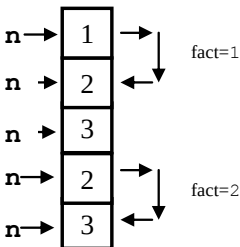
Metoda se autoapelează. De această dată, parametrul `n` ia valoarea **2**. În stivă se creează un nou nivel care reține `n` cu valoarea **2**.



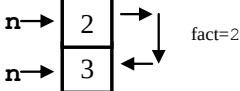
Metoda se autoapelează. De această dată parametrul `n` ia valoarea **1**. În stivă se creează un nou nivel, care reține `n` cu valoarea **1**.



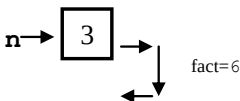
Metoda se autoapelează. Parametrul `n` ia valoarea **0**. În stivă se creează un nou nivel, care reține `n` cu valoarea **0**. Funcția ia valoarea **1**, autoapelul nu mai are loc. Se revine pe nivelul **3**.



Pe nivelul **3** se efectuează calculul $1*1=1$. Se revine apoi pe nivelul **2**.



Pe nivelul **2** se efectuează calculul $2*1=2$. Se revine apoi pe nivelul **1**.



Pe nivelul **1** se efectuează calculul $3*2=6$. Se revine apoi în `main()`.

Aici se afișează **6**, adică s-a calculat **3!**.

3.7. Exemple de utilizare a metodelor recursive

3.7.1. Exemple elementare

1. Se citește $x \in \mathbb{Z}$. Se cere programul pentru calculul funcției:

$$F(x) = \begin{cases} x - 1, & x \geq 12 \\ F(F(x + 2)), & x < 12 \end{cases}$$

```
public class Clasa1 {
    static int manna (int x)
    { if (x>=12) return x-1;
      else return manna(manna(x+2));
    }
    public static void main(String[] args) {
        int n=Integer.parseInt(cin.Token());
        System.out.println(manna(n));
    }
}
```

2. Se dau două numere naturale, a și b . Se cere să se calculeze cel mai mare divizor comun al lor.

Pentru rezolvare, utilizăm o definiție recursivă a celui mai mare divizor comun pentru două numere naturale a și b .

$$\text{cmmdc}(a, b) = \begin{cases} a, & a = b \\ \text{cmmdc}(a - b, b), & a > b \\ \text{cmmdc}(a, b - a), & a < b \end{cases}$$

Această definiție este transcrisă în metoda recursivă `cmmdc`.

```
public class Clasa1 {
    static int cmmdc (int a,int b)
    { if (a==b) return a;
      else
        if (a>b) return cmmdc(a-b,b);
        else return cmmdc(a,b-a);
    }
    public static void main(String[] args) {
        int a=Integer.parseInt(cin.Token());
        int b=Integer.parseInt(cin.Token());
        System.out.println(cmmdc(a,b));
    }
}
```

3. Să se scrie o metodă recursivă pentru a calcula suma cifrelor unui număr natural.



Ideea: se izolează ultima cifră, iar lui n i se atribuie câtul întreg dintre vechea valoare și 10. Această idee folosește pentru a găsi o relație de recurență, necesară elaborării variantei recursive:

$$S(n) = \begin{cases} 0, & n = 0 \\ n \% 10 + S(n / 10), & \text{altfel} \end{cases}$$

Programul de mai jos, calculează suma utilizând relația prezentată.

```
public class Clasa1 {
    static int s(int n)
    { if (n==0) return 0;
      else return n%10 +s(n/10); }
    public static void main(String[] args) {
        int n=Integer.parseInt(cin.Token());
        System.out.println(s(n)); }
}
```

4. Să se scrie o metodă recursivă pentru a transforma un număr natural n , din baza 10 în baza k ($1 < k < 10$).

Să ne amintim algoritmul clasic de trecere din baza 10 în baza k . Numărul se împarte la k , se reține restul, câtul se împarte la k , se reține restul ... până când câtul este mai mic decât împărțitorul. Rezultatul se obține prin scrierea în ordine inversă a resturilor obținute. Practic, afișarea restului se face după autoapel.

```
public class Clasa1 {
    static void transform(int n,int b)
    { int rest=n%b;
      if (n>=b) transform(n/b,b);
      System.out.print(rest);}
    public static void main(String[] args) {
        int n=Integer.parseInt(cin.Token());
        int b=Integer.parseInt(cin.Token());
        transform(n,b);}
}
```

3.7.2. Backtracking recursiv

1. Se citește n , număr natural. Să se afișeze permutările mulțimii $\{1, 2, \dots, n\}$. Se cere să se aplice un algoritm de generare recursiv.

Rezolvare. Vom folosi o metodă standard, recursivă de generare a permutărilor. Observați faptul că, de această dată, `st` și `n` sunt date membru.

```
public class Clasa1 {
    static int[] st=new int[10];
    static int n;
    static void init(int k)
    {st[k]=0;}
    static boolean sucesor (int k)
    { if (st[k]<n)
      { st[k]++;
        return true;}
      else return false;
    }
    static boolean valid (int k)
    { boolean ev=true;
      for (int i=1;i<=k-1;i++)
        if (st[k]==st[i]) ev=false;
      return ev;
    }
    static boolean solutie(int k)
    { return k==n+1;}
    static void tipar()
    { for (int i=1;i<=n;i++)
      System.out.print(st[i]);
      System.out.println();
    }
    static void back(int k)
    { if (solutie(k)) tipar();
      else
      { init(k);
        while(sucesor(k))
          if (valid(k)) back(k+1);
      }
    }
    public static void main(String[] args) {
      n=Integer.parseInt(cin.Token());
      back(1);
    }
}
```

2. Se citesc `n` și `p`, numere naturale. Să se afișeze toate aranjamentele de `n` luate câte `p`.

Rezolvare. Singura diferență față de algoritmul anterior este că, de această dată, o soluție are lungimea `p`. Aceasta atrage modificarea metodelor `solutie()` și `tipar()`. În plus, va trebui sa-l citim și pe `p`. În rest, programul este același.

```
public class Clasa1 {
```

```

static int[] st=new int[10];
static int n,p;
...
static boolean solutie(int k)
{ return k==p+1;}
...
static void tipar()
{ for (int i=1;i<=p;i++)
    System.out.print(st[i]);
  System.out.println();
}
...
public static void main(String[] args) {
  n=Integer.parseInt(cin.Token());
  p=Integer.parseInt(cin.Token());
  back(1);
}

```

3. Se citesc n și p , numere naturale. Să se afișeze toate combinațiile de n luate câte p .

Rezolvare. Programul este aproape la fel cu programul de generare a aranjamentelor. Practic, elementele se generează în st în ordine strict crescătoare. Aceasta elimină și posibilitatea de generare dublă a lor (de exemplu $\{2,1\}$, $\{1,2\}$). În plus, dacă elementele sunt generate în ordine strict crescătoare, dispăre condiția prin care se verifică dacă sunt distincte sau nu. Aceasta înseamnă că metoda `valid()` devine inutilă. În acest caz, ori determinăm ca metoda `valid()` să returneze întotdeauna `true`, păstrând structura metodei `back()` ca mai jos, ori mai eficient, n-o mai apelăm.

```

static void init(int k)
{ if (k==0) st[k]=0;
  else st[k]=st[k-1];}
static boolean valid (int k)
{ return true; }

```

3.8. Supraîncărcarea metodelor

⇒ În Java, metodele pot fi *supraîncărcate*. Aceasta înseamnă că putem avea două sau mai multe metode cu același nume, dar care diferă prin tipul și/sau numărul parametrilor. Priviți programul de mai jos, care conține două metode cu același nume, cu același număr de parametri (1), dar de tip diferit (`int`, respectiv `float`).



```

public class Clasa1 {
  static void eu_sunt(int n)
  { System.out.println("Parametru de tip int "+ n); }
  static void eu_sunt(double n)
  { System.out.println("Parametru de tip double "+n ); }
}

```

```
public static void main(String[] args) {
    eu_sunt(3);
    eu_sunt(-2.7);
}
}
```

- ✓ Faptul că metodele pot fi supraîncărcate are consecințe urișe în programare. Dacă în cazul programării clasice (neorientate pe obiecte) se poate trăi bine și fără această facilități, în cazul programării orientate pe obiecte, această facilități devine esențială, așa cum va rezulta din capitolele următoare.
- ✓ Un exemplu este chiar metoda `println()`, atât de utilizată până în acest moment. Ea afișează orice i-am transmite ca parametru: o valoare de tip `int`, o valoare de tip `float`, un caracter sau un șir de caractere. De fapt, există mai multe metode `println()` care diferă doar prin tipul parametrului.

Probleme propuse

1. "Numai ultima cifră". Se citesc n , număr natural <100 , și n numere naturale x_1 , nenule, mai mici ca 30000. Se cere:

- ultima cifră a numărului $x_1 + x_2 + \dots + x_n$;
- ultima cifră a numărului $x_1^{x_2 x_3 \dots x_n}$.

Exemplu: $n=3$; $x_1=11$; $x_2=4$; $x_3=3$. Programul tipărește:

- 8, pentru că ultima cifră a sumei $11+4+3$ este 8;
 - 1, pentru că ultima cifră a lui 11^{12} este 1.
2. Să se citească două matrice și să se facă suma lor. Programul se va realiza astfel:
- se scrie o metodă de citire a unei matrice cu m linii și n coloane;
 - se scrie o metodă de afișare a unei matrice cu m linii și n coloane;
 - se scrie o metodă care adună două matrice;
 - conținutul metodei `main()` rezultă din apelul acestor metode.
3. Să se calculeze coeficienții polinomului $P(x) = (x+a)^n$, în două variante:
- a) Programul utilizează o metodă care înmulțește un polinom oarecare de grad k cu polinomul $x+a$.

- b) Programul utilizează o metodă care calculează coeficienții, așa cum rezultă din formula de mai jos (binomul lui Newton):

$$(x+a)^n = \sum_{k=0}^n C_n^k a^k x^{n-k}$$

4. Să se tipărească toate numerele prime aflate între doi întregi citiți. Programul va folosi o metodă care returnează 1 sau 0 dacă numărul este prim sau nu.
5. Scrieți un program care tipărește numerele întregi găsite între două valori citite care se divid cu suma cifrelor lor. Programul va utiliza o metodă care returnează suma cifrelor unui număr întreg primit ca parametru.
6. Pentru fiecare metodă de sortare cunoscută, scrieți câte o metodă care sortează descrescător un vector cu componente numere reale.
7. Scrieți o metodă care interclasează doi vectori sortați descrescător.
8. Scrieți o metodă care identifică într-un vector cu numere reale, sortate crescător, dacă o anumită valoare este sau nu reținută de vector. Metoda va utiliza *algoritmul de căutare binară*.
9. Un polinom $P(x)$ de grad n , cu coeficienții reali este memorat sub formă de vector (cu $n+1$) componente. Se cere să se scrie o metodă care calculează $P(a)$, unde a este un număr real citit.

Indicație. Se va utiliza schema lui Horner. Exemplu: $P(x)=x^2+5x+6$ Se cere $P(4)$.

```
1  5  6
1  9 42
```

În rândul 2, 1 este coeficientul polinomului. Apoi:

```
1*4+5=9;
9*4+6=42.
```

10. Ce afișează programul următor?

```
public class Clasa1 {
    static int f(int x)
    { return x+1;}
    static void t(int a)
    { System.out.println(++a);}
    public static void main(String[] args) {
        t(f(f(f(1)))); }
}
```

a) 3
b) 4
c) 5
d) 6

11. Ce afișează programul următor?

```
public class Clasa1 {
    static int a;
    static int f()
    { int a=3;
      return a;}
}
```

a) 233;
b) 23;
c) 32;
d) 232


```

static void g()
{ int a=3;
  System.out.print(a);}

public static void main(String[] args) {
  a=2;
  System.out.print(a);;
  g();
  System.out.print(f());}
}

```

12. În programul de mai jos, înlocuiți cele trei puncte cu una din secvențele de mai jos. Pentru care dintre ele programul afișează produsul celor două numere citite.

```

public class Clasa1 {
  static int a,b,t=0;
  static void increment ()
  { t++; }

  public static void main(String[] args) {
    int i,j;
    a=Integer.parseInt(cin.Token());
    b=Integer.parseInt(cin.Token());
    ...
    increment()
    System.out.println(t);
  }
}

```

- | | |
|--|--|
| a) | b) |
| for (i=1;i<=b;i++)
for (j=1;j<=b;j++) | for (i=1;i<=a;i++)
for (j=1;j<=b-1;j++) |
| c) | d) |
| for (i=1;i<=b;i++)
for (j=1;j<=a-1;j++) | for (i=1;i<=b;i++)
for (j=1;j<=a;j++) |

13. Care dintre instrucțiunile de mai jos, înlocuită în program (în locul celor 3 steluțe) **nu** calculează a^2 , pentru un număr natural, a citit?

```

public class Clasa1 {
  static int increment (int x)
  { return x+1; }

  static int decrement (int x)
  { return x-1; }

  public static void main(String[] args) {
    int a=Integer.parseInt(cin.Token());
    ***}
}

```

- a) `System.out.println(increment(increment(a)*decrement(a)));`
- b) `System.out.println(increment(a)*decrement(a)+1);`
- c) `System.out.println(1+increment(a)*decrement(a));`

14. Calculați recursiv suma a n numere naturale citite.

15. Calculați recursiv expresiile:

- a) $1 \times 2 + 2 \times 3 + \dots + n \times (n+1)$;
- b) $1 + 1/2 + \dots + 1/n$;
- c) $1 / (2 \times 3) + 2 / (3 \times 4) + \dots + n / ((n+1)(n+2))$;

16. Se citesc n și k (numere naturale $n > k$). Calculați, recursiv, C_n^k , prin utilizarea formulei de recurență: $C_n^k = C_{n-1}^{k-1} + C_{n-1}^k$. Este eficient?

17. Scrieți un program iterativ care rezolvă problema anterioară utilizând aceeași formulă.

18. Calculați recursiv, C_n^k prin utilizarea formulei:

$$C_n^k = \begin{cases} 1, & k = 0; \\ \frac{n-k+1}{k} C_n^{k-1} & \text{altfel.} \end{cases}$$

19. Se citește un număr natural n . Se cere să se scrie o metodă recursivă care returnează cea mai mică bază în care se poate considera n .

20. Scrieți o metodă booleană recursivă care testează dacă un număr natural $n > 1$ este prim.

21. Scrieți o metodă recursivă care returnează suma elementelor pare ale unui vector citit.

Exemplu: $n=4$, $v=(2, 2, 5, 6)$. Se returnează 10.

22. Scrieți o metodă recursivă booleană prin care se testează dacă un număr natural x se regăsește între componentele unui vector v cu n numere naturale.

23. Scrieți o metodă recursivă care returnează, oglinditul unui număr natural.

Exemplu: pentru $n=123$ se returnează 321.

24. Scrieți o metodă recursivă care descompune în toate modurile posibile un număr natural n în două numere n_1 și n_2 , $n_1 \leq n_2$ a căror sumă este n .

25. Pentru un vector cu n componente 0 sau 1 care are semnificația de număr binar, se cere să se scrie o metodă recursivă care afișează numărul în baza 10.

Exemplu: pentru $n=4$ și $V=(1011)$ se va returna 11.

26. Scrieți o metodă recursivă care afișează valoarea unui polinom în punctul a . Coeficienții polinomului sunt dați într-un vector. Astfel, pentru $V=(1,2,3)$ avem polinomul $P=x^2+2x+3$.

27. Fie funcția definită pe $N^* \times N^*$. Se citesc n și k . Se cere să se scrie o metodă recursivă care evaluează funcția:

$$S(n, k) = \begin{cases} 0, & k > n \\ 1, & k \in \{1, n\} \\ S(n-1, k-1) + kS(n-1, k) & 1 < k < n \end{cases}$$

28. Calculați funcția definită pe $N^* \times N$:

$$F(n, k) = \begin{cases} 0, & k = 0 \text{ sau } k > n; \\ 1 & k = n; \\ F(n-1, k-1) - nF(n-1, k) & 1 < k < n. \end{cases}$$

29. Calculați iterativ și recursiv cel mai mare divizor pentru două numere naturale m și n , utilizând **algoritmul lui Euclid**:

$$\text{cmmdc}(m, n) = \begin{cases} \text{cmmdc}(n, m \bmod n), & n \neq 0 \\ m, & n = 0 \end{cases}$$

30. Ce se afișează pentru `System.out.println(t(12));` ?

```
static int t(int n)
{ if (n!=0) return 10*t(n/10)+ n%10;
  else return 0; }
```

a) 12; b) 21; c) eroare; d) 0.

31. Ce calculează metoda următoare?

```
static int t(int n)
{ if (n!=0) return (n%2)*n +t(n-1);
  else return 0; }
```

- a) suma primelor n numere naturale impare;
- b) suma primelor n numere naturale pare;
- c) suma numerelor naturale impare strict mai mici decât n ;
- d) suma numerelor naturale impare mai mici sau egale cu n .

32. În metoda de mai jos înlocuiți linia ... cu una dintre instrucțiunile următoare, astfel încât metoda să-și încheie execuția fără eroare pentru orice valoare admisibilă a argumentului:

```
static int x(int n)
{ if (n!=0)
  { System.out.println(n);
    ... }
  else return 0;
}
```

- | | |
|-------------------|-------------------|
| a) return x(n-2); | b) return x(n%2); |
| c) return x(n-1); | d) return x(n/2); |

33. Dacă metoda este apelată prin an(4), de câte ori se autoapelează?

```
static int an(int n)
{ if (n==0) return 1;
  else return 3*an(n-1)+7;
}
```

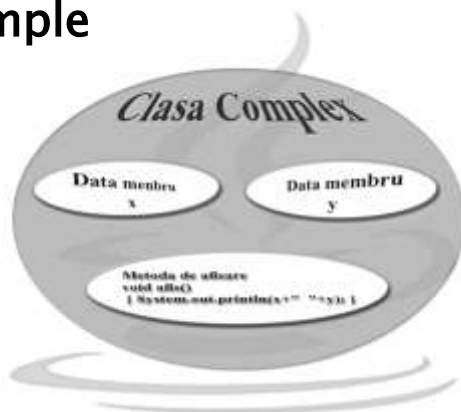
- | | |
|--------------|----------------------|
| a) de 4 ori; | b) de 3 ori; |
| c) de 5 ori; | d) depinde de stivă. |

Răspunsuri:

10. c) 11. a) 12. d) 13. c) 30 a) 31. d) 32. d) 33. a)

CAPITOLUL 4

Clase – primele noțiuni, exemple



Din cuprins:

Ce este o clasă ?

Constructori

Date membru statice și metode statice

Cuvântul cheie `this`

Referințe către obiecte

Masive de obiecte

Aplicații ale noțiunilor prezentate

Garbage Collector

Cum sunt memorate clasele ?

Pachete

Clase interioare

O problemă de terminologie

Probleme propuse

4.1. Ce este o clasă ?

La matematică v-ați întâlnit cu numere complexe! Ele sunt de forma: $z=x+iy$, unde $x,y \in \mathbb{R}$. Prin urmare, pentru fiecare număr complex ar trebui să reținem două valori reale, pentru x și pentru y . Decidem că în acest caz, pentru a le reține, avem nevoie de două variabile de tip `double`.

Priviți programul de mai jos care conține două clase: `Complex` și `test..`

```
class Complex {
    // date membru
    double x,y;
    // metoda
    void afis()
    { System.out.println(x+" "+y); }
}

public class test {
    public static void main(String[] args) {
        // declar un numar complex
        Complex z1;
        z1= new Complex (); // Complex z1=new Complex();
        // Datelor membru li se atribuie valori
        z1.x=3;
        z1.y=-4.7;
        // Afisez datele membru
        z1.afis();
    }
}
```

Să analizăm programul.

Clasa `Complex`:

1. Clasa conține două date membru, x și y , ambele de tip `double`.
2. Clasa conține o metodă care are rolul de a afișa într-un mod convenabil datele membru ale clasei.

Clasa `test` - metoda `main()`

1. `Complex z1;` - variabila `z1` este de tip `Complex`. Prin urmare, ea poate reține o referință (adresă) către un obiect de tip `Complex`.
2. `z1=new Complex();` - se creează un obiect de tip `Complex` și referința către el este reținută de `z1`.
3. `z1.x=3; z1.y=-4.7;` - datelor membru x și y ale obiectului a cărui referință este reținută de `z1` li se atribuie, respectiv, valorile 3 și 4.7.
4. `z1.afis();` - se afișează numărul complex reținut de obiectul referit de `z1`.

📢 Foarte important !

A) La baza programării pe obiecte stă noțiunea de clasă. Clasa încorporează atât date membru, cât și metode. A defini o clasă înseamnă a crea un tip (în sensul extins). Un obiect rezultă ca instanțiere a unei clase. Obiectul va avea ca tip numele clasei respective. Pentru a putea accesa datele membru și metodele clasei respective se utilizează variabile de tip referință către obiectele clasei respective.

B) Odată definită clasa, putem crea obiecte de tipul clasei respective. Într-o formă simplificată (mai precis, în absența unui constructor), crearea unui obiect de tipul unei clase se face prin construcția de mai jos, unde **v** este o variabilă referință către obiectele clasei respective.

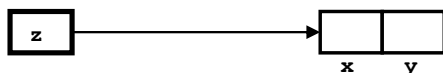
Nume_clasa v=new Nume_Clasa()

C) Un **constructor** al unei clase este o metodă specială a clasei respective care are rolul de a alocă în memorie spațiul necesar obiectului dar și de a inițializa datele membru ale acestuia. Orice clasă are un constructor implicit (adică o metodă pe care nu a scris-o cel care a creat clasa).

D) Operatorul **new** are dublu rol:

- apelează constructorul clasei respective;
- returnează referința (adresa) la care a fost creat obiectul respectiv.

Exemplu: `Complex z=new Complex();` - **z** reține o referință către obiectul creat prin **new**.



E) Accesul la variabilele interne ale obiectului se realizează prin numele obiectului, urmat de operatorul `.'` și numele variabilei. În exemplu, **z** va reține numărul $x+2i$, iar aceste valori vor fi afișate,

```

z1.x=1;
z1.y=2;
System.out.println(z1.x);
System.out.println(z2.y);
  
```

F) O clasă poate conține una sau mai multe **metode**. Apelul unei metode de acest tip se face prin numele variabilei care reține referința către obiect, urmat de `.'` și de numele metodei. **Exemplu:** `z1.afis();`

G) Datele membru ale unei clase sunt inițializate implicit cu valoarea 0, dacă sunt de un tip numeric, sau caracterul de cod 0, dacă sunt de tip **char**, sau cu **null** dacă sunt de tip referință.

4.2. Constructori

Așa cum am arătat, orice clasă are un constructor implicit. Rolul lui este de a alocă spațiu în memorie pentru obiect. Constructorul este apelat de operatorul `new`. Acesta din urmă are și rolul de a returna referința către obiect, referință care este memorată de o variabilă de tip referință către obiect.

Unei clase i se pot crea proprii constructori, așa cum rezultă din exemplul următor, unde clasei prezentate în paragraful precedent i s-au adăugat doi constructori:

```
class Complex {
    // date membru
    double x,y;
    // Constructor 1
    Complex (double x1)
    { x=x1; }
    // Constructor 2
    Complex (double x1, double y1)
    { x=x1; y=y1;}
    // metoda
    void afis()
    { System.out.println(x+" "+y);}
}

public class test {
    public static void main(String[] args) {
        Complex z1 = new Complex (2,3), z2 =new Complex (1);
        z1.afis();
        z2.afis();
    }
}
```

Primul constructor are un singur parametru și se folosește pentru declarații de numere complexe care au partea imaginară 0. Al doilea constructor se utilizează pentru a declara numere complexe cu parte reală și parte imaginară. În ambele cazuri nu mai este nevoie ca datele membru să fie inițializate.

De reținut:

- A) Un constructor nu are tip;
- B) Un constructor are întotdeauna numele clasei căreia îi aparține;
- C) Dacă unei clase i se atașează un constructor, altul decât cel implicit, atunci nu se mai poate instanția un obiect prin utilizarea constructorului implicit. De exemplu, pentru clasa definită în acest paragraf o instrucțiune de genul `Complex z=new Complex()` dă eroare de sintaxă.

D) Unei clase i se pot atașa mai mulți constructori, după cum ați văzut și în exemplu. Ei trebuie să difere prin numărul parametrilor și /sau tipul parametrilor, cu alte cuvinte, ca și metodele, aceștia sunt supraîncărcați.

E) Constructorul implicit este apelat în mod transparent (fără a scrie o secvență specială) de către constructorul propriu.

4.3. Date membru statice și metode statice

Atât datele membru cât și metodele unei clase pot fi **statice**. Aceasta înseamnă că datele și metodele declarate statice pot fi apelate, așa cum suntem deja obișnuiți, pornind de la un obiect al clasei respective, dar și pornind de la numele clasei, așa cum observați în exemplul următor.

Clasa **Matematica** conține o dată membru inițializată (**Pi**). De asemenea, ea mai conține două metode: **Patrat()** și **Cub()** prin care se calculează x^2 și x^3 . În exemplu se calculează și afișează aria unui cerc în două feluri:

- apelând data membru și metoda pornind de la numele clasei (**Matematica**);
- apelând data membru și metoda pornind de la numele unui obiect al clasei **Matematica**.

```
class Matematica {
    static double Pi=3.1415;
    static double Patrat (double x)
    { return x*x; }
    static double Cub (double x)
    { return x*x*x; }
}

public class Clasa1 {
    public static void main(String[] args) {
        int Raza=3;
        double Aria=Matematica.Pi*Matematica.Patrat(Raza);
        System.out.println(Aria);
        //altfel
        Matematica ob=new Matematica();
        Aria=ob.Pi*ob.Patrat(Raza);
        System.out.println(Aria);
    }
}
```

Foarte important !

A) Datele membru statice și metodele statice **nu** sunt memorate de fiecare obiect al clasei respective. Ele sunt memorate o singură dată, în cadrul clasei respective.

B) Sunt clase care au date membru și/sau metode statice și nestatice.



O clasă indispensabilă, este clasa **Math**, care este prezentată în această carte. Ea conține o mulțime de metode “matematice” care se utilizează mult în orice limbaj de programare. Clasa **Matematica**, din exemplu, este construită pe aceleași principii ca și clasa **Math**.

4.4. Cuvântul cheie **this**

În interiorul claselor se poate utiliza cuvântul cheie **this**. Semnificația sa este: *referință către obiectul curent*. Iată una dintre aplicațiile sale:

Aplicație. Dacă o metodă (sau un constructor) are o variabilă cu un anumit nume, de exemplu **x**, și dacă, clasa are o dată membru cu același nume, pentru exemplul nostru tot **x**, atunci pentru a nu se crea confuzie, data membru va fi adresată prin **this.x**, iar variabila din metodă (sau constructor) prin **x**.

Exemplu: pentru clasa **Complex** din paragraful precedent, cei doi constructori ar putea fi scrisi și așa:

```
Complex (double x)
{ this.x=x; }
```

```
Complex (double x, double y)
{ this.x=x; this.y=y; }
```

Alte aplicații ale cuvântului cheie **this** vor fi prezentate la momentul potrivit.

4.5. Referințe către obiecte

Metodele pot avea parametri de tip referință la obiecte și pot returna referințe către obiecte. Evident, referințele sunt transmise prin valoare. Dar, pornind de la ele, se poate accesa obiectul ca și când ar fi transmis prin referință.

Exemplu: adăugăm clasei **Complex** metoda **Complex adun(Complex z)**; Metoda returnează o referință către un obiect de tip **Complex** care reține numărul complex rezultat ca sumă între numărul reținut de obiectul curent și numărul complex transmis ca parametru.

```
class Complex {
    double x,y;

    Complex (double x)
    { this.x=x; }

    Complex (double x, double y)
    { this.x=x; this.y=y; }
```

```

void afis()
{ System.out.println(x+" "+y); }

Complex adun(Complex z)
{ return new Complex (x+z.x,y+z.y); }
}

public class test {
    public static void main(String[] args) {
        Complex z1 = new Complex (2,3);
        Complex z2 =new Complex(3,4);
        Complex z=z1.adun(z2);
        z.afis(); }
}

```

Programul afișează suma numerelor complexe reținute **z1** și **z2** adică numărul complex $z=5+7i$, afișat ca: 5 7. Observați faptul că variabila care reține referința către obiectul sumă a fost inițializată cu ajutorul metodei **adun()** a obiectului **z1**.



Clasele pot conține variabile de tip referință la propriile obiecte (sau la obiectele altei clase).

Exemplu (lista liniară simplu înlănțuită): pornind de la această observație, construim o listă liniară simplu înlănțuită. Priviți programul următor care creează și afișează o listă liniară simplu înlănțuită:

```

class Nod {
    int info;
    Nod adr_urm;
}

public class Lista {
    static Nod v=null; // nu este obligatorie inițializarea
    static void adaug(int nr)
    { Nod c=new Nod();
      c.info=nr;
      c.adr_urm=v;
      v=c; }

    static void afis()
    { Nod c=v;
      while (c!=null)
      { System.out.println(c.info);
        c=c.adr_urm; }
    }

    public static void main(String[] args) {
        for (int i=1;i<=9;i++) adaug(i);
        afis(); }
    }

```

Un nod reține o informație oarecare, s-o numim **info** și să presupunem că este de tip **int** și o referință către nodul următor (**adr_urm**). Evident, referința este către obiecte ale aceleiași clase. Clasa respectivă am numit-o **Nod**.

Clasa care construiește lista (**Lista**) conține o dată membru de tip **static** de tip referință către un obiect al clasei **Nod**. De asemenea, clasa conține și două metode **adaug()** și **afis()**.

- Metoda **adaug()** are rolul de a adăuga un nod listei liniare. Ea primește ca argument numărul care trebuie reținut de **info**. Observați cum s-a procedat: s-a creat un nou obiect al clasei **Nod**, iar referința către el este reținută de **c**. Data membru a acestui obiect, **info**, este inițializată cu valoarea primită ca parametru de metodă, iar **adr_urm** cu referința către nodul anterior, reținută de **v**. În final, **v** reține referința către ultimul nod creat. În acest fel, *lista va reține nodurile în ordinea inversă creării lor*. Am preferat acest algoritm pentru simplitatea lui.
- Metoda **afis()** listează valorile reținute de data membru **info** a fiecărui nod.

4.6. Masive de obiecte

Întrebarea este: se pot crea masive care rețin obiecte? De exemplu, putem să creăm un vector care reține obiecte de tipul **Complex**? Răspunsul este afirmativ! *Elementele unui masiv pot reține referințe către obiecte.*

Exemplu: în programul următor se creează și se afișează un vector cu componente de tip **Complex**.

1.0	2.0
2.0	3.0
3.0	4.0

```
class Complex {
    double x,y;

    Complex (double x)
    { this.x=x; }

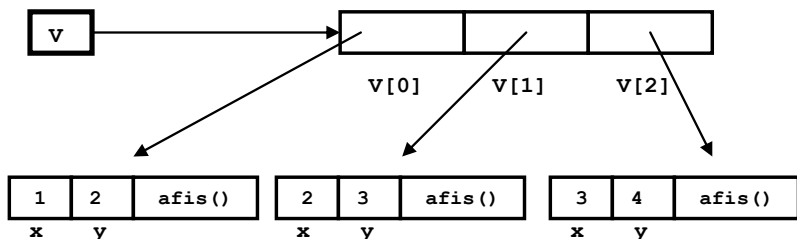
    Complex (double x, double y)
    { this.x=x; this.y=y;}

    void afis()
    { System.out.println(x+" "+y);}

    Complex adun(Complex z)
    { return new Complex (x+z.x,y+z.y);}
}

public class test {
    public static void main(String[] args) {
        Complex[] V=new Complex[3];
        for (int i=0;i<3;i++) V[i]=new Complex(i+1,i+2);
        for (int i=0;i<3;i++) V[i].afis();
    }
}
```

Pentru a înțelege modul în care a fost construit vectorul de obiecte rezultate ca instanțe ale clasei **Complex**, analizați figura următoare:



Alte exemple:

- 1) În secvența următoare, **v1** reține o referință către un vector cu 10 componente care pot reține referințe către obiecte ale clasei **Complex**. Se instanciază un obiect al clasei **Complex** și referința sa este atribuită lui **v1[2]**. Ultimele 3 instrucțiuni au rolul de a proba cele prezentate.

```
Complex[] v1 = new Complex[10];
v1[2] = new Complex(10,11);
v1[2].afis();
```

- 2) În secvența următoare, **v1** reține o referință către o matrice cu 8 linii și 5 coloane. Fiecare componentă a matricei poate reține o referință către un obiect al clasei **Complex**. Se instanciază un obiect al clasei **Complex** și referința sa este atribuită lui **v1[2][3]**.

```
Complex[][] v1 = new Complex[8][5];
v1[2][3] = new Complex(10,11);
v1[2][3].afis();
```

- 3) **v1** reține o referință către o matrice cu 5 linii și un număr neprecizat de coloane. Un element al matricei va reține o referință către un obiect al clasei **Complex**. Linia de indice 3, va avea 6 elemente. Elementul aflat în linia de indice 3 și coloana de indice 4 va reține o referință către un obiect al clasei **Complex**. Apoi, se probează...

```
Complex[][] v1 = new Complex[5][];
v1[3] = new Complex[6];
v1[3][4] = new Complex(100,200);
v1[3][4].afis();
```

4.7. Aplicații ale noțiunilor prezentate

Este momentul să învățăm să creăm clase și apoi să le utilizăm.. Înainte de a prezenta aplicațiile precizăm că, dacă lucrăm în folder-ul **bin**, așa cum am lucrat până acum, putem compila o clasă oarecare fără ca ea să conțină metoda **main()**. Fișierul obținut (are extensia **.class**) se va găsi în **bin**. (dacă cel cu extensia **.java** este tot acolo). Orice altă clasă care conține

`main()` din `bin`, poate folosi clasa compilată, fără a o mai declara într-un fel. De altfel, această problemă va fi tratată pe larg în acest capitol.

4.7.1. Lucrul cu numere raționale

Să se creeze o clasă cu ajutorul căreia să se poată lucra ușor cu numere raționale. Un număr rațional q este de forma m/n cu m și n numere întregi și n diferit de 0.

- Clasa va avea două date membru, m și n , ambele tip `int`.
- Constructorul clasei este `Rational (int m, int n)`. Obiectul este creat numai dacă $n \neq 0$. Datorită faptului că un număr rațional poate fi reținut în mai multe feluri (de exemplu $2/3=4/6=8/12=$) se va reține numărul simplificat (pentru exemplul dat, $m=2$ și $n=3$, chiar dacă introducem 8 și 12). Pentru a putea realiza simplificarea fracției, m și n se împart la cel mai mare divizor comun al lor.
- Clasa va dispune de o metodă, `static int cmmdc(int m, int n)`, apelată de constructor, prin care se calculează cel mai mare divizor comun al valorilor reținute de datele membru m și n .
- Metoda `Rational add(Rational r)` creează un obiect al clasei `Rational` care este rezultatul adunării dintre obiectul curent și obiectul a cărui referință a fost primită ca parametru și returnează referința către el. Faptul că a fost apelat constructorul cu datele membru ale obiectului sumă, are ca efect faptul că obiectul care reține suma este deja simplificat (m și n au fost împărțite la `cmmdc` al lor).
- Metoda `Rational sub(Rational r)` creează un obiect al clasei `Rational` care este rezultatul scăderii dintre obiectul curent și obiectul a cărui referință a fost primită ca parametru și returnează referința către el.
- Metoda `Rational mul(Rational r)` creează un obiect al clasei `Rational` care este rezultatul înmulțirii dintre obiectul curent și obiectul a cărui referință a fost primită ca parametru și returnează referința către el.
- Metoda `Rational div(Rational r)` creează un obiect al clasei `Rational` care este rezultatul împărțirii dintre obiectul curent și obiectul a cărui referință a fost primită ca parametru și returnează referința către el.
- Metoda `boolean maiMare (Rational r)` compară obiectul curent cu obiectul a cărui referință a fost primită ca parametru și returnează `true` dacă obiectul curent este strict mai mare decât cel a cărui referință a fost transmisă ca parametru și `false` în caz contrar.

- Metoda `boolean maiMic (Rational r)` compară obiectul curent cu obiectul a cărui referință a fost primită ca parametru și returnează `true` dacă obiectul curent este strict mai mic decât cel a cărui referință a fost transmisă ca parametru și `false` în caz contrar.
- Metoda `boolean egal (Rational r)` compară obiectul curent cu obiectul a cărui referință a fost primită ca parametru și returnează `true` dacă obiectul curent este egal cu cel a cărui referință a fost transmisă ca parametru și `false` în caz contrar.

Mai jos puteți observa modul în care arată clasa `Rational`:

```
class Rational {
    int m,n;

    static int cmmdc(int m, int n)
    { if (n!=0) return cmmdc(n, m%n);
      else return m;}

    Rational (int m, int n)
    { if (n!=0)
      { int d=cmmdc(m,n);
        this.m=m/d; this.n=n/d;}
      else System.out.println("Numitorul este nul");}

    Rational add(Rational r)
    { return new Rational (m*r.n+r.m*n, n*r.n); }

    Rational sub(Rational r)
    { return new Rational (m*r.n-r.m*n, n*r.n); }

    Rational mul(Rational r)
    { return new Rational (m*r.m,n*r.n); }

    Rational div(Rational r)
    { if (r.m!=0) return new Rational (m*r.n,n*r.m);
      else return new Rational (0,1);}

    boolean maiMare(Rational r)
    { return (double) m/n > (double)r.m/r.n; }

    boolean maiMic(Rational r)
    { return (double) m/n < (double)r.m/r.n; }

    boolean egal(Rational r)
    { return (double) m/n == (double)r.m/r.n; }
}
```

↳ În continuare, ne propunem să rezolvăm câteva aplicații în care intervin numere raționale:

1. Se citesc două numere raționale q_1 și q_2 ($\neq 0$). Se cere să se afișeze q_1+q_2 , q_1-q_2 , q_1*q_2 , q_1/q_2 . Rezultatele vor fi tot două numere raționale.

Rezolvare. Variabila q_3 este de tip referință către un obiect al clasei `Rational`. După fiecare operație, ea va reține referința către obiectul rezultat în urma operației.

```

class I {
    public static void main(String[] args) {
        int m=Integer.parseInt(cin.Token());
        int n=Integer.parseInt(cin.Token());
        Rational q1=new Rational (m,n);
        m=Integer.parseInt(cin.Token());
        n=Integer.parseInt(cin.Token());
        Rational q2=new Rational (m,n);
        Rational q3=q1.add(q2);
        System.out.println("Suma="+q3.m+"/"+q3.n );
        q3=q1.sub(q2);
        System.out.println("Diferenta="+q3.m+"/"+q3.n );
        q3=q1.mul(q2);
        System.out.println("Produsul="+q3.m+"/"+q3.n );
        q3=q1.div(q2);
        System.out.println("Catul="+q3.m+"/"+q3.n );
    }
}

```

2. Se citesc k numere raționale. Se cere să se afișeze suma lor sub formă de număr rațional.

Rezolvare. Obiectul s , va reține inițial o referință către un număr rațional nul ($m=0$, $n=1$). Se citesc, pe rând, cele k numere raționale. Pentru fiecare număr rațional citit, se construiește obiectul suma între s și obiectul care reține numărul rațional, iar referința către el este atribuită lui s . În final se afișează datele mebru ale obiectului s .

```

class I {
    public static void main(String[] args) {
        System.out.print("k=");
        int k=Integer.parseInt(cin.Token());
        Rational s=new Rational(0,1);
        for (int i=0;i<k;i++)
        { System.out.print("m=");
          int m=Integer.parseInt(cin.Token());
          System.out.print("n=");
          int n=Integer.parseInt(cin.Token());
          Rational q=new Rational (m,n);
          s=s.add(q);
        }
        System.out.println("Suma="+s.m+"/"+s.n );
    }
}

```

3. Se citește un vector cu k componente numere raționale (obiecte ale clasei **Rational**). Se cere să se afișeze vectorul sortat crescător

Rezolvare. După citirea vectorului vom utiliza metoda de sortare prin interschimbare. Pentru comparare se utilizează metoda `maiMare()`.


```

class I {
    public static void main(String[] args) {
        System.out.print("k=");
        int k=Integer.parseInt(cin.Token());
        Rational [] V=new Rational [k];
        for (int i=0;i<k;i++)
        { System.out.print("V["+i+"] .m=");
          int m=Integer.parseInt(cin.Token());
          System.out.print("V["+i+"] .n=");
          int n=Integer.parseInt(cin.Token());
          V[i]=new Rational (m,n);
        }
        boolean gasit;
        Rational man;
        do
        { gasit=false;
          for (int i=0;i<k-1;i++ )
            if (V[i].maiMare(V[i+1]))
            { man=V[i];
              V[i]=V[i+1];
              V[i+1]=man;
              gasit=true;
            }
          } while (gasit);
        for (int i=0;i<k;i++)
          System.out.println("V["+i+"] .m="+V[i].m+" V["+i+"] .n="+V[i].n);
    }
}

```

4.7.2. Lucrul cu mulțimi de numere naturale

Să se creeze o clasă numită **Multime**, care să ne faciliteze lucrul cu mulțimi de numere naturale. Numerele naturale care pot fi elemente ale mulțimilor sunt din intervalul $[0,999]$, adică cel mult 1000 de numere. De exemplu, o astfel de mulțime ar fi $\{1,3,5\}$.

O mulțime se reține prin *vectorul caracteristic*. Dacă un număr aparține mulțimii, componenta respectivă va reține 1, altfel componenta va reține 0. De exemplu, pentru mulțimea dată ca exemplu, vectorul caracteristic va fi:

0	1	0	1	0	1	0		0
0	1	2	3	4	5	6		999

În continuare, prezentăm componentele care alcătuiesc clasa respectivă:

- ⇒ **caract** – referință către un vectorul caracteristic.
- ⇒ **int card()** – metodă care returnează numărul de elemente al mulțimii (cardinalul). Practic, numără componentele nenule din vectorul caracteristic.

- ⇒ `boolean ap(int k)` - metodă care testează apartenența lui `k` la mulțime. Practic, se verifică dacă `caract[k]` reține 1. În caz de apartenență, metoda returnează `true`, altfel returnează `false`.
- ⇒ `Multime reun(int k)` - metodă care returnează o referință la mulțimea obținută din reuniunea mulțimii asociate obiectului curent cu elementul `k`.
- ⇒ `Multime reun(Multime T)` - metodă care returnează o referință la mulțimea obținută din reuniunea mulțimii asociate obiectului curent cu mulțimea asociată obiectului `a` a cărui referință este transmisă ca parametru. După cum observați, avem de fapt, două metode cu același nume (am folosit supraîncărcarea metodelor).
- ⇒ `Multime inters(Multime T)` - metodă care returnează o referință la mulțimea obținută din intersecția mulțimii asociate obiectului curent cu mulțimea asociată obiectului `a` a cărui referință este transmisă ca parametru.
- ⇒ `Multime dif(Multime T)` - metodă care returnează o referință la mulțimea obținută din diferența între mulțimea asociată obiectului curent și mulțimea asociată obiectului `a` a cărui referință este transmisă ca parametru.
- ⇒ `Multime dif(Multime T)` - metodă care returnează o referință la mulțimea obținută din diferența dintre mulțimea asociată obiectului curent și mulțimea alcătuită dintr-un singur element, adică elementul `k` transmis ca parametru.
- ⇒ `boolean inclus(Multime T)` - metodă care returnează `true` dacă mulțimea asociată obiectului curent este inclusă în mulțimea a cărei referință a fost transmisă ca parametru și `false` în caz contrar.

Analizați clasa `Multime`:

```
class Multime {
    byte[] caract=new byte[1000];

    int card()
    { int s=0;
      for (int i=0;i<1000;i++)
          if (caract[i]==1) s++;
      return s;}

    boolean ap(int k)
    { if (caract[k]==1) return true;
      else return false;}

    Multime reun (int k)
    { caract[k]=1;
      return this;}
```

```

Multime reun (Multime T)
{ Multime man=new Multime();
  for (int i=0;i<1000;i++)
    if (caract[i]==1 || T.caract[i]==1) man.caract[i]=1;
  return man; }

Multime inters (Multime T)
{ Multime man=new Multime();
  for (int i=0;i<1000;i++)
    if (caract[i]==1 && T.caract[i]==1) man.caract[i]=1;
  return man; }

Multime dif (Multime T)
{ Multime man=new Multime();
  for (int i=0;i<1000;i++)
    if (caract[i]==1 && T.caract[i]==0) man.caract[i]=1;
  return man; }

Multime dif (int k)
{ caract[k]=0;
  return this; }

boolean inclus(Multime T)
{ boolean inclus=true;
  for (int i=0;i<1000;i++)
    if (caract[i]==1 && T.caract[i]==0) inclus=false;
  return inclus; }
}

```



Exerciții de utilizare a clasei **Multimi**:

1. Se citesc n numere naturale mai mici decât 1000. Se cere să se afișeze numărul numerelor distincte și numerele propriu-zise. De exemplu, dacă $n=4$ și numerele sunt 5, 6, 5, 6 se va afișa 2 și 5, 6.

Rezolvare. Numerele citite sunt adăugate (reunite) la o multime initial vide. Evident, dacă un număr apare de mai multe ori, el se va găsi în multime o singură dată. Numărul de numere distincte va fi dat de numărul de elemente al mulțimii (cardinalul ei), iar numerele distincte sunt cele care se regăsesc în multime.

```

class I {
  public static void main(String[] args) {
    Multime A= new Multime();
    System.out.print("n=");
    int n=Integer.parseInt(cin.Token());
    for (int i=0;i<n;i++)
      A=A.reun(Integer.parseInt(cin.Token()));
    System.out.println("Numere distincte "+A.card());
    for (int i=0;i<1000;i++)
      if (A.ap(i)) System.out.println(i);
  }
}

```

2. Se citesc două mulțimi de numere naturale **A** și **B**. Se cere să se afișeze dacă cele două mulțimi sunt egale sau nu.

Rezolvare. O metodă de verificare a egalității a două mulțimi, **A** și **B**, este de a verifica dacă $A \subseteq B$ și $B \subseteq A$. Desigur, s-ar putea proceda și altfel, prin a verifica dacă cei doi vectori caracteristici sunt identici.

```
class I {
    public static void main(String[] args) {
        Multime A= new Multime();
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
        for (int i=0;i<n;i++)
            A=A.reun(Integer.parseInt(cin.Token()));
        Multime B=new Multime();
        System.out.print("m=");
        int m=Integer.parseInt(cin.Token());
        for (int i=0;i<m;i++)
            B=B.reun(Integer.parseInt(cin.Token()));
        if (A.inclus(B) && B.inclus(A) ) System.out.println("Egale");
        else System.out.println("Diferite");
    }
}
```

3. Se citesc două mulțimi de numere naturale **A** și **B**. Se cere să se afișeze dacă cele două mulțimi sunt disjuncte (nu au elemente comune) sau nu.

Rezolvare. Două mulțimi sunt disjuncte dacă intersecția lor este vidă. Programul este asemănător programului precedent, dar secvența prin care se testează dacă mulțimile sunt disjuncte este cea de mai jos.

```
if (A.inters(B).card()==0) System.out.println("Disjuncte");
else System.out.println("Nedisj");
```

4. Se citesc **n** mulțimi de numere naturale mai mici decât 1000. Să se memoreze mulțimile într-un vector de mulțimi și să se calculeze și afișeze mulțimea care rezultă din reuniunea celor **n** mulțimi.

Rezolvare. Se creează un vector de mulțimi.

```
class I {
    public static void main(String[] args) {
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
        Multime [] V= new Multime[n];
        for (int i=0;i<n;i++)
        { V[i]=new Multime();
          System.out.print("Multime["+i+"] "+ "nr elem=");
          int m=Integer.parseInt(cin.Token());
          for (int j=0;j<m;j++)
              V[i]=V[i].reun(Integer.parseInt(cin.Token()));
        }
        Multime R= new Multime();
```

```

for (int i=0;i<n;i++)
    R=R.reun(V[i]);
for (int i=0;i<1000;i++)
    if (R.ap(i))System.out.print (i+" ");
}
}

```



4.8. Garbage Collector

Modul în care se alocă obiectele a fost prezentat în acest capitol. Pe parcursul rulării unui program se pot alocă multe obiecte, iar altele devin inutile, adică nu mai există referință către ele. Dacă numărul obiectelor alocate este foarte mare, este posibil ca programul să nu mai dispună de suficientă memorie și, prin urmare, executarea sa să nu mai poată continua.

Pentru a rezolva această problemă, programele **java** conțin o secvență (fir de executare), numită **Garbage Collector**, care intervine automat (fără ca programul nostru să conțină instrucțiuni scrise special în acest scop) și eliberează memoria ocupată de obiectele nereferite. Aceasta poate duce, uneori, la o încetinire a executării programelor. Momentul în care intervine **Garbage Collector** nu este stabilit de programator.



4.9. Cum sunt memorate clasele ?

Până în prezent am utilizat, de regulă, programe scrise conținute într-un singur fișier text sursă, cu extensia **.java**, și care se află în subfolderul **bin** al folder-ului **Java**. Un astfel de text include toate clasele create de noi. Cunoaștem faptul că numele unui astfel de fișier trebuie să coincidă cu numele clasei care conține metoda **main()**.

Fie programul de mai jos, care se găsește în fișierul sursă **exmp.java**. Textul conține 3 clase **A**, **B**, și **exmp**.

```

class A
{ void tip() { System.out.println("clasa A"); }
}

class B
{ void tip() { System.out.println("clasa B"); }
}

public class exmp {
    public static void main(String[] args) {
        A x=new A(); x.tip();
        B y=new B(); y.tip();
    }
}

```

În urma compilării, fiecărei clase inclusă în fișierul text îi va corespunde un fișier cu numele clasei respective și extensia **class**.

Compilăm **exmp** prin **javac exmp.java**. Analizați folder-ul **bin**. În urma compilării, el va conține trei fișiere **A.class**, **B.class**, **exmp.class**. Pentru rularea programului vom da comanda **java exmp**.

- ✓ În acest fel, comunicăm programului **java** faptul că fișierul care conține clasa **exmp** conține și metoda **main()**. Aceasta înseamnă că se știe de unde trebuie să înceapă rularea (care este clasa care conține metoda **main()**). Mai mult, programul va identifica automat celelalte clase de care are nevoie (**A.class** și **B.class**) în folder-ul **bin**.
- ✓ Modul în care se rețin clasele are drept consecință și faptul că, odată compilată o clasă, ea se va găsi în folder-ul **bin** și va putea fi utilizată și de alte programe, care nu conțin în textul sursă clasa respectivă (pe acest principiu am utilizat clasa **cin**).
- ✓ Vă dați seama că o astfel de modalitate de lucru în care toate fișierele text, cu extensie **java**, toate clasele compilate se găsesc în același folder (**bin**) creează numeroase dezavantaje în practică, acolo unde trebuie să avem programele memorate separat. Din acest motiv, trebuie să prezentăm modul în care putem memora clasele separat.

Revenim la programul aflat în fișierul text **exmp.java**. Obsevați faptul că acest program nu utilizează clase definite în altă parte. Vom crea un folder, sa-l numim **sursa**, în care am mutat fișierul **exmp.java** (a fost șters din **bin**).. De asemenea, vom mai crea un folder, numit **sursac** în care dorim să avem programul compilat. Ambele folder-e se găsesc în rădăcină (**c:**).

Pentru a compila fișierul **exmp.java** aflat în folder-ul **sursa** și pentru ca rezultatul să se găsească în folder-ul **sursac**, vom apela **javac** astfel:

```
javac -d c:\sursac c:\sursa\exmp.java
```

- **-d c:\sursac** - specifică folder-ul în care se va găsi programul compilat;
- **c:\sursa\exmp.java** - fișierul care conține sursa java, dar aici, cuprinde și calea către el (este pe **c:**, în folder-ul **sursa** și se numește **exmp.java**).



Dacă am fi apelat **javac** prin **javac c:\sursa\exmp.java**, atunci programul compilat s-ar găsi acolo unde este și fișierul text care conține programul sursă (adică în folder-ul **sursa**).

Pentru a rula programul care se găsește compilat în folder-ul **sursac**, se apelează **java** astfel:

```
java -classpath c:\sursac exmp
```

- **-classpath c:\sursac** specifică folder-ul unde se găsește programul compilat;
- **exmp** – numele clasei care conține **main()**.



Nu uitați, puteți folosi variabila de sistem **CLASSPATH** (vezi cap. 1) !



4.10. Pachete

Un pachet este alcătuit dintr-una sau mai multe clase și eventual, unul sau mai multe subpachete. Pentru moment, vom neglija subpachetele și vom considera pachetul alcătuit exclusiv din clase. Ideea de bază, care a stat la elaborarea unui set de reguli de creare și exploatare a pachetelor, este de a permite celor ce scriu programe să utilizeze clasele respective sau numai unele metode conținute de aceste clase. Acum, ne propunem să învățăm să creăm și să utilizăm pachete. Clasele care alcătuiesc pachetul respectiv se pot găsi într-unul sau mai multe fișiere sursă.

Vom porni de la un exemplu, în care avem două clase **A** și **B**. Fiecare clasă conține câte o metodă.

Pasul 1. Stabilim numele pachetului, În exemplul nostru, acesta se numește **primulPachet**.

Pasul 2. Creăm un folder care va reține fișierele compilate. În exemplu, acesta va fi pe **C:** și se numește **Pachete**.

Pasul 3. Fișierele sursă (cu extensia **.java**) se grupează într-un folder oarecare. În exemplu acesta se va numi **Surse** și se va afla pe **c:**.

- Pentru a preciza că o anumită clasă aparține unui pachet, prima linie a sursei trebuie să fie **package nume_pachet;**
- Pentru a avea acces la o clasă din afara ei, trebuie ca respectiva clasă să fie publică. Aceasta înseamnă că înaintea cuvântului cheie **class** trebuie să se găsească modificatorul **public**.
- Pentru a avea acces la o anumită metodă a clasei respective, trebuie ca metoda să fie publică. În concluzie și metodele conținute în clasele respective vor fi precedate de modificatorul **public**.

În concluzie, în **C:\Surse** vom avea fișierele **A.java** și **B.java** de mai jos:

A.java:

```
package primulPachet;
public class A
{ public void tip() { System.out.println("clasa A"); }
}
```

B.java:

```
package primulPachet;
public class B
{ public void tip() { System.out.println("clasa B"); }
}
```

Pasul 4. Compilăm pe rând cele două clase (-d precizează unde se vor găsi fișierele compilate):

```
javac -d c:\pachete c:\surse\A.java
javac -d c:\pachete c:\surse\B.java
```

Să observăm că, în folder-ul **pachete** s-a creat *automat* un subfolder care are numele pachetului (**primulPachet**). Cu asta pachetul este creat.

Urmează să scriem un program care utilizează clasele pachetului creat. Fie programul de mai jos, a cărui sursă se găsește în folder-ul **C:\Prog**. Dorim ca programul compilat să se regăsească în același folder.

- După cum stim, este obligatoriu ca numele fișierului să coincidă cu numele clasei care conține **main()**, dar să aibă extensia **.java**.
- De asemenea, pentru a preciza că acest program utilizează pachetul numit **primulPachet**, prima linie a proiectului va conține clauza **import primulPachet.***; . Iată programul:

```
import primulPachet.*;
public class exmp {
    public static void main(String[] args) {
        System.out.println("Utilizarea primului pachet");
        A x=new A(); x.tip();
        B y=new B(); y.tip();
    }
}
```

Pasul ①. Compilăm programul, având grijă să specificăm locul unde găsesc clasele. Întrucât se dorește ca programul compilat să se găsească în același loc cu sursa, parametrul **-d** lipsește.

```
javac -classpath c:\pachete C:\prog\exmp.java
```

Pasul ②. Comandăm executarea programului. Trebuie să precizăm folder-ele care conțin clasele: **c:\prog** (pentru **exmp.class**) și **c:\pachete** (**A.class**, **B.class**). Căile se scriu separate prin **;**, fără spațiu între ele.

```
java -classpath c:\prog;c:\pachete exmp
```


📢 Important !

1. Clasele care alcătuiesc pachetul trebuie să fie publice. De asemenea, metodele lor, care pot fi apelate de utilizatorii pachetelor trebuie să fie publice.
2. Fiecare clasă a pachetului trebuie să se găsească într-un fișier text cu extensia .java și numele clasei respective. Rezultă de aici că o clasă trebuie să fie singura din fișierul respectiv.

Să recapitulăm: am creat un pachet care se numește `primulPachet`. El se găsește în folder-ul `Pachete`.

Am arătat faptul că un pachet poate conține, pe lângă clase și unul sau mai multe **subpachete**. Subpachetele sunt, la rândul lor pachete, dar sunt incluse într-un pachet din considerente de ordonare logică a claselor.

Pentru a înțelege modul în care se creează și exploatează un subpachet, vom adăuga pachetului `primulPachet` un subpachet numit `pachetel`, care conține o singură clasă, `C`, aflată în fișierul `C.java` din `bin`.

```
package primulPachet.pachetel; //numele subpachetului
public class C
{
    public void tip() {
        System.out.println("clasa din structuri.pachetel");
    }
}
```

Compilăm clasa cu: `javac -d C:\Pachete C.java` Automat, în folder-ul `primulPachet`, pe lângă fișierele `A.class`, `B.class` va apărea subfolder-ul `pachetel` care conține fișierul `C.class`.

Programul următor, aflat în fișierul `Inv.java` din `bin` va utiliza subclasa `C` din `pachetel`.

```
import primulPachet.pachetel.*;
class Inv {
    public static void main(String[] args) {
        C x=new C();
        x.tip();
    }
}
```

Compilăm programul prin: `javac -classpath C:\Pachete Inv.java`
După această comandă în `bin` avem fișierul `Inv.class` (a fost necesar să se cunoască locul unde este clasa `C`).

Lansarea în executare se face cu comanda `java -classpath C:\Pachete;C:\j2sdk1.4.2_04\bin Inv`. A doua cale din `classpath` este către folder-ul `bin` (acolo am pe calculatorul meu instalat mediul `Java`). Fiecare trece calea pentru locul unde are instalat pe propriul calculator mediul `Java`.

📢 Foarte important !

1. Prin `import primulPachet.pachetel.*;` -solicităm să avem acces la toate clasele din subpachetul `pachetel`. Cum acest subpachet nu conține decât o clasă (C), am putea scrie:

```
import primulPachet.pachetel.C;
```

2. În acest caz, nu se poate solicita accesul la clasa subpachetului prin `import primulPachet.*;`, pentru că o astfel de solicitare asigură accesul la toate clasele pachetului `primulPachet`, nu și la toate clasele subpachetelor sale.

4.11. Clase interioare

În Java există posibilitatea ca o clasă să includă una sau mai multe clase. Ca și datele membru sau metodele unei clase, **clasele interne** pot fi statice sau nestatice.

A) Clase nestatice. Acestea se includ în structura de clasă. Pentru fi activate, clasa "mamă" va conține o referință către un obiect al clasei interne. Prin intermediul acestei referințe se pot accesa datele membru și metodele clasei interne. Vezi exemplul de mai jos, unde referința către obiectul clasei interne este `ref`. În acest caz, obiectul clasei "mamă" conține o referință către obiectul clasei interne. În exemplu, se apelează metoda `M1()` a clasei "mama" și metoda `M2()` a clasei interne. De asemenea, se afișează data membru a clasei interne, numită `b`.

```
class A {
    I ref=new I();
    int a=1;
    void M1()
    { System.out.println("a="+a);}
    class I {
        int b=2;
        void M2()
        { System.out.println("Metoda din clasa inclusa");}
    }
}

public class C {
    public static void main(String[] args) {
        A x=new A();
        x.M1();
        x.ref.M2();
        System.out.println(x.ref.b);
    }
}
```

B) Clase statice. Mai jos, aveți un astfel de exemplu. Observați modul de apel al metodelor și datelor membru:

```
class A {
    int a=1;
    void M1()
    { System.out.println("a="+a); }
    static class I {
        static int b=2;
        static void M2()
        { System.out.println("Metoda din clasa inclusa"); }
    }
}

public class C {
    public static void main(String[] args) {
        A x=new A();
        x.M1();
        A.I.M2();
        System.out.println(A.I.b); }
}
```

Observații:

1. Dacă o clasă conține o clasă interioară, atunci după compilare se obțin două clase (două fișiere cu extensia `.class`). Prima cu numele clasei (în exemplul dat, `A.class`), a doua cu numele clasei care include o altă clasă, urmat de caracterul "\$" și de numele clasei incluse (în exemplu `A$I.class`).

2. Pentru clasele interioare nestatice, datele membru și metodele claselor interioare se pot accesa și fără a utiliza acea referință către clasa interioară, ca în exemplul următor, referitor la programul prezentat la începutul acestui paragraf.

```
A x=new A();
A.I y= x.new I();
y.M2();
```

3. Dintr-o clasă interioară se pot accesa datele membru ale clasei care o conține. Testați!

4.12. O problemă de terminologie

Am văzut că metodele pot întoarce referințe către obiecte. De asemenea, parametrii metodelor pot fi de tip referință către obiecte. Prin abuz de limbaj, pentru a nu complica exprimarea, vom spune, uneori, că metodele întorc obiecte, vom vorbi despre obiectul (obiectele) primite ca parametru de o metodă. Însă, acestea trebuie *înțelese ca referințe către obiecte*.

Probleme propuse

Exercițiile 1. și 2. se referă la clasa de mai jos, care se găsește deja compilată în bin:

```
class a {  
    int a=1;  
    int a()  
    { int a=2;  
      return a;}  
}
```

1. Care dintre programele de mai jos este corect din punct de vedere sintactic?

a)
class ex {
 public static void
 main(String[] args) {
 a a=new a();}
 }

b)
class ex {
 public static void
 main(String[] args) {
 a x=new a();
 }

c)
class ex {
 public static void
 main(String[] args) {
 x a=new a();}
 }

d)
class ex {
 public static void
 main(String[] args) {
 a x=new ();}
 }

2. Fiind dată `x`, o variabilă care reține o referință către un obiect al clasei `a`, care dintre instrucțiunile de mai jos afișează valoarea 2?

a) `System.out.println(x.a);`

b) `System.out.println(x.a.a);`

c) `System.out.println(x.a());`

d) Nici una dintre instrucțiunile de mai sus nu afișează valoarea 2.

3. Fiind dată clasa de mai jos, ce afișează instrucțiunea aflată în `main()`: `A.M(3.5)`?

```
class A {  
    static void M(int x) {System.out.println(1);}  
    static void M(float x) {System.out.println(2);}  
    static void M(double x) {System.out.println(3);}  
    static void M(char x) {System.out.println(4);}  
}
```

a) 1;

b) 2;

c) 3;

d) 4.

4. Ce afișează programul următor?

```
class A {  
    int a,b;  
    A(int a, int b)  
    { this.a=a;  
      this.b=b;}  
    A (int x) {b=x;}  
    void Afis() {System.out.print (a+" "+b);}  
}  
  
class ex {  
    public static void    main(String[] args) {  
        A x=new A(3);  
        x.Afis();  
    }  
}
```

- a) Eroare; b) 3 0; c) 0 0; d) 0 3.

5. Creați o clasă numită **Lista**, care operează cu liste liniare simplu înlănțuite. Un element al listei liniare simplu înlănțuite este descris de clasa **Nod**:

```
class Nod  
{ Nod adr_urm;  
  int nr; }
```

Clasa **Lista** va conține:

- ❖ Data membru **v** - adresa primului nod al listei. În cazul în care lista este nulă, **v** reține **null**.
- ❖ Metoda **void adaug (int nr)** - adaugă la sfârșitul listei un nod care va reține valoarea numerică **nr**, primită de metodă ca parametru.
- ❖ Metoda **void afis()** - care afișează valorile reținute de noduri.

6. Adăugați clasei **Lista** o metodă **void stergP()** care permite ștergerea primului nod introdus.

7. Adăugați clasei **Lista** o metodă **void stergU()** care permite ștergerea ultimului nod introdus.

8. Adăugați clasei **Lista** o metodă **void adaugI(int nr)** care permite ca la începutul listei să se adauge un nod care să conțină valoarea **nr**, primită de metodă ca parametru.

9. Scrieți un program care creează o listă liniară simplu înlănțuită. Programul va utiliza clasa **Lista**, creată anterior. Numerele se citesc de la tastatură și se vor găsi în listă în ordinea în care au fost introduse. Șirul de numere se consideră încheiat atunci când se citește numărul 0 (acesta nu este introdus în listă).

10. La fel ca la problema 9, numai că elementele se vor găsi în listă în ordinea inversă introducerii lor.

11. Adăugați clasei **Lista** o metodă **void concat(Lista)** care să concateneze lista curentă cu o altă listă. Scrieți un program care o testează.

12. Realizați o clasă numită **Stiva**, care operează cu stive alocate ca liste liniare simplu înlănțuite. Un element al listei este descris de clasa **Nod**:

```
class Nod
{ Nod adr_prec;
  int nr; }
```

Clasa **Stiva** va conține următoarele:

- ❖ Data membru **varf** - reține adresa ultimului nod introdus în stivă.
- ❖ Metoda **void push(int nr)** - adaugă un nod stivei și acesta va reține valoarea numerică **nr**, primită de metodă ca parametru.
- ❖ Metoda **int pop()** - extrage un nod din stivă și returnează valoarea pe care acesta o reținea. Dacă stiva este vidă, returnează -1.

13. Creați un folder numit **Pac** pe **c:**. Acest folder va conține pachetul **citiri**, care include clasa **cin.java** (cea pe care am utilizat-o de atâtea ori pentru citirea datelor).

14. Scrieți un program care citește și afișează o variabilă întreagă. Programul va utiliza pachetul **citiri**.

15. Realizați un pachet care conține clasele **Nod** și **Stiva**, create la exercițiul anterior. Pachetul se va numi **structuri** și se va găsi în folder-ul **Pac**, creat în **C:**.

16. Prin utilizarea clasei **Stiva**, scrieți un program care citește **n** numere naturale și le afișează în ordinea inversă citirii.

17. Creați o clasă numită **Coadă**, care implementează structura de tip coadă. Fiecare element al cozii va fi de tip **Complex** (deci va reține un număr întreg).

18. Adăugați pachetului **structuri** (vezi problema 15) clasa **Coadă**.

19. Scrieți un program care utilizează clasa **Coadă** din pachetul **structuri**.

20. Realizați un program care creează și parcurge în preordine, inordine și postordine un arbore binar. Fiecare nod al arborelui reține un număr real. Datele se citesc de la tastatură.

21. Creați un pachet care conține clasele pe care le-ați utilizat în programul precedent.

Răspunsuri:

1. a) 2. c) 3. c) o constantă reală care nu se termină cu **f**, este considerată automat ca fiind de tipul **double**. Prin urmare, este apelată automat metoda cu parametru de tip **double**.

4. d) Clasa este înzestrată cu doi constructori (supraîncărcati). Este apelat constructorul cu un singur parametru. Data membru **a** este inițializată implicit cu 0.

5.

```
class Lista
{ Nod v;
  void adaug(int nr)
  { // daca lista este vida
    if (v==null)
      { v=new Nod();
        v.nr=nr;
        v.adr_urm=null;}
    else
      // daca lista este nevida, aceasta este parcursa mai
      // intai, dupa care se adauga nodul
      { Nod c=v;
        while (c.adr_urm!=null) c=c.adr_urm;
        c.adr_urm=new Nod();
        c.adr_urm.nr=nr;
        c.adr_urm.adr_urm=null;}
      }
}
```

6. Nu este nevoie ca memoria ocupată de nodul șters să fie eliberată. De aceasta se ocupa **Garbage Collector**.

```
void stergP()
{ if (v!=null) v=v.adr_urm; }
```

7. La fel ca mai sus, nu este nevoie ca memoria ocupată de nodul șters să fie eliberată.

```
void stergU()
{ Nod c=v;
  while (c.adr_urm.adr_urm!=null) c=c.adr_urm;
  c.adr_urm=null;}
```

8.

```
void adaugI(int nr)
{ Nod c=new Nod();
  c.nr=nr;
  c.adr_urm=v;
  v=c;
}
```

9.

```

class ex {
    public static void main(String[] args) {
        Lista l1=new Lista();
        int Numar_citit;
        do
        {   System.out.print("numar_citit=");
            Numar_citit=Integer.parseInt(cin.Token());
            if (Numar_citit!=0) l1.adaug(Numar_citit);
        } while (Numar_citit!=0);
        l1.afis();
    }
}

```

10. Identic, numai că, în loc de

```
        if (Numar_citit!=0) l1.adaug(Numar_citit);
```

va fi `if (Numar_citit!=0) l1.adaugI(Numar_citit);`.

11.

```

void concat(Lista varf)
{   Nod c=v;
    while (c.adr_urm!=null) c=c.adr_urm;
    c.adr_urm=varf.v;
}

```

12.

```

class Stiva
{   Nod varf;

    void push(int nr)
    {   Nod c=new Nod();
        c.nr=nr;
        c.adr_prec=varf;
        varf=c;
    }

    int pop()
    {   if (varf!=null)
        {   int x=varf.nr;
            varf=varf.adr_prec;
            return x; }
        else return -1;
    }
}

```

13.

Creăm folder-ul `Pac`. Fișierul `cin.java` se găsește în `bin`. Îl modificăm astfel:

- prima linie va conține `package citiri;`
- clasa devine publică și fiecare metodă a ei tot publică.


```
s=s+s1;
System.out.println(s);
```

2. Același lucru este afișat și de secvența de mai jos:

```
String s =new String ("Un sir "+"Alt sir" );
System.out.println(s);
```



După cum observați, concatenarea șirurilor nu este comutativă. Aceasta înseamnă că prezintă importanță ordinea în care șirurile sunt puse în expresie.

⇒ Metoda `String concat(String s)` returnează șirul obținut prin concatenarea șirului reținut de obiectul curent cu `s`.

Exemplu: secvența următoare afișează `Un sir Alt sir`:

```
String s =new String ("Un sir ");
String s1 =new String ("Alt sir" );
System.out.println(s.concat(s1));
```

5.2.5. Parametrii metodei `main()`

Acum suntem în măsură să analizăm parametrii metodei `main()`:

```
public static void main(String[] args).
```

Antetul conține un vector cu elemente de tip `String`. În exemplu, numele vectorului este `args`.

1. Câte elemente are vectorul? Acest număr se poate afla ușor, ca la orice vector: `args.length`.

2. Unde se introduc șirurile de caractere primite ca parametri de către `main()`? Aceste șiruri se introduc pe linia de comandă a programului `Java` și sunt separate prin spații.

Exercițiu: programul de mai jos calculează suma valorilor introduse ca parametri. Evident, mai întâi acestea sunt convertite către `int`. Exemple de utilizare:

a) Dacă apelul este `java t 3 4` se va afișa 7.

b) Dacă apelul este `java t 1 2 3 4` se va afișa 10.

```
class t {
    public static void main(String[] args) {
        int s=0;
        for (int i=0;i<args.length;i++)
            s+=Integer.parseInt(args[i]);
        System.out.println("suma argumentelor este =" +s);
    }
}
```

5.2.6. Aplicații

1. Să se citească de la tastatură un șir de caractere, apoi să se afișeze.

Rezolvare. Vom utiliza clasa `cin` (cea pe care am folosit-o și până acum). De altfel, clasa `cin` o vom utiliza pentru toate exercițiile din acest paragraf.

```
public static void main(String[] args) {
    String s = new String ();
    s = cin.linie();
    System.out.println(s);
}
```

2. Se citesc două șiruri de caractere. Să se listeze indicele fiecărei apariții a primului șir citit ca subsir al celui de-al doilea șir citit. Iată cum arată execuția programului:

```
mama
tata mama tata mama tata mama
5
15
25
```

Rezolvare: Ideea este următoarea: după ce subsirul a fost identificat, se afișează poziția de început, apoi se extrage din șir subsirul de început ce conține inclusiv prima apariție a subsirului, după care procedeul se repetă până când subsirul căutat nu mai este găsit. O problemă aparte este dată de faptul că trebuie, de fiecare dată, să afișăm poziția de început a subsirului în șirul inițial și nu în șirul rămas. Din acest motiv, utilizăm mai multe variabile: `poz` - poziția de început a subsirului în șirul rămas; `ind` - suma lungimilor șirurilor extrase. De fiecare dată se va afișa ca indice de apariție a subsirului, `ind+poz`.

```
public static void main(String[] args) {
    String subsir = new String (cin.linie());
    String sir = new String (cin.linie());
    int poz=0, ind=0, lsubs=subsir.length();
    while (poz!=-1)
    { poz=sir.indexOf(subsir);
      if (poz!=-1)
      { System.out.println(ind+poz);
        ind+=poz+lsubs;
        sir=sir.substring(poz+lsubs); }
    }
}
```

3. Se citesc două șiruri de caractere. Să se afișeze șirul format din cel de-al doilea șir din care au fost extrase toate aparițiile primului șir citit. Alăturat observați un exemplu de execuție:

```
12
123 412 5612
3 4 56
```

Rezolvare: Se identifică prima apariție a subsirului, apoi, prin extragerea subsirului, se obține un nou șir, iar se identifică prima apariție a subsirului ș.a.m.d. Șirul astfel obținut se afișează atunci când nu se mai conține nici o apariție a subsirului citit.

```

public static void main(String[] args) {
    String subsir =new String (cin.linie());
    String sir =new String (cin.linie());
    int poz=0, ind=0, lsubs=subsir.length();
    while (poz!=-1)
    {
        poz=sir.indexOf(subsir);
        if (poz!=-1)
            sir=sir.substring(0,poz)+sir.substring(poz+lsubs);
    }
    System.out.println(sir);
}

```

4. Se citește un șir de caractere. Șirul conține mai multe cuvinte separate de unul sau mai multe blank-uri. Se cere să se elimine blank-urile de la începutul și sfârșitul șirului. De asemenea, se cere ca, în noul șir, cuvintele să fie separate obligatoriu printr-un singur blank.

Rezolvare. După eliminarea blank-urilor de început și de sfârșit sub un `for` se identifică primele două blank-uri consecutive, unde primul blank are indicele `i`. Se elimină primul dintre ele, apoi valoarea lui `i` scade cu 1, pentru ca în șirul nou format să se reia căutarea de la primul blank, cel neeliminat (nu uitați, la testul următor valoarea lui `i` crește cu 1, pentru că totul se execută sub `for`).

```

public static void main(String[] args) {
    String sir =new String (cin.linie());
    sir=sir.trim();
    for (int i=1;i<=sir.length()-2;i++)
        if (sir.charAt(i)==' ' && sir.charAt(i+1)==' ')
            { sir=sir.substring(0,i)+sir.substring(i+1);
              i--;
            }
    System.out.println(sir);
}

```

5. Se citesc `n` cuvinte, câte unul pe linie. Se cere să se afișeze cuvintele în ordine lexicografică (ordinea alfabetică, întâlnită în dicționar).

Răspuns. Vom utiliza un vector de obiecte de tip `String`. În rest, este o sortare obișnuită...

```

public static void main(String[] args) {
    System.out.print("n=");
    int n=Integer.parseInt(cin.Token());
    String [] V=new String [n];
    for (int i=0;i<n;i++)
        { System.out.print("Cuvantul["+i+"]=");
          V[i]=cin.linie();
        }
    boolean gasit;
    String man;
}

```

```

do
{
    gasit=false;
    for (int i=0;i<n-1;i++)
        if (V[i].compareTo (V[i+1])>0)
        {
            man=V[i];
            V[i]=V[i+1];
            V[i+1]=man;
            gasit=true;
        }
    } while (gasit);
    for (int i=0;i<n;i++)
        System.out.println(V[i]);
}

```

5.3. Clasa StringTokenizer

Clasa `StringTokenizer` se găsește în pachetul `java.util` (vezi `import`) și conține câteva metode care permit extragerea unităților lexicale (token-uri) din care este alcătuit un șir de caractere. Implicit, acestea se consideră separate prin caracterele albe (' ', '\n', '\t', '\r' '\f'). Iată cele mai importante metode:

- ⇒ `StringTokenizer (String s)` – constructor, creează obiectul care tratează șirul de caractere referit de `s`.
- ⇒ `StringTokenizer (String s, String delim)` – constructor, creează obiectul care tratează șirul de caractere referit de `s`., dar token-urile sunt separate de caracterele care alcătuiesc `delim`.
- ⇒ `boolean hasMoreTokens()` – returnează `true` dacă șirul mai conține token-uri necitite și `false` în caz contrar.
- ⇒ `String nextToken()` – returnează șirul de caractere care alcătuiesc următorul token (citește un token).
- ⇒ `int countTokens()` – returnează numărul de token-uri rămase necitite (cu `nextToken()`).

Exemplu: programul următor citește o linie introdusă de la tastatură (vezi clasa `cin`) și afișează token-urile întâlnite. De exemplu dacă linia introdusă de la tastatură este: `mama are 350000 lei`, se afișează:

```

import java.util.*;
class t {
    public static void main(String[] args) {
        String s=new String (cin.linie());
        StringTokenizer t=new StringTokenizer(s);
        while (t.hasMoreTokens())
            System.out.println(t.nextToken());
    }
}

```

```

mama
are
350000
lei

```

5.4. Clase înfășurătoare

Pentru fiecare tip primitiv s-a construit câte o clasă înfășurătoare. Un obiect al clasei înfășurătoare poate reține o valoare a tipului primitiv. Alăturat, puteți observa corespondența între numele tipurilor primitive și numele claselor înfășurătoare.

int	-	Integer
short	-	Short
long	-	Long
byte	-	Byte
char	-	Character
float	-	Float
double	-	Double
boolean	-	Boolean

⇒ Fiecare clasă înfășurătoare este înzestrată cu o metodă constructor.

Exemple:

```
int n=4;
Integer nr=new Integer(n);
Double x=new Double (-12.34);
Character c=new Character ('e');
Boolean este=new Boolean(true);
```

⇒ Constructorii acestor clase sunt supraîncărcați, în sensul că există și constructorii care au ca parametru de intrare referințe către obiecte de tip **String**.

Exemple:

```
Integer nr=new Integer("10");
Double x=new Double ("-12.34");
Boolean este=new Boolean("true");
```

⇒ Fiecare clasă înfășurătoare conține o metodă care returnează valoarea reținută de obiectul instanțiat de ea.

Exemple: se extrag valorile reținute de obiectele create în primele exemple:

```
int n1=nr.intValue();
double x1=x.doubleValue();
char c1=c.charValue();
boolean estel=este.booleanValue();
```

⇒ Fiecare clasă care reține o valoare numerică conține câte o metodă *statică* (atenție la apel!) pentru *conversia către un tip primitiv* a unui obiect de tip **String**. Astfel avem:

a) `int parseInt(String s); // in Integer`

Exemplu: `int t=Integer.parseInt("10");`

b) `parseFloat(String S); // in Float`

Exemplu: `float b=Float.parseFloat("-12.34");`

c) `double parseDouble (String s) // in Double`

Exemplu: `double b=Double.parseDouble("-12.34");`

d) `parseLong(String s); // in Long`

Exemplu: `long t=Long.parseLong("1000");`

e) `parseByte(String s) // in Byte`

Exemplu: `byte x=Byte.parseByte("12");`

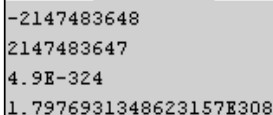
- ✓ Dacă șirul de caractere nu poate fi convertit către valoarea numerică de tipul dorit, se generează o excepție în urma căreia executarea programului se întrerupe.
- ✓ Conversia inversă, de la un tip primitiv către un șir se poate realiza ușor, așa cum am mai întâlnit de multe ori concatenând un șir (eventual vid) cu o valoare numerică.

Exemplu: secvența următoare afișează șirul 10:

```
int i=10;  
String s=new String(i+"");  
System.out.println(s);
```

- ⇒ Clasele înfășurătoare conțin constantele **MIN_VALUE** și **MAX_VALUE** care rețin cea mai mică și cea mai mare valoare a tipului respectiv. În cazul variabilelor de un tip real (**float**, **double**) **MIN_VALUE** are semnificația de cea mai mică valoare pozitivă care poate fi reținută de tipul respectiv.

```
System.out.println(Integer.MIN_VALUE);  
System.out.println(Integer.MAX_VALUE);  
System.out.println(Double.MIN_VALUE);  
System.out.println(Double.MAX_VALUE);
```



```
-2147483648  
2147483647  
4.9E-324  
1.7976931348623157E308
```

- ⇒ Clasele înfășurătoare ale tipurilor reale (**double**, **float**) conțin constanta **NaN** (**Not a Number**). Valoarea **NaN** se obține dacă, *într-o expresie de tip real*, se împarte 0 la 0, sau se extrage radical (indice 2) dintr-o valoare negativă, sau se aplică logaritmul unui număr negativ etc.

Exemple:

1. Secvența de mai jos afișează **NaN**:

```
double x=0,y=0,z;  
z=x/y;  
System.out.println(z);
```

2. Secvența de mai jos afișează **NaN**:

```
int x=-2;  
System.out.println(Math.sqrt(x));
```

⇒ Clasele înfășurătoare ale tipurilor reale (`double`, `float`) conțin metodele `boolean isNaN (double e)`, respectiv `boolean isNaN (float e)`. Cele două metode returnează `true` dacă se obține `NaN`.

Exemple:

1. Secvența de mai jos afișează: **Ambii operanzi sunt 0:**

```
int x=0; double y=0;
if (Double.isNaN(x/y))
    System.out.println("Ambii operanzi sunt 0");
```

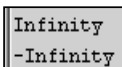
2. Secvența de mai jos afișează: **Radical dintr-un numar negativ:**

```
if (Double.isNaN(Math.sqrt(x)))
    System.out.println("Radical dintr-un numar negativ");
```

✓ Comparările precum cele de mai jos nu au efect. De exemplu, secvența de mai jos nu afișează nici un mesaj:

```
if (Math.sqrt(x)==Double.NaN)
    System.out.println("Radical din numar negativ");
```

⇒ Clasele înfășurătoare ale tipurilor reale (`double`, `float`) conțin constantele `POSITIVE_INFINITY`, `NEGATIVE_INFINITY`. Constanta `POSITIVE_INFINITY` se obține dacă, *într-o expresie de tip real*, se împarte o valoare pozitivă la 0, iar constanta `NEGATIVE_INFINITY` se obține dacă, *într-o expresie de tip real*, se împarte la 0 o valoare negativă.



```
double x=2;
System.out.println(x/0);
x=-2;
System.out.println(x/0);
```



Dacă expresia este de tip întreg, atunci se generează o *excepție* (excepțiile vor fi studiate separat), iar programul este întrerupt, ca în secvența de mai jos.

```
int x=2;
System.out.println(x/0);
```

⇒ Clasele înfășurătoare ale tipurilor reale (`double`, `float`) conțin metoda booleană `boolean isInfinite (double)` (sau `float`) prin care se testează dacă s-a obținut una din constantele de mai sus.

Exemplu: secvența de mai jos afișează **Impartire la 0.**

```
double x=2;
if (Double.isInfinite(x/0))
    System.out.println("Impartire la 0");
```

5.5. Lucrul cu numere mari

Prin numere mari vom înțelege acele valori numerice care depășesc limitele de memorare ale tipurilor primitive. Operațiile cu astfel de numere sunt anevoioase prin faptul că numărul trebuie memorat pe cifre (într-un vector). În Java există două clase care permit lucrul cu numere mari: clasa **BigInteger** și clasa **BigDecimal**.

A) Clasa **BigInteger** lucrează cu numere întregi mari. Cele mai importante metode sunt:

⇒ **BigInteger(String s)** - constructor. Are rolul de a crea un obiect de tip **BigInteger** pornind de la un obiect de tip **String**.

Exemplu: `BigInteger n2=new BigInteger("-2");`

⇒ **String toString();** - are rolul de a converti numărul reținut de obiectul curent către un șir de caractere.

⇒ **BigInteger add (BigInteger n);** - adună la numărul reținut de obiectul curent numărul reținut de obiectul transmis ca parametru.

⇒ **BigInteger subtract (BigInteger n);** - scade din numărul reținut de obiectul curent numărul reținut de obiectul transmis ca parametru.

⇒ **BigInteger multiply (BigInteger n);** - înmulțește numărul reținut de obiectul curent cu numărul reținut de obiectul transmis ca parametru.

⇒ **BigInteger divide (BigInteger n);** - împărțire întreagă între numărul reținut de obiectul curent și numărul reținut de obiectul a cărui referință este reținută de **n**.

⇒ **BigInteger remainder (BigInteger n);** - returnează un obiect care reține restul împărțirii numărului reținut de obiectul curent la numărul obiectului transmis ca parametru.

⇒ **BigInteger pow (BigInteger n);** - dacă obiectul curent reține **m** și obiectul transmis ca parametru valoarea **n**, atunci metoda returnează obiectul care reține **mⁿ**.

⇒ **BigInteger negate();** - dacă obiectul curent reține **m** atunci metoda returnează obiectul care reține **-m**.

⇒ **int signum();** - dacă obiectul curent reține **m** se returnează obiectul care reține **sgn(m)**, adică: -1 dacă **m<0**, 0 dacă **m=0** sau 1 dacă **m>0**.

- ⇒ `int CompareTo(BigInteger n);` - dacă obiectul curent reține `m`, iar obiectul referit reține `n`, atunci metoda returnează `-1` dacă `m < n`, `0` dacă `m = n`, `1`, dacă `m > n`.

Exemplu: se cere să se calculeze 2^{1000} . Numărul depășește cu mult capacitatea de memorare a tipului `long`. Prin urmare, vom utiliza clasa `BigInteger`:

```
import java.math.*;
class t {
    public static void main(String[] args) {
        BigInteger n1=new BigInteger("1");
        BigInteger n2=new BigInteger("2");
        for (int i=1;i<=1000;i++) n1=n1.multiply(n2);
        System.out.println(n1.toString());
    }
}
```

B) Clasa BigDecimal - cele mai importante metode sunt:

- ⇒ `BigDecimal(String s)` - constructor. Are rolul de a crea un obiect de tip `BigDecimal` pornind de la șirul `s`.
- ⇒ `BigDecimal(BigInteger val)` - constructor, creează un obiect pornind de la un altul de tip `BigInteger`;
- ⇒ `BigDecimal(double val)` - constructor, creează un obiect pornind de la o valoare de tip `double`.
- ⇒ `BigDecimal abs()` - returnează obiectul care reține modulul numărului reținut de obiectul curent.
- ⇒ `String toString()` - returnează numărul reținut de obiectul curent convertit către `String`.
- ⇒ `BigDecimal add(BigDecimal n)` - returnează obiectul care reține suma numerelor reținute de obiectul curent și cel transmis ca parametru. Rezultatul va avea numărul maxim de zecimale pe care le au cei doi operanzi.
- ⇒ `BigDecimal subtract(BigDecimal n)` - returnează obiectul care reține valoarea rezultată din scăderea numerelor reținute de cei doi operanzi (obiectul curent, descăzut și obiectul transmis ca parametru, scăzător). Numărul de zecimale al rezultatului se obține ca la `add`.
- ⇒ `BigDecimal multiply(BigDecimal n)` - obiectul care reține produsul numerelor reținute de cei doi operanzi. Numărul de zecimale al rezultatului este dat suma numerelor de zecimale ale operanzilor.
- ⇒ `BigDecimal divide(BigDecimal val, int rotunjire)` - împarte cei doi operanzi. Numărul de zecimale al rezultatului este egal cu numărul de zecimale al deîmpărțitului.

Exemplu: fie obiectele de mai jos:

```
BigDecimal n1=new BigDecimal("1.000");  
BigDecimal n2=new BigDecimal("3.000000");  
  
n1=n1.divide(n2,0); -n1 reține 0.334  
n1=n1.divide(n2,1); -n1 reține 0.333
```

- ⇒ **int scale()** - returnează numărul zecimalelor ale numărului reținut de obiectul curent.
- ⇒ **BigDecimal setScale(int nrzec)** - returnează un obiect care memorează numărul reținut de obiectul curent cu **nrzec** zecimale.
- ⇒ **BigDecimal negate()** - returnează un obiect care reține numărul memorat de obiectul curent, dar care a fost negativat.
- ⇒ **int signum()** - returnează **sgn(x)** vezi metoda similară de la **BigInteger**;
- ⇒ **BigDecimal abs()** - returnează obiectul care reține modulul numărului memorat de obiectul curent.
- ⇒ **int compareTo(BigDecimal val)** - compară numerele reținute de cei doi operanzi. Vezi metoda similară de la **BigInteger**.

Probleme propuse

1. Scrieți un program care să citească $0 < a < b$, numere naturale și să afișeze un număr aleator în intervalul $[a, b]$.
2. Scrieți un program care citește **x**, număr real, și **n** > 0, număr natural și calculează $P = \sin(x) * \sin(2x) * \dots * \sin(nx)$. Se presupune că **x** reprezintă o valoare exprimată în radiani.
3. Se citește de la tastatură un șir de caractere. Șirul conține cel puțin 10 caractere. Se cere să se afișeze numai primele 10 caractere din șir.
4. Se citește de la tastatură un șir de caractere. Dacă șirul are mai mult de 4 caractere să se afișeze numai primele 4, altfel se va afișa șirul citit.
5. Scrieți o clasă, numită **Afis**, care conține metoda **String aNum(int lung, String s)** metodă ce are rolul de a întoarce șirul reținut de **s**, pe lungimea **lung**. Dacă numărul de caractere ale șirului este mai mare decât **lung**, se întoarce subșirul format din primele **lung** caractere ale șirului, altfel se întoarce un șir pe lungimea **lung**. Șir alcătuit din caracterele șirului inițial la care se adaugă de un număr corespunzător de blank-uri. **Exemple** pentru **lung=5**:
 - Șirul: citit **"Mihai"** - se returnează **"Mihai"**.

- Șirul: "tastatura" - se returnează "tasta".
- Șirul: "aer" - se returnează "aer".



O astfel de metodă este utilă pentru afisarea unui șir de caractere pe o anumită lungime.

6. Scrieți un program care citește n , număr natural și n șiruri de caractere. Programul va afișa pe câte o linie, primele 5 caractere ale fiecărui șir citit. Dacă șirul are mai puțin de 5 caractere, se va afișa șirul, urmat de un număr de blank-uri astfel încât lungimea totală afișată să fie de 5 caractere.

7. Adăugați clasei **Afis** metoda **String num(int lung, int v)** care are rolul de a returna șirul de lungime **lung**, alcătuit din șirul obținut din convertirea întregului **v** în șir și, în cazul în care acest șir are lungimea mai mică decât **lung**, el va fi precedat de un număr corespunzător de blank-uri. În situația în care lungimea șirului obținut din conversia numărului este mai mare decât **lung**, metoda va returna un mesaj de eroare. Exemple pentru **lung=3**.

- Numărul 125 - se returnează "125".
- Numărul 1 - se returnează " 1".
- Șirul: "3152" - se returnează "Numarul nu poate fi afisat";

8. Scrieți un program care citește n , număr natural și n numere naturale. Programul va afișa, pe câte o linie, fiecare dintre cele n numere naturale citite. Numerele se vor afișa pe 5 caractere și vor fi aliniate la dreapta.

Exemplu: $n=3$, numerele citite sunt: 12234, 45, 4. Se va afișa:

```
12234
  45
   4
```



În practică, șirurile alcătuite din caractere se afișează aliniate la stânga, iar valorile numerice se afișează aliniate la dreapta.

9. Adăugați clasei **Afis** metoda **String num(int lung, int zec, double v)** care returnează șirul rezultat din conversia unui număr real (indiferent de numărul de zecimale). Șirul are lungimea **lung**, numărul este afișat aliniat la dreapta și va avea **zec** zecimale. În ipoteza în care numărul nu poate fi afișat prin utilizarea a **lung** caractere, se va returna un mesaj de eroare.

- ❖ **Num(5,2,23.3456)** returnează "23.34";
- ❖ **Num(5,2,3.3456)** returnează " 3.34";
- ❖ **Num(5,2,-9)** returnează "-9.00";
- ❖ **Num(5,2,123)** returnează "Numarul nu poate fi afisat".

10. Scrieți un program care citește n , număr natural și n numere reale. Programul va afișa pe câte o linie fiecare număr citit. Numerele se vor afișa pe 8 caractere, vor fi aliniate la dreapta și vor avea obligatoriu două zecimale (indiferent de numărul de zecimale pe care îl au).

11. Să se creeze o clasă numită **Elev**. Un obiect al clasei respective conține numele elevului, nota la matematică, nota la informatică și nota la engleză. Clasa va conține un constructor care citește de la tastatură valorile respective și o metodă **void afisez()**, care afișează toate datele pe o linie astfel:

- ❖ nume elev pe 20 de caractere, aliniat la stânga;
- ❖ nota la matematică pe 5 caractere, aliniată la dreapta;
- ❖ nota la informatică pe 5 caractere, aliniată la dreapta;
- ❖ nota la engleză pe 5 caractere, aliniată la dreapta;
- ❖ media generală, pe 8 caractere, cu două zecimale, aliniată la dreapta.

12. Scrieți un program care, prin utilizarea clasei **Elev**, citește numele și notele pentru n elevi și le afișează aliniat, împreună cu media generală.

13. Scrieți un program care citește un șir introdus de la tastatură. Cuvintele din șir sunt separate printr-unul sau mai multe spații. Se cere să se afișeze șirul obținut din cuvintele din șirul inițial, dar care, de această dată, sunt separate printr-un singur spațiu.

Exemplu:

se citește: **mama merge la plimbare**

se afișează: **mama merge la plimbare**

14. Calculați $n!$ pentru $1 \leq n \leq 1000$.

15. Rezolvați ecuația de gradul 1 $ax+b=0$, unde a și b sunt numere reale atât de mari că nu pot fi reținute de variabile de tip **double**.

16. Scrieți un program care calculează e^x , cu k zecimale exacte (k se citește de la tastatură). Calculul se va face prin utilizarea formulelor de mai jos:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!} + R_n(x) \qquad |R_n(x)| \leq e^{|x|} \frac{|x|^{n+1}}{(n+1)!}$$

Dacă, de exemplu $k=30$, atunci se va efectua calculul pentru n determinat astfel încât:

$$R_n(x) < \frac{1}{10^{30}}$$

17. Aflați prin metoda înjumătățirii intervalului rădăcina reală a ecuației $x^3+x+1=0$. Se cere ca rădăcina să fie afișată cu primele 50 de zecimale exacte.

Indicații / rezolvări:

```
1. class prima {
    public static void main(String[] args) {
        System.out.print("a="); int a=Integer.parseInt(cin.Token());
        System.out.print("b="); int b=Integer.parseInt(cin.Token());
        System.out.println(a+Math.floor(Math.random()*(b-a+1)));
    }
}
```

```
3. class prima {
    public static void main(String[] args) {
        String afis=new String(cin.linie());
        System.out.println(afis.substring(0,10));
    }
}
```

4. Dacă încercați programul precedent, pentru un șir de caractere mai mic de 10 caractere veți primi eroare!

```
class prima {
    public static void main(String[] args) {
        String afis=new String(cin.linie());
        if (afis.length()>4)
            System.out.println(afis.substring(0,4));
        else System.out.println(sir);}
}
```

```
5. class Afis {
    String aNum(int lung, String sir)
    { if (sir.length()>lung)
        return sir.substring(0,lung);
      else
        { String blank="";
          for (int i=1;i<=lung-sir.length();i++) blank=blank+" ";
          return sir+blank; }
    }
}
```

```
6. class prima {
    public static void main(String[] args) {
        System.out.print("n="); int n=Integer.parseInt(cin.Token());
        String[] t=new String [n];
        for(int i=0;i<n;i++)
            t[i]=cin.linie();
        Afis comanda=new Afis();
        for(int i=0;i<n;i++)
            System.out.println(" "+comanda.aNum(5,t[i])); }
}
```

```
7.
String num(int lung, int v)
{ String sir=new String (" "+v);
  if (sir.length()>lung) return "Numarul nu poate fi afisat";
  else
    { while (sir.length() !=lung) sir=" "+sir;
      return sir;}
}

8.
class prima {
public static void main(String[] args) {
  System.out.print("n="); int n=Integer.parseInt(cin.Token());
  Afis comanda=new Afis();
  int[] t=new int [n];
  for(int i=0;i<n;i++)
    t[i]=Integer.parseInt(cin.Token());
  for(int i=0;i<n;i++)
    System.out.println(comanda.num(5,t[i]));
}
}

9.
String num(int lung, int zec, double v)
{ String sir=new String (" "+v);
  int indice=sir.indexOf(".");
  if (sir.length()-indice-1<zec)
    while (sir.length()-indice-1!=zec) sir=sir+"0";
  else
    sir=sir.substring(0,indice+zec+1);
  if (sir.length()>lung) return "Numarul nu poate fi afisat";
  else
    { while (sir.length() !=lung) sir=" "+sir; return sir; }
}

10.
class prima {
public static void main(String[] args) {
  System.out.print("n="); int n=Integer.parseInt(cin.Token());
  Afis comanda=new Afis();
  double[] t=new double [n];
  for(int i=0;i<n;i++)
    t[i]=Double.parseDouble(cin.Token());
  for(int i=0;i<n;i++)
    System.out.println(comanda.Num(8,2,t[i]));
}
}

11.
class Elev
{ String Nume;
  int nota_mate,nota_info,nota_engl;
  Elev()
  { System.out.println("Introduceti datele");
    System.out.print("Numele elev=");Nume=cin.linie();
```

```

        System.out.print("Nota mate=");
        nota_mate=Integer.parseInt(cin.Token());
        System.out.print("Nota info=");
        nota_info=Integer.parseInt(cin.Token());
        System.out.print("Nota engl=");
        nota_engl=Integer.parseInt(cin.Token());
    }
    void afisez()
    { Afis elv=new Afis();
      System.out.print(elv.aNum(20,Nume));
      System.out.print(elv.num(5,nota_mate));
      System.out.print(elv.num(5,nota_info));
      System.out.print(elv.num(5,nota_engl));
      double med=(double) (nota_mate+nota_info+nota_engl)/3;
      System.out.println(elv.num(8,2,med));}
}

```

12.

```

class prima {
    public static void main(String[] args) {
        System.out.print("n=");int n=Integer.parseInt(cin.Token());
        Elev[] tabel=new Elev[n];
        for (int i=0;i<n;i++) tabel[i]=new Elev();
        for (int i=0;i<n;i++) tabel[i].afisez();}
}

```

13.

```

import java.util.*;
class t {
    public static void main(String[] args) {
        String s=new String (cin.linie());
        StringTokenizer t=new StringTokenizer(s);
        String sir_corect= new String();
        while (t.hasMoreTokens())
            sir_corect=sir_corect+t.nextToken()+" ";
        sir_corect=sir_corect.trim();
        System.out.println(sir_corect);}
}

```

14.

```

import java.math.*;
class t {
    public static void main(String[] args) {
        System.out.print("n=");int n=Integer.parseInt(cin.Token());
        BigInteger P=new BigInteger("1"),I=new BigInteger("1"),incr=new
            BigInteger("1");
        for(int i=1;i<=n;i++)
            { P=P.multiply(I); I=I.add(incr);}
        System.out.println(P.toString());}
}

```

17. Dacă notăm cu $f(x)=x^3+x+1$, avem $f(-1)=-1$, $f(0)=1$. Cum $f'(x)=3x^2+1$, f este strict crescătoare și are o unică rădăcină în intervalul $(-1,0)$. Lugimea intervalului este 1. De acum putem aplica metoda înjumătățirii intervalului.

CAPITOLUL 6

Extinderea claselor



Din cuprins:

Un exemplu de extindere al unei clase
O modalitate de simulare a extinderii multiple
Referințe către obiectele superclaselor, referințe
către obiectele subclaselor
Probleme propuse

6.1. Noțiuni generale

1. Pornind de la o clasă dată se poate crea o altă clasă, care o *extinde* (*moștenește*) pe aceasta. Noua clasă va avea toate datele membru și toate metodele clasei de la care s-a pornit, cărora li se pot adăuga noi date membru și/sau noi metode. În **Java**, clasa care este extinsă se numește *superclasă*, iar noua clasă se numește *subclasă*.

Analizați exemplul de mai jos:

```
class C1
{ int x,y; }

class C2 extends C1
{ int z;
  void afis()
  { System.out.println(x+" "+ y+" "+z); }
}

public class C {
  public static void main(String[] args) {
    C2 ref= new C2();
    ref.x=3;
    ref.y=4;
    ref.z=5;
    ref.afis();
  }
}
```

Aici s-a extins clasa C1. Ea conține două date membru **x** și **y**, ambele de tip **int**. În exemplul dat, C1 este *superclasă*.

Noua clasă, C2, va conține datele membru ale clasei C1 (în exemplu **x** și **y**) dar și o nouă dată membru (**z**, de tip **int**). De asemenea, clasa C2 conține și metoda **afis()**, care are rolul de a afișa conținutul datelor membru ale unui obiect. În exemplu, clasa C2 este *subclasă*. Programul creează un obiect al clasei C2, numit **ref**, îi inițializează datele membru și le afișează.

2. Pentru a marca faptul că o clasă extinde o altă clasă se folosește cuvântul **extends** (în exemplu **class C2 extends C1**).

3. Inițializarea unui obiect al subclasei are ca efect apelul, în această ordine, al constructorului *superclasei* și apelul constructorului *subclasei*. Este normal să fie așa, pentru că subclasa include datele membru și metodele superclasei.

Exemplu: analizați programul de mai jos, în care fiecare clasă conține câte un constructor. Programul afișează:

```
Constructor 1
Constructor 2
```

```

class Unu{
    Unu () { System.out.println("Constructor 1"); }
}

class Doi extends Unu
{ Doi() { System.out.println("Constructor 2"); };
}

public class C {
    public static void main(String[] args) {
        Doi x=new Doi();
    }
}

```

4. Există posibilitatea ca anumite date membru sau metode să aibă același nume atât în *superclasă* cât și în *subclasă*. În astfel de cazuri, *implicit*, la întâlnirea numelui datei membru (sau metodei) *se apelează data membru (metoda) a subclasei*. În cazul în care se dorește ca să fie apelată data membru (metoda) a *superclasei*, se utilizează cuvântul cheie **super**. *El are semnificația de obiect curent al superclasei*. În exemplul de mai jos, **x** este dată membru atât a superclasei cât și a subclasei. **Exemplu:** programul următor va afișa 1 2 3.

```

class C1
{ int x=1,y; }

class C2 extends C1
{ int x;
  void afis()
  { System.out.println(super.x+" "+ y+" "+x); }
}

class C{
  public static void main(String[] args) {
    C2 pt= new C2();
    pt.x=3;
    pt.y=2;
    pt.afis();
  }
}

```

➡ **super** nu poate fi folosit în afara clasei.

5. În cazul în care superclasa este înzestrată cu un constructor care primește parametri, atunci este obligatoriu ca acesta să fie apelat de constructorul subclasei. De ce? Așa cum am arătat, constructorul subclasei apelează automat constructorul superclasei. În cazul în care apelul nu este explicit, este apelat constructorul implicit (fără parametri) al superclasei. Or, am învățat faptul că, în condițiile existenței unui constructor cu parametri, nu mai este posibil să apelăm constructorul implicit, pentru că apare eroare. Mai mult, *prima instrucțiune a constructorului subclasei trebuie să fie apelul constructorului superclasei*. Apelul constructorului superclasei se face prin:

super (lista_parametri).

Exemplu: Analizați programul următor, în care clasa **Elev** conține numele și prenumele unui elev, iar clasa **Olimpic** extinde clasa **Elev** și conține în plus o dată membru, materia la care elevul este olimpic și o metodă prin care se afișează numele, prenumele elevului și materia respectivă.

```
class Elev
{ String nume, prenume;
  Elev (String nume, String prenume)
  { this.nume=new String(nume);
    this.prenume=new String(prenume); }
}

class Olimpic extends Elev
{ String materia;
  Olimpic(String nume, String prenume, String materia)
  { super(nume,prenume); // apel constructor al superclasei
    this.materia=new String(materia);}
  void afis()
  { System.out.println(nume+" "+prenume+" "+materia); }
}

public class test {
  public static void main(String[] args) {
    //primul mod de creare a unui obiect olimpic
    Olimpic A=new Olimpic("Pascu", "Marius", "informatica");
    //al doilea mod de creare a unui obiect olimpic
    String n=new String("Grosu");
    String p=new String("Bogdan");
    String mat=new String("matematica");
    Olimpic B=new Olimpic(n,p,mat); A.afis(); B.afis();
  }
}
```

6. În Java, o clasă poate extinde (mosteni) o singură superclasă. Nu există un mecanism de mostenire multiplă, așa precum există în alte limbaje. Din acest motiv, *ierarhia claselor are o structură arborescentă*.

6.2. Un exemplu de extindere al unei clase

Fie clasa **Complex**, de mai jos. Ea are două date membru, **x** și **y**, care rețin partea reală și partea imaginară a unui număr complex. De asemenea, clasa este înzestrată cu un constructor și metoda **afis()**, care afișează numărul complex reținut de obiect.

```
class Complex
{ double x,y;
  Complex (double x, double y)
  { this.x=x;
    this.y=y;}
  void afis()
  {System.out.println(x+" "+y);}
}
```

Clasa este extinsă la o alta, **Complex_op**, care conține și metode prin care se efectuează operațiile de mai jos cu numere complexe:

$$z_1 + z_2 = x_1 + i \cdot y_1 + x_2 + i \cdot y_2 = x_1 + x_2 + i \cdot (y_1 + y_2).$$

$$z_1 - z_2 = x_1 + i \cdot y_1 - x_2 - i \cdot y_2 = x_1 - x_2 + i \cdot (y_1 - y_2).$$

$$z_1 \cdot z_2 = (x_1 + i \cdot y_1)(x_2 + i \cdot y_2) = x_1 \cdot x_2 - y_1 y_2 + i \cdot (x_1 y_2 + x_2 y_1).$$

$$\frac{z_1}{z_2} = \frac{x_1 + i \cdot y_1}{x_2 + i \cdot y_2} = \frac{(x_1 + i \cdot y_1) \cdot (x_2 - i \cdot y_2)}{(x_2 + i \cdot y_2) \cdot (x_2 - i \cdot y_2)} = \frac{x_1 x_2 + y_1 y_2 + i(x_2 y_1 - x_1 y_2)}{x_2^2 + y_2^2}$$

Iată ce conține, în plus, clasa **Complex_op**:

- ⇒ Doi constructori. Primul dintre ei acceptă partea reală și partea imaginară a unui număr complex. Al doilea, acceptă numai partea reală (număr real inclus, evident, în mulțimea numerelor complexe).
- ⇒ Metodele **add**, **sub**, **mul**, **div** - efectuează operațiile de adunare, respectiv scădere, înmulțire, împărțire a două numere complexe. Ele returnează numărul complex rezultat în urma operației.

```
class Complex_op extends Complex
{
    Complex_op (double x, double y)
    { super(x,y); }

    Complex_op (double x)
    { super(x,0); }

    Complex_op add(Complex_op z)
    { Complex_op t=new Complex_op (x+z.x,y+z.y);
      return t; }

    Complex_op sub(Complex_op z)
    { Complex_op t=new Complex_op (x-z.x,y-z.y);
      return t; }

    Complex_op mul(Complex_op z)
    { Complex_op t=new Complex_op (x*z.x-y*z.y, x*z.y+z.x*y);
      return t; }

    Complex_op div(Complex_op z)
    { double num=z.x*z.x+z.y*z.y;
      Complex_op t;
      if (num!=0)
          t=new Complex_op ((x*z.x+y*z.y)/num, (z.x*y-x*z.y)/num);
      else t=new Complex_op(0,0);
      return t;
    }
}
```

Exerciții rezolvate

1. Să se creeze două obiecte de tip `Complex_op` și să se afișeze suma, diferența, produsul și câtul lor.

```
public class test {
    public static void main(String[] args) {
        // instantiez z si il afisez
        Complex_op z=new Complex_op(2,3);
        z.afis();
        // instantiez z1 si il afisez
        Complex_op z1= new Complex_op(10);
        z1.afis();
        // Afisez suma dintre z si z1
        z.add(z1).afis();
        // Afisez diferenta dintre z si z1
        z.sub(z1).afis();
        // Afisez produsul dintre z si z1
        z.mul(z1).afis();
        // Afisez catul dintre z si z1
        z.div(z1).afis();
    }
}
```

2. Se citesc a și b două numere reale care reprezintă partea reală și partea complexă a numărului $z=a+b*i$ și n , un număr natural mai mare sau egal cu 1. Se cere să se afișeze z^n .

```
public class test {
    public static void main(String[] args) {
        System.out.print("a=");
        double a=Double.parseDouble(cin.Token());
        System.out.print("b=");
        double b=Double.parseDouble(cin.Token());
        Complex_op z= new Complex_op(a,b);
        z.afis();
        // initializez produsul cu 1;
        Complex_op P= new Complex_op(1);
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
        for (int i=0;i<n;i++) P=P.mul(z);
        P.afis();
    }
}
```

3. Să se extindă clasa `Complex_op`, la o altă, numită `Cmpl`, care să conțină, în plus, o metodă prin care returnează modulul numărului complex și o altă metodă booleană care returnează `true`, în cazul în care numărul este real (y are valoarea 0).

Rezolvare. Întrucât constructorii nu se mostenesc, a fost necesară rescrierea lor:

```

class Cmpl extends Complex_op
{
    Cmpl (double x, double y)
    { super(x,y); }
    Cmpl (double x)
    { super(x,0); }
    double modul()
    { return Math.sqrt(x*x+y*y); }
    boolean real()
    { return y==0; }
}

```

4. Se citește un vector de numere complexe. Se cere să se sorteze crescător elementele vectorului în funcție de modulul lor.

Rezolvare: datorită faptului că avem nevoie de modulul numerelor complexe, vom apela la ultima clasă creată: `Cmpl`. Observați ierarhia acestor clase: la bază este `Complex`, apoi `Complex_op`, iar ultima este `Cmpl`. Programul utilizează și metoda `Afis()`, care aparține clasei `complex`.

```

public class test {
    public static void main(String[] args) {
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
        Cmpl[] V=new Cmpl[n];
        for (int i=0;i<n;i++)
        {
            System.out.print("V["+i+"].x=");
            double x=Double.parseDouble(cin.Token());
            System.out.print ("V["+i+"].y=");
            double y=Double.parseDouble(cin.Token());
            V[i]=new Cmpl(x,y);
        }
        boolean gasit;
        Cmpl man;
        do
        {
            gasit=false;
            for (int i=0;i<n-1;i++)
                if (V[i].modul()>V[i+1].modul())
                {
                    man=V[i];
                    V[i]=V[i+1];
                    V[i+1]=man;
                    gasit=true;
                }
        } while (gasit);
        for (int i=0;i<n;i++) V[i].afis();
    }
}

```

6.3. O modalitate de simulare a extinderii multiple

Asa cum deja știți, în Java o clasă poate extinde (mosteni) o singură clasă. Se poate totuși, simula moștenirea multiplă. Priviți exemplul următor, în care clasa **Mosten** extinde două clase **A** și **B**. Ea va avea datele membru și metodele celor două clase. Ideea e foarte simplă:

1. Clasa va reține referințe către obiecte ale claselor pe care le extinde;
2. Pornind de la aceste referințe se pot accesa datele membru și metodele claselor care sunt extinse, după cum se poate observa în exemplu:

```
class A {
    int a=1;
    void m1() { System.out.println("a="+a);}
}

class B {
    int b=2;
    void m2() { System.out.println("b="+b);}
}

class Mosten {
    A x=new A();
    void m1() { x.m1(); }
    B y=new B();
    public void m2() { y.m2(); }
}

public class test {
    public static void main(String[] args) {
        Mosten t=new Mosten();
        t.m1();
        System.out.println(t.x.a);
        t.m2();
        System.out.println(t.y.b);
    }
}
```

🔍 Observații:

1. În fapt, se instanează 3 obiecte: primul este al clasei **A**, al doilea este al clasei **B** și al treilea este al clasei **Mosten**. Accesul la datele membru și metodele obiectelor de tip **A** și **B** se face prin referințele către acestea, referințe continute de obiectul de tip **Mosten**.
2. Accesul la datele membru ale claselor extinse se face puțin mai greu (vedeți exemplul de mai sus). Cu toate acestea, problema nu este atât de gravă. În practica OOP, se obișnuiește ca datele membru să fie accesate numai prin metodele clasei respective. Dezavantajul acestui procedeu este dat de faptul că, în cazul unor clase cu zeci de metode care urmează a fi "moștenite", trebuie avută multă răbdare pentru a implementa, ca mai sus, toate metodele.

6.4. Referințe către obiectele superclaselor și referințe către obiectele subclaselor

Fie clasele de mai jos, unde clasa **Y**, extinde clasa **X**:

```
class X
{ int k=1;
  void met() {System.out.println("Metoda x"); }
}

class Y extends X
{ int v=2;
  void metoda() {System.out.println("Metoda y"); }
}
```

1. O variabilă de tip referință către un obiect al superclasei poate reține o referință către un obiect al subclasei. În acest caz, pornind de la referință, se pot accesa *numai* datele membru și metodele superclasei, care sunt, evident, și ale subclasei.

Exemplu: programul următor afișează 1 și **Metoda x**:

```
public class test {
  public static void main(String[] args) {
    X ref=new Y();
    System.out.println(ref.k);
    ref.met(); }
}
```

În aceste condiții, fiecare dintre instrucțiunile următoare ar produce eroare:

```
System.out.println(ref.v);
ref.metoda();
```

2. Totuși, există posibilitatea ca, în cazul în care o variabilă de tip referință către superclasă conține o referință către un obiect al subclasei, pornind de la ea, cu ajutorul operatorului de conversie explicită, să putem accesa datele membru și metodele acelui obiect. Vedeti exemplul de mai jos, unde se afișează 2 și **Metoda y**:

```
public class test {
  public static void main(String[] args) {
    X ref=new Y();
    Y ref1=(Y)ref;
    System.out.println(ref1.v);
    ref1.metoda();
  }
}
```




Observație foarte importantă: în cazul în care o metodă, nu și o dată membru, a superclasei este redefinită în subclasă, adică a fost scrisă o alta cu același nume, atunci este posibil ca prin procedeul de mai sus, fără a utiliza operatorul de conversie explicită, să apelăm metoda subclasei. Acest caz este tratat separat. Vedeți *polimorfismul*.

3. O variabilă de tip referință către un obiect al subclasei, **nu** poate reține o referință către superclasă.

Exemplu: dacă în program am avea instrucțiunea: `Y ref=new X();`, ea ar produce eroare la compilare.

Vă puteți întreba, la ce folosește faptul că o variabilă de tip referință către un obiect al superclasei poate reține și o referință către un obiect al subclasei? Ideea de bază este următoarea: se poate crea o clasă, care poate fi extinsă de mai multe clase. Aceasta înseamnă că o structură oarecare (vector, matrice, stivă, coadă, listă liniară, etc) cu elemente de tip referință către obiectele superclasei, poate lucra cu referințe către obiectele subclaselor. *Sau, altfel spus, prin acest procedeu se poate asigura independența structurii față de datele propriu-zise.* Ideea este fundamentală în OOP.

Exemplu: dorim să creăm o structură de tip stivă care să lucreze cu orice fel de date, dar structura să rămână aceeași. În acest exemplu, nodurile conțin fie numere întregi, fie numere reale.

Rezolvare: Pentru a putea implementa această cerință vom considera o clasă, s-o numim **Baza**, care are o singură dată membru: o variabilă care poate reține referințe către propriile obiecte. Clasa **Int** extinde clasa **Baza**. Ea are și data membru **x**, care reține un întreg, precum și un constructor. Clasa **Real** extinde și ea clasa **Baza**. Ea are și data membru **x**, care reține un număr real, precum și un constructor.

```
class Baza
{ Baza adr_prec; }

class Int extends Baza
{ int x;
  Int(int x)
  { this.x=x; }
}

class Real extends Baza
{ double x;
  Real(double x)
  { this.x=x; }
}
```

🕒 Observații:

1. Evident că obiectele clasei **Int**, precum și obiectele clasei **Real** au și data membru **adr_prec**, care este moștenită.

2. Practic, prin extinderea clasei **Baza** putem obține orice alte clase ale căror obiecte să poată fi memorate de stivă.

Clasa **Stiva** conține:

- variabila **varf** - care poate reține o referință către un obiect al clasei **Baza**. Rolul ei este de a memora adresa ultimului nod introdus în stivă.
- metoda **void push(Baza Nod)** - introduce un nod în stivă;
- metoda **Baza pop()** - are rolul de a extrage un nod din stivă și de a-l returna.

```
class Stiva
{
    Baza varf;
    void push(Baza Nod)
    {
        Nod.adr_prec=varf;
        varf=Nod;
    }
    Baza pop()
    {
        if (varf!=null)
        {
            Baza x=varf;
            varf=varf.adr_prec;
            return x;
        }
        else return null;
    }
}
```

În programul următor se introduc în stivă 3 numere întregi, după care se extrag și se afișează:

```
class test {
    public static void main(String[] args)
    {
        Stiva st=new Stiva();
        st.push(new Int(3));
        st.push(new Int(5));
        st.push(new Int(6));
        Int N;
        while ((N=(Int)st.pop())!=null)    System.out.println(N.x);
    }
}
```

În programul următor se introduc în stivă 3 numere reale, după care se extrag și se afișează:

```
class test {
    public static void main(String[] args)
    {
        Stiva st=new Stiva();
        st.push(new Real(3.2));
        st.push(new Real(5.5));
        st.push(new Real(6.3));
        Real N;
        while ((N=(Real)st.pop())!=null)    System.out.println(N.x);
    }
}
```


7. Scrieți o secvență în `main()`, care afișează valorile reținute de data membru **A** a clasei **Unu** și data membru **A** a clasei **Doi**. Cerință suplimentară: se va utiliza o singură dată operatorul `new`.

8. Fiind date clasele de mai jos, care este corpul corect al constructorului clasei **Doi**?

```
class Unu
{ int A;
  Unu (int A)
  {this.A=A; }
}
```

```
class Doi extends Unu
{ int B,C;
  Doi (int A, int B, int C)
  { ... }
  void afis() { System.out.println(A+" "+B+" "+C); }
}
```

a)	b)	c)	d)
<code>this.B=B;</code>	<code>super(A);</code>	<code>this.A=A;</code>	Nici unul
<code>this.C=C;</code>	<code>this.B=B;</code>	<code>this.B=B;</code>	nu este
<code>super(A);</code>	<code>this.C=C;</code>	<code>this.C=C;</code>	corect

9. Fiind dată clasa de mai jos (**Baza**), scrieți o clasă (**Lista**) care creează o listă liniară simplu înlănțuită în care elementele sunt obiectele clasei **Baza**. Evident, elementele acestei liste nu au nici o altă informație, în afara celei de adresă.

```
class Baza { Baza adr_urm; }
```

Lista va conține:

- **Baza v;** - data membru; reține vârful listei;
- **Baza varf()** - returnează vârful listei liniare (adresa primului element);
- **void adaug (Baza Nod)** - adaugă la sfârșitul listei un element.

10. Creați clasa **Int**, care să extindă clasa **Baza** și să conțină în plus:

- **int val;** - dată membru;
- **Int (int val)** - constructor;
- **int valoare()** - returnează valoarea reținută de **val**.

11. Scrieți un program care creează o listă liniară simplu înlănțuită cu **n** elemente (**n** citit de la tastatură) de tip **Int**. Valorile reținute de obiectele de tip **Int** sunt, de asemenea, citite de la tastatură. Lista va fi construită cu ajutorul clasei **Lista**.

12. Scrieti un program care creează o listă liniară simplu înălțuită cu n elemente (n citit de la tastatură) de tip `String`. Șirurile de caractere reținute de obiectele de tip `Sir` sunt, de asemenea, citite de la tastatură. Lista va fi construită cu ajutorul clasei `Lista`.

13. Scrieti un program care creează o listă liniară simplu înălțuită cu n elemente (n citit de la tastatură), obiecte de tip `Persoana`. Clasa `Persoana` va fi construită de dvs. și va conține numele și vârsta persoanei. Lista va fi construită cu ajutorul clasei `Lista`.

Indicații / răspunsuri:

1.

```
class Cit extends cin
{
    static int Intreg()
    { return Integer.parseInt(Token()); }
    static double Real()
    { return Double.parseDouble(Token()); }
    static char Caracter()
    { return linie().charAt(0); }
    static String Sir()
    { return linie(); }
}
```
2.

```
class t {
    public static void main(String[] args) {
        System.out.print("n="); int n=Cit.Intreg();
        System.out.println(n);
        System.out.print("x="); double x=Cit.Real();
        System.out.println(x);
        System.out.print("ch="); char ch=Cit.Caracter();
        System.out.print(ch);
    }
}
```
3. b) 4. a) 5. c) 6. c)
7.

```
Doi x=new Doi();
Unu y=x;
System.out.println(y.A+" "+x.A);
```
8. b)
9.

```
class Lista
{
    Baza v;
    Baza varf()
    { return v; }
    void adaug (Baza Nod)
    {
        Baza c=v;
        if (c!=null)
        { while (c.adr_urm!=null) c=c.adr_urm;
          c.adr_urm=Nod; }
        else v=Nod;
    }
}
```

```

10. class Int extends Baza
    { int val;
      int (int val)
      { this.val=val; }
      int valoare()
      { return val; }
    }

11. public class C {
    public static void main(String[] args) {
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
        Lista l= new Lista();
        for (int i=0;i<n;i++)
            { System.out.print("val=");
              int val=Integer.parseInt(cin.Token());
              Int Nod=new Int(val);
              l.adaug(Nod); }
        Baza varf=l.varf();
        Int Nod=(Int)varf;
        while (Nod!=null)
            { System.out.println(Nod.valoare());
              Nod=(Int)Nod.adr_urm; }
    }
}

```

12. Rezolvăm problema în doi pași:

a) Construim clasa Sir, care extinde clasa Baza:

```

class Sir extends Baza
{ String sir;
  Sir (String sir)
  { this.sir=sir; }
  String valoare()
  { return sir; }
}

```

b) Scriem programul propriu-zis:

```

public class C {
    public static void main(String[] args) {
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
        Lista l= new Lista();
        for (int i=0;i<n;i++)
            { System.out.print("val=");
              String val=cin.linie();
              Sir Nod=new Sir(val);
              l.adaug(Nod); }
        Baza varf=l.varf();
        Sir Nod=(Sir)varf;
        while (Nod!=null)
            { System.out.println(Nod.valoare());
              Nod=(Sir)Nod.adr_urm; }
    }
}

```

13.

```

class Persoana extends Baza
{
    String nume;
    int varsta;

    Persoana ()
    {
        System.out.println("Nume=");
        nume=cin.linie();
        System.out.println("varsta=");
        varsta=Integer.parseInt(cin.Token());
    }

    String nume()
    { return nume; }

    int varsta()
    { return varsta; }
}

public class C {
    public static void main(String[] args) {
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
        Lista l= new Lista();
        for (int i=0;i<n;i++)
        { Persoana Nod=new Persoana();
          l.adauga(Nod);
        }
        Baza varf=l.varf();
        Persoana Nod=(Persoana)varf;
        while (Nod!=null)
        { System.out.println(Nod.nume() +" "+Nod.varsta());
          Nod=(Persoana)Nod.adr_urm;
        }
    }
}

```

CAPITOLUL 7

Noțiuni avansate de programare orientată pe obiecte



Din cuprins:

Polimorfism. Exemple
Clase abstracte. Exemple
Interfețe. Aplicații
Modificatori. Exemple
Probleme propuse

7.1. Polimorfism

Prin *polimorfism* vom înțelege posibilitatea ca atât superclasa, cât și subclasa să aibă metode cu același nume (metodele pot fi redefinite).

Deși, în aparență, înțelegerea *polimorfismului* nu este dificilă, în realitate, apar anumite probleme pe care ne propunem să le rezolvăm în acest paragraf. Mai mult, *polimorfismul* reprezintă unul din punctele cheie ale programării orientate pe obiecte și este un instrument puternic de programare.

Fie **B** o clasă care extinde o clasă **A**. Prin urmare, toate datele membru și metodele clasei **A** se regăsesc și în clasa **B**. Am mai învățat faptul că o metodă a clasei **A** poate fi redefinită, adică, în clasa **B** scriem o metodă cu același nume (în exemplu, `void tip()`). Vom trata două cazuri, cazul în care *metodele sunt nestatice* și cazul în care *metodele sunt statice*.

l) Metodele și datele membru sunt *nestatice*.

Fie **O** un obiect al clasei **A** și **O1** un obiect al clasei **B**. Vedeți exemplul următor, al cărui rezultat îl puteți observa alăturat.

```
Cazul 1
clasa A
1
Cazul 2
clasa B
2
Cazul 3
clasa B
1
```

```
class A
{ int i=1;
  void tip() { System.out.println("clasa A"); }
}

class B extends A
{ void tip() { System.out.println("clasa B"); }
}

public class test {
  public static void main(String[] args) {
    // Cazul 1
    A O=new A(); O.tip();
    // Cazul 2
    B O1=new B(); O1.tip();
    // Cazul 3
    A O2=new B(); O2.tip();
  }
}
```

- În cazul 1 se apelează metoda `tip()` a clasei **A**. Se va afișa **clasa A** și valoarea 1 (pentru `i`). Este normal să fie așa, obiectul este al clasei **A**, referința este către un obiect al clasei **A**.
- În cazul 2 se apelează metoda `tip()` a clasei **B**. Se va afișa **clasa B** și valoarea 2 (pentru `i`). Este normal să fie așa, obiectul este al clasei **B**, referința este către un obiect al clasei **B**.

- În cazul 3, și acesta este cazul interesant pe care-l studiem în acest moment, se va apela metoda `tip()` a clasei `B`, prin urmare se va afișa `clasa B` și valoarea `1` (pentru `i`). Să observăm că *obiectul este al clasei B, dar variabila care reține referința către el este o referință către obiecte ale clasei A*. Cu toate că a fost apelată metoda clasei `B`, a fost afișat conținutul variabilei `i` a clasei `A`.



Foarte important: în cazul unei metode a superclasei, redefinită în subclasă, două elemente prezintă importanță: *tipul variabilei referință* (către obiecte ale superclasei sau către obiecte ale subclasei) și *tipul obiectului creat* (al superclasei sau al subclasei). Se disting, astfel două cazuri:

- A) *tipul referinței* și *tipul obiectului creat* coincid (sunt fie ale superclasei, fie ale subclasei). În acest caz este apelată metoda care este din tipul comun. De asemenea, se afișează variabila care aparține obiectului de tip comun. Vezi, mai sus, cazurile 1 și 2.
- B) *tipul referinței* este al superclasei, iar *tipul obiectului creat* este al subclasei (invers nu este posibil, așa cum am arătat). În acest caz, *dacă metoda redefinită este nestatică*, Java ia în considerare *tipul obiectului creat* și deci, va fi apelată metoda subclasei. Vezi, mai sus, cazul 3. În schimb, se obține accesul la data membru a superclasei.
- II) Metodele și datele membru sunt *statice*. Reluăm problema pentru acest caz. Vedeti programul de mai jos, iar rezultatul afișat de el, alăturat. În cazurile 1 și 2 comportamentul este același, dar în cazul 3 *este apelată metoda superclasei*, deci se afișează `clasa A`. Observați faptul că, în acest caz, comportamentul este diferit.

```
Cazul 1
clasa A
1
Cazul 2
clasa B
2
Cazul 3
clasa A
1
```

```
class A
{ static int i=1;
  static void tip() { System.out.println("clasa A"); }
}

class B extends A
{ static int i=2;
  static void tip() { System.out.println("clasa B"); }
}

public class test {
  public static void main(String[] args)
  { // Cazul 1
    System.out.println("Cazul 1");
    A O=new A(); O.tip();
    System.out.println(O.i);
    // Cazul 2
    System.out.println("Cazul 2");
    B Ol=new B(); Ol.tip();
    System.out.println(Ol.i);
  }
}
```

```
// Cazul 3
System.out.println("Cazul 3");
A O2=new B(); O2.tip();
System.out.println(O2.i);
}
}
```

În terminologia programării orientate pe obiecte veți întâlni termenul de **legare** (statică, dinamică). Care este semnificația lui? Pornind de la referința către obiect, se apelează o metodă redefinită. Am văzut că, în unele cazuri se apelează metoda superclasei, în altele, se apelează metoda subclasei. Cu alte cuvinte, se apelează (*leagă*) o metodă sau alta. Aceasta este semnificația termenului de **legare** (adică apel al unei metode sau alta).

⇒ Legarea este **statică** dacă înainte de executarea programului se poate ști care metodă este apelată și este **dinamică** dacă numai în momentul executării se decide metoda care va fi apelată. În **Java** este permisă și legarea dinamică.

Priviți exemplul de mai jos, în care **legarea dinamică** este evidentă. Dacă se introduce 0 se afișează **clasa A**, iar dacă se introduce o valoare diferită de 0 se afișează **clasa B**.

```
class A
{ int i=1;
  void tip() { System.out.println("clasa A"); }
}

class B extends A
{ void tip() { System.out.println("clasa B"); }
}

public class Clasal {
  public static void main(String[] args) {
    A x;
    int n=Integer.parseInt(cin.Token());
    if (n==0) x=new A();
        else x=new B();
    x.tip();
  }
}
```

7.2. Un exemplu de polimorfism

Să se creeze o clasă numită **Muncitor**. Clasa va conține data membru **ore_lucrate** și un constructor prin care acesteia i se atribuie o valoare. De asemenea, clasa conține o metodă nestatică **salariu()** prin care se calculează suma câștigată de acesta (se obține ca produs între numărul orelor lucrate și suma încasată pentru o oră lucrată, în exemplu 20000 lei). Dintre muncitori, se selectează șeful de echipă, care are în

subordine mai mulți muncitori. Se știe că șeful de echipă primește mai mult pentru o oră lucrată (în exemplu 50.000). Se construiește clasa `Sef echipa`, care este subclasă a clasei `Muncitor`. Metoda `salariu()` este redefinită. Dintre șefii de echipă, unii sunt selectați pentru a urma o școală de maistri. După ce devin maistri și au în subordine mai mulți șefi de echipă. Să se construiască clasa `Maistru`, care are ca superclasă clasa `Sef echipa`. Un maistru primește pe oră mai mult decât un șef de echipă (în exemplu 100000 lei), deci metoda `salariu()` este redefinită. Se cere să se scrie un program care citește un număr natural n între 1 și 3. Dacă n este 1 se construiește un obiect al clasei `muncitor`, dacă n este 2 se construiește un obiect al clasei `sef echipa`, iar dacă n este 3 se construiește un obiect al clasei `Maistru`. De asemenea, se citește numărul de ore lucrat de persoana respectivă. În final, se va afișa salariul încasat de persoana citită.

```
class Muncitor
{ int ore_lucrate;
  Muncitor(int ore_lucrate)
  { this.ore_lucrate=ore_lucrate;}
  double salariu()
  { return ore_lucrate*20000;}
}

class Sef_echipa extends Muncitor
{ Sef_echipa(int ore_lucrate)
  { super(ore_lucrate); }
  double salariu()
  { return ore_lucrate*50000;}
}

class Maistru extends Sef_echipa
{ Maistru(int ore_lucrate)
  { super(ore_lucrate); }
  double salariu()
  { return ore_lucrate * 100000;}
}

public class test {
  public static void main(String[] args) {
    Muncitor x=null;
    int tip=Integer.parseInt(cin.Token());
    int ore=Integer.parseInt(cin.Token());

    switch (tip)
    { case 1: x=new Muncitor(ore); break;
      case 2: x=new Sef_echipa(ore); break;
      case 3: x=new Maistru(ore); break; }
    System.out.println(x.salariu());
  }
}
```

🔔 Observații:

- ✓ Exemplul dat se referă la polimorfism și la legarea dinamică. Metoda `salariu` este prezentă în toate cazurile. Este rulată o metodă sau alta în funcție de valoarea citită, prin urmare avem legare dinamică.
- ✓ Există multe alte metode de rezolvare ale acestei probleme. Judecând după problema în sine, ar rezulta că utilizarea polimorfismului poate fi evitată. Așa este, dar în cazul programelor complexe (de exemplu în programarea vizuală), utilizarea lui se dovedește extrem de utilă.

7.3. Clase abstracte

Limbajul **Java** pune la dispoziția programatorilor un mecanism care se dovedește uneori extrem de eficient și anume clasele abstracte.

O clasă *abstractă* conține date membru (nu în mod obligatoriu) și mai multe metode, dar unele pot fi, la rândul lor abstracte. Pentru o metodă *abstractă*, în clasă se trece doar prin antetul ei. Metodele *abstracte* urmează să fie redefinite în subclase. Orice clasă abstractă trebuie să fie precedată de cuvântul cheie **abstract** și orice metodă abstractă din cadrul ei trebuie și ea să fie precedată de același cuvânt cheie. Clasele abstracte nu se folosesc direct, adică nu se instanciază obiecte ale lor, ci se utilizează doar pentru a fi extinse (sunt prin definiție superclase).

Exemplu. În programul de mai jos clasa **Ex** este abstractă. Ea conține două metode abstracte, `afis1()` și `afis2()`, dar și o metodă `decid()`, care în funcție de un parametru apelează una dintre cele două metode abstracte. Pentru a putea folosi clasa **Ex**, o extindem obținând astfel clasa **Ex1**. În clasa **Ex1**, redefinim cele două metode. Totuși, clasa **Ex1** nu redefineste metoda `decid()`, deci orice obiect al ei va folosi metoda din clasa abstractă. Acest mecanism, permite ca în clasa abstractă să existe un anumit “motor” (metodă) care apelează alte metode care urmează a fi redefinite. Programul de mai jos apelează mai întâi **Afis 1** și apoi **Afis 2**:

```
abstract class Ex
{
    abstract void afis1();
    abstract void afis2();

    void decid(int b)
    {
        if (b>0) afis1();
        else afis2();
    }
}

class Ex1 extends Ex
{
    void afis1()
    {
        System.out.println("Afis 1");
    }
    void afis2()
    {
        System.out.println("Afis 2");
    }
}
```

```

public class test {
    public static void main(String[] args) {
        Ex1 a=new Ex1();
        a.decide(1);
        a.decide(-1);
    }
}

```

7.4. Un exemplu de clasă abstractă

Cei care au studiat backtracking în variantă iterativă, standardizată, au văzut faptul că există o unică metodă care apelează mai multe metode care au același nume, parametri și același rol. A rezolva o problemă sau alta înseamnă a rescrie metodele specifice (`init()`, `succesor()`, s.a.m.d.) dar motorul (metoda `back()`) rămâne aceeași. De aici și **ideea**: o clasă abstractă (`Bkt`) va reține metoda `back()`, care nu este abstractă, iar celelalte metode sunt abstracte. A rezolva o problemă înseamnă a crea o subclasă în care redefinim metodele care o rezolvă (`init()`, `succesor()`, `valid()`,...) conform cerințelor problemei.

Programul de mai jos, utilizează clasa `bkt`, pentru a o extinde în vederea rezolvării problemei particulare de generare a tuturor permutărilor mulțimii $\{1, 2, \dots, n\}$, pentru n citit.

```

abstract class Bkt
{
    int[] st;
    int n;

    abstract void init(int k);
    abstract boolean am_Succesor (int k);
    abstract boolean e_Valid(int k);
    abstract boolean solutie(int k);
    abstract void tipar();

    void back()
    {
        boolean AS;
        int k=1;
        st=new int[n+1];
        init(k);
        while (k>0)
        {
            do { } while ((AS=am_Succesor(k)) && !e_Valid(k));
            if (AS)
            {
                if (solutie(k)) tipar();
                else {k++;init(k);}
            }
            else k--;
        }
    }
}

class Permut extends Bkt {
    Permut (int n)
    {
        this.n=n;
    }

    void init(int k)
    {
        st[k]=0;
    }
}

```

```

boolean am_Sucesor (int k)
{ boolean rasp=false;
  if (st[k]<n) { st[k]++; rasp=true;}
  return rasp;}

boolean e_Valid(int k)
{ boolean rasp=true;
  for (int i=1;i<k;i++)
    if (st[i]==st[k]) rasp=false;
  return rasp; }

boolean solutie(int k)
{ return k==n;}

void tipar()
{ for (int i=1;i<=n;i++) System.out.print(st[i]);
  System.out.println();}
}

public class test {
  public static void main(String[] args) {
    System.out.print("n=");
    int n=Integer.parseInt(cin.Token());
    Permut x=new Permut(n);
    x.back();
  }
}

```

Exercițiu: se cere să se rezolve problema celor n dame. Folosind clasele anterioare și polimorfismul, problema devine foarte ușoară. Obținem clasa Dame care extinde clasa Permut. O singură metodă trebuie redefinită, metoda `boolean e_Valid(int k)`.

```

class Dame extends Permut
{ Dame (int n)
  { super(n); }

  boolean e_Valid(int k)
  { boolean rasp=true;
    for (int i=1;i<k;i++)
      if ((st[i]==st[k]) || (Math.abs(st[k]-st[i])==Math.abs(k-i)) )
        rasp=false;
    return rasp;
  }
}

```

Iar programul este asemănător celui anterior:

```

public class test {
  public static void main(String[] args) {
    System.out.print("n=");
    int n=Integer.parseInt(cin.Token());
    Bkt x=new Dame(n);
    x.back();
  }
}

```

7.5. Interfețe

Prin interfață vom înțelege un **proiect de clasă**. O interfață poate conține doar **constante** și **antete de metode**. Ele sunt acceptate de limbaj și se presupune că, ulterior, interfețele vor fi *implementate* de una sau mai multe clase. O interfață arată ca o clasă, numai că, în locul cuvântului **class**, se folosește cuvântul **interface**. Atunci când scriem o clasă care *implementează* o anumită interfață, se folosește cuvântul cheie **implements**, ca în exemplul de mai jos. O interfață poate fi implementată de una sau mai multe clase.

Exemplu: Mai jos, avem o interfață numită **InterfA**. Interfața este implementată de două clase, **A** și **B**. În **main()** probăm toate acestea prin crearea a două obiecte, unul care instanțiază clasa **A**, altul care instanțiază clasa **B**. Programul afișează:

```
Afis din clasa A=6 5
Afis din clasa B=7 5
```

iar codul este:

```
interface InterfA {
    int i=5;
    void afis();
}

class A implements InterfA{
    int i=6;
    public void afis()
    { System.out.println("Afis din clasa A="+i+" "+InterfA.i);}
}

class B implements InterfA {
    int i=7;
    public void afis()
    { System.out.println("Afis din clasa B="+i+" "+InterfA.i);}
}

public class test {
    public static void main(String[] args) {
        A x=new A(); x.afis(); B y=new B(); y.afis(); }
}
```

Mai observăm că:

- ❖ Constantele declarate în interfață (în exemplu constanta **i**, cu valoarea 5) se apelează prin numele interfeței, urmat de '.' și de numele constantei.
- ❖ Constantele pot fi redefinite de date membru (în exemplu de **i** cu valoarea 6 și **i** cu valoarea 7). Atât constanta cât și redefinirile ei pot fi apelate simultan.

1. O variabilă de tip *referință către interfață* poate memora o referință către orice clasă care implementează respectiva interfață. Mai mult, legarea este dinamică, așa cum rezultă din exemplul următor.

Exemplu: pentru clasele implementate anterior, se poate obține același efect dacă `main()` se scrie astfel:

```
public static void main(String[] args) {
    int tip=Integer.parseInt(cin.Token());
    if (tip==1) { interfA x=new A(); x.afis(); }
        else { interfA y=new B(); y.afis(); }
}
```

2. O interfață poate extinde mai multe interfețe. Să observăm faptul că o clasă poate extinde o singură clasă, dar o interfață poate extinde mai multe interfețe, ca mai jos:

```
interface A
{ void metA();}

interface B
{ void metB();}

interface C
{ void metC();}

interface Extinsa extends A,B,C
{ void MetE();}
```

3. A implementa o interfață extinsă înseamnă a implementa **toate** metodele interfețelor care au stat la baza extinderii, plus metodele a căror antet se găsește în interfața extinsă. În exemplul următor se implementează clasa **Extinsa**:

```
class Ex implements Extinsa {
    public void metA() {System.out.println("metA");}
    public void metB() {System.out.println("metB");}
    public void metC() {System.out.println("metC");}
    public void metE() {System.out.println("metE");}
}

public class test {
    public static void main(String[] args) {
        Ex x=new Ex();
        x.metA(); x.metB(); x.metC(); x.metE(); }
}
```

4. O clasă poate implementa oricâte interfețe, ca în exemplul următor:

```
interface A
{ void metA();}

interface B
{ void metB();}

interface C
{ void metC();}
```

```

class Ex implements A,B,C {
    public void metA() {System.out.println("metA");}
    public void metB() {System.out.println("metB");}
    public void metC() {System.out.println("metC");}
}

public class test {
    public static void main(String[] args) {
        Ex x=new Ex();
        x.metA(); x.metB(); x.metC();}
}

```

5. Cu ajutorul interfețelor se poate *simula* moștenirea multiplă (se știe că în Java nu este permisă moștenirea multiplă). **Exemplu:** Avem 3 clase rezultate în urma implementării a 3 interfețe. Prin procedeul de mai jos, clasa **Mosten** preia metodele celor 3 clase:

```

interface A
{ void metA();}

class Aclass implements A
{ public void metA() {System.out.println("metA");}
}

interface B
{ void metB();}

class Bclass implements B
{ public void metB() {System.out.println("metB");}
}

interface C
{ void metC();}

class Cclass implements C
{ public void metC() {System.out.println("metC");}
}

class Mosten implements A,B,C {
    public void metA() { A x= new Aclass(); x.metA();}
    public void metB() { B x= new Bclass(); x.metB();}
    public void metC() { C x= new Cclass(); x.metC();}
}

public class test {
    public static void main(String[] args) {
        Mosten x=new Mosten();
        x.metA(); x.metB(); x.metC();}
}

```

6. Mai mult, se pot moșteni metode ale claselor care *implementează o aceeași interfață*. Priviți exemplul următor în care clasa **Mosten** utilizează, la un moment dat, o singură clasă dintre cele 3 (A, B, C) care implementează interfața numită **Interf**:

```

interface Interf
{ void met();}

```

```

class A implements Interf
{ public void met() {System.out.println("Metoda din A");}
}

class B implements Interf
{ public void met() {System.out.println("Metoda din B");}
}

class C implements Interf
{ public void met() {System.out.println("Metoda din C");}
}

class Mosten {
    Interf x;
    Mosten (Interf x) { this.x=x;}
    void met() { x.met();}
}

public class test {
    public static void main(String[] args) {
        Mosten v1=new Mosten(new A()); v1.met();
        Mosten v2=new Mosten(new B()); v2.met();
        Mosten v3=new Mosten(new C()); v3.met();
    }
}

```

7.6. Aplicații ale interfețelor

Vom prezenta câteva exemple, pur didactice, care să explice rolul interfețelor. Atunci când veți studia programarea vizuală vă veți convinge de importanța lor.

Exemplul 1. În multe programe se solicită citirea unui număr natural. Aceasta se poate face prin afișarea unui mesaj, apoi urmează citirea propriu-zisă a acestuia. Pentru a permite ca programul să rămână aproape neschimbat atunci când citim un număr natural într-un fel sau altul, metoda care citește se va găsi într-o interfață, care este implementată diferit de mai multe clase. În acest exemplu interfața `CitScriu` conține antetul unei metode de citire a unui număr natural. Două clase, `IO1` și `IO2` implementează această interfață. Clasa `Suma` este înzestrată cu un constructor care are ca parametru o referință către obiectul care efectuează citirea și o metodă, `afis()`, care citește două numere naturale și afișează suma lor. Rulați programul și prin crearea obiectului `Suma`, cu ajutorul instrucțiunii: `Suma s=new Suma (new IO2());`.

```

interface CitScriu {
    int citeste();}

class IO1 implements CitScriu {
    public int citeste()
    { System.out.print("Introduceti valoarea ");
      return Integer.parseInt(cin.Token());
    }
}

```

```

class IO2 implements CitScriu {
    public int citeste()
    { System.out.print("Tastati valoarea dorita ");
      return Integer.parseInt(cin.Token()); }
}

class Suma {
    CitScriu x;
    Suma(CitScriu x)
    { this.x=x; }
    void afis()
    { int n=x.citeste();
      n+=x.citeste();
      System.out.println("Suma este "+n); }
}

public class test {
    public static void main(String[] args) {
        Suma s=new Suma (new IO1());
        s.afis();
    }
}

```

Exemplul 2. Uneori creatorul unei clase care utilizează metoda (metodele) unei interfețe dorește să lase utilizatorilor clasei “plăcerea” de a implementa respectivele metode. Într-un astfel de caz el furnizează fișierul compilat al clasei respective. Iată, pentru exemplul anterior, cum arată clasa **Suma**:

```

abstract class Suma implements CitScriu{
    void afis()
    { int n=citeste();
      n+=citeste();
      System.out.println("Suma este "+n); }
}

```

Cel care utilizează clasa respectivă va scrie un program precum cel de mai jos:

```

class SumaM extends Suma implements CitScriu
{ public int citeste() {
    System.out.print("v=");
    return Integer.parseInt(cin.Token()); }
}

public class test {
    public static void main(String[] args) {
        SumaM s=new SumaM (); s.afis(); }
}

```

Exemplul 3. Prin utilizarea interfețelor se pot transmite metode (funcții) ca parametri. În exemplu, interfața **Funcția** conține antetul unei funcții reale de parametru real, numită **f**. Interfața poate fi implementată de mai multe clase, unde fiecare clasă definește funcția **f**. Clasa **Afis** conține o variabilă de tip referință către interfața **Funcția**, numită **ref**. Această variabilă este utilizată pentru a memora o referință către un obiect al uneia din clasele

care implementează interfața, deci către funcția dorită. Prin urmare, cu ajutorul clasei **Afis** se poate calcula funcția dorită. În exemplu, interfața a fost implementată de două clase, care conțin funcții diferite: **Liniara** și **Patratica**. Clasa **Afis** rămâne nemodificată, indiferent de funcția pe care o evaluează. Compilați interfața **Funcția** și clasele **Liniara**, **Patratica** și **Afis**.

```
interface Functia
{ double f(double x);}

class Liniara implements Functia
{ public double f(double x) {return 3*x+1;}
}

class Patratica implements Functia
{ public double f(double x) {return x*x;}
}

class Afis {
    Functia ref;
    Afis(Functia ref)
    { this.ref=ref;}

    public double funct(double x)
    { return ref.f(2); }
}
```

Să presupunem că un utilizator dorește să afișeze $f(1)$, $f(2)$... $f(10)$ pentru funcția liniară $3x+1$. Atunci el va scrie programul:

```
public class test{
    public static void main(String[] args) {
        Afis a =new Afis(new Liniara());
        for (int i=1;i<=10;i++) System.out.println (a.funct(i));
    }
}
```

Să presupunem că utilizatorul dorește să afișeze $f(1)$, $f(2)$... $f(10)$ pentru funcția x^2 . Atunci el va scrie programul:

```
public class test{
    public static void main(String[] args) {
        Afis a =new Afis(new Patratica());
        for (int i=1;i<=10;i++) System.out.println (a.funct(i));
    }
}
```



Observație: mai general, prin utilizarea interfețelor se transmit de fapt obiecte rezultate din instanțieri ale claselor care au implementat respectiva interfață.

Exemplul 4. Nu este de neglijat faptul că într-o interfață se pot defini multe constante. Urmăriți exemplul următor în care am declarat și “utilizat” constante pentru zilele săptămânii:

```

interface Saptamana {
    int Luni=1,Marti=2, Miercuri=3,Joi=4,
        Vineri=5, Sambata=6, Duminica=7;
}

class Ziua implements Saptamana {
    void ceFac()
    { for (int zi=Luni;zi<=Duminica;zi++)
        if (zi>=Marti && zi<=Joi)
            System.out.println("Ceva munca");
        else
            System.out.println("Sarbatoare");
    }
}

public class test {
    public static void main(String[] args) {
        Ziua x=new Ziua();
        x.ceFac();
    }
}

```

În practică, constantele joacă un rol extrem de important asigurând o citire ușoară și chiar scriere eficientă a programelor. În programarea vizuală veți întâlni o mulțime de constante pentru culori, alinieri de text, etc.

7.7. Modificatori

Forma generală prin care o clasă este definită o puteți observa mai jos. Tot ce este trecut între paranteze drepte este opțional.

```

[<Modificatori clasa>] class <nume clasa> [extends <nume
clasa>] implements [lista interfete]
{ [...] }

```

Observăm că, până acum, nu am prezentat modificatorii de clasă. Aceștia sunt: **public**, **abstract** și **final** și se folosesc în această ordine.

1. Modificatorul **public**. Dacă este trecut acest modificator, atunci clasa respectivă poate fi accesată de orice cod **Java** care accesează pachetul în care se găsește acea clasă. Dacă acest modificator este absent, atunci clasa poate fi accesată doar din pachetul în care se găsește (revedeți crearea pachetelor). Clasele publice trebuie scrise în fișiere care le poartă numele și au extensia **.java**.



Observație foarte importantă. În cazul în care, la crearea clasei, nu se folosește clauza **package** (nu se realizează un pachet), se consideră că respectiva clasă face parte dintr-un pachet fără nume. Pachetul fără nume este alcătuit din totalitatea claselor fără clauza **package** care se găsesc într-un folder. Din acest motiv, cât timp am lucrat numai cu clase aflate în folder-ul **bin**, nu a fost nevoie ca acestea să conțină clauza **package**.

2. **abstract** - marchează o clasă abstractă (revedeți clasele abstracte). În absență, se consideră că respectiva clasă nu este abstractă.

3. Modificatorul **final** este trecut atunci când *nu* se dorește ca respectiva clasă să fie extinsă (derivată, moștenită), caz în care această operație este interzisă.

În rest, am învățat faptul că o clasă poate extinde o singură clasă, dar poate implementa mai multe interfețe (vedeți: **extends**, **implements**).

Pentru definirea datelor membru în Java avem forma generală:

```
[<Modificatori>] <tip data>
<Nume data membru1> [=valoare]
[Nume data membru2] [=valoare]...
```

Modificatorii sunt:

1. **public** - data membru poate fi accesată de oriunde este accesibilă (deci și din afara pachetului) și poate fi moștenită;

2. **protected** - data membru poate fi accesată de codul aflat în același pachet și poate fi moștenită;

3. **private** - data membru poate fi accesată numai din clasa în care se află și nu este moștenită (sau dacă vrei, este moștenită, dar nu poate fi accesată din clasa care o moștenește, ceea ce este același lucru);

4. modificatorul implicit (**package**) - are efect în absența modificatorilor de mai sus. Data poate fi accesată de cod din același pachet și poate fi moștenită numai de clase ale aceluiași pachet;

5. **final** - data membru este considerată constantă, deci nu poate fi modificată;

6. **static** - data membru nu este memorată de fiecare obiect al clasei respective (modificatorul a mai fost prezentat).

Modificatorii **public**, **protected**, **private** și cel implicit (**package**) se numesc *modificatori de acces*. Evident, o definiție de dată membru va conține numai unul din cei trei modificatori sau, în absența lor, se consideră că avem modificatorul implicit.

Pentru declararea unei metode se utilizează forma generală:

```
[<Modificatori>] <Tip metoda> <Nume> (<lista parametri>)
[throws <TipClasa1> [<TipClasa2>] ...] <metoda>
```

Modificatorii sunt prezentați mai jos, iar clauza **throws** va fi studiată în capitolul care tratează excepțiile.

1. Modificatorii de acces - au aceeași semnificație ca la datele membru.

2. **final** - specifică faptul că într-o clasă derivată, metoda nu poate fi redefinită.

3. **static** - are ca efect faptul că metoda nu este reținută de fiecare obiect al clasei respective. Acest modificador a fost prezentat.

Exemple de utilizare a modifcatorilor

1. Modificatorii sunt absenți. Clasa poate fi accesată din interiorul pachetului, poate fi derivată, data membru **i** poate fi moștenită ca și metoda **met()**. De asemenea, data membru și metoda pot fi accesate de date aflate în același pachet.

```
class A
{ int i=2;
  void met() {System.out.println("Test");}
};
```

2. Clasa **test** trebuie să se găsească în fișierul **test.java** (deoarece este publică). Pentru ea nu se pot crea subclase (pentru că are modificadorul **final**). Data membru **i** este statică, adică nu se regăsește în fiecare obiect al clasei respective, iar metoda este nestatică, deci se regăsește în fiecare obiect al clasei respective.

```
public final class test
{ static int i=2;
  void met() {System.out.println("Test");}
};
```

În aceste condiții, definiția clasei de mai jos este eronată, chiar dacă se găsește în același pachet cu clasa **test**.

```
class B extends test {}
```

3. Fie clasele de mai jos:

```
class A
{ private int i=2;
  void met() {System.out.println("Test");}
  void metp() {System.out.println(i);}
};

class B extends A
{ void met1() {System.out.println("Test1");}
}
```

În aceste condiții, dacă în clasa de mai jos, aflată în același pachet, se înlocuiesc cele trei puncte cu instrucțiunile următoare atunci:

```
public class test {
  public static void main(String[] args) {
    A x=new A();
    ... }
}
```


- a) `System.out.println(x.i);` - avem eroare, deoarece `i` are modificatorul `private`, deci nu poate fi accesat din afara clasei;
- b) `x.met();` - se afișează `Test` pentru că metoda nu este privată;
- c) `B y=new B(); y.met1();` - se afișează `Test1`;
- d) `B y=new B(); y.met();` - se afișează `Test`;
- e) `x.metp();` - se afișează 2. Deși `i` este privat, el este afișat de `met()` care are acces la datele private ale propriei clase.

4. Fie clasa de mai jos. Atunci:

```
class A
{ final int i=2;
  final void met() {System.out.println("Test");}
  void metp() {System.out.println(i);}
}
```

- a) Clasa următoare, aflată în același pachet, este eronată pentru că data membru `i` este cu modificatorul `final`, este deci considerată constantă și prin urmare nu `i` se poate modifica valoarea:

```
public class test {
  public static void main(String[] args) {
    A x=new A();
    x.i=4;
  }
}
```

- b) Clasa următoare, aflată în același pachet, este eronată pentru că extinde clasa `A` și redefineste o metodă care conține modificatorul `final`:

```
class B extends A
{ void met() {System.out.println("Test1");}
}
```

- c) Clasa următoare, aflată în același pachet, este corectă pentru că, deși extinde clasa `A`, redefineste o metodă care nu are modificatorul `final`:

```
class B extends A
{ void metp() {System.out.println("Test2");}
}
```

Probleme propuse

Testele de la 1 la 4 se referă la clasele următoare. De fiecare dată, instrucțiunea (sau setul de instrucțiuni) la care se referă întrebarea va fi scrisă în locul liniei lipsă (punctată):

```
class A
{ int x=1;
  void met() {System.out.println("Clasa A");}
}

class B extends A
{ int x=2;
  void met() {System.out.println("Clasa B");}
}

class C extends B
{ int x=3;
  void met() {System.out.println("Clasa C");}
}

public class test {
  public static void main(String[] args) {
    ...
  }
}
```

1. Ce se afișează în urma executării secvenței:

```
A x=new B();x.met();
```

a) Clasa A; b) Clasa B; c) Clasa C; d) EROARE.

2. Ce se tipărește în urma executării secvenței:

```
A x=new B(); System.out.println(x.x);
```

a) 1; b) 2; c) 3; d) EROARE.

3. Ce se afișează în urma executării secvenței:

```
B x=(A)new B(); x.met();
```

a) Clasa A; b) Clasa B; c) Clasa C; d) EROARE.

4. Ce se tipărește în urma executării secvenței:

```
B x=new C(); x.met();
```

a) Clasa A; b) Clasa B; c) Clasa C; d) EROARE.

5. Ce afișează programul de mai jos, dacă n citit este 3?

```
class Unu
{ int a=1;
  void afis()
  {System.out.println(a); }
}

class Doi extends Unu
{ int a=2;
  void afis()
  { System.out.println(a); }
}

class t {
  public static void main(String[] args) {
    Unu x;
    int n=Integer.parseInt(cin.Token());
    if (n==1) x=new Unu();
        else x=new Doi();
    x.afis();
    System.out.println(x.a);
  }
}
```

a) 1 1; b) 2 2; c) 1 2; d) 2 1.

6. Utilizând polimorfismul, scrieti un program care citește un număr natural n și dacă este prim, afișați succesorul său în mulțimea numerelor prime, dacă este impar (fără a fi prim) afișați succesorul său în mulțimea numerelor impare, iar dacă este par, afișați succesorul său în mulțimea numerelor pare.

Exemple:

$n=7$ se afișează 11;

$n=21$ se afișează 23;

$n=8$ se afișează 10.

Testele 7 și 8 se referă la programul următor:

```
interface Testare {
  int k=1; int f();
}

class Test1 implements Testare
{ int k=2;
  public int f() {return Testare.k;}
}

class Test2 implements Testare
{ int k=3;
  public int f() {return k;}
}
```

```
public class test {
    public static void main(String[] args) {
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
        Testare x;
        if (n%2==0) x=new Test1();
            else x=new Test2();
        System.out.println(x.f()+" "+x.k); }
}
```

7. Ce se afișează dacă se introduce valoarea 8?

- a) 1 2; b) 1 1; c) 3 3; d) 2 1.

8. Ce se tipărește dacă se citește valoarea 21?

- a) 1 3; b) 2 1; c) 3 3; d) 3 1.

9. Fiind date interfața de mai jos și 3 implementări ale ei, se cere să se scrie un program care afișează valoarea celor 3 funcții conținute de implementări pentru o valoare citită de la tastatură:

```
interface functie {
    int f(int x);
}

class f1 implements functie
{ public int f(int x) { return x*x-x; }
}

class f2 implements functie
{ public int f(int x) { return 8*x-1; }
}

class f3 implements functie
{ public int f(int x) { return 3*x*x-x+10; }
}
```

10. Pentru k valori citite de la tastatură scrieți un program care afișează valoarea funcției $g(x)$, definită mai jos, unde f_1 , f_2 , f_3 sunt cele de la problema anterioară.

$$g(x) = \max\{f_1(x), f_2(x), f_3(x)\}$$

Răspunsuri:

1. b); 2. a); 3. d); 4. c); 5 d);

```
6. class Numar
{ int nr;
    Numar(int nr) { this.nr=nr;}
    int valoare () { return nr;}
    boolean impar()
    { if (nr%2==1) return true;
      else return false;}
```

```

boolean prim ()
{
    boolean p=true;
    for (int i=2;i<=Math.sqrt(nr);i++)
        if (nr%i==0) { p=false;
                        break;}
    return p;
}

int sucesor() {return nr+1;}
}

class Par extends Numar
{
    Par(int nr)
    { super(nr); }
    int sucesor() { return nr+2;}
}

class Impar extends Numar
{
    Impar(int nr)
    { super(nr); }
    int sucesor() { return nr+2;}
}

class Prim extends Impar
{
    Prim(int nr)
    { super(nr); }
    int sucesor()
    {
        Numar val=new Numar(nr+2);
        while (!val.prim()) val=new Numar(val.sucesor());
        return val.valoare();
    }
}

public class test {
    public static void main(String[] args) {
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
        Numar x=new Numar(n);
        if (x.prim()) x=new Prim(x.valoare());
        else
            if (x.impar()) x=new Impar(x.valoare());
            else x=new Par(x.valoare());
        System.out.println(x.sucesor());
    }
}

```

7. b); 8. d);

```

9. public class test {
    public static void main(String[] args) {
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
        functie f=new f1(); System.out.println(funcut(f ,n));
        f=new f2(); System.out.println(funcut(f ,n));
        f=new f3(); System.out.println(funcut(f ,n));
    }
    public static int funcut(funcutie ref, int x)
    { return ref.f(x); }
}

```

CAPITOLUL 8

Excepții

```
try {.....}  
catch (ER1 ob1) {.....}  
[ catch (ER2 ob2) {.....}  
  catch (ER3 ob3) {.....}  
  ...  
  catch (ERn obn) {.....} ]  
[ finally {.....}]
```



Din cuprins:

Noțiunea de excepție
Mecanismul de tratare a excepțiilor
Metode de tratare a excepțiilor
Exemple de tratare a excepțiilor
Probleme propuse

8.1. Noțiunea de excepție

Prin excepție vom înțelege o situație nedorită în care poate ajunge un program în timpul rulării. Există nenumărate exemple de excepții:

- se dorește citirea unui număr natural, dar din greșeală, utilizatorul introduce un caracter;
- datele introduse pentru diverse câmpuri nu sunt în plaja de valori admisă pentru câmpul respectiv;
- un vector are n componente (indicii sunt de la 0 la $n-1$) și se încearcă, de exemplu, să se atribuie o valoare componente de indice n ;
- se încearcă deschiderea unui fișier inexistent;
- ...

Pentru a rezolva astfel de situații, în Java există un mecanism extrem de bine pus la punct (mecanism de tratare a excepțiilor).

8.2. Mecanismul de tratare a excepțiilor

Pentru a prezenta acest mecanism vom porni de la un exemplu extrem de simplu, atât de simplu încât ar putea fi tratat fără probleme și în absența acestuia.

Exemplu: să se citească și să se afișeze un număr natural $1 \leq n \leq 5$.

Practic, problema se reduce la citirea unui întreg și afișarea lui. Dar, în timpul executării, din greșeală, se poate introduce o valoare numerică care nu este între aceste valori. Cum procedăm, altfel decât în mod clasic?

Priviți exemplul următor, în care, dacă numărul citit este între 1 și 5, acesta este afișat, altfel se afișează mesajul $1 \leq n \leq 5$.

```
class Eroare extends Exception
{ void afis()
  { System.out.println("1<=n<=5"); }
}

class t {
  public static int citesc() throws Eroare
  { int n=Integer.parseInt(cin.Token());
    if (n>=1 && n<=5) return n;
    else throw new Eroare();
  }
}
```

```
public static void main(String[] args) {
    try
    {
        int numar=citesc();
        System.out.println(numar);}
    catch (Eroare er) {er.afis();}
}
```

a) Clasa **Eroare** extinde clasa **Exception** (este obligatoriu să fie precizată această extensie. Ea conține o singură metodă, **void afis()**, care are rolul de a afișa mesajul de eroare.

b) La sfârșitul antetului metodei **int citesc()** se găsește cuvântul cheie **throws**, urmat de numele clasei care tratează eroarea respectivă. Prin aceasta ***se anunță*** că respectiva metodă poate întoarce ***fie un întreg*** (în caz de introducere reușită a datelor, fie ***o referință către un obiect al clasei Eroare*** (în caz de introducere nereușită a datelor). Iată, deci, că în acest caz, ***metodele pot întoarce și date de alt tip decât cel dat de tipul metodei***. Acest aspect este esențial. Instrucțiunea **throw** returnează referința către un obiect al clasei **Eroare**.

c) Utilizarea metodei **citesc()** se face, ***în mod obligatoriu***, cu ajutorul instrucțiunii **try** (încearcă). Secvența unde poate apărea excepția, se găsește în instrucțiunea compusă subordonată lui **try** (în exemplu **int numar=citesc()**). Dacă citirea reușește, programul își încheie execuția, altfel, se verifică dacă obiectul întors de metodă este de tip **Eroare** (vezi clauza **catch**). În caz afirmativ, se execută metoda **afis()** a acestui obiect (**er** este referința către obiect).

În sinteză, putem afirma:

- clauza **throws** are rolul de a specifica faptul că o metodă poate întoarce și referințe către obiectele altor clase (dacă extind clasa **Exception**). Dacă sunt mai multe clase, atunci numele acestora sunt separate prin virgulă.
- instrucțiunea **throw** - returnează referința către unul din obiectele claselor trecute după clauza **throws**.
- instrucțiunea **try** are forma generală (ceea ce este trecut între paranteze drepte este facultativ):

```
try { secvența de instrucțiuni care pot conduce către excepții }
catch (Er1 ob1) { secvența care tratează excepția cu clasa Er1 }
[ catch (Er2 ob2) { secvența care tratează excepția cu clasa Er2 }
  catch (Er3 ob3) { secvența care tratează excepția cu clasa Er2 }
  ...
  catch (Ern obn) { secvența care tratează excepția cu clasa Ern } ]
[ finally { secv. se execută indiferent dacă a fost eroare sau nu} ]
```


Exemplu: extindem, ce-i drept, în mod artificial exemplul anterior. Pentru cele două excepții ($n < 1$ și $n > 5$) vom utiliza două metode diferite aflate în clase diferite (**Eroare1** și **Eroare2**). În acest caz metodele sunt de tip constructor, deci corpurile **catch** sunt vide.

```
class Eroare1 extends Exception
{ Eroare1() { System.out.println("1<=n"); }
}

class Eroare2 extends Exception
{ Eroare2() { System.out.println("n<=5"); }
}

class t {
    public static int citesc() throws Eroare1,Eroare2
    { int n=Integer.parseInt(cin.Token());
      if (n<1) throw new Eroare1();
      else
        if (n>5) throw new Eroare2();
      else return n;
    }

    public static void main(String[] args) {
        try
        { int numar=citesc();
          System.out.println(numar);
        }
        catch (Eroare1 er1) {}
        catch (Eroare2 er2) {}
    }
}
```

1. Una dintre rațiunile existenței unui mecanism de tratare a excepțiilor este dată de faptul că, în anumite cazuri, excepția poate fi tratată în mod corespunzător, după care secvența se poate relua (fără ca programul să-și încheie execuția). Revenim asupra exemplului dat, în care se citesc numere naturale din intervalul $[1, 5]$. Programul următor citește și afișează un vector cu 3 astfel de componente. În momentul în care s-a introdus un număr greșit citirea se reia. Programul își încheie execuția atunci când au fost citite toate cele 3 componente ale vectorului.

```
class Eroare extends Exception
{ void afis()
  { System.out.println("1<=n<=5"); }
}

class e {
    public static int citesc() throws Eroare
    { int n=Integer.parseInt(cin.Token());
      if (n>=1 && n<=5) return n;
      else throw new Eroare();
    }
}
```

```

public static void main(String[] args) {
    int[] V=new int[3];
    for (int i=0;i<3;i++)
        try { V[i]=citesc(); }
        catch (Eroare er) { er.afis(); i--; }
    System.out.println("Am citit");
    for (int i=0;i<3;i++)
        System.out.print(V[i]+" ");
}
}

```

2. Dacă clasa care tratează excepția extinde clasa **Exception**, atunci este obligatoriu ca apelul metodei ce poate returna o excepție să se facă în construcția **try**, ca în exemplul anterior, sau metoda care apelează o astfel de metodă să conțină în antet clauza **throws**.

Exemplul 1: dacă în metoda **main()** de mai sus, am avea **V[]=citesc()**, fără construcția **try**, atunci am primi eroare de compilare.

Exemplul 2. **citesc()** poate fi apelată ca mai jos, unde metoda **main()** conține clauza **throws**. Oricum, în acest caz, dacă numărul nu este în intervalul [1.5] excepția este tratată printr-un mesaj implicit, în care se precizează numele clasei căreia îi aparține obiectul returnat.

```

public static void main(String[] args) throws Eroare{
    System.out.println(citesc());}

```

Clasa **Exception** conține (printre altele):

- un constructor implicit (fără parametru): **Exception()**;
- un constructor cu parametru de tip **String**: **Exception(String mes)**;
- o metodă care returnează mesajul **mes**, primit de constructor: **String getMessage()**.

Exemplu: În aceste condiții, programul din exemplul anterior poate fi scris și ca mai jos (mai simplu):

```

class e {
    public static int citesc() throws Exception
    { int n=Integer.parseInt(cin.Token());
      if (n>=1 && n<=5) return n;
      else throw new Exception("1<=n<=5");
    }
    public static void main(String[] args) {
        int[] V=new int[3];
        for (int i=0;i<3;i++)
            try { V[i]=citesc(); }
            catch (Exception er) {i--; System.out.println(er.getMessage());}
        System.out.println("Am citit");
        for (int i=0;i<3;i++)
            System.out.print(V[i]+" ");
    }
}

```

Uneori, este deranjant faptul că instrucțiunea capabilă să producă o excepție trebuie pusă automat sub **try**, iar clasa căreia îi aparține trebuie să conțină clauza **throws**. Mai mult, chiar în Java există metode care pot produce excepții, dar care pot fi folosite fără **try**. De exemplu, metoda **parseInt** a clasei **Integer**. Priviți programul următor, în care dacă șirul de caractere introdus poate fi convertit către un întreg se afișează numărul, altfel programul se termină cu excepție. Și toate acestea fără **try**...

```
class e {  
    public static void main(String[] args) {  
        int n=Integer.parseInt(cin.linie());  
    }  
}
```

Să analizăm antetul complet al metodei **parseInt**:

```
public static int parseInt(String s)  
    throws NumberFormatException
```

Observăm faptul că, în cazul în care apare o excepție, se apelează la clasa **NumberFormatException**. De aici, putem deduce o primă observație: dacă dorim ca programul nostru să furnizeze un mesaj în caz de excepție, putem proceda ca mai jos, adică să folosim construcția **try**, și în caz că se returnează un obiect al clasei **NumberFormatException**, să afișăm un mesaj de eroare lămuritor.

```
class e {  
    public static void main(String[] args) {  
        System.out.print("n="); int n=1;  
        try  
        { n=Integer.parseInt(cin.Token());}  
        catch (NumberFormatException e) {System.out.println ("Valoare  
nenumERICA");}  
        System.out.println(n);  
    }  
}
```

Mai mult, citirea se poate efectua până când valoarea introdusă este corectă, ca în programul următor:

```
class e {  
    public static void main(String[] args) {  
        int i=1, n=1;  
        while (i==1)  
        { System.out.print("n=");  
          try  
          { n=Integer.parseInt(cin.Token()); i=0;}  
          catch (NumberFormatException e)  
          { System.out.println ("Valoare nenumERICA");}  
        }  
        System.out.println(n);  
    }  
}
```

- ✓ Dacă în secvența subordonată lui **try** se depistează o excepție, se abandonează executarea tuturor instrucțiunilor. În exemplu, aceasta implică faptul că **i** rămâne 1, deci se reia secvența subordonată lui **while**. Dacă nu a apărut excepția, **i** devine 0 și prin urmare, executarea lui **while** este considerată încheiată.
- ✓ Dacă nu l-am fi declarat de la început pe **n** cu valoare inițială, oricare ar fi ea, programul ar fi semnalat eroare la compilare, pentru că se ajunge la afișarea lui. Ori, în Java, nu este permisă afișarea conținutului unei variabile neinițializate.

Totuși, nu se rezolvă problema pusă: cum este posibil ca **parseInt** să poată fi folosită fără construcția **try**?

Obiectele claselor care extind clasa **RuntimeException** sau **Error** pot fi apelate fără ca antetul lor să conțină clauza **throws**, ca mai jos:

```
class Eroare extends RuntimeException
{ void afis()
  { System.out.println("1<=n<=5"); }
}

class e {
  public static int citesc() // fara throws
  { int n=Integer.parseInt(cin.Token());
    if (n>=1 && n<=5) return n;
    else throw new Eroare();
  }
  public static void main(String[] args) {
    ...
  }
}
```

Pot exista următoarele cazuri:

a) Dacă în **main()** avem secvența de mai jos, atunci distingem două cazuri:

```
int numar=citesc();
```

a1) Se introduce un număr între 1 și 5 - programul nu generează excepție și totul este OK!

a2) Se introduce un număr în afara intervalului. Va fi afișată o excepție într-un format implicit și **nu** se va afișa mesajul "1<=n<=5".

b) Dacă în **main()** avem secvența:

```
try
{ int numar=citesc();
  System.out.println(numar);
}
catch (Eroare er) { er.afis(); }
```

b1) Se introduce un număr între 1 și 5 - programul nu generează excepție și totul este OK!

b2) Se introduce un număr în afara intervalului. Va fi afișată excepția cu mesajul "1<=n<=5".

8.3. Metodele care pot returna, în caz de excepții, obiecte ale unor alte clase

Priviți antetul metodei `parseInt` a clasei `Integer`. Din considerente didactice, până acum antetul nu a fost prezentat integral. În cazul în care se întâlnește o excepție, se returnează un obiect al clasei `NumberFormatException`:

```
public static int parseInt(String s)
    throws NumberFormatException
```

Aceasta poate ajuta programatorul să "gestioneze" situația, ca în exemplul de mai jos, unde se citește o variabilă întreagă care este afișată. În cazul în care valoarea introdusă nu este corectă, se reia citirea.

```
class e {
    public static void main(String[] args) {
        int i=0;
        while (i==0)
            try
            { int numar=Integer.parseInt(cin.Token());
              System.out.println(numar);
              i=1;
            }
            catch (NumberFormatException e)
            { System.out.println("Numar eronat"); }
    }
}
```

Iată și antetul complet al metodelor `parseFloat` și `parseDouble`:

1.

```
public static double parseDouble(String s)
    throws NumberFormatException
```
2.

```
public static float parseFloat(String s)
    throws NumberFormatException
```

⇒ În cazul împărțirii la 0 pentru tipuri întregi, se returnează un obiect al clasei `ArithmeticException`. Vedeți exemplul următor:

```
class t {
    public static void main(String[] args) {
        System.out.print("n="); int n=Integer.parseInt(cin.Token());
        System.out.print("m=");
        int m=Integer.parseInt(cin.Token());
        try { System.out.println(n/m); }
        catch (ArithmeticException e)
            { System.out.println ("Impartire la 0 ");}
    }
}
```

În cazul în care se încearcă să se atribuie o valoare unei componente inexistente a unui vector, se returnează pentru tipuri întregi un obiect al clasei `ArrayIndexOutOfBoundsException`. Vedeți exemplul următor:

```
class t {
    public static void main(String[] args) {
        int []v=new int[3];
        System.out.print("indice=");
        int indice=Integer.parseInt(cin.Token());
        try { v[indice]=1;}
        catch (ArrayIndexOutOfBoundsException e)
            { System.out.println ("Eroare indice");}
    }
}
```

📌 Observație foarte importantă

Clasele `ArrayIndexOutOfBoundsException`, `ArithmeticException` ca și multe altele de acest tip, sunt rezultate în urma unor derivări ale clasei `Exception`. Cum un parametru de tip referință către un obiect al superclasei poate reține referințe către obiectele subclaselor sale, atunci putem scrie clauza `catch` de forma:

```
catch (Exception e) {...}
```

8.4. Exemple de tratare a excepțiilor

1. Validare pentru numere reale. Să se scrie un program care citește `n`, număr natural și `n` numere reale. Se cere să se calculeze suma numerelor citite. Dacă un număr real se introduce greșit (nu respectă formatul de număr real) se reia citirea lui.

Rezolvare. Metoda `parseDouble`, a clasei `Double`, returnează fie un număr de format `double`, fie un obiect al clasei `NumberFormatException`.

```
class e {
    public static void main(String[] args) {
        double s=0;
        System.out.print("n=");
        int n=Integer.parseInt(cin.Token());
```

```

for (int i=0;i<n;i++)
    try { double x=Double.parseDouble(cin.Token());s+=x;}
    catch (NumberFormatException e)
        { System.out.println("Numar eronat");i--;}
System.out.println("S="+s);
}
}

```

2. Validare alfabetică. Se citește un șir de caractere alcătuit din mai multe cuvinte separate prin blank-uri. Dacă șirul citit conține numai literele alfabetului (mici sau mari) sau blank-uri, acesta se va afișa, altfel se va tipări mesajul `sir nealfabetic`.

Rezolvare. Metoda `citSir()` citește șirul și îl validează. Dacă șirul este valid, el este returnat, altfel se returnează un obiect al clasei `Nealfabetic`. În `main()` se testează dacă a fost returnat șirul (`try`), caz în care acesta este afișat, sau a fost returnat un obiect al clasei `Nealfabetic`, caz în care se afișează mesajul de eroare `sir nealfabetic` prin utilizarea metodei `mesaj()` a obiectului.

```

class Nealfabetic extends RuntimeException {
    String mesaj()
    { return "sir nealfabetic"; }
}

class e {
    static String citSir()
    { String sir=new String(cin.linie());
      boolean gasit=false;
      for (int i=0;i<sir.length();i++)
          if ( (sir.charAt(i)<'A' || sir.charAt(i)>'Z') &&
              (sir.charAt(i)<'a' || sir.charAt(i)>'z') &&
              sir.charAt(i)!=' ') gasit=true;
      if (gasit) throw new Nealfabetic();
      else return sir;
    }

    public static void main(String[] args) {
        boolean corect=false;
        System.out.print("Tastati sirul ");
        String s="";
        try { s=citSir(); System.out.println(s); }
        catch (Nealfabetic e) { System.out.println(e.mesaj()); }
    }
}

```

Probleme propuse

1. Se citesc n , număr natural și n numere reale. Se cere să se afișeze suma numerelor citite. Pentru citirea numerelor, programul va utiliza mecanismul excepțiilor (în caz de introducere greșită a datelor - care nu corespund formatului de număr real, se va afișa un mesaj corespunzător și se va relua citirea numerelor).

2. La fel ca mai sus numai că cele **n** numere citite trebuie să fie mai mici decât 10. De asemenea, se va folosi mecanismul excepțiilor atât pentru formatul real, cât și pentru valorile admise.

3. Scrieți un program care citește două numere reale **a** și **b** și afișează rezultatul împărțirii lui **a** la **b**. Dacă ambele numere sunt 0, se va afișa un mesaj lămuritor, dacă numai **b** este 0, se va tipări alt mesaj lămuritor, iar dacă împărțirea se poate face, se va afișa rezultatul împărțirii. Evident, programul va utiliza mecanismul excepțiilor și metode ale claselor înfășurătoare (de revăzut).

4. La fel ca mai sus numai că se vor utiliza excepțiile și la citirea numerelor **a** și **b**, iar în caz că împărțirea nu se poate efectua se va repeta citirea numerelor **a** și **b**.

Rezolvări:

1.

```
class test {
    public static void main(String[] args) {
        int n=1; double nr=0; int semnal=0;
        // citesc pe n
        while (semnal==0)
        { try
            { System.out.print("n=");
              n=Integer.parseInt(cin.Token()); semnal=1;
            } catch (Exception e) {System.out.println("Eroare la introducerea
              lui n");}
        }
        // citesc cele n numere
        double s=0;
        for (int i=0;i<n;i++)
        { nr=0;
          try
          { System.out.print("valoarea este=");
            nr=Double.parseDouble(cin.Token());}
          catch (Exception e) { System.out.println ("Valoare
            nenumERICA"); i--;}
          s+=nr;}
        System.out.println("Suma="+s);
    }
}
```

```
2. class Er1 extends Exception {
    void afis(){System.out.println ("valoarea trebuie sa fie mai
      mica decat 10");}
}
```

```
class Er2 extends Exception {
    void afis(){ System.out.println ("valoarea trebuie sa fie
      numerica");}
}
```



```

class test {
    public static void main(String[] args) {
        int n=1; double nr=0; int semnal=0;
        // citesc pe n
        while (semnal==0)
        { try
            { System.out.print("n=");
              n=Integer.parseInt(cin.Token()); semnal=1;}
            catch (Exception e) {System.out.println("Eroare la introducerea
              lui n");}
        }
        // citesc cele n numere
        double s=0;
        for (int i=0;i<n;i++)
        { nr=0;
          try { nr=citesc();}
          catch (Er1 e) { e.afis(); i--;}
          catch (Er2 e) { e.afis(); i--;}
          s+=nr;}
        System.out.println("Suma="+s);
    }

    public static double citesc() throws Er1,Er2
    { double n=0;
      try { n=Double.parseDouble(cin.Token());}
      catch (Exception e) { throw new Er2();}
      if (n>10) throw new Er1();
      else return n;
    }
}

3.
class Er1 extends Exception {
    void afis() { System.out.println ("Impartire la 0");}
}

class Er2 extends Exception {
    void afis() { System.out.println ("ambii operanzi sunt 0");}
}

class test {
    public static void main(String[] args) {
        double s=0;
        System.out.print("a=");double a=Double.parseDouble(cin.Token());
        System.out.print("b=");double b=Double.parseDouble(cin.Token());
        try {double z=divide(a,b); System.out.println(z);}
        catch (Er1 e) { e.afis();}
        catch (Er2 e) { e.afis();}
    }

    static double divide (double a, double b) throws Er1,Er2
    { if (Double.isNaN(a/b)) throw new Er2();
      else
        if (Double.isInfinite (a/b)) throw new Er1();
        else return a/b;
    }
}

```

CAPITOLUL 9

Alte clase esențiale în Java



Din cuprins:

Clasa Vector

Fișiere

Fire de execuție

Probleme propuse

9.1. Clasa Vector

Clasa **Vector** oferă posibilitatea lucrului cu o structură asemănătoare vectorului, dar în plus, oferă posibilitatea adăugării sau ștergerii unor elemente în interiorul vectorului. Clasa **Vector** se găsește în pachetul **java.util**. Astfel, structura seamănă mult cu o listă liniară simplu înlănțuită. Idei de bază:

① Elementele vectorului sunt *referințe către obiecte, de orice tip* dorim. Atenție, am spus referințe către obiecte și nu tipuri primitive. Pentru tipurile primitive există acele clase înfășurătoare, pe care vă sfătuiesc să le revedeți.

② La baza ierarhiei arborescente a claselor din Java se găsește clasa **Object**. Aceasta înseamnă că o variabilă de tip referință către un obiect al clasei **Object** poate reține orice referință către un obiect al unei clase descendente. Prin urmare, un element al "vectorului" poate reține o referință către orice alt tip de obiect. De aici rezultă faptul că printr-un obiect al clasei **Vector** se pot manipula obiecte de orice tip.

Principalele metode ale clasei **Vector** sunt:

⇒ **Vector()** - constructor ce creează un vector fără nici un element (vector vid).

⇒ **void addElement (Object O);** - adaugă la sfârșitul vectorului un element care reține o referință către un anumit obiect.

⇒ **Object elementAt(i)** - returnază referința către obiectul reținut de componenta de indice **i** a vectorului.

⇒ **Object firstElement()** - returnează referința către primul element al vectorului.

⇒ **Object lastElement()** - returnează referința către ultimul element al vectorului.

⇒ **int size()** - returnează numărul de componente ale vectorului.

⇒ **void insertElementAt(Object O, int i)** - inserează un obiect în vector pe poziția **i**. Celelalte elemente ale vectorului sunt deplasate către dreapta cu o poziție. Poziția **i** trebuie să fie cuprinsă între 0 și **size()-1**.

⇒ **void removeElementAt(int i)** - șterge un element al vectorului aflat pe poziția **i**. Celelalte elemente ale vectorului sunt deplasate către stânga cu o poziție. Poziția **i** trebuie să fie cuprinsă între 0 și **size()-1**.

⇒ `setElementAt(Object O,int i)` - setează componenta de indice `i` să rețină o referință către un nou obiect. Vechiul obiect a cărui poziție era reținută de componenta `i` se pierde. Poziția `i` trebuie să fie cuprinsă între 0 și `size()-1`.

⇒ `void removeAllElements()` - șterge toate elementele vectorului.

⇒ `void removeElement(Object O)` - șterge componenta care are ca referință un obiect identic cu cel transmis prin parametru. Toate celelalte componente ale vectorului sunt deplasate către stânga.

⇒ `int indexOf(Object O)` - întoarce indicele primei apariții a obiectului în vector.

Exemplu: în programul următor se citesc și se afișează `n` numere naturale. Cele `n` numere naturale sunt reținute de un obiect al clasei `Vector`.

```
import java.util.*;
class t {
    public static void main(String[] args) {
        Vector v=new Vector();
        System.out.print("n="); int n=Integer.parseInt(cin.Token());
        for (int i=0;i<n;i++)
            { System.out.print("V["+i+"]=");
              int valoare=Integer.parseInt(cin.Token());
              v.addElement(new Integer(valoare)); }
        for (int i=0;i<n;i++)
            { Integer val=(Integer)v.elementAt(i);
              System.out.println(val.intValue()); }
    }
}
```

🔍 Observații asupra programului:

a) Elementele vectorului sunt referințe către obiecte ale clasei `Integer`. Fiecare obiect al clasei `Integer` reține un număr citit.

```
int valoare=Integer.parseInt(cin.Token());
v.addElement(new Integer(valoare));
```

b) La citire se returnează referința către obiectul de tip `Integer`. Deoarece, de fapt, un element al vectorului reține o referință către un obiect de tip `Object`, referința este convertită către referință către `Integer`.

```
Integer val=(Integer)v.elementAt(i);
```

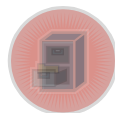
③ Pentru a afișa numărul citit, se utilizează metoda `intValue()` a clasei `Integer`, metodă care returnează valoarea reținută de obiect.

```
System.out.println(val.intValue());
```

④ Linii de la observațiile 3 și 4 pot fi înlocuite cu linia de mai jos:

```
System.out.println(((Integer)v.elementAt(i)).intValue());
```

9.2. Fișiere

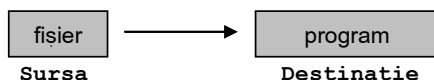


9.2.1. Noțiunea de flux

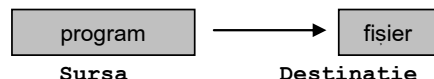
În Java, fișierele sunt abordate pornind de la noțiunea de flux. *Prin flux vom înțelege o cale pe care o urmează datele pentru a ajunge de la sursă la destinație.* Putem imagina un flux ca o șosea pe care circulă mașini. Astfel, în Java, datele ar fi mașinile care circulă pe șosea.

Există două tipuri de fluxuri:

A) Fluxuri de intrare - de la sursă (un fișier), către destinație (programul), unde datele sunt prelucrate.



B) Fluxuri de ieșire - de la sursă, (programul), către destinație (fișier), unde se rețin rezultatele..



Clasele pe care le studiem în acest paragraf se găsesc în pachetul `java.IO`.

9.2.2. Fluxuri de octeți

În acest caz, datele sunt date de octeți.

A) Fluxurile de intrare sunt tratate de clasa `FileInputStream`. Sursa este dată de un fișier, iar destinația este memoria. Principalele metode ale clasei `FileInputStream` sunt:

1. Constructorul - are rolul de a deschide fișierul al cărui nume este dat de parametrul de tip `String`. În cazul în care fișierul nu este găsit (excepție), se returnează un obiect al clasei `FileNotFoundException`.

```
public FileInputStream(String nume)
    throws FileNotFoundException
```

2. Citirea unui octet se realizează cu metoda `read()`:

```
public int read()
    throws IOException
```

3. Închiderea fișierului se realizează cu metoda `close()`:

```
public void close()
    throws IOException
```

Observații:

- ✓ Clasa `IOException` este o subclasă a clasei `Exception`. Prin urmare, orice metodă care apelează o altă metodă care poate returna un obiect al acestei clase trebuie să aibă în antet clauza `throws` sau apelul să fie făcut sub `try` (vezi excepții).
- ✓ Numele fișierului poate conține și calea către el. Aceasta trebuie pusă în cazul în care programul nu se găsește în același folder cu fișierul.

Exemple:

1. Creați în **Notepad** un fișier text oarecare, să-l numim `in.txt`. El va conține un text oarecare, scris pe mai multe linii. Scrieți un program care afișează conținutul fișierului.

```
import java.io.*;
public class f {
    public static void main (String[ ] args) throws IOException {
        FileInputStream f=new FileInputStream("in.txt");
        int ch=0;
        while ((ch=f.read())!=-1) System.out.print((char)ch);
    }
}
```



Sfârșitul unui fișier se detectează prin faptul că ultimul octet reține -1.

2. Programul următor are același efect ca și cel precedent, dar fișierul de intrare se găsește pe `C:\`, iar în cazul în care fișierul nu este găsit, se afișează un mesaj corespunzător:

```
import java.io.*;
public class f {
    public static void main (String[ ] args)
    {
        try { FileInputStream f=new FileInputStream("C:\\in.txt");
            int ch=0;
            while ((ch=f.read())!=-1) System.out.print((char)ch); }
        catch (IOException e) {System.out.println("Fișier negăsit"); }
    }
}
```

B) Fluxurile de ieșire sunt tratate de clasa `FileOutputStream`. Sursa este dată de o variabilă de tip `int` a programului, iar destinația este un fișier. Principalele metode ale clasei `FileOutputStream` sunt:

1. **Constructorul** - are rolul de a deschide fișierul al cărui nume este dat de parametrul de tip `String` și de a-l pregăti pentru scriere:

```
public FileOutputStream(String name)
    throws FileNotFoundException
```

2. **Scrierea unui octet** se realizează cu metoda `write()`:

```
public void write(int b)
    throws IOException
```

3. **Închiderea fișierului** se realizează cu metoda `close()`:

```
public void close()
    throws IOException
```

Exemplu: programul de mai jos copiază fișierul `in.txt`. Fișierul rezultat în urma copierii se numește `out.txt` și se găsește în același folder cu sursa.

```
import java.io.*;
public class f {
    public static void main (String[ ] args) throws IOException {
        FileInputStream f=new FileInputStream("javac.exe");
        FileOutputStream g=new FileOutputStream("javac1.exe");
        int ch=0;
        while ((ch=f.read())!=-1) g.write(ch);
        f.close();
        g.close();
    }
}
```

🔍 Observații:

- ✓ Deschideți fișierul astfel obținut cu **Notepad**. Veți observa că nu există nici o modificare față de precedentul.
- ✓ Cu acest procedeu se pot copia și executabilele. Testați. Copiați un executabil și apoi, fișierul astfel obținut îl lansați în executare. Evident, noul fișier trebuie să aibă extensia **.exe**.

9.2.3. Fluxuri de caractere

Vom utiliza fluxuri de caractere pentru a lucra cu fișiere text. Acestea pot conține una sau mai multe linii.

A) Fluxuri de intrare - se folosesc pentru citirea fișierelor text.

Pentru a transforma un flux de octeți într-un flux de caractere, vom utiliza constructorul clasei `InputStreamReader` (care extinde clasa `Reader`):

```
⇒ public InputStreamReader(InputStream in)
```

Întrucât clasa `FileInputStream` extinde clasa `InputStream`, parametrul `in` (de tip `InputStream`) poate fi o referință către un obiect al clasei `FileInputStream`.

Pentru a putea lucra cu ușurință cu caracterele, vom utiliza clasa **BufferedReader** care permite atașarea fluxului de caractere a unei memorii, numită **buffer**. Practic, buffer-ul permite memorarea mai multor caractere citite, care urmează să fie prelucrate. Constructorul clasei **BufferedReader** este:

```
⇒ public BufferedReader(Reader in1)
```

Întrucât clasa **InputStreamReader** extinde clasa **Reader**, parametrul *in1* poate fi o referință către un obiect al clasei **InputStreamReader**.

Clasa **BufferedReader** conține metoda **readLine()** care citește din fișier (flux) o întreagă linie de caractere:

```
⇒ public String readLine()  
    throws IOException
```

Acestea fiind spuse, puteți analiza programul de mai jos care afișează toate liniile unui fișier text. De exemplu, dacă creați cu **Notepad** un fișier cu mai multe linii, numit *in.txt*, programul îl afișează:

```
import java.io.*;  
public class f {  
    public static void main (String[ ] args) throws IOException  
    { FileInputStream f=new FileInputStream("in.txt");  
      InputStreamReader fchar=new InputStreamReader (f);  
      BufferedReader buf = new BufferedReader(fchar);  
      String linie;  
      while ((linie=buf.readLine() )!=null) System.out.println(linie);  
      fchar.close();  
    }  
}
```

B) Fluxuri de ieșire - se folosesc pentru scrierea fișierelor text.

Clasa **FileOutputStream** creează un flux de ieșire către un fișier al cărui nume este transmis ca parametru. Ea extinde clasa **OutputStream** și are constructorul:

```
public FileOutputStream(String nume)  
    throws FileNotFoundException
```

Clasa **PrintStream** conține mai multe metode **print** și **println** care funcționează exact ca **System.out.print()** și **System.out.println()**. Practic, valoarea care este scrisă în fișier este mai întâi convertită către tipul **String**, așa cum suntem deja obișnuiți. Clasa este înzestrată cu constructorul:

```
public PrintStream(OutputStream out)
```


Exemplu: programul de mai jos creează un fișier text (numit `out.txt`) care conține 3 linii, iar fiecare linie conține câte un șir de caractere:

```
import java.io.*;
public class f {
    public static void main (String[ ] args) throws IOException
    { FileOutputStream f=new FileOutputStream("out.txt");
      PrintStream fchar=new PrintStream (f);
      fchar.println("O linie");
      fchar.println("Alta linie");
      fchar.print("A treia linie");
      fchar.close();
    }
}
```

9.2.4. Intrări de la tastatură

Pentru a gestiona intrările de la tastatură, în program se utilizează constanta de tip `in` a clasei `System`. Practic, ea reține referința către fluxul de intrare de la tastatură (acesta din urmă este de tipul `InputStream`).

```
public static final InputStream in
```

Cel mai bun exemplu de utilizare a acestei clase este dat de metoda `linie()` a clasei `cin` (cea pe care am folosit-o de atâtea ori până acum).

```
import java.io.*;
import java.util.*;
class cin {
    static InputStreamReader f = new InputStreamReader (System.in);
    static BufferedReader tamp=new BufferedReader(f);
    static String linie() {
        try { return tamp.readLine();}
        catch (IOException e) { return ("Eroare");}
    }
    static String Token(){
        StringTokenizer v=new StringTokenizer(linie());
        if (v.hasMoreTokens())return v.nextToken();
        else return"";
    }
}
```

9.3. Fire de execuție

Ați observat, probabil, că în anumite programe apar mai multe cadre în care se afișează anumite animații. Vă dați seama că fiecare animație trebuie executată separat. Imaginați-vă că un program conține, să zicem, 6 astfel de animații. Aparent, fiecare dintre ele se execută independent. Cum a fost scris acel program?

Dacă acel program a fost scris în Java, atunci s-au utilizat firele de execuție. Practic, un fir de execuție înseamnă o secvență de instrucțiuni. Procesorul își împarte timpul pentru a executa o parte dintr-un fir de execuție, apoi o parte din alt fir de execuție, s.a.m.d. Mai mult, de câte ori el trebuie să intre în așteptare pentru un fir de execuție (de exemplu, se așteaptă introducerea unei date), reia executarea altui fir de execuție, s.a.m.d. Vă dați seama că un astfel de mecanism este extrem de complex, dar în Java se poate folosi cu ușurință pentru că acesta este foarte bine pus la punct. Ce trebuie să știm pentru a crea fire de execuție?

1. Un fir de execuție este un obiect al unei clase care extinde (moștenește) clasa **Thread**.
2. Secvența de instrucțiuni a acestui fir de execuție este conținută de metoda **run()** a clasei.
3. Un fir de execuție se lansează utilizând metoda **start()** a clasei din instanțierea căreia a rezultat firul de execuție respectiv. Evident, metoda **start()** este moștenită din clasa **Thread**.
4. Un fir de execuție poate fi pus în așteptare prin utilizarea metodei **sleep(long m)**, metodă moștenită de la clasa **Thread**. Ea poate genera o excepție, așa că trebuie pusă sub construcția **try**. Evident, în acest timp, procesorul poate relua executarea unui alt fir de execuție. Timpul este dat în milisecunde.

Exemple:

1. Programul următor conține un fir de execuție, care incrementează valoarea unei variabile (date membru) apoi o afișează, după care stă în așteptare o secundă, s.a.m.d., până când valoarea variabilei devine mai mare decât 10:

```
class Fir extends Thread {
    int i=0;
    public void run()
    { while (i!=10)
        { i++;
          System.out.println("i="+i);
          try { sleep(1000);}
          catch (Exception e) {}
        }
    }
}

public class pv {
    public static void main(String args[]) {
        Fir x=new Fir();
        x.start();
    }
}
```

2. Programul următor conține două fire de execuție. Fiecare dintre ele afișează valoarea unei variabile locale. Se va afișa $i=1$, $j=1$, $i=2$, $j=2$, ș.a.m.d. Observați puterea firelor de execuție. Practic, când un fir de execuție este întrerupt, se salvează valoarea variabilelor locale și se memorează punctul în care firul a fost întrerupt.

```
class Fir extends Thread {
    public void run()
    { int i=0;
      while (i!=10)
      { i++;
        System.out.println("i="+i);}
    }
}

class AltFir extends Thread {
    public void run()
    { int j=0;
      while (j!=10)
      { j++;
        System.out.println("j="+j);}
    }
}

public class pv {
    public static void main(String args[]) {
        Fir x=new Fir();
        AltFir y=new AltFir();
        x.start();
        y.start();
    }
}
```

Probleme propuse

1. Implementați o stivă prin utilizarea clasei **Vector**. Avantajul acestei implementări este evident: se poate utiliza stiva pentru a reține orice tip de obiect.
2. Memorați în stiva astfel obținută 3 numere întregi și afișați-le în ordinea extragerii.
3. Memorați în stiva astfel obținută n șiruri de caractere citite de la tastatură și afișați-le în ordinea extragerii.
4. Creați o înregistrare care reține pentru o persoană numele și vârsta. Citiți datele pentru n astfel de persoane, adăugați-le în stivă și afișați-le în ordinea extragerii.
5. Implementați o coadă prin utilizarea clasei **Vector**.

6. Refaceți problemele de la 2 la 4 prin utilizarea structurii de tip coadă.
7. Fișierul text `numere.in` conține pe prima linie mai multe numere naturale. Care este suma lor?
8. Fișierul text `numere.in` conține pe prima linie mai multe numere naturale, dar și alte șiruri de caractere. Numerele sunt separate prin spații de celelalte șiruri de caractere (vezi exemplul de mai jos). Care este suma lor?

Exemplu: 1 asf 12 JKHS 17 aha 10 ngsvss 5 se afisează 45.

9. Refaceți programul de la problema 9, astfel încât numele fișierului să fie preluat de linia de comandă din fereastra `CMD`.

Indicații / rezolvări:

1.

```
import java.util.*;

class Stiva {
    Vector st=new Vector();
    void push(Object x)
    { st.addElement (x);}
    Object pop ()
    { if (st.size()>0)
        { Object retur=st.lastElement();
          st.removeElementAt(st.size()-1);
          return retur;
        }
        else return null;
    }
}
```

2.

```
class test {
    public static void main(String[] args) {
        Stiva st=new Stiva();
        st.push(new Integer (1));
        st.push(new Integer (2));
        st.push(new Integer (3));
        Integer numar;
        do
        { numar=(Integer)st.pop();
          if (numar!=null)
              System.out.println(numar.intValue());
        } while (numar!=null);
    }
}
```

```
7.
import java.io.*;
import java.util.*;
public class test {
    public static void main (String[ ] args) throws IOException
    { FileInputStream f=new FileInputStream("numere.in");
      InputStreamReader fchar=new InputStreamReader (f);
      BufferedReader buf = new BufferedReader(fchar);
      String linie=buf.readLine();
      StringTokenizer t=new StringTokenizer(linie);
      int s=0;
      while (t.hasMoreTokens())
          s+=Integer.parseInt(t.nextToken());
      System.out.println(s);
      fchar.close();
    }
}
```

```
8.
import java.io.*;
import java.util.*;
public class test {
    public static void main (String[ ] args) throws IOException
    { FileInputStream f=new FileInputStream("numere.in");
      InputStreamReader fchar=new InputStreamReader (f);
      BufferedReader buf = new BufferedReader(fchar);
      String linie=buf.readLine();
      StringTokenizer t=new StringTokenizer(linie);
      int s=0;
      int nr;
      while (t.hasMoreTokens())
          try { nr=Integer.parseInt(t.nextToken()); s+=nr;}
          catch (Exception e) {}
      System.out.println(s);
      fchar.close();
    }
}
```

```
9.
...
public static void main (String[ ] args) throws IOException
{ if (args.length==0)
    System.out.println ("Fisierul?");
  else
  { String nume_f=args[0];
    FileInputStream f=new FileInputStream(nume_f);
    ...
  }
}
```

CAPITOLUL 10

Inițiere în programarea vizuală



Din cuprins:

Prima fereastră

Mecanismul prin care se atașează componente
ferestrei. Clasa `Container`

Clasele `JComponent` și `Toolkit`

Poziționarea componentelor

Studiul principalelor componente

Mai mult despre clasa `Container`

Cutii de dialog

Clasele `Graphics` și `Graphics2D`

Evenimente

Animarea imaginilor

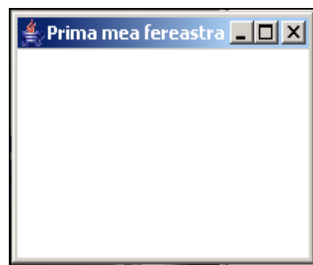
Probleme propuse

10.1. Prima fereastră

O fereastră este un obiect al clasei `JFrame`. Clasa `JFrame` este reținută în pachetul `javax.swing`. Mai jos puteți observa un program care afișează o fereastră:

```
import javax.swing.*;
public class pv {
    public static void main(String args[]) {
        JFrame fer=new JFrame("Prima mea
                               fereastră");

        fer.setSize(200,300);
        fer.setLocation(300,400);
        fer.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE);
        fer.setVisible(true);
    }
}
```



↳ Să observăm că fereastra conține meniul implicit, poate fi maximizată, minimizată (adusă pe bara de task-uri), dar poate fi și închisă.

Din analiza programului deducem câteva metode pe care le are clasa `JFrame`:

- ⇒ `JFrame()` - constructor. Dacă o fereastră este creată printr-un astfel de constructor ea apare fără titlu.
- ⇒ `JFrame(String titlu)` - constructor. Construiește o fereastră care afișează un anumit titlu (în exemplu, *Prima mea fereastră*).
- ⇒ `void setSize(int width, int height)` - stabilește lățimea și înălțimea ferestrei.
- ⇒ `void setLocation (int x, int y)` - stabilește poziția unde va fi afișat colțul din stânga sus al ferestrei (și implicit poziția ferestrei), în raport cu colțul din stânga sus al ecranului. Parametrul `x` precizează distanța pe orizontală a colțului ferestrei, iar `y` distanța pe verticală a acestuia (ambele sunt date în pixeli).
- ⇒ `void setDefaultCloseOperation(int a)` - stabilește ce se întâmplă atunci când se închide fereastra (s-a păstrat acel buton `x`). Parametrii sunt constante de tip întreg ale clasei. Cea mai importantă constantă este `EXIT_ON_CLOSE` și prin ea, se cere închiderea ferestrei și încheierea execuției programului.
- ⇒ `void setResizable(boolean ac)` - dacă parametrul este `false` nu se pot modifica dimensiunile ferestrei.
- ⇒ `setVisible(boolean x)` - stabilește dacă fereastra este vizibilă (apare pe ecran) sau nu (deși există, nu este afișată).

Dacă dorim să fim eficienți, putem construi o clasă, de exemplu numită **Fereastră**, al cărei constructor să returneze obiectul cu toate datele de mai sus (titlul, dimensiunile ferestrei, poziția sa pe ecran).

Exemplu: programul de mai jos afișează o fereastră:

```
import javax.swing.*;
class Fereastra extends JFrame
{ Fereastra(String Nume, int lat, int inalt, int dreapta, int
  stanga)
  { super(Nume);
    setSize(lat,inalt);
    setLocation(dreapta, stanga);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setVisible(true);
  }
}

public class pv {
  public static void main(String args[]) {
    Fereastra f= new Fereastra ("Prima", 100,200, 25, 100);}
}
```

10.2. Mecanismul prin care se atașează componente ferestrei. Clasa **Container**

☛ **Foarte important!** Pentru ca unei ferestre (obiect de tip **JFrame**) să i se poată atașa alte componente (cum ar fi butoanele, liste, etc. – care sunt obiecte de alte tipuri), este necesar ca aceasta să conțină o referință către o structură specială, care la rândul ei va reține referințe către obiectele (componentele) care sunt atașate ferestrei. Structura care reține referințele către obiectele care se află pe fereastră este un obiect al clasei **Container**.

Alăturat putem observa ierarhia claselor din care a rezultat clasa **JFrame**. Prin urmare, un obiect de tip **JFrame** sau dintr-o subclasă a lui **JFrame** conține datele membru și metodele clasei **Container**.

```
Object
  Component
    Container
      Window
        java.awt.Frame
          javax.swing.JFrame
```

Accesul la container-ul unei ferestre se face utilizând o metodă a clasei **JFrame**, numită **getContentPane()**.

⇒ **Container getContentPane()** – returnează o referință către container-ul ferestrei. Pornind de la această referință, putem realiza două lucruri:

1. *Putem adăuga ferestrei componentele dorite.* Pentru aceasta se folosește metoda `add()` a clasei `Container`:

- ❖ **Component** `add(Component comp)` - adaugă o componentă ferestrei. **Important:** toate componentele sunt derivate din clasa `Component`. Prin urmare, parametrul `comp` poate reține referințe către componente de orice tip.

2. *Putem spune cum să fie aranjate în fereastră componentele adăugate.*

Important: Mecanismul din Java care rezolvă această problemă presupune existența unor așa numiți **gestionari de poziționare**. Gestionarii de poziționare sunt obiecte ale unor clase specifice. Ei aranjează automat componentele unui container. Pentru a atașa unui container un **gestionar de poziționare**, se utilizează metoda clasei `Container` numită `setLayout()`:

- ❖ **void** `setLayout(LayoutManager gest)` - atașează unui container un gestionar de poziționare. `LayoutManager` - este o interfață. Toți gestionarii de poziționare pe care îi vom studia într-un paragraf separat au rezultat ca urmare a implementării acestei interfețe. Aceasta înseamnă că metoda poate fi utilizată pentru atașarea oricărui gestionar de poziționare.



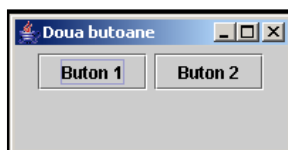
Pentru început, vom folosi gestionarul de poziționare `FlowLayout`, care are un constructor fără parametri. Pe scurt, acesta așează componentele în fereastră, pe linie, una după alta. În cazul în care o linie s-a umplut, se trece la linia următoare.

! Clasa `Container` se găsește în pachetul `java.awt`.

Exemplu: programul care urmează afișează două butoane! Apăsarea butoanelor nu are nici un efect.

```
import java.awt.*;
import javax.swing.*;
class Fer extends JFrame
{
    public Fer(String titlu) {
        super(titlu);
        setSize(200,100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        JButton A=new JButton("Buton 1"); x.add(A);
        JButton B=new JButton("Buton 2"); x.add(B);
        setVisible(true);}
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Doua butoane");}
}
```





Butoanele atașate sunt componente de tip `JButton`, care au un constructor de tip `JButton(String s)`. Șirul `s` va apărea pe suprafața butonului.

10.3. Un mecanism prin care butoanele răspund evenimentului de “apăsare”

Până în prezent știm să construim o fereastră, știm să-i atașăm unul sau mai multe butoane, dar nu știm să facem de așa manieră încât la “apăsarea” butonului (click pe suprafața lui) să aibă loc o anumită acțiune. Este exact ce ne propunem să prezentăm în acest paragraf.

În Java există o interfață numită `ActionListener`, “ascultătorul” de evenimente de tip `ActionEvent`. Un exemplu de eveniment de tip `ActionEvent` este “apăsarea” unui buton. Interfața conține antetul unei singure metode: `actionPerformed (ActionEvent e)`.

Pentru ca o componentă să poată răspunde la un eveniment de tipul `ActionEvent`, trebuie să implementeze clasa `ActionListener`. Aceasta înseamnă că:

1. Clasa care include componenta (fereastra) să conțină clauza

```
implements ActionListener;
```

2. Să fie implementată metoda `actionPerformed()`. Această metodă se va executa automat atunci când este apăsătorul butonul. Prin urmare, implementarea ei va scrie codul necesar acțiunii dorite.

`ActionEvent` este o clasă care conține metoda:

- ❖ `String getActionCommand()` – returnează șirul de caractere asociat componentei care a transmis evenimentul. Metoda poate fi utilizată pentru a depista componenta care a transmis evenimentul.

Exemplu: extindem programul prezentat în paragraful anterior. Când se apasă un buton, în fereastra `CMD` va apărea șirul reținut de butonul apăsătorul.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class Fer extends JFrame implements ActionListener
{ public Fer(String titlu) {
    super(titlu);
    setSize(200,100);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    Container x=getContentPane();
```

```

x.setLayout(new FlowLayout ());
JButton A=new JButton("Buton 1");
x.add(A);
JButton B=new JButton("Buton 2");
x.add(B);
A.addActionListener(this);
B.addActionListener(this);
setVisible(true);
}

public void actionPerformed (ActionEvent e)
{ if( e.getActionCommand().compareTo("Buton 1")==0)
    System.out.println("Ai apasat Buton 1");
  else
    System.out.println("Ai apasat Buton 2");
}
}

public class pv {
  public static void main(String args[]) {
    Fer fp=new Fer("Doua butoane");
  }
}

```

10.4. Clasa JComponent

Prin componentă vom înțelege un obiect care are o reprezentare grafică. Exemple de componente: butoane, liste, edit-uri, etichete (am preferat denumirea lor din cursul de Tehnologia Informației. Dacă o componentă este obiect, înseamnă că o componentă rezultă în urma instanțierii unei clase. Fiecare tip de componentă pe care o vom studia rezultă în urma instanțierii unei clase specifice ei. De exemplu, un buton rezultă în urma instanțierii clasei `JButton`, o etichetă rezultă în urma instanțierii clasei `JLabel`, un edit rezultă în urma instanțierii clasei `JTextField`, ș.a.m.d.

 **Important:** clasele tuturor componentelor enumerate mai sus sunt subclase ale clasei `JComponent`. Aceasta înseamnă că putem folosi metodele clasei `JComponent` pentru orice componentă.

 Clasa `JComponent` este descendentă, dar nu direct, a unei alte clase numită `Component`. Atunci când prezentăm unele metode ale clasei `JComponent`, care sunt moștenite de la clasa `Component` nu vom mai face referire la acest lucru, ci le vom trata ca făcând parte din clasa `JComponent`.

În continuare prezentăm câteva metode ale clasei `JComponent`:

⇒ **`void setBackground(Color c)`** - metoda stabilește culoarea de fond a componentei. Să observăm că parametrul ei este de tip `Color`.

- ⇒ `void setForeground(Color c)` - setează culoarea caracterelor (în cazul în care componenta conține un text.

Clasa Color conține anumite constante care indică culoarea și mai multe metode prin care se poate stabili o culoare. **Exemple** de constante de culoare: `black`, `red`, `white`, `yellow`, etc. De asemenea, clasa conține constructorul `Color(float r, float g, float b)` prin care se poate forma o culoare în sistemul RGB (Red, Green, Blue), sistem studiat la tehnologia informației.

- ⇒ `setFont(Font f)` setează font-ul cu care se scrie, stilul său și mărimea. Parametrul este un obiect al clasei `Font`.

Clasa `Font` are, printre altele, constructorul:

```
Font(String nume, int stil, int marime).
```

unde:

`nume` - este numele font-ului;

`stil` - stilul. Valorile uzuale sunt: `Font.ITALIC` (italic), `Font.BOLD` (bold), `Font.PLAIN` (clasic). Se pot folosi și sume, cum ar fi pentru italic și bold: `Font.ITALIC+Font.BOLD`.

`marime` - mărimea font-ului.

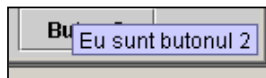
Exemplu: butonul cu fond roșu, "Apasa" conține text scris cu verde, se utilizează font-ul `Arial`, mărimea 20 și este scris italic+bold. Să observăm faptul că dimensiunea butonului este stabilită automat, în funcție de mărimea textului pe care îl conține.

```
JButton A=new JButton("Apasa");  
A.setBackground(Color.red);  
A.setFont(new Font("Arial",Font.ITALIC+Font.BOLD, 20));  
A.setForeground(Color.GREEN);
```

- ⇒ `void setToolTipText(String text)`; - metoda setează un șir de caractere care va fi afișat atunci când cursorul mouse-ului staționează asupra componentei. Șirul are rolul unui mesaj lămuritor despre funcția respectivei componente.

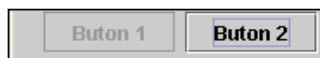
Exemplu:

```
JButton B=new JButton("Buton 2");  
B.setToolTipText("Eu sunt butonul 2");
```



- ⇒ `void setEnabled(boolean v)` - Face ca o componentă să fie activată (`v` reține `true`) sau nu (`v` reține `false`). O componentă dezactivată nu mai răspunde comenzilor și are un aspect specific prin care utilizatorul este anunțat de faptul că aceasta este dezactivată.

Exemplu: în imaginea alăturată butonul 1 este dezactivat și butonul 2 este activat.



⇒ `void setVisible(boolean v)` - dacă parametrul reține `true`, componenta este vizibilă, altfel ea este invizibilă.

10.5. Clasa Toolkit

Se știe faptul că programele scrise în Java trebuie să ruleze independent de rezoluția calculatorului. O clasă care face posibil acest lucru este clasa `Toolkit`. În acest sens, cele mai importante metode ale ei sunt:

⇒ `public static Toolkit getDefaultToolkit()` - returnează un obiect `Toolkit` (conține date despre setările curente);

⇒ `Dimension getScreenSize()` - returnează o referință către un obiect de tip `Dimension` care conține lungimea (`width`) și înălțimea (`height`) în pixeli a ecranului.

Exemplu: programul de mai jos creează o fereastră care, inițial, ocupă întreg ecranul.

```
import java.awt.*;
import javax.swing.*;
class Fer extends JFrame
{
    public Fer(String titlu) {
        super(titlu);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Toolkit ec=Toolkit.getDefaultToolkit();
        Dimension dim = ec.getScreenSize();
        int i = dim.height; int l = dim.width;
        setSize(dim.width, dim.height); setVisible(true);}
}
public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Fereastra pe intreg ecranul");}
}
```

10.6. Poziționarea componentelor

Pentru a aseza componentele în poziția dorită, putem să utilizăm **poziționarea absolută** (dar *este nerecomandată* pentru că programul trebuie să ruleze corect pe o diversitate de platforme) sau unul dintre gestionarii de poziționare existenți. Cum facem acest lucru? Veți vedea în paragraful acesta! Oricum, trebuie să știți că mecanismul este puțin mai complicat decât cel cu care ați fost obișnuiți în ipoteza că ați studiat programarea vizuală (de exemplu în **Borland Delphi** sau **Borland Builder**). Nu uitați, programele Java trebuie să funcționeze pe orice platformă!

10.6.1. Poziționarea absolută

Pentru a utiliza poziționarea absolută a componentelor trebuie să lucrăm în absența unui gestionar de poziționare, iar coordonatele sunt date în pixeli. Pentru aceasta trebuie să cunoaștem următoarele:

1. Secvența de mai jos se utilizează pentru a lucra în absența unui gestionar de poziționare.

```
Container x=getContentPane();  
x.setLayout(null);
```

2. În poziționarea absolută componentele trebuie dimensionate și poziționate, altfel nu sunt vizibile. Testați...

Pentru dimensionarea componentelor se utilizează următoarele metode ale clasei **JComponent**:

⇒ **setBounds(int x, int y, int lat, int inal);** metoda poziționează și dimensionează componenta. Parametrii **x** și **y** dau poziția componentei în raportată la colțul din stânga sus al componentei care o găzduiește, iar **lat** și **lung** o dimensionează.

Metoda de mai sus poate fi înlocuită cu următoarele două metode:

⇒ **setLocation(int x, int y)** - metodă care are rolul de poziționare;

⇒ **setSize (int lat, int lung)** - metodă care are rolul de dimensionare.

Exemplu: utilizăm poziționarea absolută și așezăm un buton în fereastră:

```
Container x=getContentPane();  
x.setLayout(null);  
JButton A=new JButton ("Exemplu");  
A.setBounds(10,10, 100, 40);
```

10.6.2. Gestionarul de poziționare **FlowLayout**

Componentele sunt afișate pe linii, în ordinea în care au fost declarate. Pe fiecare linie ele sunt afișate de la stânga la dreapta (atâtea câte încap).

În continuare, puteți observa cum a fost creată fereastra alăturată:



```
class Fer extends JFrame
{
    public Fer(String titlu) {
        super(titlu);
        setSize(300,150);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        JButton A=new JButton("Buton 1"); x.add(A);
        JButton B=new JButton("Buton 2"); x.add(B);
        JButton C=new JButton("Buton 3"); x.add(C);
        JButton D=new JButton("Buton 4"); x.add(D);
        JButton E=new JButton("Buton 5"); x.add(E);
        JButton F=new JButton("Buton 6"); x.add(F);
        JButton G=new JButton("Buton 7"); x.add(G);
        setVisible(true);
    }
}
```

Clasa **FlowLayout** conține constante de aliniere pe linie, din care mai importante sunt: **CENTER**, aliniere în centru, opțiune implicită, **LEFT**, la stânga și **RIGHT**, la dreapta.

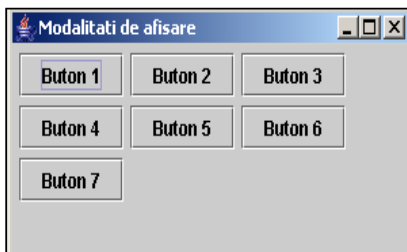
⇒ Clasa **FlowLayout** este înzestrată cu 3 constructori:

a) **FlowLayout()** - distanța între rânduri este de 5 unități, distanța pe orizontală între componente este de 5 unități și componentele sunt aliniate pe linie la centru (**CENTER**).

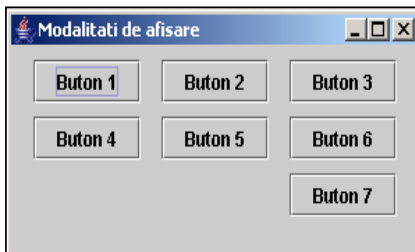
b) **FlowLayout(int aliniere)** - se cere explicit ca alinierea să fie într-un anumit fel (**CENTER**, **RIGHT**, **LEFT**, acestea sunt constante ale clasei **FlowLayout**).

c) **FlowLayout(int aliniere, int dist_oriz, int dist_vert)** - se specifică și distanța pe orizontală între componente și distanța pe verticală dintre ele.

Exemple de utilizare a constructorilor:



FlowLayout (FlowLayout.LEFT)



FlowLayout (FlowLayout.RIGHT,15,8)

De reținut:

1. Dimensionarea componentelor și poziționarea lor este făcută automat de către gestionar.
2. Metodele utilizate în poziționarea absolută pentru poziționarea și dimensionarea componentelor (`setBounds()`, `setLocation()`, `setSize()`) deși sunt acceptate la compilare, nu au nici un efect.
3. Există, totuși, o metodă care dimensionează componentele și este acceptată de gestionarul `FlowLayout`. Ea aparține clasei `JComponent`:

⇒ `setPreferredSize(Dimension dim);` Clasa `Dimension` are constructorul `Dimension(int lat, int inalț)` prin care se specifică lățimea, respectiv înălțimea componentei.

Exemplu: prin utilizarea gestionarului `FlowLayout` se adaugă unei ferestre două butoane, primul de dimensiune stabilită, al doilea de dimensiune implicită:

```
Container x=getContentPane();
x.setLayout(new FlowLayout());
JButton A=new JButton ("Buton 1");
A.setPreferredSize(new Dimension(100, 100));
JButton B=new JButton ("Buton 2");
x.add(A);
x.add(B);
```



10.6.3. Gestionarul de poziționare GridLayout

Clasa `GridLayout` - aranjează componentele într-o altă logică. Ideea de bază este aceea că se împarte suprafața ferestrei în mai multe dreptunghiuri de suprafață egală și în fiecare dreptunghi astfel obținut se așează o componentă care este de cele mai multe ori extinsă ca suprafață, astfel încât să ocupe întreaga suprafață a dreptunghiului care îi revine. Pentru a înțelege modul de aranjare a componentelor vom prezenta pe scurt cei trei constructori ai clasei:

a) `GridLayout()` - dacă avem n componente care trebuie așezate, suprafața ferestrei este împărțită într-o singură linie și n coloane. Fiecare componentă este așezată într-un dreptunghi, iar componentele sunt fără spațiu între ele.

b) `GridLayout(int nr_linii, int nr_coloane)` - suprafața împărțită în $nr_linii \times nr_coloane$ dreptunghiuri. Fiecare dreptunghi reține o componentă.

Vedeți exemplul alăturat.



GridLayout (3,3)

c) `GridLayout(int nr_linii, int nr_col, int sp_o, int sp_v)` - se procedează ca anterior, numai că se trec și spațiile pe orizontală și pe verticală între dreptunghiurile generate.



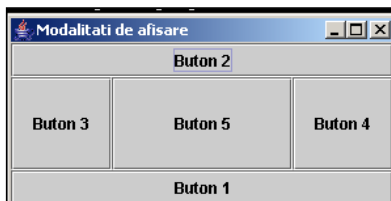
`GridLayout (3,3,5,8)`

! Pentru `GridLayout` nu are nici un efect metoda `setPreferredSize()`.

10.6.4. Gestionarul de poziționare `BorderLayout`

Clasa `BorderLayout` - împarte suprafața ferestrei în 5 părți: nord (**NORTH**), sud (**SOUTH**), est (**EAST**), vest (**WEST**) și centru (**CENTER**). În fiecare parte se poate așeza o componentă. Mai jos puteți observa cum am obținut așezarea componentelor ca în fereastra alăturată.

```
x.setLayout(new BorderLayout ());
JButton A=new JButton("Buton 1");
x.add(A,BorderLayout.SOUTH);
JButton B=new JButton("Buton 2");
x.add(B,BorderLayout.NORTH);
JButton C=new JButton("Buton 3");
x.add(C,BorderLayout.WEST);
JButton D=new JButton("Buton 4");
x.add(D,BorderLayout.EAST);
JButton E=new JButton("Buton 5");
x.add(E,BorderLayout.CENTER);
```



De reținut:

1. Pentru adăugarea unei componente container-ului, se folosește metoda `add` cu 2 parametri, al doilea fiind cel care precizează poziția în care este adăugată componenta. Vezi, mai sus!
2. Componentele sunt extinse pentru a ocupa întreaga suprafață alocată. Vedeți figura de mai sus.
3. Dacă lipsește componenta din nord, sau cea din sud, spațiul este ocupat de linia din centru.
4. Dacă lipsește componenta din centru, spațiul este lăsat liber. Dacă lipsește componenta din est, sau cea din vest, spațiul este ocupat de componenta din centru.
5. Metoda `setPreferredSize()` are efect parțial. Astfel avem: dacă este aplicată unei componente din nord sau din sud, se ia în considerare numai înălțimea transmisă de metodă. Dacă este aplicată unei componente din est, vest, sau centru se ia în considerare numai lățimea componentei.

10.6.5. Gestionarul GridBagLayout

Gestionarul `GridBagLayout()` este o extindere a gestionarului `GridLayout()`. Și el împarte suprafața container-ului în mai multe dreptunghiuri, unde fiecare dreptunghi va fi ocupat de o componentă. Evident, dreptunghiurile sunt aranjate în linii și coloane. Dar, aici, proporțiile între linii și coloane pot fi diferite, iar o componentă poate ocupa unul sau mai multe dreptunghiuri. De asemenea, o componentă poate fi plasată în cadrul dreptunghiului (dreptunghiurilor) în mai multe feluri. Să vedem cum se pot realiza toate acestea.

⇒ Clasa `GridBagLayout` - un obiect al acestei clase este de fapt, administratorul de poziționare. Printre altele, clasa conține:

- ❖ `GridBagLayout()` - constructor;
- ❖ `setConstraints(Component comp, GridBagConstraints cons)` - metodă. Pentru a ști unde se poziționează fiecare componentă, vom utiliza un mecanism specific. El constă în a scrie, pentru fiecare componentă în parte, un set de restricții (cu ajutorul lor se determină poziționarea acesteia). Restricțiile de poziționare pentru fiecare componentă în parte sunt memorate de câte un obiect de tip `GridBagConstraints`. Metoda pe care o tratăm în acest moment anunță gestionarul de poziționare, componenta și restricțiile ei de poziționare.

Acum trebuie să vedem cum stabilim restricțiile de poziționare pentru o componentă oarecare. Pentru aceasta vom utiliza clasa

`GridBagConstraints.`

Ea are constructorul:

⇒ `GridBagConstraints(int gridx, int gridy,
int gridwidth, int gridheight,
double weightx, double weighty,
int anchor, int fill, Insets insets,
int ipadx, int ipady)`

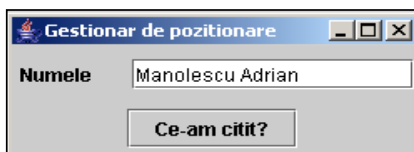
Semnificația parametrilor este următoarea:

- `gridx, gridy` - specifică dreptunghiul care conține componenta. Pentru fiecare dreptunghi din exemplul alăturat am trecut, în această ordine: `gridx, gridy`.

0,0	1,0
0,1	1,1
0,2	1,2

- **gridwidth, gridheight** - numărul de dreptunghiuri pe linie, respectiv pe coloană care sunt ocupate de componentă.
- **weightx** și **weighty** - lățimea ocupată de dreptunghi pe linie, respectiv pe coloană. Acestea nu sunt date în pixeli. Pentru a ușura înțelegerea vom lua ca exemplu linia. Presupunem că avem câte două dreptunghiuri pe linie, ca în exemplul de mai sus. Dacă pentru primul dreptunghi **weightx** reține 1 și pentru al doilea, **weightx** reține 2, atunci lățimea primului dreptunghi este $1/3$ din lățimea finală, iar a celui de-al doilea dreptunghi este $2/3$ din lățimea finală.
- **anchor** - se referă la poziționarea componentei în cadrul dreptunghiului. Poate lua una din valorile: **CENTER**, **NORTH**, **NORTHEAST**, **EAST**, **SOUTHEAST**, **SOUTH**, **SOUTHWEST**, **WEST**, **NORTHWEST**.
- **fill** - se referă la cazul în care componenta ocupă mai mult sau mai puțin spațiu decât dreptunghiul rezervat ei și dorim s-o ajustăm astfel încât să încapă exact. În acest caz se pune problema micșării sau măririi ei. Poate lua valorile: **NONE** (nu se ajustează), **HORIZONTAL** (se ajustează pe lățime, dar înălțimea rămâne nemodificată), **VERTICAL** (se ajustează pe înălțime, dar lățimea rămâne nemodificată), **BOTH** (se ajustează atât lățimea cât și înălțimea).
- **insets** - referință la un obiecte de tip **Insets**. Aceasta are rolul de a adăuga în fiecare parte a componentei un spațiu exterior (dat în pixeli). Constructorul clasei **Insets** are forma **Insets(int sus, int stanga, int jos, int dreapta)**. O componentă care are margini exterioare este separată de altă componentă cel puțin prin acele margini.
- **ipadx, ipady** - adaugă spațiu interior pe orizontală, respectiv pe verticală. Dacă, de exemplu, **ipadx** reține 1, se adaugă de fapt un pixel la stânga și un pixel la dreapta. Diferența între spațiul interior și cel exterior este dată de faptul că se presupune că spațiul interior este al componentei, deci pe această suprafață componenta răspunde la evenimente.

Exemplu: programul următor afișează o fereastră care conține o etichetă, un edit și un buton. Cele trei componente sunt afisate prin utilizarea gestionarului de poziționare **GridBagLayout**. Butonul ocupă două dreptunghiuri, iar raportul dintre spațiul ocupat de etichetă și cel ocupat de edit este de $1/4$. Nu trebuie să vă îngrijoreze faptul că în exemplu s-au utilizat și alte componente. Toate vor fi studiate începând cu paragraful următor.



```

import java.awt.*;
import javax.swing.*;
class Fer extends JFrame
{
    public Fer(String titlu) {
        super(titlu);
        setSize(250,100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        GridBagLayout pozitii=new GridBagLayout();
        JLabel et=new JLabel ("Numele ");
        JTextField txt=new JTextField(15);
        JButton bt=new JButton("Ce-am citit?");
        Container x=getContentPane();
        // pozitionare eticheta
        GridBagConstraints p_et=new
            GridBagConstraints(0,0,1,1,1,2,GridBagConstraints.EAST,
                GridBagConstraints.NONE,new Insets(0,0,0,0), 0,0);
        pozitii.setConstraints(et,p_et);
        // pozitionare edit
        GridBagConstraints p_txt=new
            GridBagConstraints(1,0,1,1,4,2,GridBagConstraints.EAST,
                GridBagConstraints.NONE,new Insets(0,0,0,0), 0,0);
        pozitii.setConstraints(txt,p_txt);
        // pozitionare buton
        GridBagConstraints p_bt=new
            GridBagConstraints(0,1,2,1,1,1,GridBagConstraints.CENTER,
                GridBagConstraints.NONE,new Insets(0,0,0,0), 0,0);
        pozitii.setConstraints(bt,p_bt);
        x.setLayout(pozitii);
        x.add(et); x.add(txt); x.add(bt);
        setVisible(true);
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Gestionar de pozitionare");
    }
}

```

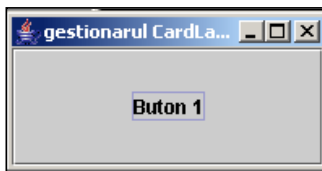
10.6.6. Gestionarul CardLayout

Gestionarul `CardLayout` este unul cu totul aparte. El se caracterizează prin faptul că, la un moment dat, afișează o singură componentă.

- ⇒ `CardLayout()` - constructor;
- ⇒ `next(Container parinte)` - afișează următoarea componentă. Dacă este afișată ultima componentă, se afișează din nou prima componentă;
- ⇒ `last(Container parinte)` - se afișează ultima componentă;

- ⇒ `first(Container parinte)` - se afișează prima componentă;
- ⇒ `addLayout(String nume, Component c)` - se adaugă o componentă căreia i se și atribuie un nume.

Exemplu: în programul următor se afișează cu gestionarul `CardLayout` 3 butoane. Aceasta înseamnă că, la un moment dat, numai unul este vizibil. La apăsarea butonului vizibil se afișează următorul buton, ș.a.m.d.



```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fer extends JFrame implements ActionListener
{
    CardLayout g=new CardLayout();
    Container x=getContentPane();
    public Fer(String titlu) {
        super(titlu); setSize(200,100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        x.setLayout(g);
        JButton A=new JButton("Buton 1"); x.add("Buton 1",A);
        A.addActionListener(this);
        JButton B=new JButton("Buton 2"); x.add("Buton 2",B);
        B.addActionListener(this);
        JButton C=new JButton("Buton 3"); x.add("Buton 3",C);
        C.addActionListener(this); setVisible(true);
    }
    public void actionPerformed (ActionEvent e)
    { g.next(x); }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("gestionarul CardLayout");
    }
}
```

10.7. Studiul principalelor componente

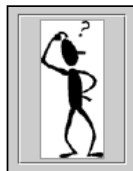
10.7.1. Componente de tip `JButton`

Cu ajutorul componentelor de acest tip se creează butoane. Vom prezenta cele mai importante metode (constructori):

- ⇒ `JButton()` - creează un buton fără text, fără imagine asociată.
- ⇒ `JButton(String text)` - creează un buton care afișează un text.
- ⇒ `JButton(Icon icon)` - creează un buton care reține o imagine de mici dimensiuni.

Exemplu:

```
Icon icon = new ImageIcon("om.gif");
JButton A=new JButton(icon);
```



⇒ `JButton(String text, Icon icon)` - creează un buton cu text și imagine.

```
Icon icon = new ImageIcon("om.gif");
JButton A=new JButton( "Java...",icon);
```



⇒ metoda `setVericalTextPosition(constanta)` setează poziția pe verticală a textului în raport cu imaginea:

- deasupra (`JButton.TOP`),
- dedesubt (`JButton.BOTTOM`),
- în centru (`JButton.CENTER`).

⇒ metoda `setHorizontalTextPosition(constanta)` setează poziția pe orizontală a textului în raport cu imaginea:

- stânga (`JButton.LEFT`),
- dreapta (`JButton.RIGHT`),
- în centru (`JButton.CENTER`).

Exemplu:

```
Icon icon = new ImageIcon("om.gif");
JButton A=new JButton( "Java...",icon);
A.setVerticalTextPosition(JButton.TOP);
A.setHorizontalTextPosition(JButton.CENTER);
```



☞ Mărimea imaginii determină mărimea butonului.

10.7.2. Componente de tip JLabel

Componentele de acest tip se utilizează pentru a afișa texte și imagini, cu rol lămuritor pentru alte componente. Clasa este înzestrată cu următoarele metode mai importante:

⇒ `JLabel(string S)` - se utilizează pentru a afișa un șir de caractere.

Exemplu: dacă constructorul ferestrei conține secvența de mai jos, atunci pe fereastră apare:

```
Container x=getContentPane();
x.setLayout(new BorderLayout ());
JLabel A=new JLabel ("Un text ");
x.add(A);
```



⇒ **JLabel(Icon image)** - se utilizează pentru a afișa o imagine:

Exemplu: la fel ca mai sus, numai că pe fereastră se afișează o imagine:

```
Container x=getContentPane();
x.setLayout(new BorderLayout ());
Icon icon = new ImageIcon("idee.gif");
JLabel A=new JLabel (icon);
x.add(A);
```



⇒ **JLabel(String text, Icon icon, int horizontalAlignment)** afișează o imagine și un text. Ansamblul (imagine + text) se aliniază pe orizontală, conform ultimului parametru (**CENTER**, **LEFT**, **RIGHT**).

⇒ **setVerticalTextPosition(c)**, **setHorizontalTextPosition(c)** pot fi folosite și la **JLabel** la fel ca la **Jbutton**:

Exemplu:

```
Container x=getContentPane();
x.setLayout(new GridLayout());
Icon icon = new ImageIcon("idee.gif");
JLabel A=new JLabel
    ("Asta era!!!",icon,JLabel.CENTER);
A.setHorizontalTextPosition(JLabel.CENTER);
A.setVerticalTextPosition(JLabel.TOP);
x.add(A);
```



10.7.3. Componente de tip JPanel

Componentele **JPanel** (paneluri) sunt de tip container, adică au rolul de a conține pe suprafața lor alte componente. Clasa este înzestrată cu următoarele metode:

⇒ **JPanel()** - constructor, obiectul rezultat are gestionarul de poziționare **FlowLayout()**.

⇒ **JPanel(LayoutManager layout)** - constructor, obiectul rezultat are gestionarul de poziționare transmis ca parametru.

⇒ **setLayout(LayoutManager layout);** - setează gestionarul de poziționare.

⇒ **add(Component c)** - adaugă o componentă obiectului.

Exemplu: fereastra programului următor conține o componentă de tip `JPanel`. Pe ea sunt atașate 3 butoane. La apăsarea unui buton, componenta de tip `JPanel` va avea culoarea afișată de buton.

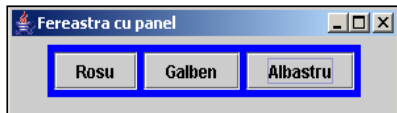
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fer extends JFrame implements ActionListener
{
    JPanel panel;

    public Fer(String titlu) {
        super(titlu);
        setSize(300,300);
        fer.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        panel=new JPanel();
        JButton A=new JButton("Rosu");
        panel.add(A); A.addActionListener(this);
        JButton B=new JButton("Galben");
        panel.add(B); B.addActionListener(this);
        JButton C=new JButton("Albastru");
        panel.add(C);
        C.addActionListener(this);
        x.add(panel);
        setVisible(true);}

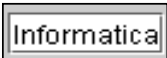
    public void actionPerformed (ActionEvent e)
    {
        String culoare=e.getActionCommand();
        if (culoare.compareTo("Rosu")==0)
            panel.setBackground(Color.red);
        else
            if (culoare.compareTo("Galben")==0)
                panel.setBackground(Color.yellow);
            else
                panel.setBackground(Color.blue);
    }
}

public class cv {
    public static void main(String args[]) {
        Fer fp=new Fer("Fereastra cu panel");
    }
}
```

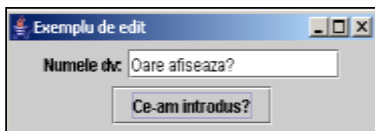


10.7.4. Componente de tip `JTextField`

Componentele de tip `JTextField` (edit-uri) sunt utilizate pentru ca utilizatorul să introducă sau să afișeze șiruri de caractere de la tastatură, șiruri pe care programul urmează să le prelucereze. Cele mai importante metode ale acestui obiect sunt:

- ⇒ **TextField(int nr)** - creează un edit vid, dar care are o lățime suficientă pentru a vizualiza simultan **nr** caractere. Șirul de caractere introdus poate avea orice lungime. 
- ⇒ **TextField(String s)** - creează un edit care inițial afișează un șir de caractere. 
- ⇒ **TextField(String S, int nr)** - creează un edit care inițial afișează un șir de caractere și care are o lățime suficientă pentru a afișa simultan un număr de **nr** caractere.
- ⇒ **String getText()** - returnează șirul de caractere reținut de edit la un moment dat.
- ⇒ **void setText(String s)** - are rolul de a determina ca edit-ul să conțină șirul de caractere specificat.

Exemplul 1. Programul următor afișează fereastra alăturată. Utilizatorul introduce un șir de caractere și, la apăsarea butonului se afișează, în fereastra CMD, textul introdus de acesta.



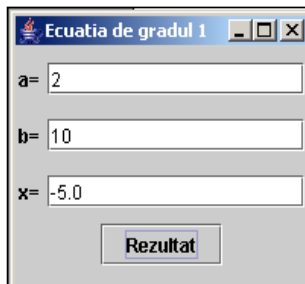
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class Fer extends JFrame implements ActionListener
{
    JTextField txt;
    public Fer(String titlu) {
        super(titlu);
        setSize(300,200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        JLabel et=new JLabel ("Numele dv:");
        x.add(et);
        txt=new JTextField(15);
        x.add(txt);
        JButton but=new JButton("Ce-am introdus?");
        x.add(but);
        but.addActionListener(this);
        setVisible(true);
    }
    public void actionPerformed (ActionEvent e)
    {
        System.out.println(txt.getText());
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Exemplu de edit");
    }
}
```

Exemplul 2. Programul următor rezolvă ecuația de gradul 1 și afișează o fereastră precum cea alăturată.

Rezolvare:

- Pentru a grupa elementele vom utiliza trei componente de tip `JPanel` și un obiect de tip `JButton`. Fiecare componentă de tip `JPanel` va reține o componentă de tip `JLabel` (de ex afișează "a=") și una de tip `JTextField` (de exemplu, pentru introducerea lui a).
- Datele citite sunt validate prin utilizarea mecanismelor de excepție.
- La apăsarea butonului se afișează rezultatul.



```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fer extends JFrame implements ActionListener
{
    JTextField a,b,r; JButton but;

    public Fer(String titlu) {
        super(titlu); setSize(200,200);
        fer.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        //primul panel
        JPanel panel1 = new JPanel();
        JLabel et=new JLabel ("a=");
        panel1.add(et);
        a=new JTextField(15);
        panel1.add(a);
        x.add(panel1);
        //al doilea panel
        JPanel panel2 = new JPanel();
        JLabel et1=new JLabel ("b=");
        panel2.add(et1);
        b=new JTextField(15);
        panel2.add(b);
        x.add(panel2);
        // al treilea panel
        JPanel panel3 = new JPanel();
        JLabel et2=new JLabel ("x=");
        panel3.add(et2);
        r=new JTextField(15);
        panel3.add(r);
        x.add(panel3);
        // butonul Rezulta
        but=new JButton("Rezultat");
        x.add(but);
        but.addActionListener(this);
        setVisible(true);
    }
}
```

```

public void actionPerformed (ActionEvent e)
{
    double aR=0,bR=0,xR=0, er=0;
    try // incerc sa-l citesc pe A
    {
        aR=Double.parseDouble(a.getText());
    }
    catch (NumberFormatException exc)
    {
        er=1;
        System.out.println("a este introdus gresit");
        r.setText("");
    }
    try // incerc sa-l citesc pe B
    {
        bR=Double.parseDouble(b.getText());
    }
    catch (NumberFormatException exc)
    {
        er=1;
        System.out.println("b este introdus gresit");
        r.setText("");
    }
    if (er==0)// daca aR si bR au fost citite
    {
        if (aR!=0)
            r.setText(""+(double)-bR/aR);
        else
            if (bR==0) r.setText("Infinitate de solutii");
            else
                r.setText("Nu are solutie");
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Ecuatia de gradul 1");
    }
}

```

10.7.5. Componente de tip JComboBox

Obiectele clasei **JComboBox** sunt așa numitele liste, din care utilizatorul poate selecta o anumită opțiune (unică). Principalele metode ale clasei **JComboBox** sunt:

- ⇒ **JComboBox ()** – constructor.
- ⇒ **void addItem(Object s)** – adaugă listei o opțiune. Cum clasa **Object** este la baza ierarhiei de clase Java, parametrul efectiv poate fi o referință către orice obiect, inclusiv către tipul des utilizat **String**.
- ⇒ **Object getSelectedItem()** – returnează șirul de caractere corespunzător opțiunii selectate.
- ⇒ **int getItemCount()** – returnează numărul de opțiuni.
- ⇒ **int getSelectedIndex()** – returnează indexul opțiunii selectate.
- ⇒ **Object getItemAt(int ind)** – returnează obiectul reținut de item-ul de indice **ind** (dacă trebuie să obținem un obiect de tip **String**, atunci convertiți către **String** prin operatorul de conversie).

Exemplu: Programul următor afișează o listă. Selectați din listă o opțiune și apăsați butonul. În fereastra CMD va fi afișată opțiunea selectată.

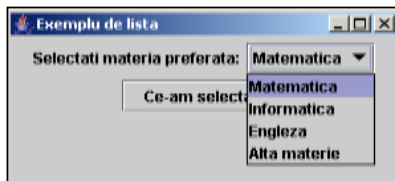
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fer extends JFrame implements ActionListener
{
    JComboBox lista;

    public Fer(String titlu) {
        super(titlu); setSize(300,200);
        fer.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        JLabel et=new JLabel ("Selectati materia preferata:");
        x.add(et);
        lista=new JComboBox ();
        lista.addItem("Matematica");
        lista.addItem("Informatica");
        lista.addItem("Engleza");
        lista.addItem("Alta materie");
        x.add(lista);
        JButton but=new JButton("Ce-am selectat?");
        x.add(but);
        but.addActionListener(this);
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    { System.out.println(lista.getSelectedItem()); }
}

public class cp {
    public static void main(String args[]) {
        Fer fp=new Fer("Exemplu de lista");
    }
}
```



10.7.6. Componente de tip `JCheckBox` și `JRadioButton` și gruparea lor

Obiectele clasei `JCheckBox` sunt așa-numitele butoane de validare. Fereastra poate conține unul sau mai multe astfel de butoane. Principalele metode ale acestei clase sunt:

- ⇒ `JCheckBox(String s)` - creează un buton de validare care conține șirul de caractere transmis ca parametru.
- ⇒ `JCheckBox(String s, boolean v)` - creează un buton de validare care conține șirul de caractere transmis ca parametru și care este marcat sau nu în funcție de valoarea pe care o reține parametrul `v`, respectiv `true` sau `false`.

- ⇒ `JCheckBox(String s, Icon f)` - creează un buton de validare care conține șirul de caractere transmis ca parametru și o imagine.
- ⇒ `boolean isSelected()` - returnează `true` dacă butonul a fost selectat și `false` în caz contrar.
- ⇒ `String getText()` - returnează șirul de caractere reținut de buton.

Exemplu. Programul următor afișează mai multe butoane de validare, din care unul (cel care afișează Informatica) este selectat de la început! La apăsarea butonului **Ce-am selectat?** este afișată selecția făcută.

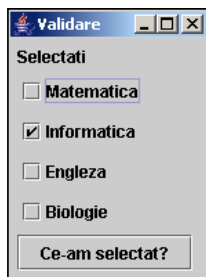
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fer extends JFrame implements ActionListener
{
    JCheckBox b1,b2,b3,b4;

    public Fer(String titlu) {
        super(titlu);
        setSize(150,200);
        Container x=getContentPane();
        x.setLayout(new FlowLayout (FlowLayout.LEFT));
        fer.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JLabel et=new JLabel ("Selectati");
        x.add(et);
        b1=new JCheckBox("Matematica");x.add(b1);
        b2=new JCheckBox("Informatica",true);x.add(b2);
        b3=new JCheckBox("Engleza"); x.add(b3);
        b4=new JCheckBox("Biologie");x.add(b4);
        JButton but=new JButton("Ce-am selectat?");
        x.add(but);
        but.addActionListener(this);
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    {
        if (b1.isSelected()) System.out.println(b1.getText());
        if (b2.isSelected()) System.out.println(b2.getText());
        if (b3.isSelected()) System.out.println(b3.getText());
        if (b4.isSelected()) System.out.println(b4.getText());
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Validare");
    }
}
```



- ⇒ Obiectele de tip `JRadioButton` sunt asemănătoare cu cele de tip `JCheckBox`, negrupate au același comportament, numai că arată ca niște butoane radio.

Butoanele pot fi grupate, adică pot fi toate tratate ca un întreg. Un exemplu convingător este dat de necesitatea grupării butoanelor radio (unde, la un moment dat, numai un singur buton poate fi selectat). Pentru a grupa butoanele se folosește un obiect al clasei **ButtonGroup**. Cele mai importante metode ale acestei clase sunt:

- ⇒ **ButtonGroup()** - constructor;
- ⇒ **void add(AbstractButton b)** - adaugă un buton grupului (clasa **AbstractButton** este superclasă pentru **JButton**, **JRadioButton**, **JCheckBox**).
- ⇒ **Enumeration getElements();** - returnează un pointer la grupul de butoane.
- ⇒ **int getButtonCount()** - returnează numărul de butoane din grup.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;

class Fer extends JFrame implements ActionListener
{
    ButtonGroup grup;

    public Fer(String titlu) {
        super(titlu); setSize(150,200);
        fer.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout(FlowLayout.LEFT));
        JLabel et=new JLabel ("Selectati"); x.add(et);
        JRadioButton b1=new JRadioButton("Matematica"); x.add(b1);
        JRadioButton b2=new JRadioButton("Informatica",true); x.add(b2);
        JRadioButton b3=new JRadioButton("Engleza"); x.add(b3);
        JRadioButton b4=new JRadioButton("Biologie"); x.add(b4);
        JButton but=new JButton("Ce-am selectat?"); x.add(but);
        but.addActionListener(this);
        grup = new ButtonGroup();
        grup.add(b1);grup.add(b2);
        grup.add(b3);grup.add(b4);
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    {
        Enumeration enum=grup.getElements();
        while (enum.hasMoreElements())
        {
            JRadioButton b=(JRadioButton)enum.nextElement();
            if(b.isSelected()) System.out.println(b.getText());
        }
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Validare");
    }
}
```

10.7.7. Meniuri

În **Java**, meniurile se construiesc prin utilizarea obiectelor mai multor clase:

A) Clasa JMenuBar - obiectele acestei clase se utilizează pentru a obține meniuri clasice, aflate în partea de sus a ferestrei. Cele mai importante metode ale acestei clase sunt:

- ⇒ **JMenuBar()** - constructor;
- ⇒ **add(JMenu)** - adaugă meniului componente de tip **JMenu**, vedeți în continuare.

B) Clasa JMenu - obiectele acestei clase au rolul de a înscrie în meniu anumite opțiuni, care la rândul lor pot conține alte opțiuni. Cele mai importante metode ale acestei clase sunt:

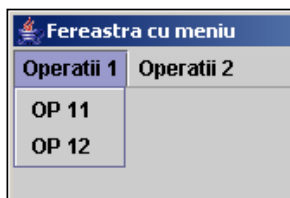
- ⇒ **JMenu(String s)** - constructor. Obiectul va afișa șirul **s**
- ⇒ **add(JMenuItem meniu)** - adaugă o componentă de tip **JMenuItem**. Vedeți mai jos clasa **JMenuItem**;
- ⇒ **add(Component c)** - adaugă o altă componentă (de un tip derivat din tipul **Component**).

C) Clasa JMenuItem - o astfel de componentă care apare în meniu, arată ca o opțiune și are rolul unui buton, adică selectată, conduce la o anumită acțiune. Cele mai importante metode ale acestei clase sunt:

- ⇒ **JMenuItem(String s)** - constructor, opțiunea afișează șirul **s**;
- ⇒ **JMenuItem(Icon ic)** - constructor, opțiunea afișează o mică imagine;
- ⇒ **JMenuItem(String s, Icon ic)** - constructor, opțiunea afișează un șir de caractere și o mică imagine.

Exemplu: programul următor construiește un meniu clasic. Pentru fiecare opțiune selectată se afișează în fereastra **CMD** șirul afișat de opțiune.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```



```

class Fer extends JFrame implements ActionListener
{
    JTextArea t;
    public Fer(String titlu) {
        super(titlu);
        setSize(400,400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        //Bara meniu
        JMenuBar bara = new JMenuBar();
        // Creez meniul principal
        JMenu meniul = new JMenu("Operatii 1");
        bara.add(meniul);
        JMenu meniul2 = new JMenu("Operatii 2");
        bara.add(meniul2);
        // adaug itemi pentru fiecare optiune din meniul principal
        JMenuItem item11 = new JMenuItem("OP 11");
        item11.addActionListener(this);
        meniul.add(item11);
        JMenuItem item12 = new JMenuItem("OP 12");
        item12.addActionListener(this);
        meniul.add(item12);
        JMenuItem item21 = new JMenuItem("OP 21");
        item21.addActionListener(this);
        meniul2.add(item21);
        setJMenuBar(bara);
        Container x=getContentPane();
        x.setLayout(new GridLayout ());
        setVisible(true);
    }
    public void actionPerformed (ActionEvent e)
    {
        System.out.println(e.getActionCommand());
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Fereastra cu meniu");
    }
}

```



Metoda clasei `JFrame` `setJMenuBar(JMenuBar b)` are rolul de a atașa bara de meniu ferestrei.

Meniurile pot conține și submeniuri. Pentru aceasta, unei componente de tip `JMenu` i se adaugă, pe lângă alte componente de tip `JMenuItem`, o altă componentă, tot de tip `JMenu`. Acestea din urmă i se adaugă componente de tip `JMenuItem`.

Exemplu: pentru a obține meniul alăturat, adăugați programului de mai sus secvența următoare:

```

//submeniuri
JMenu altele = new JMenu("Altele");
meniul.add(altele);

```




```

JMenuItem item131 = new JMenuItem("OP 131");
altele.add(item131);
item131.addActionListener(this);

JMenuItem item132 = new JMenuItem("OP 132");
altele.add(item132);
item132.addActionListener(this);

```



Evident, procedeul se poate repeta pentru a obține oricâte submeniuri doriți.

Acceleratori. În multe aplicații, anumite opțiuni din meniuri sunt înzestrate cu acceleratori (shortcut-uri de meniu) prin care o anumită comandă se poate obține fie din meniu, fie dintr-o combinație de taste. Și în Java se pot crea acceleratori și acest mecanism este prezentat în continuare. Metoda:

```
void setAccelerator(KeyStroke keyStroke)
```

are rolul de a atașa unui obiect de tip `JMenuItem` un accelerator, adică de a invoca acțiunea specifică acestuia atunci când utilizatorul a apăsât fie o tastă, fie o combinație de taste. În primul caz (o tastă), se folosește metoda:

```
static KeyStroke getKeyStroke(char keyChar),
```

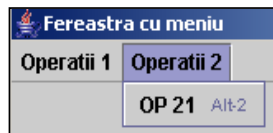
iar în al doilea caz metoda:

```
static KeyStroke getKeyStroke(int keyCode, int modifiers)
```

Cum obținem parametrii pentru a doua metodă `getKeyStroke()`?

- ❖ Primul parametru este o constantă a clasei `KeyEvent`. Iată câteva astfel de constante: `static int VK_1` (tasta 1), `static int VK_2` (tasta 2), `static int VK_F1` (tasta F1), `static int VK_V` (tasta v) s.a.m.d.
- ❖ Al doilea parametru este o constantă a clasei `ActionEvent`: `SHIFT_MASK` (codul 1), `CTRL_MASK` (codul 2), `META_MASK` (codul 4), `ALT_MASK` (codul 8).

Exemplu: în primul program de la începutul paragrafului, pentru `item21` doresc să am un accelerator, adică să obțin același efect prin apăsarea combinației de taste `ALT+2` ca și în cazul în care operăm selecția din meniu:



```

JMenuItem item21 = new JMenuItem("OP 21");
item21.setAccelerator(KeyStroke.getKeyStroke(
    KeyEvent.VK_2, ActionEvent.ALT_MASK));
item21.addActionListener(this);

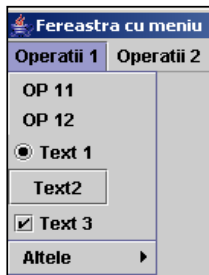
```

În acest caz, dacă apăsăm **ALT+2**, obținem același efect ca atunci când operăm selecția din meniu. Dacă dorim să obținem o combinație din trei taste, de exemplu **CTRL+ALT+2**, vom folosi (datorită codificării din **ActionEvent**):

```
getKeyStroke( KeyEvent.VK_2, ActionEvent.ALT_MASK
+ ActionEvent.CTRL_MASK));
```

Alte tipuri de componente. Unui meniu i se pot adăuga și alte componente dintre cele prezentate. Iată un exemplu în care se adaugă în meniu mai multe butoane:

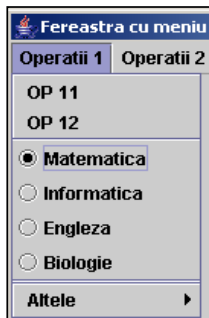
```
JRadioButton b1=new JRadioButton("Text 1");
meniul.add(b1);
JButton b2=new JButton("Text2");
meniul.add(b2);
JCheckBox b3=new JCheckBox("Text 3");
meniul.add(b3);
```



Grupuri de butoane. Meniurilor li se pot adăuga grupuri de butoane (revedeți Gruparea butoanelor). Mai mult, pentru a identifica un grup în meniu, se utilizează separatori (linii orizontale) care sunt obiecte ale clasei **JSeparator()**.

Exemplu:

```
meniul.add(new JSeparator());
JRadioButton b1=new JRadioButton("Matematica");
meniul.add(b1);
JRadioButton b2=new JRadioButton("Informatica");
meniul.add(b2);
JRadioButton b3=new JRadioButton("Engleza");
meniul.add(b3);
JRadioButton b4=new JRadioButton("Biologie");
meniul.add(b4);
meniul.add(new JSeparator());
ButtonGroup grup = new ButtonGroup();
grup.add(b1);
grup.add(b2);
grup.add(b3);
grup.add(b4);
```



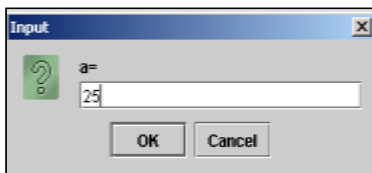
10.7.8. Cutii (casete) de dialog predefinite

Cutiile de dialog predefinite sunt extrem de utile pentru introducerea sau afișarea unor date. Pentru a putea folosi astfel de cutii, vom utiliza clasa **JOptionPane** care conține mai multe *metode statice* care afișează astfel de cutii. Cutiile sunt predefinite, conținând automat anumite componente (nu mai este necesar ca acestea să fie create de programator).

10.7.8.1. Cutii de dialog de intrare

Iată câteva metode prin care putem apela astfel de cutii:

- ⇒ `public static String showInputDialog(Object mes)` creează și afișează în centrul ecranului o cutie de dialog care afișează mesajul transmis ca parametru. În cazul în care utilizatorul apasă **Cancel**, se returnează pointer-ul `null`, motiv pentru care trebuie tratată această excepție.



Exemplu:

```
try {
    String intr = JOptionPane.showInputDialog("a=");
    if (intr.length()==0)
        System.out.println("Intr vida");
    else System.out.println(intr);}
catch (NullPointerException exc) {
    System.out.println("S-a apasat Cancel");}
```

- ⇒ `public static String showInputDialog(Object mes, Object val)` la fel ca mai sus, numai că edit-ul memorează de la început valoarea transmisă de al doilea parametru (valoare inițială). În exemplul de mai jos, valoarea inițial afișată este "0".

```
String intr = JOptionPane.showInputDialog("a=", "0");
```

- ⇒ `public static String showInputDialog(Component par, Object mes)` - ca mai sus, numai că se afișează o cutie de dialog care are o componentă părinte. În exemplul următor, afișarea se face cu ajutorul clasei care descrie fereastra principală, deci părintele este `this` (acest obiect):

```
String intr = JOptionPane.showInputDialog(this, "a=");
```

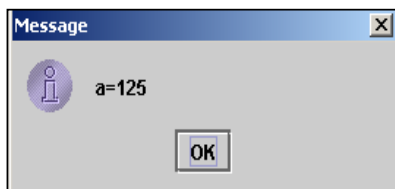
- ⇒ `public static String showInputDialog(Component par, Object mes, Object val)` - ca mai sus, numai că se afișează o cutie de dialog în care edit-ul reține și o valoare inițială.

10.7.8.2. Cutii de dialog pentru afișarea mesajelor

Astfel de cutii de dialog se utilizează pentru a transmite utilizatorului anumite mesaje de eroare, sau rezultate.

- ⇒ `public static void showMessageDialog(Component par, Object mes)` - afișează o cutie de dialog cu un mesaj dorit. Exemplu: clasa asociată ferestrei se găsește secvența:

```
int a=125;  
JOptionPane.showMessageDialog(this  
, "a="+a);
```



10.7.8.3. Cutii de dialog de confirmare

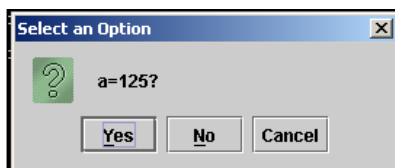
O astfel de cutie de dialog afișează trei butoane **Yes**, **No** și **Cancel**.

⇒ `public static int showConfirmDialog(Component par, Object mes);` Metoda returnează un întreg a cărui valoare este dată de butonul pe care-l apasă utilizatorul: 0 pentru **Yes**, 1 pentru **No** sau 2 pentru **Cancel**.

🔍 **Observație:** de fapt, se returnează indicele butonului apăsat: **Yes** are 0 pentru că este primul buton, **No** are 1 pentru că este al doilea buton, iar **Cancel** are 2 deoarece este al treilea buton.

Exemplu: secvența următoare testează, prin afișare, valorile returnate de `showConfirmDialog()`:

```
int a=125;  
int rasp=  
JOptionPane.showConfirmDialog(this,"a="+a+"?");  
System.out.println(rasp);
```



10.7.8.4. Cutii de dialog cu opțiuni

Aceste cutii de dialog sunt mai complexe. Ele permit să selectăm un set de butoane (vom vedea cum) și să le schimbăm numele conform propriilor dorințe!

⇒ `public static int showOptionDialog(Component f_par, Object mesaj, String titlu, int tip_optiune, int tip_mesaj, Icon icon, Object[] vect, Object buton_selectat_initial)`

Parametrii metodei sunt:

- ❖ `Component f_par` - fereastra părinte;
- ❖ `Object mesaj` - șirul de caractere care apare afișat în cutie;
- ❖ `String titlu` - titlul ferestrei;

- ❖ `int tip_optiune` - se alege una dintre opțiunile următoare pentru a preciza tipul butoanelor (`DEFAULT_OPTION`, `YES_NO_OPTION`, `YES_NO_CANCEL_OPTION`, `OK_CANCEL_OPTION`);
- ❖ `int tip_mesaj` - are ca efect selectarea unei mici imagini care este afișată pe cutie. Astfel avem valorile:



ERROR_MESSAGE



INFORMATON_MESSAGE



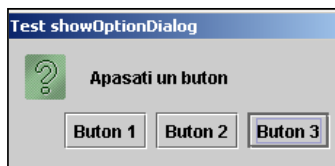
WARNING_MESSAGE



QUESTION_MESSAGE

- ❖ `Icon icon` - icon-ul ferestrei;
- ❖ `Object[] vect` - vector de obiecte cu numele butoanelor;
- ❖ `Object buton_selectat_initial` - butonul care este selectat inițial.

Exemplu: în programul următor se afișează o astfel de cutie în care numele butoanelor sunt buton 1, buton 2, buton 3. La apăsarea unui buton se afișează un mesaj care precizează numele butonului apăsător.



```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fer extends JFrame implements ActionListener
{
    public Fer(String titlu) {
        super(titlu); setSize(200,100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane(); x.setLayout(new FlowLayout ());
        JButton A=new JButton("Apasa"); x.add(A);
        A.addActionListener(this); setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    {
        Object[] butoane = {"Buton 1","Buton 2","Buton 3"};
        int n = JOptionPane.showOptionDialog(this,
            "Apasati un buton", "Test showOptionDialog",
            JOptionPane.YES_NO_CANCEL_OPTION,
            JOptionPane.QUESTION_MESSAGE, null, butoane, butoane[2]);
        if (n==0) JOptionPane.showMessageDialog(this, " butonul 1");
        else
            if (n==1) JOptionPane.showMessageDialog(this, " butonul 2");
            else
                JOptionPane.showMessageDialog(this, " butonul 3");
    }
}

public class pvl {
    public static void main(String args[]) {
        Fer fp=new Fer("Fereastra cu buton") }
}
```

10.7.9. Componente de tip JTextArea

Componentele de acest tip permit utilizatorului să introducă text pe mai multe linii. Se poate utiliza un singur font, de o singură mărime! Un astfel de text, cu un singur font, care nu permite să fie formatat se numește text plan.

Cele mai importante metode ale clasei sunt:

- ⇒ **JTextArea()** - constructor.
- ⇒ **void append(String S)** - adaugă la sfârșitul șirului reținut de componentă un alt șir.
- ⇒ **void insert(String s, int pozitie)** - inserează în text, începând de la **pozitie** un șir de caractere. Dacă **pozitie** depășește indicele ultimului caracter, atunci inserarea se face la sfârșitul șirului reținut de componentă.
- ⇒ **void replaceRange(String str, int start, int end)** - înlocuiește șirul dintre indicii **start** și **end** cu șirul transmis ca parametru. Evident, dacă acesta din urmă este vid, se șterge întreg textul.
- ⇒ **setFont(Font f)** - setează fontul utilizat.
- ⇒ **String getText()** - returnează șirul de caractere conținut de componentă (inclusiv dacă el conține **newline**, altfel spus, mai multe paragrafe).
- ⇒ **void setText(String S)** - determină ca obiectul de tip **JTextArea** să rețină șirul **S**.
- ⇒ **void setBackground(Color c)** - setează culoarea de fond a suprafeței care afișează literele.
- ⇒ **String getSelectedText()** - returnează șirul de caractere selectat.
- ⇒ **void selectAll()** - selectează întreg textul reținut de componentă.
- ⇒ **int getSelectionStart()** - returnează indicele primului caracter selectat.
- ⇒ **int getSelectionEnd()** - returnează indicele ultimului caracter selectat.
- ⇒ **void setLineWrap(boolean v)** - dacă reține **true**, setează obiectul ca textul scris fără **newline** (adică un singur paragraf) să se afișeze pe mai multe rânduri (în cazul în care nu încapă pe lățime). Dacă reține **false**, acesta este afișat pe o singură linie, chiar dacă, la un moment dat, nu este toată vizibilă.

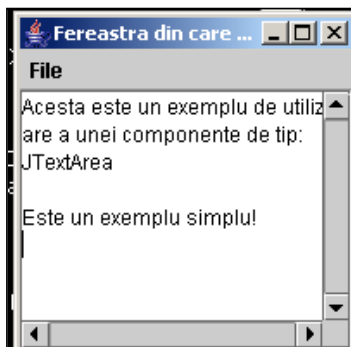
Obiectele de tip `JTextArea` nu se afișează pe fereastră în mod direct, ci prin intermediul altor obiecte de tip `JScrollPane`, asemănătoare obiectelor de tip `JPanel`, dar care permit atașarea unor bare de scroll, așa cum se vede mai jos:

```
JScrollPane panel = new JScrollPane(t);  
panel.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);  
panel.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
```

Exemplu: programul următor afișează o componentă de tip `JTextArea` și un meniu. Utilizatorul introduce un text la alegerea sa. Textul poate fi salvat cu ajutorul meniului (**File** și **Save**) și poate fi recuperat tot cu ajutorul meniului (**File** și **Open**). Pentru simplificarea programului fișierul în care se salvează este întotdeauna `test.txt`. Înainte de a studia programul, recomand o scurtă recapitulare a lucrului cu fișiere.

```
import java.io.*;  
import java.util.*;  
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;
```

```
class Fer extends JFrame implements ActionListener  
{  
    JTextArea t;  
    public Fer(String titlu) {  
        super(titlu); setSize(200,200);  
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        Container x=getContentPane();  
        x.setLayout(new GridLayout());  
        JMenuBar bara = new JMenuBar();  
        JMenu meniul = new JMenu("File");  
        bara.add(meniul);  
        JMenuItem item11 = new JMenuItem("Open");  
        item11.addActionListener(this); meniul.add(item11);  
        JMenuItem item12 = new JMenuItem("Save");  
        item12.addActionListener(this); meniul.add(item12);  
        setJMenuBar(bara);  
        t=new JTextArea(); t.setLineWrap(true);  
        JScrollPane panel = new JScrollPane(t);  
        panel.setVerticalScrollBarPolicy(  
            JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);  
        panel.setHorizontalScrollBarPolicy(  
            JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);  
        x.add(panel);  
        setVisible(true);  
    }  
}
```



```

void scriuFisier() throws IOException
{
    FileOutputStream f=new FileOutputStream("test.txt");
    PrintStream fchar=new PrintStream (f);
    StringTokenizer tok=new StringTokenizer(t.getText(),"\n");
    while (tok.hasMoreTokens()) fchar.println(tok.nextToken());
    fchar.close();
}

void citFisier() throws IOException
{
    FileInputStream f=new FileInputStream("test.txt");
    InputStreamReader fchar=new InputStreamReader (f);
    BufferedReader buf = new BufferedReader(fchar);
    t.setText(""); String linie;
    while ((linie=buf.readLine())!=null) t.append(linie+"\n");
}

public void actionPerformed (ActionEvent e)
{
    if ( e.getActionCommand().compareTo("Save")==0)
        // Daca s-a selectat Save
        { try {scriuFisier(); }
          catch (Exception excp) {} }
    else
        // Daca s-a selectat Open
        { try {citFisier(); }
          catch (Exception excp) {} }
}
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Fereastra din care pot introduce textul");
    }
}

```

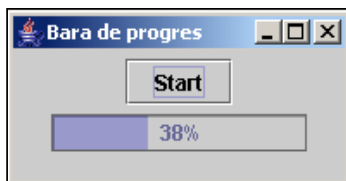
10.7.10. Componente de tip JProgressBar

Astfel de componente, numite bare de progres, se utilizează atunci când programul efectuează anumite operații consumatoare de timp și se dorește informarea utilizatorului asupra timpului rămas pentru executarea operației respective. Clasa **JProgressBar** conține următoarele metode mai importante:

- ⇒ **JProgressBar()** - constructor;
- ⇒ **void setMinimum(int a)** - memorează valoarea la care componenta indică faptul că nu s-a efectuat nimic;
- ⇒ **void setMaximum(int b)** - memorează valoarea la care componenta indică faptul că operația este terminată;
- ⇒ **void setValue(int c)** - setează valoarea pe care o reține componenta la un moment dat. Ea trebuie să fie între valorile **a** și **b**, setate cu **setMinimum()** și **setMaximum()**.

- ⇒ `void setStringPainted(boolean b)` - dacă primește ca parametru `true`, setează bara ca să afișeze un șir de caractere.
- ⇒ `void setString(String s)` - setează șirul de caractere care va fi afișat de bară. Evident, trebuie ca să fi fost executată metoda anterioară cu parametru `true`.

Exemplu: programul următor, la apăsarea butonului `Start`, "pune în mișcare" o bară de progres. Înainte de a-l studia, revedeți *fire de execuție*. Bara "se mișcă" într-un fir de execuție separat de program. Lansarea firului de execuție se face la apăsarea butonului. Firul de execuție este oprit, temporar, la fiecare actualizare a valorii reținute de bară.



```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class AcBara extends Thread {
    JProgressBar bara;
    void ref(JProgressBar bara)
    { this.bara=bara; }
    public void run()
    { for (int i=0;i<=1000;i++)
        { bara.setValue(i);
          bara.setString(""+i/10+"%");
          try {sleep(10);}
          catch (Exception e) { }
        }
    }
}

class Fer extends JFrame implements ActionListener
{ JProgressBar bara; AcBara t;
  public Fer(String titlu) {
    super(titlu);
    setSize(200,100);
    Container x=getContentPane();
    x.setLayout(new FlowLayout ());
    JButton A=new JButton("Start"); x.add(A);
    A.addActionListener(this);
    bara = new JProgressBar();
    bara.setMinimum(0); bara.setMaximum(1000);
    bara.setStringPainted(true);
    x.add(bara);
    setVisible(true);
  }
  public void actionPerformed (ActionEvent e)
  { if (t==null)
    { t= new AcBara(); t.ref(bara); t.start(); }
  }
}
```

```
public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Bara de progres");
        WindowListener t=new Evfer();
        fp.addWindowListener(t);
    }
}
```



Întrebare: de ce a fost necesar ca “punerea în mișcare” a barei să se facă într-un fir de execuție și n-am descris secvența de instrucțiuni în metoda care se rulează la apăsarea butonului (`actionPerformed()`) ?

Răspuns: chiar merită încercat! Exercițiu! Dar, veți rămâne dezamăgiți. Practic, când apăsați butonul, acesta rămâne apăsat până se desfășoară întreaga activitate consumatoare de timp. În acest timp bara rămâne nemodificată. La sfârșit, ea va indica brusc 100%. De ce?

Cum credeți că se actualizează valoarea arătată de bară atunci când se utilizează `setValue()`? Nu este o simplă memorare, așa cum pare la prima vedere! Se memorează valoarea și se generează un eveniment, în urma căruia se declanșează o metodă a barei care actualizează desenul pentru a indica progresul. Atunci când apăsam butonul se declanșează un eveniment. Până când acesta nu este tratat, adică până când nu se rulează metoda `actionPerformed()`, celelalte evenimente sunt în așteptare. Prin urmare, la sfârșit, sunt tratate rapid toate celelalte evenimente de redesenare a barei, dar acțiunea durează atât de puțin, încât noi vedem că bara arată deodată 100%.

10.7.11. Componente de tip `JSpinner`

Componentele de tip `JSpinner` sunt un tip special de liste, în care o valoare se poate selecta cu ajutorul unor săgeți. Principalele metode ale acestei clase sunt:

- ⇒ `JSpinner(SpinnerModel model)` - constructor, unde parametrul `SpinnerModel` este o interfață.
- ⇒ `void setValue(Object value)` - componenta va afișa valoarea setată. Atenție la parametrul, este de tip `Object`. Această clasă stă la baza ierarhiei de clase din Java. Prin urmare, parametrul poate primi o referință către orice obiect, în particular pentru un întreg. Pentru a folosi metoda vom crea un obiect de tip `Integer` pe care îl transmitem ca parametru.

Exemplu:

```
s =new JSpinner (m);
s.setValue(new Integer (5));
```

⇒ `object getValue()` - returnează valoarea selectată de utilizator. Valoarea selectată este de tip `Object`. Pentru a o transforma într-un întreg, vom proceda ca mai jos:

Exemplu: pentru componenta `JSpinner s`;

```
int k=((Integer) s.getValue()).intValue();
```

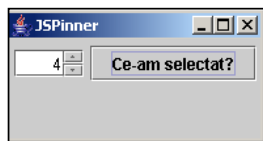
unde `=(Integer) s.getValue()` este o referință către un obiect `Integer`, căreia i se aplică metoda `intValue()` care returnează valoarea reținută de ea.

O implementare a interfeței `SpinnerModel` este dată de clasa `SpinnerNumberModel`, care are constructorul:

```
SpinnerNumberModel(int v_initala, int v_min,  
                    int v_max, int pas)
```

Constructorul setează o plajă de valori cuprinsă între `v_min` și `v_max`, o valoare care va fi afișată inițial de componentă (`v_initala`) și pasul de afișare al valorilor (`pas`).

Exemplu: programul următor afișează o componentă de tip `JSpinner` și un buton. La apăsarea butonului se afișează valoarea selectată în componentă.

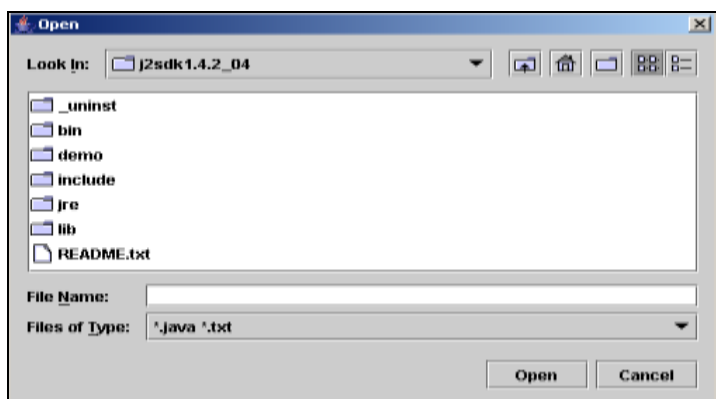


```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class Fer extends JFrame implements ActionListener
{
    JSpinner s;
    public Fer(String titlu) {
        super(titlu); setSize(200,100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout());
        SpinnerNumberModel m= new SpinnerNumberModel (0,-10, 10, 1);
        s =new JSpinner (m); x.add(s);
        JButton A=new JButton("Ce-am selectat?");
        x.add(A);
        A.addActionListener(this);
        setVisible(true);
    }
    public void actionPerformed (ActionEvent e)
    { System.out.println(((Integer) s.getValue()).intValue());}
}
public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("JSpinner");
    }
}
```

10.7.12. Cutii de tip **Open** / **Save**

Astfel de cutii se utilizează în operațiile de încărcare și salvare a fișierelor. Cu ajutorul lor se selectează fișierele și folder-urile necesare acestor operații. Ele pot fi obținute cu metodele clasei `JFileChooser()`. Cele mai importante metode ale acestei clase sunt:

- ⇒ `JFileChooser()` - constructor;
- ⇒ `int showOpenDialog(Component fer_parinte)` - afișează o cutie de dialog de tip **Open**, așa cum este cea de mai jos:



Valoarea returnată este 0, dacă s-a apăsât butonul **Open** sau este 1 dacă s-a apăsât butonul **Close**. Rolul ei este ca, atunci când a fost selectat un anumit fișier și s-a apăsât **Open**, obiectul de tip `JFileChooser` să conțină date despre fișierul selectat.

- ⇒ `int showSaveDialog(Component fer_parinte)` - afișează o cutie de dialog de tip **Save**, asemănătoare cu cutia precedentă. Valoarea returnată este 0 dacă s-a apăsât butonul **Save** și 1 dacă s-a apăsât butonul **Cancel**. Ca și cutia precedentă (de tip **Open**), rolul ei este ca obiectul de tip `JFileChooser` să conțină date despre fișierul salvat.



Atenție: cutia de tip **Open** nu deschide fișierul, iar cutia de tip **Save** nu-l salvează. Rolul lor este de a obține informațiile necesare deschiderii și salvării fișierelor.

- ⇒ `File getSelectedFile()` - metoda returnează, atât în cazul salvării, cât și în cazul deschiderii fișierelor, date despre fișierul selectat (furnizate de `showOpenDialog()` și `showSaveDialog()`). Pentru a putea accesa aceste informații, trebuie să studiem câteva metode ale tipului `File`.

- ⇒ `void addChoosableFileFilter(Filter x);` - metoda se utilizează pentru a adăuga un filtru cutiilor de tip **Open** și **Save**. Despre filtre vom învăța în cadrul acestui paragraf.
- ⇒ `setCurrentDirectory(File f)` - setează cutia astfel încât să se deschidă într-un anumit folder.

Clasa File. Cele mai importante metode ale acestei clase sunt:

- ❖ `File()` - constructor;
- ❖ `File(String nume)` - construiește un obiect care conține numele unui fisier (folder).
- ❖ `boolean isDirectory()` - returnează **true** dacă fișierul reținut de obiect este folder (folder-ele sunt fișiere speciale), sau **false** în caz contrar.
- ❖ `String getName()` - returnează numele fișierului.
- ❖ `String getPath()` - returnează numele întreg al fișierului, inclusiv calea către el.

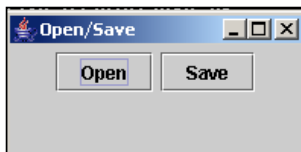
Filtre. În partea de jos a cutiilor de tip **Open** sau **Save** se găsește o listă (**File Name**). Dacă programul nostru nu conține un filtru, implicit se afișează doar opțiunea **All Files**, adică cutiile vizualizează toate folder-ele și toate fișierele, indiferent de tipul lor. În cazul în care dorim ca programul să conțină filtre, adică să se vizualizeze doar fișierele de un anumit tip (sau tipuri), vom utiliza clasa **FileFilter**.

Clasa FileFilter este **abstractă**. Pentru a crea un filtru este necesar să rescriem două metode ale sale:

- ⇒ `public String getDescription()` - trebuie să returneze un șir de caractere care va fi afișat de lista **File of Type**, listă conținută de cutiile **Open** și **Save**.
- ⇒ `public boolean accept(File f)` - metodă care returnează **true** pentru un fișier care, fie are o extensie dorită, fie este de tip folder, altfel returnează **false**. Dacă metoda returnează **true**, fișierul este aflat de cutiile **Open/Save**, altfel acesta nu este afișat.

Exemple de utilizare a cutiilor **Open** și **Save**:

1. Programul următor afișează o fereastră cu două butoane: **Open** și **Save**. La apăsarea unuia dintre ele se afișează cutia respectivă. După ce utilizatorul selectează fișierul (pentru **Open**) sau selectează folder-ul și scrie în editul corespunzător numele fișierului, în fereastra **CMD** se va afișa numele fișierului selectat, inclusiv calea. Cutiile **Open** și **Save** vor afișa doar folder-ele și fișierele cu extensiile **.java** și **.txt**.



```

import java.io.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Filtre extends javax.swing.filechooser.FileFilter {
    // redefinesc cele doua functii pentru filtru
    public boolean accept(File f) {
        if (f.isDirectory()) { return true;}
        String nume_fis = f.getName();
        return nume_fis.endsWith(".java") || nume_fis.endsWith(".txt") ;
    }

    public String getDescription() {
        return "*.java" + " "+"*.txt" ;
    }
}

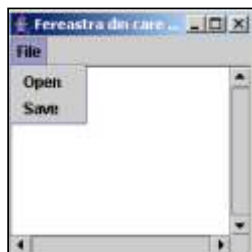
class Fer extends JFrame implements ActionListener
{
    public Fer(String titlu) {
        super(titlu); setSize(200,100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        JButton A=new JButton("Open"); x.add(A);
        JButton B=new JButton("Save"); x.add(B);
        A.addActionListener(this);
        B.addActionListener(this);
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    {
        if (e.getActionCommand().compareTo("Open")==0)
            // daca am apasat Open
            {
                JFileChooser c_op = new JFileChooser();
                c_op.addChoosableFileFilter(new Filtre());
                int valRet = c_op.showOpenDialog(this);
                if (valRet==0)
                    {
                        File f=c_op.getSelectedFile();
                        System.out.println(f.getPath());
                    }
            }
        else
            // daca am apasat Save
            {
                JFileChooser c_save = new JFileChooser();
                c_save.addChoosableFileFilter(new Filtre());
                int valRet = c_save.showSaveDialog(this);
                if (valRet==0)
                    {
                        File f=c_save.getSelectedFile();
                        System.out.println(f.getPath());
                    }
            }
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Open/Save");
    }
}

```

2. Programul de mai jos este un editor primitiv de fișiere text. El nu conține în meniu decât două opțiuni: **Open** și **Save**. Oricare dintre ele are ca efect afișarea cutiilor respective și salvarea sau deschiderea cu afișare a unui fișier cu una din extensiile `.java` și `.txt`. Afișarea fișierului se face cu ajutorul unei componente de tip `JTextArea`.



```
import java.io.*;
import java.util.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Filte extends javax.swing.filechooser.FileFilter
{
    public boolean accept(File f) {
        if (f.isDirectory()) {return true;}
        String nume_fis = f.getName();
        return nume_fis.endsWith(".java") || nume_fis.endsWith(".txt");
    }

    public String getDescription() {
        return "*.java" + " " + "*.txt" ;
    }
}

class Fer extends JFrame implements ActionListener
{
    JTextArea t;

    public Fer(String titlu) {
        super(titlu); setSize(200,200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new GridLayout());
        JMenuBar bara = new JMenuBar();
        JMenu meniul = new JMenu("File"); bara.add(meniul);
        JMenuItem item11 = new JMenuItem("Open");
        item11.addActionListener(this); meniul.add(item11);
        JMenuItem item12 = new JMenuItem("Save");
        item12.addActionListener(this); meniul.add(item12);
        setJMenuBar(bara);
        t=new JTextArea(); t.setLineWrap(true);
        JScrollPane panel = new JScrollPane(t);
        panel.setVerticalScrollBarPolicy
            (JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
        panel.setHorizontalScrollBarPolicy
            (JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
        x.add(panel); setVisible(true);
    }

    void scriuFisier(String nume) throws IOException
    {
        FileOutputStream f=new FileOutputStream(nume);
        PrintStream fchar=new PrintStream (f);
        StringTokenizer tok=new StringTokenizer(t.getText(),"\n");
        while (tok.hasMoreTokens()) fchar.println(tok.nextToken());
        fchar.close();
    }
}
```

```

void citFisier(String nume) throws IOException
{
    FileInputStream f=new FileInputStream(nume);
    InputStreamReader fchar=new InputStreamReader(f);
    BufferedReader buf = new BufferedReader(fchar);
    t.setText("");
    String linie;
    while ((linie=buf.readLine() )!=null) t.append(linie+"\n");
}

public void actionPerformed (ActionEvent e)
{
    if( e.getActionCommand().compareTo("Open")==0)
    {
        JFileChooser c_op = new JFileChooser();
        c_op.addChoosableFileFilter(new Filtre());
        int valRet = c_op.showOpenDialog(this);
        if (valRet==0)
        {
            File f=c_op.getSelectedFile();
            { try { citFisier(f.getPath()); }
              catch (Exception excp) {}
            }
        }
    }
    else
    {
        JFileChooser c_save = new JFileChooser();
        c_save.addChoosableFileFilter(new Filtre());
        int valRet = c_save.showSaveDialog(this);
        if (valRet==0)
        {
            File f=c_save.getSelectedFile();
            { try { scriuFisier(f.getPath()); }
              catch (Exception excp) {}
            }
        }
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Fereastra din care pot introduce textul");
    }
}

```

10.7.13. Componente de tip JColorChooser

În programarea vizuală culorile au un rol fundamental. Pentru a usura selectarea culorilor, Java pune la dispoziția programatorilor clasa **JColorChooser**. În linii mari, această clasă permite afișarea unui panou prin care se selectează ușor culorile (fie direct, fie folosind sistemul **RGB**, fie folosind sistemul **HSL**). Iată cele mai importante metode ale acestei clase:

- ⇒ **JColorChooser()** - constructor. În panoul culorilor este automat selectată culoarea albă;
- ⇒ **JColorChooser(Color c)** - constructor. În panoul culorilor este automat selectată culoarea transmisă ca parametru;

- ⇒ `static Color showDialog (Component parinte, String mes, Color c)` - lansează, modal, o cutie de dialog care conține panoul culorilor. După cum observați, metoda returnează culoarea selectată (dacă se apasă butonul OK, iar dacă se apasă Cancel se returnează null). Parametrii sunt: `parinte` (fereastra părinte), `mes` (mesaj ce se afișează pe bara de titlu a cutiei) și `c` (culoarea inițial setată în cutie). Utilizarea acestei metode este cel mai simplu mecanism prin care utilizatorul poate selecta culoarea.
- ⇒ `Color getColor()` - returnează culoarea selectată în cutie.

Exemplu. Din fereastra principală, la apăsarea unui buton se afișează o cutie de dialog specifică pentru selectarea culorii. Culoarea selectată va fi culoarea de fond a ferestrei principale.

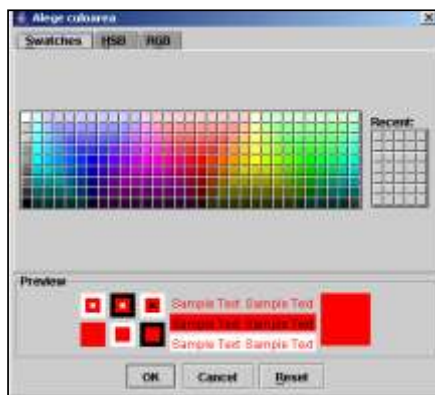
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fer extends JFrame
implements ActionListener
{
    Container x;

    public Fer(String titlu) {
        super(titlu);
        setSize(200,100);
        x=getContentPane();
        x.setLayout(new FlowLayout ());
        JButton A=new JButton("Alta culoare");
        x.add(A);
        A.addActionListener(this);
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    {
        Color c=JColorChooser.showDialog
        (this, "Alege culoarea", Color.red);
        if (c!=null) x.setBackground(c);
    }
}

public class pvl {
    public static void main(String args[]) {
        Fer fp=new Fer("Schimbarea culorii ferestrei");
    }
}
```



10.7.14. Componente de tip JToolBar

O astfel de componentă creează o “bară cu unelte”. Pe ea se așează numeroase alte componente, întocmai ca pe un panel. De regulă se găsește în partea de sus a ferestrei, dar întotdeauna sub meniu.

Clasa **JToolBar** conține următoarele metode mai importante:

- ⇒ **JToolBar()** - constructor, creează o bară de unelte poziționată orizontal;
- ⇒ **JToolBar(int orientare)** - creează o bară de unelte orientată fie orizontal, **orientare** va avea valoarea **JToolBar.HORIZONTAL**, fie vertical, **orientare** va avea valoarea **JToolBar.VERTICAL**. Diferența de orientare afectează modul în care sunt așezate pe ea diferitele componente (pe orizontal sau pe vertical).
- ⇒ **Component add(Component comp)** - adaugă o componentă;
- ⇒ **void addSeparator()** - adaugă un separator (spațiu liber) între componente.

Exemplu: mai jos puteți observa programul care afișează o fereastră cu o bară de unelte și în rest, un panel (componentă de tip **JPanel**):



```
import java.awt.*;
import javax.swing.*;

class Fer extends JFrame
{
    public Fer(String titlu) {
        super(titlu);
        setSize(600,800);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        JToolBar bara=new JToolBar();
        bara.setBackground(Color.red);
        bara.setLayout(new FlowLayout(FlowLayout.LEFT));
        JPanel panel=new JPanel();
        panel.setBackground(Color.yellow);
        JComboBox lista=new JComboBox();
        x.setLayout(new BorderLayout());
        x.add(bara,BorderLayout.NORTH);
        x.add(panel,BorderLayout.CENTER);
        JButton A=new JButton("buton 1");
        bara.add(A);
        A.setPreferredSize(new Dimension (70,30));
        JButton B=new JButton("buton 2");
        B.setPreferredSize(new Dimension (70,30));
        bara.add(B);
    }
}
```

```

        bara.addSeparator();
        lista.addItem("Optiunea 1");
        lista.addItem("Optiunea 2");
        bara.add(lista);
        lista.setPreferredSize(new Dimension (100,30));
        setVisible(true);
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Bara cu unelte");
    }
}

```

🔍 Observații:

1. Pentru a poziționa bara orizontal se preferă gestionarul **GridLayout** și bara se așează în nord.
2. În aceste condiții, panelul s-a aranjat în centru. Cum celelalte trei componente lipsesc, componenta din centru ocupă toată linia din mijloc, iar linia din mijloc ocupă și partea de sud!
3. Pentru bară se preferă gestionarul **FlowLayout** (**FlowLayout.LEFT**) care aranjează componentele pe bară de la dreapta la stânga, dar aliniate stânga!
4. Pentru a ne asigura că butoanele și lista au aceeași înălțime, am preferat dimensionarea lor prin **setPreferredSize()**.
5. Între cele două butoane și listă a fost introdus un separator. Observați-l în imaginea de pe pagina anterioară!

10.7.15. Componente de tip **JTable**

Cu ajutorul componentelor de acest tip se creează tabele. În continuare prezentăm cele mai importante metode ale acestui tip.

⇒ **JTable(Object[][] celule, Object[] cap_t)** - constructor. Parametrul **celule** este o matrice de referințe către obiectele afișate de celulele tabelului, iar **cap_t** este un vector de referințe către obiectele afișate de capul de tabel. Numărul coloanelor matricei de referințe **celule** trebuie să coincidă cu numărul elementelor vectorului de referințe **cap_t** și reprezintă numărul coloanelor din tabel.

Înainte de a da primul exemplu de tabel, trebuie precizat că această componentă se plasează, de obicei, pe o componentă de tip **JScrollPane**.

⇒ **public void setPreferredSize(Dimension dim)** are rolul de preciza dimensiunea tabelului și faptul că acesta trebuie să conțină o bara scroll verticală (dacă tabelul are suficiente linii astfel încât să nu încapă în totalitate în spațiul generat de **dim**).

Exemplu: tabelul alăturat a fost obținut cu secvența de mai jos:

```
class Fer extends JFrame
{ public Fer(String titlu) {
    super(titlu);setSize(600,800);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    Object[][] date = {
        {"x(0,0)", "x(0,1)", "x(0,2)"},
        {"x(1,0)", "x(1,1)", "x(1,2)"}};
};
String[] cap_tabel = {"Coloana 1", "Coloana 2", "Coloana 3"};
Container x=getContentPane();
x.setLayout(new FlowLayout());
JTable table = new JTable(date, cap_tabel);
JScrollPane panel = new JScrollPane(table);
table.setPreferredScrollableViewportSize
    (new Dimension(200, 30));
x.add(panel);setVisible(true);
}
```

Coloana 1	Coloana 2	Coloana 3	
x(0,0)	x(0,1)	x(0,2)	▲
x(1,0)	x(1,1)	x(1,2)	▼



Dacă dorim, putem inversa ordinea coloanelor în tabel prin *drag and drop*. Încercați!

Tabelul este, de fapt, format din două componente: tabelul propriu-zis (de tip `JTable`) și capul de tabel (de tip `JTableHeader`).

⇒ `JTableHeader getTableHeader()` – metoda returnează o referință către capul de tabel.

Imediat ce creăm un tabel ne punem problema culorilor de fond și de scriere, atât pentru capul de tabel cât și pentru tabelul propriu-zis. Dacă ambele sunt componente, atunci putem utiliza metodele clasei `JComponent`: `setBackground()`, `setForeground()` și `setColor()`.

Exemplu: adăugați exemplului anterior secvența de mai jos și veți obține efectul scontat!

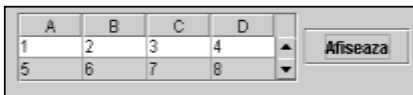
```
// pentru tabel
table.setFont(new Font("Arial",Font.ITALIC, 10));
table.setBackground(Color.GREEN);
table.setForeground(Color.RED);
// pentru capul de tabel
JTableHeader cap= table.getTableHeader();
cap.setFont(new Font("Arial",Font.BOLD, 12));
cap.setBackground(Color.YELLOW);
cap.setForeground(Color.RED);
```



Observați faptul că fiecare celulă a tabelului este editabilă. Aceasta înseamnă că dacă executăm click pe celula respectivă, aceasta se comportă ca un edit oarecare. De aici, rezultă faptul că un tabel poate fi folosit și pentru introducerea datelor, nu doar pentru afisarea lor!

- ⇒ `Object getValueAt(int linie, int coloana)` întoarce o referință către obiectul memorat de celula respectivă;
- ⇒ `void setValueAt(Object ref, int linie, int coloana)` - memorează referința către obiect în linia și coloana dată de indicii transmiși.
- ⇒ `JTable(int n_linii, int n_coloane)` - este un alt constructor al tabelului, în care se creează un tabel `n_linii` și `n_coloane`. Liniile sunt numerotate începând de la 0. De asemenea, coloanele sunt numerotate începând de la 0.

Exemplu: programul următor creează un tabel fără date cu 2 linii și 4 coloane. Utilizatorul introduce date în fiecare celulă a tabelului. La apăsarea butonului se afișează, în fereastra **CMD**, valorile reținute de fiecare celulă. Observați faptul că numele coloanelor a fost dat automat.



```
import java.awt.*;
import javax.swing.*;
import javax.swing.table.*;
import java.awt.event.*;

class Fer extends JFrame implements ActionListener
{
    JTable table;

    public Fer(String titlu) {
        super(titlu);setSize(600,800);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout());
        table = new JTable(3, 4);
        JScrollPane panel = new JScrollPane(table);
        table.setPreferredScrollableViewportSize
            (new Dimension(200, 30));
        x.add(panel);
        JButton A=new JButton("Afiseaza"); x.add(A);
        A.addActionListener(this);
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    {
        if (table.getCellEditor() != null)
            table.getCellEditor().stopCellEditing();
        for (int i=0;i<2;i++)
            for (int j=0;j<4;j++)
                System.out.println(i+" "+j+" "+table.getValueAt(i,j));
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Tabel");
    }
}
```

Să urmărim metoda `actionPerformed()`. Ce rost are acel `if`? Atunci când se scriu datele într-o celulă, până nu s-a trecut la celula următoare (click pe suprafața ei), data conținută de ea nu este memorată și prin urmare, la afișare va apărea `null` (obiectul `null`). Pentru a evita aceasta, s-a utilizat metoda `getCellEditor()` a clasei `JTable`.

⇒ **`TableCellEditor getCellEditor()`** - metoda returnează o referință către un obiect dacă o celulă a tabelului este sub editare și `null` în caz contrar. **`TableCellEditor`** este o interfață care, evident, este implementată. Fără a intra în amănunte, interfața conține antetul metodei `stopCellEditing()`, metodă care are rolul de a stopa editarea și, prin urmare, de memorare a tuturor datelor introduse în tabel.

În continuare, vedem cum putem proiecta capul de tabel! Clasa **`JTable`** conține metodele:

⇒ **`TableColumnModel getColumnModel()`**. **`TableColumnModel`** este o interfață, implementată de o anumită clasă (**`DefaultTableColumnModel`**). Prin urmare, putem utiliza metodele clasei pornind de la referința întoarsă de metodă. Cu ajutorul lor proiectăm capul de tabel.

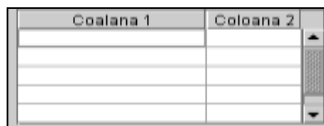
⇒ **`TableColumn getColumn(int index)`** - returnează o referință către un obiect **`TableColumn`**. Un astfel de obiect conține toate atributele unei coloane a tabelului. Iată câteva metode, mai importante ale acestui obiect:

❖ **`void setHeaderValue(Object ob)`** - setează obiectul afișat de coloana respectivă (numele coloanei).

❖ **`setPreferredWidth(int lat)`** - setează lățimea coloanei.

❖ **`setResizable(boolean v);`** - în mod normal, marginile unei coloane se pot deplasa, astfel încât lățimea coloanei poate fi stabilită și de utilizator. Dacă metoda primește ca parametru valoarea `false`, marginile nu mai pot fi stabilite de utilizator.

Exemplu: secvența de mai jos ne arată cum putem construi un tabel precum cel alăturat, care conține numele coloanelor, dar în care, primei coloane nu i se poate modifica lățimea (implicit, și celelalte coloane).



```
table = new JTable(6, 2);
TableColumnModel mColoane=table.getColumnModel();
TableColumn coloana=mColoane.getColumnModel(0);
coloana.setHeaderValue("Coloana 1");
```

```
coloana.setPreferredWidth(150);
coloana.setResizable(false);
coloana=mColoane.getColumn(1);
coloana.setHeaderValue("Coloana 2");
JScrollPane panel = new JScrollPane(table);
table.setPreferredScrollableViewportSize
        (new Dimension(200, 80));
```

Alte metode ale clasei **JTable**:

- ⇒ **void setGridColor(Color c)** - setează culoarea liniilor care separă celulele tabelului.
- ⇒ **void setRowHeight(int inalt)** - setează înălțimea liniilor tabelului (în pixeli).
- ⇒ **void setSelectionBackground(Color c)** - setează culoarea liniei selectate (atunci când se execută click pe o celulă, se selectează întreaga linie).
- ⇒ **void setSelectionForeground(Color c)** - setează culoarea cu care se scrie în linia selectată (mai puțin celula în care se editează).
- ⇒ **setIntercellSpacing(Dimension dim)** - setează spațiile între linii și coloane. Aceasta nu afectează liniile separatoare dintre celule (vecine), ci numai spațiul dintre părțile lor editabile. Primul parametru pentru **dim** este pentru linii, al doilea parametru este pentru coloane.
- ⇒ **void setShowHorizontalLines(boolean v)** - dacă parametrul este **false**, atunci nu mai sunt trasate liniile orizontale ale tabelului.
- ⇒ **setShowGrid(boolean v)** - dacă parametrul este **false**, atunci nu mai sunt trasate liniile orizontale și verticale ale tabelului.

10.8. Mai mult despre clasa **Container**

Este momentul să analizăm modul în care fereastra aplicației reține componentele grafice atasate. Logic vorbind, este nevoie de o variabilă, asemănătoare unui vector, în care fiecare componentă a sa va reține referințe către componentele grafice. De asemenea, este necesară și o variabilă care reține o referință către gestionarul de poziționare. Atunci când atașăm ferestrei componente grafice, mai întâi declarăm o referință la un obiect al clasei **Container** și o inițializăm cu adresa obiectului de tip **Container** al ferestrei (returnată de metoda clasei **JFrame**, numită **getContentPane()**):

Exemplu: `Container x=getContentPane();`

Clasa **Container** conține mai multe metode, dintre care le prezentăm pe următoarele:

⇒ **Component add(Component c);** - adaugă o componentă container-ului. Cum orice componentă este un obiect al unei clase derivate din clasa **Component**, parametrul metodei poate fi utilizat pentru a transmite referințe către orice tip de componentă. Până în prezent am folosit de multe ori această metodă.

⇒ **int getComponentCount();** returnează numărul componentelor reținute de container.



Spuneam că un container este asemănător unui vector. Astfel, prima componentă adăugată cu metoda de mai sus are indicele 0, a doua componentă are indicele 1, ș.a.m.d.

⇒ **Component getComponent(i);** - returnează referința la componenta de indice *i*.

Am precizat faptul că tipurile tuturor componentelor grafice pe care le-am studiat sunt derivate din clasa **Component**, dar nu direct. Aceasta din urmă conține și metodele:

❖ **void setName(String name);** - metoda prin care unei componente, indiferent de tipul ei, i se poate atașa un nume. Acesta poate fi folosit atunci când trebuie să detectăm o anumită componentă din container.

❖ **String getName();** - metoda returnează numele componenteii dat prin **setName()**.



Prin operatorul de conversie explicită, o referință de tip **Component** poate fi convertită către o referință la o componentă de un anumit tip, numai dacă componenta propriu-zisă are tipul respectiv. De exemplu, dacă *c* este tip **Component** și reține adresa unui obiect de tip **JButton**, atunci conversia **JButton buton (JButton)c;** este corectă. Dacă însă *c* reține adresa unei componente de tip **JLabel**, atunci prin conversia de mai sus se obține o excepție.

În cazul componentelor de tip **JPanel**, lucrurile stau puțin diferit, în sensul că pentru a depista componentele pe care le conține, vom utiliza metodele de mai jos (ale tipului **JPanel**):

⇒ Adăugarea unei componente se face cu metoda **add()** a tipului **JPanel**, așa cum suntem deja obișnuiți;

⇒ **Component[] getComponents()** returnează un vector de referințe către componente.

Exemplu: programul următor creează o fereastră cu diverse componente: un buton, o etichetă (obiect de tip `JLabel`) și un panel (obiect de tip `JPanel`). Acesta din urmă conține un buton și o etichetă. *Toate componentele au primit un nume.* La apăsarea butonului **Componente**, se afișează numele tuturor componentelor din fereastră și panel.

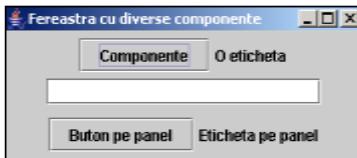
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fer extends JFrame implements ActionListener
{
    Container x; JPanel panel;

    public Fer(String titlu) {
        super(titlu); setSize(300,200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        x=getContentPane();
        x.setLayout(new FlowLayout ());
        // Adaug elemente pe fereastra si panel
        JButton A=new JButton("Componente");
        A.setName("Buton A"); x.add(A);
        JLabel et =new JLabel("O eticheta");
        et.setName("Eticheta et"); x.add(et);
        JTextField txt=new JTextField(20);
        txt.setName("edit-ul txt"); x.add(txt);
        panel=new JPanel(new FlowLayout()); panel.setName("panelul");
        JButton B=new JButton("Buton pe panel");
        B.setName("Buton pe panel"); panel.add(B);
        JLabel et1 =new JLabel("Eticheta pe panel");
        et1.setName("Eticheta pe panel"); panel.add(et1);
        x.add(panel);
        A.addActionListener(this);
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    {
        int nr_comp=x.getComponentCount();
        System.out.println
            ("Numar componente atasate ferestrei="+nr_comp);
        // afisez numele componentelor de pe fereastra
        for(int i=0;i<nr_comp;i++)
        {
            Component c=x.getComponent(i);
            System.out.println(c.getName());
        }
        Component[] vect=panel.getComponents();
        System.out.println
            ("Numar componente atasate panelului="+vect.length);
        // afisez numele componentelor de pe panel
        for(int i=0;i<vect.length;i++)
        {
            Component c=vect[i];
            System.out.println(c.getName());
        }
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Fereastra cu diverse componente");
    }
}
```



10.9. Cutii de dialog

Până în prezent am studiat numai cutiile de dialog predefinite. Desigur, cutiile predefinite sunt ușor de folosit, dar nu răspund tuturor necesităților. Uneori, suntem obligați să creăm cutii de dialog care să includă componentele pe care le dorim. În acest paragraf vom prezenta modul în care putem construi cutiile de dialog după propria dorință.

Pentru început trebuie spus că putem afișa o cutie de dialog în două feluri:

A) **Modal** - după afișarea cutiei nu mai putem acționa asupra ferestrei din care aceasta a fost lansată.

B) **Nemodal** - deși cutia este afișată, putem acționa și asupra componentelor reținute de cutie, dar și asupra componentelor din fereastra din care aceasta a fost lansată.

Pentru a construi o cutie de dialog trebuie să știm că:

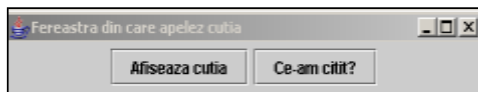
1. Cutia de dialog propriu-zisă se obține ca instanțiere a unei clase derivată din clasa `JDialog`. Clasa `JDialog` conține constructorul:

❖ `JDialog (Dialog fer_parinte, String titlu, boolean modal)` prin care se specifică fereastra părinte (fereastra care conține comanda care a lansat-o), titlul ferestrei și un parametru de tip boolean, care dacă reține `true`, afișarea va fi modală, iar dacă reține `false`, afișarea va fi nemodală.

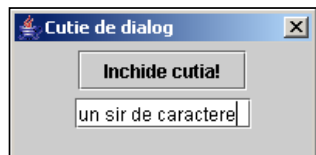
2. Clasa `JDialog`, ca și clasa `JWindow` au o superclasă comună (`Window`). Aceasta are ca efect faptul că putem construi cutia la fel cum am construit fereastra principală.

Exemplul 1 (afișare modală)

programul următor afișează fereastra alăturată:



Dacă se apasă butonul "Afișează cutia" se afișează cutia de dialog alăturată. Introduceți în edit un șir oarecare de caractere și apăsați butonul "Inchide cutia!"



Analizați modul în care am detectat șirul de caractere introdus în edit prin apăsarea butonului "Ce-am citit?". Va apărea cutia de dialog standard de mai jos:

Prezentăm programul:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Dial extends JDialog implements ActionListener
{
    // Constructia cutiei de dialog
    JButton buton=new JButton("Inchide cutia!");
    JTextField txt=new JTextField(10);
    Container y;
    String sir=txt.getText();
    Dial (JFrame parinte,String mesaj, boolean modal)
    { super (parinte,mesaj,modal);
      y=getContentPane();
      setSize(200,200);
      y.setLayout(new FlowLayout());
      y.add(buton);
      y.add(txt);
      buton.addActionListener(this);
      setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    // inchide cutia
    { setVisible(false);}
}

class Fer extends JFrame implements ActionListener
{
    Dial cutie=null;

    public Fer(String titlu) {
        super(titlu);
        setSize(400,400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        JButton A=new JButton("Afiseaza cutia"); x.add(A);
        A.addActionListener(this);
        JButton B=new JButton("Ce-am citit?");
        x.add(B);
        B.addActionListener(this);
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    {
        //Daca a fost apasat butonul "Afiseaza cutia"
        if( e.getActionCommand().compareTo("Afiseaza cutia")==0)
            //daca cutia nu a fost afisata
            if (cutie==null) cutie=new Dial(this,"Cutie de dialog",
                true);
            // daca cutia a fost afisata
            else cutie.setVisible(true);
    }
}
```



```

else
    //daca a fost apasat butonul "Ce-am citit?"
    if (cutie!=null)
    { // recuperez container-ul cutiei
      Container t=cutie.y;
      // recuperez referinta catre edit
      JTextField com=(JTextField)t.getComponent(1);
      // recuperez continutul edit-ului
      JOptionPane.showMessageDialog(this,com.getText());
    }
    // daca vreau sa vad ce-am scris fara sa fi afisat cutia
    else JOptionPane.showMessageDialog
        (this,"Mai intai afiseaza cutia de dialog");
}
}

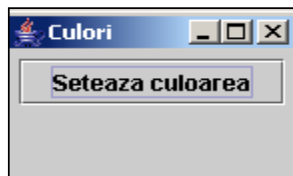
public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Fereastră din care apelez cutia");
    }
}

```

🕒 Observații:

- ✓ A închide o fereastră (cutie de dialog) înseamnă a o face invizibilă (`setVisible(false)`). A deschide o fereastră, deja creată, care a fost închisă înseamnă a o face vizibilă (`setVisible(true)`). Prin urmare, chiar dacă fereastra a fost închisă, pornind de la o referință către container-ul ei, putem accesa componentele pe care fereastra (cutia de dialog) le conține, deoarece ea există în memorie.
- ✓ Practic, datele din cutie se pot recupera pornind de la container-ul cutiei, iar dacă se dorește, din cutie se pot accesa datele ferestrei pornind de la referința către părintele cutiei, adică fereastra.

Exemplul ② (afișare nemodală). Programul de mai jos creează o fereastră cu un buton. La apăsarea lui, se afișează o cutie de dialog cu două butoane. Prin apăsarea oricărui buton al cutiei, se schimbă culoarea de fond a ferestrei principale.



Cutia de dialog este afișată nemodală! Practic, dacă apăsăm un buton al ei, culoarea ferestrei principale se schimbă instantaneu, fără să fim nevoiți să închidem cutia.

```

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Dial extends JDialog implements ActionListener
{
    Container y;
    JFrame frs;
    Dial (JFrame parinte,String mesaj, boolean modal)
    {
        super(parinte,mesaj,modal);
        y=getContentPane();
        frs=parinte;
        setSize(100,100);
        y.setLayout(new FlowLayout());
        JButton b1=new JButton("Rosu"); y.add(b1);
        b1.addActionListener(this);
        JButton b2=new JButton("Galben"); y.add(b2);
        b2.addActionListener(this);
        setVisible(true);
    }
    public void actionPerformed (ActionEvent e)
    {
        Container z=frs.getContentPane();
        if( e.getActionCommand().compareTo("Rosu")==0)
            z.setBackground(Color.red);
        else z.setBackground(Color.yellow);
    }
}

class Fer extends JFrame implements ActionListener
{
    Dial cutie=null;
    public Fer(String titlu) {
        super(titlu);
        setSize(150,100);
        Container x=getContentPane();
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        x.setLayout(new FlowLayout ());
        JButton A=new JButton("Seteaza culoarea"); x.add(A);
        A.addActionListener(this);
        setVisible(true);
    }
    public void actionPerformed (ActionEvent e)
    {
        if (cutie==null) cutie=new Dial(this,"Seteaza culoarea",
                                         false);
        else cutie.setVisible(true);
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Culori");
    }
}

```

Exemplul 3 Din fereastra principală se afișează *nemodal* o cutie de dialog care conține un buton și o componentă de tip `JColorChooser`. Utilizatorul selectează culoarea și apasă butonul. Automat (fără închiderea cutiei), fereastra principală va avea culoarea selectată.

```

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Dial extends JDialog implements ActionListener
{
    Container y;
    JColorChooser col;
    JFrame frs;

    Dial (JFrame parinte,String mesaj, boolean modal)
    {
        super(parinte,mesaj,modal);
        y=getContentPane();
        frs=parinte; setSize(500,500);
        y.setLayout(new FlowLayout());
        JButton b=new JButton("Gata!"); y.add(b);
        b.addActionListener(this);
        col = new JColorChooser(Color.yellow);
        y.add(col);
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    {
        Container z=frs.getContentPane();
        z.setBackground(col.getColor());
    }
}

class Fer extends JFrame implements ActionListener
{
    Dial cutie=null;

    public Fer(String titlu) {
        super(titlu);
        setSize(150,100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        JButton A=new JButton("Seteaza culoarea"); x.add(A);
        A.addActionListener(this);
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e)
    {
        if (cutie==null) cutie=new Dial
                                (this,"Seteaza culoarea",false);
        else cutie.setVisible(true);
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Culori");
    }
}

```

10.10. Clasa Graphics

10.10.1. Afișarea imaginilor

Pentru a afișa o imagine aflată într-un fișier, vom scrie o clasă care extinde clasa `Canvas`. Obiectele clasei `Canvas` conțin o metodă numită `paint(Graphics gcr)`. Această metodă este apelată *automat* atunci când componenta respectivă este afișată și are rolul de a desena componenta. Observați că metoda `paint()` primește ca parametru o referință către un obiect al clasei `Graphics` și, pornind de la ea, putem apela toate metodele acestei clase, metode cu ajutorul cărora putem afișa imaginea, scrie în mod grafic sau desena.

În programul următor, clasa care construiește componenta ce afișează imaginea se numește `Imagine`. Clasa `Imagine` conține:

1. O referință către un obiect de tip `Image`. Un astfel de obiect poate reține o imagine citită. Referința este utilizată în metoda `paint()` pentru a afișa imaginea respectivă.

2. Un constructor, care citește imaginea, stabilește mărimea componentei (`setSize()`) și culoarea de fundal (`setBackground()`). Pentru citirea imaginii se utilizează metoda clasei `Toolkit` numită `getImage()`:

⇒ `Image getImage(String fisier)`, unde parametrul reprezintă calea către fișierul respectiv.

3. Metoda `paint()` (redefinită, pentru a avea acces). Metoda "desenează" imaginea și folosește metoda `drawImage()` a clasei `Graphics`:

⇒ `drawImage(image im, int a, int b, int latime, int inaltime, this)`, unde:

- `a, b` - coordonatele relative (față de colțul din stânga-sus al componentei care afișează imaginea; în exemplu `Imagine`) ale colțului din stânga sus al dreptunghiului care conține imaginea.
- `latime, inaltime` - lățimea și înălțimea dreptunghiului care conține imaginea.
- `this` - obiectul curent (cel al clasei `Imagine`). Nu tratăm semnificația generală a acestui parametru.

Observații:

- ✓ Metoda poate redimensiona imaginea citită (o mărește sau o micșorează) prin valorile transmise pentru `latime` și `inaltime`.

- ✓ Fișierul imagine se găsește în **Bin** (folder-ul curent), dar îi putem atașa o cale.
- ✓ Exemplul următor conține o componentă care afișează desenul în centrul ei. Desigur, imaginea putea acoperi întreaga suprafață a componentei **Imagine**, dar din considerente didactice, pentru a distinge întreaga suprafață unde am fi putut desena, am preferat să folosim `setBackground()`.

Programul de mai jos construiește o fereastră care afișează o imagine:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Imagine extends Canvas
{
    Image im;

    Imagine()
    {
        Toolkit ec=Toolkit.getDefaultToolkit();
        im=ec.getImage("calculator.jpg");
        setSize(300,300);
        setBackground(Color.YELLOW);
    }

    public void paint (Graphics ecr)
    { ecr.drawImage(im,50,50,200,200,this); }
}

class Fer extends JFrame
{
    Imagine p=new Imagine();

    public Fer(String titlu) {
        super(titlu);setSize(150,200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        x.add(p);
        setVisible(true);
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Imagine");
    }
}
```



10.10.2. Cum scriem ?

Înainte de parcurgerea acestui paragraf, revedeți 8.15. "Afișarea imaginilor". Ați putut observa acolo că metoda **Paint** primește ca referință un obiect al clasei **Graphics**. Putem utiliza metodele acestui obiect pentru a desena ceva sau pentru a încărca o imagine care se găsește într-un fișier. Cele mai importante metode ale acestei clase sunt:

- ⇒ `void setFont (Font f)` - setează obiectul pentru ca scrierea grafică să se realizeze cu un anumit font;
- ⇒ `void setColor(Color c)` - setează culoarea de scriere (desenare);
- ⇒ `drawString(String str, int x, int y)` - scrie pe ecran un șir de caractere. Șirul este scris cu font-ul și culoarea setate la un moment dat. Parametrii `x` și `y` reprezintă coordonatele colțului din stânga sus al șirului de caractere afișat.

Exemplu: iată cum arată metoda `Paint()` pentru a putea scrie două șiruri de caractere cu font-uri și culori diferite:

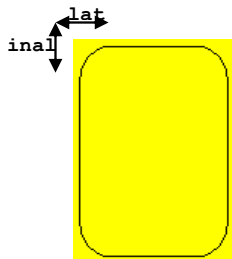
Un exemplu de scriere

Scriu cu alta culoare si alt font

```
public void paint (Graphics ecr)
{ Fontf=new Font("Arial",Font.ITALIC+Font.BOLD, 11);
  ecr.setFont(f);
  ecr.setColor(Color.red);
  ecr.drawString("Un exemplu de scriere",20, 30);
  ecr.setColor(Color.black);
  Font f1=new Font("CourierNew",Font.BOLD, 14);
  ecr.setFont(f1);
  ecr.drawString("Scriu cu alta culoare si alt font",20, 50);
}
```

10.10.3. Cum desenăm ?

- ⇒ `drawLine(int x1, int y1, int x2, int y2)` - desenează o linie (segment) între punctele de coordonate `(x1,y1)` și `(x2,y2)`.
- ⇒ `drawRect(int x, int y, int latime, int inaltime)` - desenează un dreptunghi pentru care colțul din stânga sus are coordonatele `(x,y)` și de lățime și înălțime date.
- ⇒ `drawRoundRect(int x, int y, int latime, int inaltime, int lat, int inal)` - desenează un dreptunghi care are colțurile rotunjite, (vedeti alăturat).
- ⇒ `void fillRect(int x, int y, int latime, int inaltime)`, la fel ca `drawRect()`, numai că dreptunghiul este colorat în interior.
- ⇒ `void fillRoundRect(int x, int y, int latime, int inaltime, int lat, int inal)` - la fel ca `drawRoundRect()`, numai că dreptunghiul rotunjit este colorat în interior.

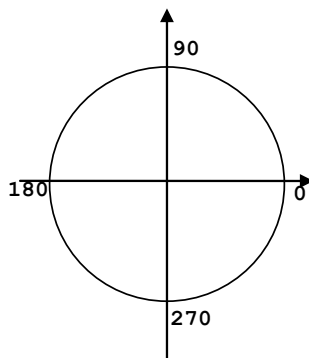


- ⇒ **void drawPolygon(int[] X, int[] Y, int nr)** - desenează un poligon care are **nr** vârfuri. Coordonatele **x** ale punctelor se găsesc în vectorul **X**, iar coordonatele **y** ale punctelor se găsesc în vectorul **Y**. Practic, primul punct are coordonatele (**X[0],Y[0]**), al doilea punct are coordonatele (**X[1],Y[1]**), s.a.m.d. În final, pentru a obține poligonul se unesc printr-un segment primul și ultimul punct. Poligonul este oarecare, deci poate fi și neconvex.

Exemplu:

```
ecr.setColor(Color.black);
int[] X= {10, 100,25, 80};
int[] Y= {10, 100,75, 30};
ecr.drawPolygon(X, Y, 4);
```

- ⇒ **void fillPolygon(int[] X, int[] Y, int nr)** - la fel ca mai sus, numai că poligonul este colorat în interior.
- ⇒ **void drawPolyline(int[] X, int[] Y, int nr)** - la fel ca mai sus, numai că nu se mai unește primul punct cu ultimul punct. Astfel se obține o succesiune de segmente.
- ⇒ **void drawOval(int x, int y, int latime, int inaltime)** - desenează un oval. Coordonatele **x** și **y** sunt ale colțului din stânga sus al dreptunghiului circumscris ovalului, iar **latime** și **inaltime** sunt lățimea și înălțimea acelui dreptunghi.
- ⇒ **fillOval(int x, int y, int latime, int inaltime)** - la fel ca mai sus, numai că ovalul este colorat în interior.
- ⇒ **drawArc(int x, int y, int latime, int inaltime, int u1, int u2)** - trasează un arc. Arcul este o porțiune dintr-un oval și primii 4 parametri sunt ai ovalului căruia îi aparține arcul. Ultimii doi parametri sunt: unghiul de unde pornește arcul și numărul de grade al arcului. Unghiurile sunt date în grade.



- ⇒ **fillArc(int x, int y, int latime, int inaltime, int u1, int u2)** - la fel ca **drawArc**, numai că se consideră sectorul (nu numai arcul) și acesta este colorat. De exemplu, în cazul în care **latime** și **inaltime** rețin aceeași valoare, ovalul este, de fapt, un cerc. În acest caz avem un sector de cerc.

⇒ `drawImage(image im, int a, int b, int latime, int inaltime, this)`, unde:

- `a, b` sunt coordonatele relative (față de componenta creată, vezi `setSize()`) ale colțului din stânga sus al dreptunghiului care conține imaginea.
- `latime, inaltime` - lățimea și înălțimea dreptunghiului care conține imaginea.
- `this` - obiectul curent (cel al clasei `Image`). Nu tratăm semnificația generală a acestui parametru.

➡ **Observație:** Metoda a mai fost folosită atunci când am afișat o imagine!

10.11. Clasa `Graphics2D`

Clasa `Graphics2D` moștenește clasa `Graphics` și prin urmare, oferă mai multe posibilități de lucru, unele dintre ele cu adevărat spectaculoase. Pentru a o utiliza se procedează ca mai jos, unde a fost rescrisă metoda `paint()` pentru o clasă care moștenește clasa `Canvas` (la fel ca atunci când am creat imagini care se afișează pe fereastră). Toate desenele și setările făcute se utilizează identic ca și când ar fi fost realizate în `Graphics`.

```
public void paint (Graphics ecr1)
{
    Graphics2D ecr=(Graphics2D) ecr1;
    ecr.setColor(Color.red);
    ...
}
```

10.11.1. Cum desenăm ?

Pentru desenare nu avem multe metode proprii (altele decât ale clasei moștenite), dar se pot utiliza metode ale altor clase aflate în pachetul `geom` (adăugați `import java.awt.geom.*`). Acestea construiesc obiectul în memorie, iar pentru afișare vom utiliza metodele clasei `Graphics2D` următoare:

- ⇒ `draw(Shape S)` - desenează obiectul fără a-l "umple" cu culoare sau gradient, ori textură. Toate aceste noțiuni sunt prezentate pe larg, în cadrul aceluiași capitol.
- ⇒ `fill (Shape S)` - desenează obiectul 'umplut' cu o culoare (gradient sau textură).

Ambele metode au parametrii de tipul **Shape**, care este o interfață. Clasele care construiesc figurile geometrice prezentate în continuare, sunt implementări ale tipului **Shape**:

Clasa **Line2D.Float** - are constructorul:

⇒ **Line2D.Float(float x1, float y1, float x2, float y2);**
construiește o linie care unește punctele de coordonate **(x1,y1)** și **(x2,y2)**.

Exemplu: construiesc și afișez o linie roșie (ecr este de tipul **Graphics2D**).

```
ecr.setColor(Color.red);  
Line2D.Float linie = new Line2D.Float(10,10, 100,100);  
ecr.draw(linie);
```

Clasa **Rectangle2D.Float** - are constructorul:

⇒ **Rectangle2D.Float(float x, float y, float latime, float inaltime)** - construiește un dreptunghi. Colțul din stânga sus are coordonatele **(x,y)**, iar dreptunghiul are lățimea și înălțimea specificate de parametri.

Exemplu: construiesc și afișez un dreptunghi cu interiorul "umplut" cu roșu:

```
ecr.setColor(Color.red);  
Rectangle2D.Float drept = new Rectangle2D.Float(10,10, 50, 50);  
ecr.fill(drept);
```

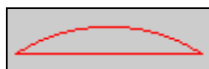
Clasa **Ellipse2D.Float** - are constructorul:

⇒ **Ellipse2D.Float(float x, float y, float latime, float inaltime)** - construiește o elipsă. Parametrii se referă la dreptunghiul circumscris elipsei.

Clasa **Arc2D.Float** - are constructorul:

⇒ **Arc2D.Float(float x, float y, float latime, float inaltime, float start, float nr, int type)** - construiește un arc. Primii 4 parametri se referă la elipsa căreia îi aparține arcul, **start** este unghiul de pornire, **nr** este numărul de grade al arcului, iar **type** este o constantă care determină modul în care arată arcul. Valoarea constantei **type** rezultă din imaginile de mai jos:

Exemplu: construim un arc de 90 de grade al unui cerc. Punctul de start al arcului este la 45 de grade:



Arc2D.CHORD



Arc2D.OPEN



Arc2D.PIE

Clasa **GeneralPath** se utilizează pentru trasarea unei linii frânte sau, în particular, pentru trasarea unui poligon. Clasa conține:

- ⇒ **GeneralPath()** - constructor;
- ⇒ **moveTo(float x, float y)** - mută un cursor imaginar în punctul de coordonate (x,y);
- ⇒ **lineTo(float x, float y)** - trasează o dreaptă între punctul în care se găsește cursorul imaginar și cel de coordonate transmise ca parametru;
- ⇒ **closePath()** - trasează dreapta care închide linia frântă (adică se creează un poligon).

Exemplu: în secvența de mai jos se afișează un triunghi. Vârfurile sale sunt (5.5). (100,100), (150,10):

```
GeneralPath path= new GeneralPath(0,0);
path.moveTo(5,5);
path.lineTo(100,100);
path.lineTo(150,10);
path.closePath();
ecr.draw(path);
```

10.11.2. Stabilirea modului de trasare a liniilor

În clasa **Graphics**, toate liniile aveau o singură grosime: 1 pixel. În **Graphics2D**, se poate stabili ca liniile să aibă o anumită grosime, mai mare sau egală cu 1. De asemenea, se pot face setări ale modului în care se termină segmentele și asupra modului în care acestea se îmbină. Pentru a realiza aceste lucruri, se utilizează o metodă a clasei **Graphics2D**, numită **setStroke()**:

- ⇒ **void setStroke(Stroke s)**; După cum vedeți, parametrul metodei este de tip **Stroke**. Dar, **Stroke** este o interfață. Ea este implementată de clasa **BasicStroke**.

Clasa **BasicStroke** are mai mulți constructori, din care, aici sunt prezentați doar doi, mai importanți:

- ⇒ **BasicStroke(float latime)** - setează lățimea liniilor (în pixeli);
- ⇒ **BasicStroke(float latime, int cap, int join)** - setează lățimea liniilor (**latime**), modul în care arată capetele segmentelor (**cap**) și modul în care acestea sunt unite (**join**). Pentru ultimii doi parametri, apelul metodei se realizează utilizând constantele clasei **BasicStroke**.

Exemplu: pentru a crea imaginea alăturată am utilizat clasa de mai jos:

```
class Imagine extends Canvas
{
    Imagine()
    {
        setSize(300,300);
    }
    public void paint (Graphics ecr1)
    {
        Graphics2D ecr=(Graphics2D) ecr1;
        ecr.setColor(Color.red);
        BasicStroke c =new BasicStroke(10, BasicStroke.CAP_ROUND,
                                         BasicStroke.JOIN_BEVEL);

        ecr.setStroke(c);
        GeneralPath path= new GeneralPath(0,0);
        path.moveTo(5,5);
        path.lineTo(75,75);
        path.lineTo(150,5);
        ecr.draw(path);
    }
}
```



În continuare, ne propunem să analizăm valorile pe care le poate lua parametrul **cap**, prezentând de fiecare dată, câte o dreaptă (segment). Pentru ultimele două cazuri, segmentului i se adaugă un pătrat, respectiv un semicerc:



CAP_BUTT



CAP_SQUARE



CAP_ROUND

Iată care sunt valorile pe care le poate lua parametrul **join** (modul de unire a segmentelor). Pentru fiecare din cazuri vom prezenta o linie frântă realizată cu **GeneralPath**:



JOIN_MITER



JOIN_BEVEL



JOIN_ROUND



Parametrul **join** are efect doar în cazul figurilor unitare (linie frântă, dreptunghi, etc.). El nu va funcționa în cazul în care trasăm două drepte (segmente) care se unesc într-un punct, pentru că nu există informații care să determine acest efect.



Prin utilizarea parametrului `join`, nu mai sunt necesare metode de tipul celor care creează un dreptunghi cu colțurile rotunjite. Acestea pot fi rotunjite prin valoarea pe care o transmitem pentru `join`.

10.11.3. Gradient

Până în prezent, atunci când am colorat o anumită imagine, am utilizat numai o singură culoare. În `Graphics2D` se poate utiliza un gradient, adică o trecere treptată de la o culoare la alta, obținându-se astfel efecte deosebite. Pentru a realiza un gradient vom folosi metoda `setPaint()` a clasei `Graphics2D`:

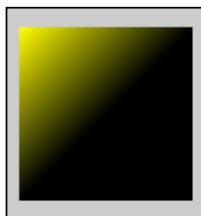
⇒ `setPaint(Paint p)` - după cum observați, parametrul este de tip `Paint`, care este o interfață implementată de mai multe clase. Pentru a stabili un gradient vom utiliza clasa `GradientPaint`.

Clasa `GradientPaint` are constructorul următor:

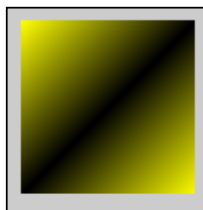
⇒ `GradientPaint(float x1, float y1, Color c1, float x2, float y2, Color c2, boolean cyclic);`

Semnificația parametrilor este următoarea: în punctul de coordonate `(x1,y1)` vom avea culoarea `c1`, în punctul de coordonate `(x2,y2)` vom avea culoarea `c2`. Între aceste puncte se va face o trecere treptată de la culoarea `c1` la culoarea `c2`. Trecerea poate fi aciclică (parametrul `cyclic` reține `false`) sau ciclică (parametrul `cyclic` reține `true`). Pentru cea aciclică, trecerea de la `c1` la `c2` se face treptat, iar pentru cea ciclică, între cele două puncte se trece de mai multe ori de la `c1` la `c2` și invers și nu există cerința ca în punctele date să fie exact culorile solicitate.

Exemplu: pentru a crea un dreptunghi cu gradient, prin trecerea de la galben la negru, vom utiliza clasa de mai jos. În exemplu, gradientul este aciclic, dar figurile prezintă și gradientul ciclic:



aciclic



ciclic

```

class Imagine extends Canvas
{
    Imagine()
    {
        setSize(300,300);
        public void paint (Graphics ecr1)
        {
            Graphics2D ecr=(Graphics2D) ecr1;
            GradientPaint g=new GradientPaint(50,50, Color.YELLOW,
                                                100,100, Color.BLACK, false);
            ecr.setPaint(g);
            Rectangle2D.Float drept = new Rectangle2D.Float
                                    (50,50, 100, 100);
            ecr.fill(drept);
        }
    }
}

```

10.11.4. Clasa BufferedImage

Uneori, preferăm să creăm un desen pe care să-l păstrăm în memorie. Pentru aceasta utilizăm clasa **BufferedImage**. Afișarea desenului se face atunci când dorim, în **paint()**, cu **drawImage()** așa cum suntem obișnuiți.

Clasa BufferedImage conține:

- ⇒ **BufferedImage(int latime, int inaltime, int imageType)** - constructor. Primii doi parametri specifică lățimea și înălțimea imaginii memorate de obiect. Ultimul parametru primește o constantă a clasei **BufferedImage** și semnifică un anumit tip de imagine cu o anumită culoare de fond.
- ⇒ **Graphics2D createGraphics()** - returnează o referință către obiect de tip **Graphics2D** cu ajutorul căruia putem desena o imagine reținută de obiectul de tip **BufferedImage**.
- ⇒ **ImageProducer getSource()** - returnează imaginea reținută de obiect.

Pentru a realiza cele propuse mai avem nevoie să utilizăm o metodă a clasei **Toolkit** (vezi 10.5):

- ⇒ **Image createImage(ImageProducer buf)** - creează un obiect de tip **Image** pornind de la o implementare a interfeței **ImageProducer**.

```

class Imagine extends Canvas
{
    Image im;
    BufferedImage imag_a;
    Graphics2D gr;
}

```



```

Imagine()
{ // creez un obiect BufferedImage
  imag_a=new BufferedImage(200, 200, BufferedImage.TYPE_INT_RGB);
  // pentru a desena ii asociez un un obiect Graphics2D
  gr=imag_a.createGraphics();
  // desenez
  Line2D.Float linie = new Line2D.Float(10,10, 100,100);
  gr.draw(linie);
  // obtin un obiect de tip Image care contine desenul
  im=Toolkit.getDefaultToolkit().createImage(imag_a.getSource());
  setSize(300,300); setBackground(Color.WHITE);
}

public void paint (Graphics ecr)
{ // desenez imaginea p
  ecr.drawImage(im,0,0,this);
}
}

class Fer extends JFrame
{ Imagine p=new Imagine();
  public Fer(String titlu) {
    super(titlu);setSize(400,400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    Container x=getContentPane();
    x.setLayout(new FlowLayout ());
    x.add(p);
    setVisible(true);
  }
}

public class pv {
  public static void main(String args[]) {
    Fer fp=new Fer("Imagine ascunsa");
  }
}

```

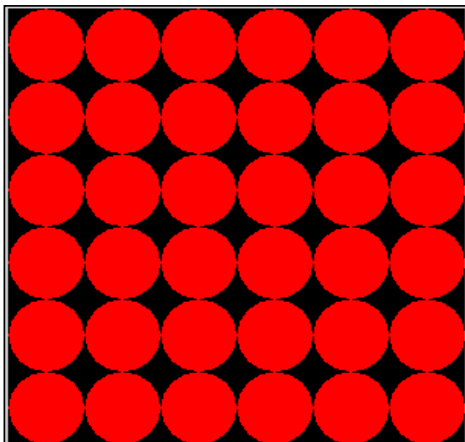
10.11.5. Texturi

Suprafețele pot fi acoperite de o aceeași culoare, de un gradient sau de o textură. Pentru a acoperi o suprafață cu o textură, vom utiliza aceeași metodă a clasei **Graphics2D** ca și când acoperim suprafața cu un gradient și anume **setPaint(Paint p)**. După cum am arătat, **Paint** este o interfață. Ea este implementată de clasa **GradientPaint** (cu ajutorul ei creăm un gradient), dar și de clasa **TexturePaint**, clasă cu ajutorul căreia creăm o textură.

Clasa **TexturePaint** are constructorul:

⇒ **TexturePaint(BufferedImage buf, Rectangle drept)**. După cum observați, ambii parametri sunt de tip referință la obiect. Obiectele aparțin de două clase distincte, **BufferedImage** și **Rectangle**.

Înainte de a prezenta cei doi parametri, să vedem principal, cum se realizează o textură (textura alăturată este obținută de programul din acest paragraf). Atunci când desenăm o figură geometrică, ea poate fi acoperită de o textură. Textura este alcătuită dintr-o imagine desenată care se repetă de câte ori încapă în cadrul figurii. Prin urmare, pentru a defini textura, trebuie să cunoaștem:



1. Care este imaginea care se repetă?

2. Care este mărimea afișată a imaginii care se repetă?

Răspunsul la prima întrebare este că, pentru a specifica imaginea se utilizează un obiect al clasei **BufferedImage**. Pentru a preciza dimensiunea imaginii care se repetă, se utilizează un obiect al clasei **Rectangle**.

Observație. A nu se confunda dimensiunile imaginii reținute de obiectul de tip **BufferedImage** cu dimensiunile la care se va afișa imaginea în textură. Dacă imaginea reținută de obiectul de tip **BufferedImage** este mai mică decât cea la care se va afișa imaginea, atunci imaginea este mărită automat. Dacă, dimpotrivă, imaginea reținută de obiect are dimensiuni mai mari decât cea afișată, se va afișa ce se poate din ea (pornind de la colțul din stânga sus).

Clasa **Rectangle** se utilizează, așa cum am precizat, pentru a specifica mărimea imaginii afișate. Unul din constructorii ei este:

⇒ **Rectangle (int latime, int inaltime).** Prin utilizarea lui precizăm dimensiunile imaginii afișate.

Exemplu: programul de mai jos afișează textura dată, de exemplu, în acest paragraf.

```
import java.awt.*;
import javax.swing.*;
import java.awt.geom.*;
import java.awt.image.BufferedImage;

class Imagine extends Canvas
{
    Imagine()
    {
        setSize(300,300);
    }
}
```

```

public void paint (Graphics ecr1)
{ Graphics2D ecr=(Graphics2D) ecr1;
  BufferedImage buf=new BufferedImage(50, 50,
                                     BufferedImage.TYPE_INT_RGB);
  Graphics2D model = buf.createGraphics();
  model.setColor(Color.red);
  Ellipse2D.Float el= new Ellipse2D.Float(0, 0, 50, 50);
  model.fill(el);
  Rectangle r = new Rectangle(50,50);
  TexturePaint tp = new TexturePaint(buf, r);
  ecr.setPaint(tp); ecr.fillRect(0,0,300,300);
}
}

class Fer extends JFrame
{ Imagine p=new Imagine();
  public Fer(String titlu) {
    super(titlu);
    setSize(500,500);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    Container x=getContentPane();
    x.setLayout(new FlowLayout ());
    x.add(p);
    setVisible(true);
  }
}

public class pv {
  public static void main(String args[]) {
    Fer fp=new Fer("Imagine");
  }
}

```

10.11.6. Dreptunghiuri 3D

Clasa `Graphics2D` ne oferă posibilitatea să desenăm dreptunghiuri în relief. În acest scop, ea conține metodele `draw3DRect()` și `fill3DRect()`:

- ⇒ `draw3DRect(int x, int y, int latime, int inaltime, boolean raised)` - desenează un dreptunghi.
- ⇒ `fill3DRect(int x, int y, int latime, int inaltime, boolean raised)` - desenează un dreptunghi "umplut cu culoare".

Primii parametri caracterizează orice dreptunghi. Ultimul parametru, de tip boolean, precizează dacă dreptunghiul este ridicat (`true`) sau coborât (`false`). Pentru a utiliza cu succes această metodă, trebuie să știm următoarele:

1. Metoda nu desenează decât un dreptunghi cu linia de grosime 1 pixel. Pentru a obține efectul scontat, trebuie s-o utilizăm de mai multe ori, realizând mai multe dreptunghiuri incluse unul în altul, dar cu laturile lipite.

2. Atunci când parametrul **raised** reține **true**, laturile din stânga și sus sunt desenate într-o culoare deschisă, iar laturile din dreapta și jos sunt desenate într-o culoare închisă. Dacă **raised** reține **false**, culorile laturilor se inversează. Exact aceste culori creează impresia de ridicat sau coborât.

3. Practic, culoarea deschisă este cea setată pentru desen, iar culoarea închisă este aleasă automat de metodă. Din acest motiv, dacă avem setată culoarea negru, efectul este nul, iar dacă avem setată culoarea alb efectul este maxim.

Exemplu: butoanele alăturate au fost realizate în clasa următoare:

```
class Imagine extends Canvas
{
    Imagine() { setSize(300,300); }
    public void paint (Graphics ecr1)
    {
        Graphics2D ecr=(Graphics2D) ecr1;
        ecr.setColor(Color.red);
        for (int i = 0; i <= 10; i++)
            ecr.draw3DRect(50 - i, 20 - i, 80 + 2 * i, 30 + 2 * i, true);
        for (int i = 0; i <= 10; i++)
            ecr.draw3DRect(200 - i, 20 - i, 80 + 2 * i, 30 + 2 * i,
                           false);
    }
}
```



10.11.7. Spațiul de coordonate al utilizatorului

Până în prezent am lucrat folosind exclusiv spațiul de coordonate al dispozitivului. Tot ce am desenat a fost raportat la acest spațiu. În **Graphics2D** se poate lucra într-un alt spațiu de coordonate, acela al utilizatorului. Mecanismul este următorul:

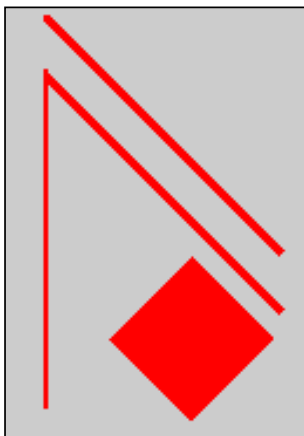
1. Inițial, spațiul de coordonate al dispozitivului coincide cu spațiul de coordonate al utilizatorului;

2. Spațiul de coordonate al utilizatorului poate deveni altul față de spațiul de coordonate al dispozitivului, în urma unor operații de translație și de rotație. Pentru a realiza aceste operații, se utilizează următoarele metode ale clasei **Graphics2D**:

- ⇒ **translate(int x1, int y1)** - are rolul ca, după executarea ei, originea spațiului de coordonate al utilizatorului să aibă originea în punctul de coordonate (**x1**, **y1**);
- ⇒ **rotate(double alfa)** - rotește spațiul de coordonate în sens invers trigonometric (sensul acelor de ceasornic cu unghiul transmis ca parametru). Unghiul **alfa** trebuie să fie exprimat în radiani. De acum, spațiul utilizatorului este cel obținut în urma rotației.

3. În urma operațiilor de translație și rotație, ce era desenat anterior rămâne nemodificat. În noul spațiu, se lucrează cu ceea ce se desenează de acum încolo.

Exemplu: Desenul de mai jos a fost obținut cu metoda următoare:



```
public void paint (Graphics ecr1)
{
    Graphics2D ecr=(Graphics2D) ecr1;
    ecr.setColor(Color.red);
    ecr.setStroke(new BasicStroke(4));
    Line2D.Float linie = new Line2D.Float(0,0,100,100);
    ecr.draw(linie);
    ecr.translate(0,25);
    ecr.draw(linie);
    ecr.rotate(Math.PI/4);
    ecr.draw(linie);
    Rectangle2D.Float drept = new Rectangle2D.Float(100,10, 50, 50);
    ecr.fill(drept);
}
```

Aceeași dreaptă (segment) a fost desenată de 3 ori:

- o dată clasic (prima din dreapta),
- apoi, în urma unei translații (a doua din dreapta),
- apoi, în urma unei rotații (linia verticală).

Se observă și faptul că după aplicarea unei rotații, pătratul se afișează rotit.

10.12. Evenimente

10.12.1. Generalități

În exemplele date până acum, majoritatea operațiilor erau efectuate ca urmare a apăsării unui buton. În realitate, mecanismul declanșării și tratării evenimentelor este cu mult mai complex. Există mai multe tipuri de evenimente și ele sunt tratate de “ascultători” diferiți. În continuare, le vom prezenta pe cele mai des utilizate în programarea vizuală.

10.12.2. Evenimente de tip **Action**

Aceste evenimente sunt produse de componente de tipurile:

- **JButton** - atunci când se execută click pe suprafața butonului;
- **JComboBox** - atunci când se operează o selecție în listă;
- **TextField** - atunci când se scrie în edit și se tastează **Enter**;
- **JCheckBox**, **JRadioButton** - atunci când se bifează sau se debifează butonul respectiv;
- **JMenuItem** - atunci când se selectează opțiunea respectivă.

Pentru a le capta, trebuie ca:

1. Fereastra care include componentele să implementeze interfața **ActionListener**. Interfața conține antetul metodei care trebuie scrisă:

⇒ **void actionPerformed(ActionEvent e)** - metodă prin care se programează acțiunea care trebuie întreprinsă, dacă evenimentul s-a produs.

Evenimentul produs are tipul **ActionEvent**. O metodă importantă a acestei clase este:

⇒ **String getActionCommand()** - returnează un șir de caractere după care se poate identifica componenta care a transmis evenimentul. Astfel, în cazul butoanelor, se returnează șirul afișat de ele; în cazul edit-ului, se returnează șirul de caractere introdus de utilizator; în cazul listelor, se returnează șirul **ComboBoxChanged**, iar în cazul componentelor de tip **JMenuItem**, se returnează șirul afișat de componentă.

2. Fiecare componentă care trebuie să emită un astfel de eveniment trebuie să fie adăugată “ascultătorului” de evenimente de tip **ActionEvent**.

Astfel, fiecare componentă care poate emite mesaje de acest tip conține metoda:

⇒ `void addActionListener(ActionListener l)` - prin care se adaugă componenta gestionarului curent (al ferestrei).

⇓ Testați programul următor pentru a va convinge!

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class Fer extends JFrame implements ActionListener
{
    public Fer(String titlu) {
        super(titlu);
        setSize(200,100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        //Butonul
        JButton A=new JButton("Buton 1");
        x.add(A);
        A.addActionListener(this);
        //Lista
        JComboBox lista=new JComboBox ();
        lista.addItem("Optiunea 1");
        lista.addItem("Optiunea 2");
        x.add(lista);
        lista.addActionListener(this);
        // Edit-ul
        JTextField t=new JTextField(10);
        x.add(t);
        t.addActionListener(this);
        // JCheckBox
        JCheckBox b=new JCheckBox("Bifeaza");
        x.add(b);
        b.addActionListener(this);
        // JCheckBox
        JRadioButton c=new JRadioButton("Radio");
        x.add(c);
        c.addActionListener(this);
        setVisible(true);
    }
    public void actionPerformed (ActionEvent e)
    { System.out.println(e.getActionCommand()); }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Test(ActionEvent");
    }
}
```

10.12.3. Evenimente de tip `MouseEvent`

Acestea sunt evenimente care se generează în urma acțiunilor executate cu mouse-ul. Componenta care recepționează astfel de evenimente trebuie să implementeze una sau două dintre interfețele prezentate mai jos:

Interfața `MouseListener` - conține antetele metodelor de mai jos, metode care se rulează atunci când se întâlnește un eveniment de mouse:

- ⇒ `mouseClicked(MouseEvent e)` - se execută atunci când se efectuează click pe suprafața componentei;
- ⇒ `mouseEntered(MouseEvent e)` - se execută atunci când cursorul mouse-ului ajunge pe suprafața grafică a componentei;
- ⇒ `mouseExited(MouseEvent e)` - se execută atunci când cursorul mouse-ului părăsește suprafața grafică a componentei;
- ⇒ `mousePressed(MouseEvent e)` - se execută atunci când am apăsât butonul mouse-ului;
- ⇒ `mouseReleased(MouseEvent e)` - se execută atunci când s-a eliberat butonul mouse-ului.

Interfața `MouseMotionListener` - conține următoarele antete:

- ⇒ `MouseDragged()` - metoda se execută *în mod continuu*, cât timp am apăsât un buton al mouse-ului și deplasăm cursorul (că la operația **Drag and Drop**);
- ⇒ `MouseMoved(MouseEvent e)` - se execută *în mod continuu*, cât timp cursorul mouse-ului se "plimbă" pe suprafața componentei.

După cum observați, parametrul oricărei metode care răspunde unei acțiuni efectuate cu mouse-ul este de tip `MouseEvent`. Printre altele, clasa `MouseEvent` conține următoarele două metode:

- ❖ `getX()` - returnează coordonata **x** a cursorului grafic;
- ❖ `getY()` - returnează coordonata **y** a cursorului grafic.

Ambele coordonate sunt relative la colțul din stânga-sus al componentei.

Evenimentele de mouse pot fi tratate de orice componentă care are la bază clasa `Component`, deci sfera de acțiune a sa este foarte largă. Pentru a implementa "ascultătorii" de evenimente de mouse, se utilizează metodele clasei `Component`:


```
⇒ void addMouseListener(MouseListener l)
    - pentru "ascultătorul" MouseListener;

⇒ void addMouseMotionListener(MouseMotionListener l)
    - pentru "ascultătorul" MouseMotionListener.
```

Exercițiu: urmăriți modul în care se generează evenimentele de mouse, rulând programul următor. Pentru fiecare eveniment produs, în fereastra CMD se afișează numele evenimentului.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.awt.image.*;

class Imagine extends Canvas implements MouseListener,
MouseMotionListener
{
    Imagine()
    {
        setSize(200,200);
        setBackground(Color.RED);
        this.addMouseListener(this);
        this.addMouseMotionListener(this);
    }
    public void mouseClicked(MouseEvent e)
    { System.out.println("MouseClicked"); }
    public void mouseEntered(MouseEvent e)
    { System.out.println("MouseEntered"); }
    public void mouseExited(MouseEvent e)
    { System.out.println("MouseExitetted"); }
    public void mousePressed(MouseEvent e)
    { System.out.println("MousePressed"); }
    public void mouseReleased(MouseEvent e)
    { System.out.println("MouseReleased"); }
    public void mouseDragged(MouseEvent e)
    { System.out.println("MouseDragged"); }
    public void mouseMoved(MouseEvent e)
    { System.out.println("MouseMoved"); }
}

class Fer extends JFrame
{
    Imagine p=new Imagine();
    public Fer(String titlu) {
        super(titlu); setSize(400,400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        x.add(p); setVisible(true);
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("MouseEvent");
    }
}
```

Aplicație - program prin care se desenează cu mouse-ul. Programul următor va exemplifica modul în care se poate utiliza mouse-ul pentru a desena.

🕒 Observați că:

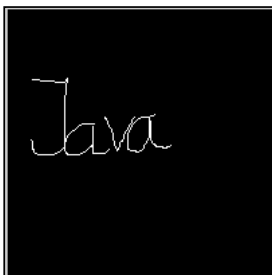
1. Desenul este reținut de o componentă dintr-o clasă care moștenește clasa `Canvas`. Clasa componentei implementează `MouseListener` și `MouseMotionListener`.

2. Desenul este practic realizat în memorie, prin utilizarea unui obiect de tip `BufferedImage`, iar metoda `Paint` nu face altceva decât să deseneze pe ecran imaginea din memorie.

3. Evenimentele de mouse sunt generate astfel:

a) `mousePressed()` - memorează în `pozvX` și `pozvY` coordonatele cursorului grafic în momentul în care se apasă butonul mouse-ului.

b) `void mouseDragged()` - trasează un segment de dreaptă între coordonatele memorate în `pozvX` și `pozvY` și cele actuale, memorate în `poznX` și `poznY`. Apoi, coordonatele actuale sunt memorate în `pozvX` și `pozvY`. După aceasta, se redesenează imaginea prin utilizarea metodei `repaint()`. Cum evenimentul se generează în mod continuu, se obține efectul scontat.



```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.awt.image.*;

class Imagine extends Canvas implements MouseListener,
                                         MouseMotionListener
{
    Image im;
    BufferedImage desen;
    Graphics gr;
    int pozvX, pozvY, poznX, poznY;

    Imagine()
    {
        desen = new BufferedImage(
            200, 200, BufferedImage.TYPE_INT_RGB);
        gr = desen.createGraphics();
        im=Toolkit.getDefaultToolkit().createImage(desen.getSource());
        setSize(200,200);
        this.addMouseListener(this);
        this.addMouseMotionListener(this);
    }

    public void paint (Graphics ecr)
    { ecr.drawImage(im,0,0,this); }

    public void update(Graphics ecr) { paint(ecr);}

    public void mouseClicked(MouseEvent e) { }
```

```

public void mouseEntered(MouseEvent e) {}
public void mouseExited(MouseEvent e) {}
public void mousePressed(MouseEvent e)
{ pozvX=e.getX();
  pozvY=e.getY();
}

public void mouseReleased(MouseEvent e) {}
public void mouseDragged(MouseEvent e)
{ poznX=e.getX();
  poznY=e.getY();
  gr.drawLine(pozvX,pozvY,poznX,poznY);
  pozvX=poznX;
  pozvY=poznY;
  im=Toolkit.getDefaultToolkit().createImage(desen.getSource());
  repaint();
}

public void mouseMoved(MouseEvent e){}
}

class Fer extends JFrame
{
    Imagine p=new Imagine();

    public Fer(String titlu) {
        super(titlu);
        setSize(400,400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        x.add(p);
        setVisible(true);
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Imagine");
    }
}

```



Observați faptul că programul conține și metoda **update** (Graphics ecr). Ce este cu ea, de ce am pus-o?

Pentru a redesena, se apelează **repaint()**. Practic, metoda apelează **update()** și, aceasta din urmă apelează **paint()**. Metoda **update()**, apelată implicit, are rolul de a șterge suprafața desenată și de a o afișa, după care se apelează **paint()**. Apoi, metoda **paint()** realizează noul desen. Desigur, toate acestea se produc într-un timp foarte scurt. Cu toate acestea, apare o "pâlpâire" a imaginii, care deranjează. Soluția? Metoda **update()** a fost rescrisă. În noua "versiune", metoda nu mai cuprinde ștergerea imaginii. Aceasta are un efect deosebit în acest caz, deoarece întotdeauna noua imagine se obține din precedenta, la care se adaugă un segment!

10.12.4. Focus, evenimente de tip `FocusEvent`

Prin faptul că o componentă are **focus-ul**, înțelegem că respectiva componentă este pregătită să primească date sau comenzi de la tastatură. Putem distinge vizual componentele care au **focus-ul**. De exemplu, dacă un buton are focus-ul, atunci el apare ca selectat (conține pe suprafața lui un dreptunghi care marchează acest fapt). Dacă este programat să răspundă la evenimente de tip **Action**, simpla apăsare a blank-ului (și nu **Enter**) are același efect ca executarea unui click pe suprafața lui. Dacă un edit (componentă de tip `JTextField`) are focus-ul, atunci ce se tastează apare în edit. De asemenea, în interiorul lui, în dreptul caracterului care urmează să fie introdus, clipește cursorul de editare. Pentru a putea lucra cu focus-ul, trebuie să mai știm următoarele:

1. O componentă primește focus-ul dacă se execută click pe suprafața ei.
2. Pentru ca focus-ul să aparțină componentei următoare, se apasă **Tab**. Ordinea în care se parcurg componentele, la apăsarea tastei **Tab**, este de la stânga la dreapta, de sus în jos. Astfel, teoretic, se poate lucra fără mouse.
3. Se poate programa ca o anumită componentă să primească focus-ul. Se va utiliza metoda:

⇒ `void requestFocus()` – aparține de clasa `Component`.

4. Pentru a răspunde la evenimente, se implementează interfața `FocusListener` cu cele două metode ale ei:

⇒ `void focusGained(FocusEvent e)` – metoda se rulează atunci când o componentă primește focus-ul;

⇒ `void focusLost(FocusEvent e)` – metoda se rulează atunci când o componentă pierde focus-ul.

Ambele metode conțin un parametru de tip `FocusEvent`. O metoda importantă a acestei clase este:

⇒ `Component getComponent()` – metoda returnează o referință la componenta care a recepționat evenimentul. Pornind de la această referință, se poate detecta componenta ca în exemplul următor:

Exemplul 1. Programul următor probează cele prezentate. Fereastra conține două butoane și un edit. Inițial, "**Butonul 1**" are implicit focus-ul. Dacă apăsați **Blank** veți obține același efect ca și când ați executat click pe suprafața butonului. Urmăriți atribuirea focus-ului prin apăsarea tastei **Tab**. Atunci când o componentă primește focus-ul, metoda `focusGained()` afișează numele ei.

```

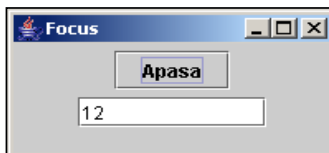
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

class Fer extends JFrame implements FocusListener, ActionListener
{
    JTextField txt;
    public Fer(String titlu) {
        super(titlu);
        setSize(200,100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        JButton A=new JButton("Buton 1"); x.add(A);
        A.setName("Buton 1");
        A.addFocusListener(this);
        A.addActionListener(this);
        JButton B=new JButton("Buton 2"); x.add(B);
        B.addFocusListener(this);
        B.setName("Buton 2");
        txt =new JTextField(10);
        txt.setName("Edit");
        txt.addFocusListener(this);
        x.add(txt);
        setVisible(true);
    }
    public void focusGained(FocusEvent e)
    { System.out.println(e.getComponent().getName());}
    public void focusLost(FocusEvent e) {}
    public void actionPerformed (ActionEvent e)
    { System.out.println("blank sau click");}
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Focus");
    }
}

```

Exemplul 2. Validare. Una dintre cele mai frecvente utilizări a focus-ului este dată de operația de validare. **Concret:** când un edit pierde focus-ul, în metoda `focusLost()` se scrie secvența de validare. Dacă data introdusă este incorectă, se afișează mesajul corespunzător și se redirectionează focus-ul către edit. În acest fel, nu se poate părăsi operația de editare până când nu am introdus o valoare corectă. În exemplu, se cere ca valoarea introdusă să fie de tip `double`.



```

import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

```

```

class Fer extends JFrame implements FocusListener
{
    JTextField txt;

    public Fer(String titlu) {
        super(titlu); setSize(200,100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        x.setLayout(new FlowLayout ());
        JButton A= new JButton("Apasa"); x.add(A);
        txt =new JTextField(10);
        txt.addFocusListener(this);
        x.add(txt); setVisible(true);
    }

    public void focusGained(FocusEvent e){}
    public void focusLost(FocusEvent e)
    {
        String t=txt.getText();
        try { double x=Double.parseDouble(txt.getText());
            System.out.println(t);
        }
        catch (Exception ex)
        { System.out.println("Numar eronat"); txt.requestFocus();}
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("Focus");
    }
}

```

10.12.5. Evenimente de selectare a item-ilor

Astfel de evenimente apar pentru componente precum `JCheckBox` sau `JComboBox`. Ele se pot trata în mod direct, imediat ce selectarea a fost implementată. Pentru aceasta, trebuie utilizată interfața `ItemListener` care conține o singură metodă:

⇒ `void itemStateChanged(ItemEvent e)` - metodă care se apelează de câte ori se produce o deselectare sau o selectare. **Atenție:** dacă, de exemplu, într-o listă se selectează o altă opțiune, mai întâi opțiunea curentă este deselectată, după care este selectată opțiunea aleasă de utilizator. Aceasta înseamnă că, în acest caz, metoda se apelează de două ori.

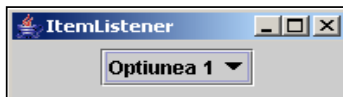
Să observăm că parametrul metodei de mai sus este de tipul `ItemEvent`. O metodă importantă a acestei clase este:

❖ `int getStateChange()` - returnează o valoare de tip `int` corespunzătoare uneia din cele două posibilități de generare a evenimentului (deselectarea sau selectarea). Cele două valori sunt `SELECTED` și `DESELECTED` (constante ale clasei `ItemEvent`).



Componentele care pot transmite astfel de evenimente conțin metoda `addItemListener(ItemLitener a)`.

Exemplu: programul următor afișează o fereastră care conține o listă. La fiecare selectare, se afișează în fereastra CMD numele opțiunii selectate. Observați faptul că, de această dată, ascultătorul de evenimente a fost implementat ca o clasă aparte (**AsLista**).



```
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

class AsLista implements ItemListener
{
    JComboBox lista;
    AsLista(JComboBox lista)
    {
        this.lista=lista;
    }

    public void itemStateChanged (ItemEvent e)
    {
        if (e.getStateChange()==ItemEvent.SELECTED)
            System.out.println(lista.getSelectedItem());
    }
}

class Fer extends JFrame
{
    JComboBox lista; JSpinner s;
    public Fer(String titlu) {
        super(titlu); setSize(200,200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x=getContentPane();
        lista=new JComboBox ();
        x.setLayout(new FlowLayout());
        lista.addItem("Optiunea 1"); lista.addItem("Optiunea 2");
        lista.addItem("Optiunea 3"); lista.addItem("Optiunea 4");
        x.add(lista);
        AsLista as=new AsLista(lista);
        lista.addItemListener(as);
        setVisible(true);
    }
}

public class pv {
    public static void main(String args[]) {
        Fer fp=new Fer("ItemListener");
    }
}
```

10.12.6. Evenimente pentru fereastră

Obiectele de tip **JFrame** pot transmite următoarele evenimente:

- ⇒ **WINDOW_ACTIVATED** - se produce atunci când fereastra devine activă. Acest eveniment se poate întâmpla atunci când fereastra a fost afișată pentru prima dată, atunci când după minimizarea ei se revine la dimensiunea inițială sau atunci când a fost activată o altă fereastră (s-a executat click pe suprafața ei) după care este reactivată fereastra studiată.

- ⇒ **WINDOW_OPENED** - se produce atunci când este deschisă fereastra.
- ⇒ **WINDOW_DEACTIVATED** - se produce când fereastra devine inactivă. Evenimentul se poate întâmpla atunci când am activat o altă fereastră sau atunci când am minimizat-o.
- ⇒ **WINDOW_CLOSING** - se produce atunci când se închide fereastra.
- ⇒ **WINDOW_ICONIFIED** - se produce atunci când am minimizat fereastra.
- ⇒ **WINDOW_DEICONIFIED** - se produce atunci când se execută click pe fereastra minimizată.
- ⇒ **WINDOW_CLOSED** - se produce după ce fereastra a fost închisă.

Pentru fiecare eveniment, interfața **WindowListener** conține antetul unei metode care trebuie apelată. Legătura dintre numele evenimentelor și cel al metodelor este evidentă.

Astfel, avem următoarele antete în **WindowListener**:

```
public void windowOpened(WindowEvent e)
public void windowClosing(WindowEvent e)
public void windowClosed(WindowEvent e)
public void windowIconified(WindowEvent e)
public void windowDeiconified(WindowEvent e)
public void windowActivated(WindowEvent e)
public void windowDeactivated(WindowEvent e)
```

Metoda clasei **JFrame**,

```
void addWindowListener(WindowListener a)
```

are rolul de a asocia ferestrei ascultătorul de evenimente.

Exemplu: programul de mai jos vă permite să studiați "pe viu" evenimentele declanșate atunci când se efectuează diverse operații cu fereastra. Numele evenimentelor este afișat în fereastra **CMD**:

```
import javax.swing.*;
import java.awt.event.*;

class InchiF extends Evfer
{
    public void windowClosing(WindowEvent e)
    {
        System.out.println("WINDOW_CLOSING");
        System.exit(0);
    }

    public void windowOpened(WindowEvent e)
    {
        System.out.println("WINDOW_OPENED");
    }

    public void windowActivated(WindowEvent e)
    {
        System.out.println("WINDOW_ACTIVATED");
    }
}
```



```

public void windowDeactivated(WindowEvent e)
{ System.out.println("WINDOW_DEACTIVATED"); }

public void windowIconified(WindowEvent e)
{ System.out.println("WINDOW_ICONIFIED"); }

public void windowDeiconified(WindowEvent e)
{ System.out.println("WINDOW_DEICONIFIED"); }
}

public class td {
    public static void main(String args[]) {
        JFrame fp=new JFrame("Fereastra care se inchide");
        WindowListener t=new InchidF();
        fp.addWindowListener(t);
        fp.setSize(200,300);
        fp.setVisible(true);
    }
}

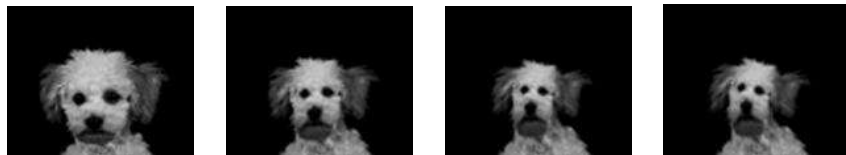
```

10.13. Animarea imaginilor

Este momentul să prezentăm modul în care se realizează în Java animarea imaginilor. În principal, dacă se dispune de o succesiune de imagini, mecanismul este simplu:

1. Se încarcă imaginile în memorie, într-un vector de componente de tip **Image**.
2. O imagine este afișată un interval de timp, apoi se afișează următoarea imagine un interval de timp, ș.a.m.d.

Exemplu: avem mai multe imagini succesive în fișierele **Poza1.jpg**, **Poza2.jpg**,..., **Poza12.jpg**. O fereastră afișează la nesfârșit imaginile în ordinea 1, 2...12, apoi 12, 11, ...1, și iar, 1, 2,...12, etc...



Poza1.jpg.....Poza4.jpg.....Poza8.jpg.....Poza12.jpg

De fapt, efectul obținut este o **iluzie optică**. Dacă animația este realizată într-un mod coerent și intuitiv, ea va fi percepută foarte ușor de privitori. Totul depinde de îndemânarea și talentul programatorului la crearea unor efecte vizuale cât mai realiste.

Programul este următorul:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fir extends Thread {
    // fir de executie pentru afisarea imaginilor la nesfarsit
    // constructorul primeste ca parametru referinta catre imagine
    Image imag ;
    int i;

    Fir (Image imag) { this.imag=imag;}

    public void run()
    { while (true)
      { while (i!=11)
        { i++; imag.afis(i);
          try { sleep(300); }
          catch (Exception e) {} }
        while (i!=0)
        { i--; imag.afis(i);
          try { sleep(300); }
          catch (Exception e) {} }
      }
    }
}

class Imagine extends Canvas
{ Image[] im= new Image[12];
  int i=0;
  Imagine()
  { Toolkit ec=Toolkit.getDefaultToolkit();
    for (int i=0;i<=11;i++)
      im[i]=ec.getImage("C:\\Foto\\Poza"+(i+1)+".jpg");
    setSize(300,300);
    setBackground(Color.YELLOW);
  }
  public void paint (Graphics ecr)
  { ecr.drawImage(im[i],0,0,300,300,this);}
  public void update(Graphics ecr)
  { paint(ecr);}
  void afis(int i)
  { this.i=i;
    repaint();}
}

class Fer extends JFrame
{ Imagine p=new Imagine();
  public Fer(String titlu) {
    super(titlu); setSize(400,400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    Container x=getContentPane();
    x.setLayout(new FlowLayout ()); x.add(p);
    Fir f=new Fir(p); f.start();
    setVisible(true);
  }
}
```

```
public class pv {  
    public static void main(String args[]) {  
        Fer fp=new Fer("Imagine");  
    }  
}
```

🕒 Observații asupra programului:

1. Clasa **image** este asemănătoare cu clasa care afișează o imagine și este dată ca exemplu în această carte (vezi 10.10.1). Deosebirile sunt date de următoarele:

a) Nu se reține o imagine, ci 12. Ele sunt încărcate de constructorul clasei.

b) Clasa are data membru **i**. Metoda **paint()** desenează imaginea cu indicele **i**.

c) Metoda **afis()** are rolul de a transmite valoarea care este memorată de **i** și de a comanda redesenarea imaginii.

d) Metoda **update()** este redefinită, pentru că dacă n-ar fi fost redefinită, înainte de a fi desenată noua imagine de **paint()**, vechea imagine ar fi fost ștearsă. Aceasta ar crea efectul de "pălăire".

2. Practic, clasa **Fir** este un fir de executare care asigură animarea imaginilor. S-a procedat în felul următor:

a) Constructorul ei primește ca parametru referința către obiectul de tip **Image** și o memorează;

b) Pornind de la această referință, se apelează continuu metoda **afis()** a clasei **Image**, metodă care redesează imaginea de indice dat.

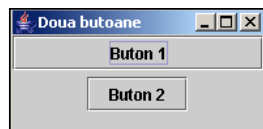
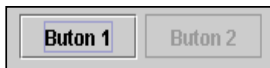
Probleme propuse

1. Scrieți un program în care o fereastră afișează un buton. La apăsarea butonului, programul va afișa în fereastra **CMD** rezoluția ecranului.

2. Scrieți un program în care o fereastră afișează două butoane: **Buton 1** și **Buton 2**. Știind că metoda **boolean isEnabled()** a clasei

JComponent returnează **true** dacă butonul este activ și **false** în caz contrar, faceți în așa fel încât, la apăsarea primului buton, dacă al doilea buton este activ să devină inactiv și invers, dacă este inactiv să devină activ.

3. Cum se pot așeza două butoane în fereastră ca în exemplul alăturat? Observați faptul că primul buton ocupă toată partea de sus, iar al doilea este așezat centrat pe orizontală, în partea de jos.



4. Cum se poate ca o fereastră să afișeze cinci butoane, precum cea de mai jos:



5. Prin utilizarea gestionarului de poziționare `GridBagLayout`, creați fereastra de mai jos:



6. Se știe că pentru a citi o parolă se utilizează o componentă de tip `JPasswordField` (clasă care are ca superclasă pe `JTextField`). Se știe de asemenea, că pentru o componentă de tip `JPasswordField`, metoda `getText()` este depreciată și în locul ei se utilizează metoda: `char[] getPassword()`. Se cere ca programul dvs. să citească o parolă și să afișeze OK dacă parola introdusă este șirul "Mama" și NO în caz contrar.



7. Realizați un program în care într-un edit să se poată introduce numere prin apăsarea unor butoane, ca alăturat. Extindeți apoi programul pentru simularea unui calculator de buzunar.



8. Să se scrie un program care conține trei metode prin care se determină dacă un număr natural n citit este prim sau nu. Prima dintre ele testează toți divizorii numărului de 2 la $n-1$, a

doua divizorii de la 2 la $\frac{n}{2}$, a treia de 2 la $\sqrt{n}$.

Pentru citirea numărului, fereastra va conține o etichetă și un edit, pentru selectarea algoritmului se va utiliza o listă, iar după introducerea numărului și selectarea algoritmului, un buton va declanșa prelucrarea corespunzătoare.

9. Programul dvs. va efectua o copie a un fișier text. Numele fișierului care se copiază va fi introdus într-un edit, iar numele fișierului destinație în alt edit.

10. Scrieți un program care generează un fișier text cu n numere naturale. Valoarea lui n va fi introdusă cu ajutorul unui edit!

11. Scrieți un program care creează, pornind de la fișierul anterior, un altul care conține aceleași numere, dar sortate crescător. Se vor implementa mai mulți algoritmi de sortare, iar selecția celui care va sorta fișierul se va face cu ajutorul unei liste aflată pe fereastra principală.

12. Scrieți un program care citește două numere reale. Programul va putea afișa suma, diferența, produsul și câtul lor. Operația care se efectuează se va selecta dintr-un meniu. Rezultatul va fi afișat cu ajutorul unor cutii de dialog predefinite.

13. Creați un fișier text în care fiecare linie conține numele unei persoane și vârsta ei. Datele se introduc secvențial. Pentru a le introduce, o fereastră va avea atașate două edit-uri (unul pentru nume și altul pentru vârstă) și un buton. La apăsarea butonului se va efectua validarea datelor (alfabetic pentru nume și număr natural între 1 și 99 pentru vârstă). Dacă datele introduse nu corespund, se va afișa un mesaj de avertizare și utilizatorul va fi invitat să introducă corect datele. Dacă datele sunt introduse corect, se va scrie linia corespunzătoare în fișierul text.

14. Adăugați o listă ferestrei de la problema precedentă din care se pot selecta mai multe opțiuni, în urma cărora programul va putea afișa:

- a) Media vârstelor persoanelor a căror date au fost trecute în fișier;
- b) Numărul de persoane care au datele trecute în fișier.

15. Scrieți un program care afișează și actualizează fișierul creat la problema 13. Pentru a realiza operațiile propuse se va utiliza o componentă de tip **JTextArea**, iar operațiile **Open**, **Save** și **Save As** se vor selecta dintr-un meniu.

16. Scrieți un program care generează un fișier text care conține cel puțin 10.000 numere naturale aleatoare. Apoi, scrieți un alt program care sortează fișierul prin utilizarea mai multor metode de sortare. Metoda de sortare se va selecta dintr-o listă. La fiecare sortare, se va estima timpul rămas prin utilizarea unei componente de tip **JProgressBar**.

17. Scrieți un program în care o componentă de tip **JTextArea** afișează un text preluat dintr-un fișier text dat. Se cere să se seteze font-ul prin selectarea sa dintr-o listă și mărimea lui prin utilizarea unei componente de tip **JSpinner**.

18. Scrieți un program în care se creează și se actualizează un fișier text. Textul se introduce cu ajutorul unei componente de tip **JTextArea**. Pentru

deschiderea fișierului și salvarea lui, se utilizează cutii de dialog de tip **Open / Save** apelabile dintr-un meniu.

19. Completați programul anterior astfel încât culoarea suprafeței componente de tip **JTextArea** să poată fi selectată cu ajutorul unei componente de tip **JColorChooser**, apelabilă cu ajutorul unui buton aflat pe bara de unelte. De asemenea, fontul va fi selectat dintr-o listă, iar mărimea lui cu ajutorul unei componente de tip **JSpinner**, ambele aflate pe bara cu unelte (componentă de tip **JToolBar**).

20. Citiți două matrice **4x4**. Afișați produsul lor. Pentru citire și afișare se vor utiliza componente de tip **JTable**.

21. Scrieți un program care creează și actualizează fișierul text **persoane.txt**. Fișierul conține, pe fiecare linie, numele și vârsta unei persoane. Pentru introducerea datelor și actualizarea lor se va utiliza o componentă de tip **JTable**.

22. La fel ca la problema 21, numai că pentru introducerea datelor sau actualizarea lor se se utilizează mai multe cutii de dialog create de dvs.

23. Scrieți un program în care fereastra va afișa, la alegere, mai multe imagini. Imaginea afișată va fi selectată dintr-o listă.

24. Scrieți un program care afișează toate soluțiile pentru problema celor n dame. În exemplu, se consideră $n=8$. După afișarea unei soluții, pentru a fi afișată următoarea soluție, se va apăsa un buton. Pentru afișare, veți utiliza o componentă de tip **JTable**.

25. Scrieți un program în care fereastra afișează în permanență un segment care se rotește în jurul unei extremități.

26. Programul va afișa un pătrat care se rotește în jurul unui colț al său.

27. Scrieți un program care vă permite să realizați desene. Cerințe: se pot utiliza mai multe culori, selectabile cu ajutorul unei componente de tip **JColorChooser**. De asemenea, programul vă va permite să stergeți porțiuni din desen.

Indicații:

1. La apăsarea butonului, răspunde metoda:

```
public void actionPerformed (ActionEvent e)
{
    Toolkit ec=Toolkit.getDefaultToolkit();
    Dimension dim = ec.getScreenSize();
    System.out.println(dim.width);
    System.out.println(dim.height);
}
```

2. **JButton B** trebuie declarat ca dată membru a clasei pentru a avea acces la el din metoda următoare, care răspunde la apăsarea primului buton:

```
public void actionPerformed (ActionEvent e)
{ if (B.isEnabled()) B.setEnabled(false);
  else B.setEnabled(true);
}
```

3. Fereastra are gestionarul de poziționare **BorderLayout**. În nord, avem primul buton. În centru avem un panel. Acesta se extinde pe întreaga suprafață rămasă. La rândul lui, panelul este gestionat de **FlowLayout**. Lui îi adăugăm al doilea buton.

```
...
Container x=getContentPane();
x.setLayout(new BorderLayout ());
JButton A=new JButton("Buton 1");
x.add(A,BorderLayout.NORTH);
JPanel panel=new JPanel();
x.add(panel,BorderLayout.CENTER);
panel.setLayout(new FlowLayout());
B=new JButton("Buton 2");
panel.add(B);
...
```

4.

```
...
Container x=getContentPane();
x.setLayout(new BorderLayout ());
JButton A=new JButton("Buton 1");
x.add(A,BorderLayout.SOUTH);
JPanel panel=new JPanel();
x.add(panel,BorderLayout.CENTER);
panel.setLayout(new GridLayout(2,2));
JButton B=new JButton("Buton 2"); panel.add(B);
JButton C=new JButton("Buton 3"); panel.add(C);
JButton D=new JButton("Buton 4"); panel.add(D);
JButton E=new JButton("Buton 5"); panel.add(E);
...
```

5.

```
class Fer extends JFrame
{
    public Fer(String titlu) {
        super(titlu); setSize(250,200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        GridBagLayout pozitii=new GridBagLayout();
        JLabel et=new JLabel ("Numele ");
        JTextField txt=new JTextField(15);
        JLabel et1=new JLabel ("Varsta");
        JTextField txt1=new JTextField(2);
        JButton bt=new JButton("Apasa");
        Container x=getContentPane();
```

```

// pozitionare et
GridBagConstraints p_et=new
GridBagConstraints(0,0,1,1,2,GridBagConstraints.WEST,
    GridBagConstraints.NONE,new Insets(0,0,0,0), 0,0);
pozitii.setConstraints(et,p_et);
// pozitionare txt
GridBagConstraints p_txt=new
GridBagConstraints(1,0,1,1,4,2,GridBagConstraints.WEST,
    GridBagConstraints.NONE,new Insets(0,0,0,0), 0,0);
pozitii.setConstraints(txt,p_txt);
// pozitionare et1
GridBagConstraints p_et1=new
GridBagConstraints(0,1,1,1,1,2,GridBagConstraints.WEST,
    GridBagConstraints.NONE,new Insets(0,0,0,0), 0,0);
pozitii.setConstraints(et1,p_et1);
// pozitionare txt1
GridBagConstraints p_txt1=new
GridBagConstraints(1,1,1,1,4,2,GridBagConstraints.WEST,
    GridBagConstraints.NONE,new Insets(0,0,0,0), 0,0);
pozitii.setConstraints(txt1,p_txt1);
// pozitiona buton
GridBagConstraints p_bt=new
GridBagConstraints(0,2,2,1,1,1,GridBagConstraints.CENTER,
    GridBagConstraints.NONE,new Insets(0,0,0,0), 0,0);
pozitii.setConstraints(bt,p_bt);
x.setLayout(pozitii);
x.add(et);
x.add(txt);
x.add(bt);
x.add(et1);
x.add(txt1);
setVisible(true);
}
}

```

6. Utilizați metoda:

```

public void actionPerformed (ActionEvent e)
{
    String parola=new String(txt.getPassword());
    if (parola.compareTo("Mama")==0)
        System.out.println("OK");
    else System.out.println("KO");
}

```

7. Procedați ca mai jos:

```

class Fer extends JFrame implements ActionListener
{
    JButton zero=new JButton("0");
    JButton unu=new JButton("1");
    JButton doi=new JButton("2");
    JButton trei=new JButton("3");
    JButton patru=new JButton("4");
}

```



```
JButton cinci=new JButton("5");
JButton sase=new JButton("6");
JButton sapte=new JButton("7");
JButton opt=new JButton("8");
JButton noua=new JButton("9");
JTextField txt=new JTextField("");
JPanel panel=new JPanel();

public Fer(String titlu) {
    super(titlu);
    setSize(200,200);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    Container x=getContentPane();
    x.setLayout(new BorderLayout ());
    panel.setLayout(new GridLayout(4,3));
    x.add(txt,BorderLayout.NORTH);
    x.add(panel,BorderLayout.CENTER);
    panel.add(unu);
    panel.add(doi);
    panel.add(trei);
    panel.add(patru);
    panel.add(cinci);
    panel.add(sase);
    panel.add(sapte);
    panel.add(opt);
    panel.add(noua);
    panel.add(zero);
    unu.addActionListener(this);
    doi.addActionListener(this);
    trei.addActionListener(this);
    patru.addActionListener(this);
    cinci.addActionListener(this);
    sase.addActionListener(this);
    sapte.addActionListener(this);
    opt.addActionListener(this);
    noua.addActionListener(this);
    zero.addActionListener(this);
    setVisible(true);
    txt.setHorizontalAlignment(JTextField.RIGHT);
}

public void actionPerformed (ActionEvent e)
{ txt.setText(txt.getText()+e.getActionCommand());
}
}
```

CAPITOLUL 11

Applet-uri



Din cuprins:

Crearea și executarea applet-urilor
Arhive Java și atributele elementului APPLET
Redefinirea unor metode ale applet-ului
Afișarea imaginilor. Clasa URL
Redarea sunetelor
Exemple de applet-uri

11.1. Generalități

Pentru a înțelege acest capitol, trebuie să aveți noțiuni despre **HTML** și modul în care se “postează” pe Internet o pagină **WEB**. Aceste noțiuni pot fi găsite, fie în “**Crearea și programarea paginilor WEB**”, fie în “**Tehnologia Informației și a Comunicațiilor**”, manual pentru clasa a IX-a, ambele scrise de autorii acestei cărți.

Important:

1. Un applet este asemănător unei ferestre. El poate conține una sau mai multe componente pe suprafața sa.
2. Pentru crearea unui applet vom utiliza o componentă de tip **JApplet**.
3. Applet-ul este atașat unui fișier **HTML**, la fel ca o imagine.
4. Applet-ul este afișat de către browser-ul calculatorului de pe care se vizualizează pagina **WEB** și este rulat tot de acesta.
5. Pentru ca browser-ul să poată rula applet-ul este necesar ca pe acel calculator să se găsească implementată (instalată) o mașină virtuală Java (**Java Virtual Machine, JVM**). Cum noi avem deja implementat soft-ul de dezvoltare Java, nu mai avem nici o problemă, deoarece **JVM** este inclus în acesta. Dar, un utilizator oarecare poate descărca și instala **JVM** de pe site-ul firmei **Sun**.
6. Măsurile de securitate impun anumite restricții. Vă dați seama ce ar fi dacă nu s-ar lua măsurile de securitate de mai jos:
 - ✱ Applet-urile nu pot scrie sau citi fișiere;
 - ✱ Nu pot apela programe aflate pe sistemul utilizatorului.

11.2. Crearea și executarea applet-urilor

Mai jos aveți un exemplu de applet. Pe suprafața sa se găsește un buton, la apăsarea căruia se afișează o cutie de dialog cu un mesaj oarecare. Observați cât de asemănător este acest program cu un altul care afișează o fereastră cu un buton. Și aici avem un container căruia i s-a adăugat un buton, am implementat **ActionListener** pentru ca butonul să răspundă la apăsare. Pentru crearea și afișarea unui applet, trebuie să respectați următorii pași:

- ① Textul applet-ului se va găsi într-un fișier text. Pentru a-l scrie și corecta, putem folosi **Notepad**-ul. Fișierul text care-l conține, va avea extensia **.java**. Numele fișierului trebuie să coincidă cu numele clasei care conține applet-ul, în exemplu **Ap.java**.

```
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

public class Ap extends JApplet implements ActionListener
{
    public void init () {
        Container x = getContentPane();
        x.setLayout(new FlowLayout ());
        JButton A=new JButton("merge");
        A.addActionListener(this);
        x.add(A);
    }

    public void actionPerformed (ActionEvent e)
    {
        JOptionPane.showMessageDialog
            (this, "La fel ca la fereastra clasica");
    }
}
```

② Compilăm programul așa cum suntem deja obișnuiți: pentru exemplul dat, vom scrie comanda `javac Ap.java`.

③ Scriem un text **HTML** (tot cu **Notepad**) care include applet-ul scris, ca mai jos. Observați faptul că prin **WIDTH** și **HEIGHT** am precizat (în pixeli) lățimea, respectiv înălțimea dreptunghiului în care se încadrează applet-ul. Fișierul are în mod obligatoriu extensia `.html`, dar numele lui poate fi oarecare (nu este obligatoriu să coincidă cu cel al applet-ului):

```
<HTML>
<BODY>
<APPLET CODE="Ap.class" WIDTH=300 HEIGHT=300>
</BODY>
</HTML>
```

④ Testăm rezultatul, executând dublu click pe fișierul cu extensia `.html`. Automat, va fi apelat browser-ul care va afișa applet-ul.

⑤ După ce fișierul **HTML** este gata (putem adăuga orice altceva în el, așa cum suntem obișnuiți), îl transferăm pe site cu ajutorul unui program **FTP** (eventual cu programul **WS_FTP**). De asemenea, transferăm pe site și applet-ul compilat (cu extensia `class`). De acum, îl putem apela de pe orice calculator cu acces la Internet și care are implementată **JVM**.



11.3. Arhive Java și atributele elementului APPLET

Ce se întâmplă în situația în care applet-ul utilizează mai multe clase? Dar dacă, în plus, sunt necesare și fișiere imagine? După cum vedem, în fișierul **HTML** se declară un singur fișier de tip **class**, dar la compilare, pentru fiecare clasă rezultă câte un fișier de acest tip...

Mediul de dezvoltare Java conține și un program care poate crea arhive. O **arhivă** este un fișier alcătuit din mai multe fișiere. Prin arhivare înțelegem operația prin care, pornind de la mai multe fișiere, se creează o arhivă în care datele sunt compresate, iar prin dezarhivare se înțelege operația prin care, pornind de la o arhivă, se refac toate fișierele care o alcătuiesc.

În Java, arhivele se creează cu programul **jar**, furnizat de mediul de dezvoltare (ca și **javac**, de exemplu). Arhivele pot fi recunoscute după extensia lor: **.jar**. Forma simplificată a apelului programului **jar** pentru crearea unei arhive alcătuită din fișierele: **f1, f2,... fn** este:

```
jar cf nume_arhiva f1 f2... fn
```



Puteti vedea toti parametrii programului **jar**, dacă-l apelați fără nici un parametru !

Exemplu: în mod forțat, applet-ul dat ca exemplu în paragraful anterior a fost creat prin utilizarea a două clase:

```
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

class Buton extends JButton
{ }

public class Ap extends JApplet implements ActionListener
{ public void init () {
    Container x = getContentPane();
    x.setLayout(new FlowLayout ());
    Buton A=new Buton();
    A.addActionListener(this);
    x.add(A);
}

    public void actionPerformed (ActionEvent e)
    { JOptionPane.showMessageDialog(this, "La fel ca la fereastra
        clasica");
    }
}
```

La compilare, rezultă două fișiere cu extensia `.class`: `Ap.class` și `Buton.class`. Formăm arhiva `arh.jar` prin comanda:

```
jar cf arh.jar Ap.class Buton.class
```

Mai jos, se poate observa cum arată de această dată fișierul **HTML**:

```
<HTML>
<BODY>
<APPLET CODE="Ap.class" ARCHIVE="arh.jar" WIDTH=300 HEIGHT=300>
</BODY>
</HTML>
```

După testare, vom transfera pe site fișierul **HTML** și arhiva.

Până acum, am prezentat pe scurt modul în care se atașează un applet în fișierul **HTML**. Am văzut faptul că pentru a realiza acest lucru, se folosește elementul **APPLET**. Atributele acestuia sunt:

- **CODE** - numele fișierului care conține clasa ce extinde **JApplet**;
- **HEIGHT**, **WIDTH** - înălțimea și lățimea dreptunghiului care afișează applet-ul;
- **ARCHIVE** - numele fișierului care conține resursele utilizate de applet (clase și fișiere imagine);
- **ALIGN** - alinierea applet-ului în pagină. Mai importante sunt valorile:
 - **LEFT** - aliniere stânga;
 - **RIGHT** - aliniere dreapta;
 - **TOP** - în partea de sus;
- **HSPACE**, **VSPACE** - spațiul, în pixeli, care trebuie păstrat între dreptunghiul care încadrează applet-ul și text sau alte componente ale aflate pe pagina WEB. (**HSPACE**, spațiul lăsat pe orizontală) și **VSPACE** (spațiul lăsat pe verticală).

11.4. Redefinirea unor metode ale applet-ului

Componentele de tip **JApplet** conțin și metodele de mai jos, care pot fi redefinite:

⇒ **void init()** - este prima care se apelează atunci când am definit un applet. Redefinită, ea poate conține cod pentru anumite operații care dorim să fie efectuate la început. Această metodă se execută o singură dată.

- ⇒ **void start()** - se apelează atunci când applet-ul devine activ, dar și în alte situații, de exemplu, atunci când a fost părăsită pagina web și se revine asupra ei.
- ⇒ **void stop()** - se apelează atunci când se oprește applet-ul sau când a fost părăsită pagina web care-l conține.
- ⇒ **void paint(Graphics ecr)** - se apelează de câte ori este cazul. Prin intermediul ei se desenează componenta. Ea este apelată după metodele **init()** și **start()**.
- ⇒ **void update(Graphics ecr)** - spre deosebire de clasa **Canvas**, unde metoda mai întâi ștergea desenul, după care apela **paint()**, de această dată, metoda are corpul vid. Din acest motiv, nu mai este necesară rescrierea ei pentru a elimina fenomenul de "pălăire" a imaginii, dar dacă dorim ca ea să șteargă imaginea, atunci va trebui redefinită, iar operația de ștergere trebuie scrisă de dezvoltatorul applet-ului.

Exemplu: pentru a vedea cum funcționează metodele de mai sus, rulați applet-ul următor. Inițial, o cutie de dialog afișează 'Init', apoi o alta afișează "Start" și, dreptunghiul care reprezintă applet-ul devine roșu:

```
import java.awt.*;
import javax.swing.*;
public class Ap extends JApplet{
    public void init()
    { JOptionPane.showMessageDialog (this, "Init");}
    public void start()
    { JOptionPane.showMessageDialog (this, "Start");}
    public void paint(Graphics g)
    { setBackground(Color.RED);}
}
```

11.5. Afișarea imaginilor. Clasa URL

Pentru a afișa o imagine, mai întâi ea trebuie transferată pe site (eventual cu **WS_FTP**).

Pentru ca applet-ul să aibă acces la ea, el va conține un obiect **URL** (**Uniform Resource Locator**). Această clasă se găsește în pachetul **java.net**. Clasa are constructorul:

- ⇒ **URL(String s)** - unde **s** reprezintă adresa **url** a imaginii. Întrucât la acea adresa s-ar putea să nu se găsească resursa respectivă, crearea unui obiect **URL** trebuie să se facă sub construcția **try**.

Pentru a citi imaginea, se utilizează metoda de mai jos a clasei **JApplet**:

⇒ **Image getImage(URL adr)** - metoda returnează imaginea.

În rest, pentru a afișa imaginea, vom utiliza metoda **drawImage()** a clasei **Graphics**, metodă pe care o cunoaștem deja. De asemenea, pentru a putea folosi applet-ul cu dimensiuni diferite (scrise în **HTML**, **APPLET**), se utilizează două metode ale clasei **JApplet**:

⇒ **int getWidth()** - returnează lățimea dreptunghiului care conține applet-ul.

⇒ **int getHeight()** - returnează înălțimea dreptunghiului care conține applet-ul.

Exemplu: applet-ul următor afișează o imagine pe întreaga sa suprafață:

```
import java.awt.*;
import javax.swing.*;
import java.net.*;

public class Ap extends JApplet{
    Image im;
    int w,h;
    URL x;

    public void init()
    { try { x=new URL("http://www.xxxx.ro/imagine1.jpg");
      catch (Exception e)
      { JOptionPane.showMessageDialog (this, "Imaginea?");
        im=getImage(x);
        w=getWidth();
        h=getHeight();
      }
    }

    public void paint (Graphics ecr)
    { ecr.drawImage(im,0,0,w,h,this);
    }
}
```



11.6. Redarea sunetelor

O componentă de tip **JApplet** poate reda sunete. Fișierele pot fi de mai multe tipuri: **.au**, **.aiff**, **.wav**. Inițial, fișierul audio este transferat pe site.

Pentru a lucra cu sunete se utilizează interfața **AudioClip** (se găsește în **java.applet**). Ea conține antetul a trei metode:

⇒ **void play()** - redă sunetele din fișier o singură dată;

⇒ **void loop()** - redă sunetele din fișier în mod continuu;

⇒ **void stop()** - oprește redarea sunetelor din fișier. În cazul în care nu se utilizează această metodă, sunetele se redau și după ce vizitatorul părăsește pagina care conține applet-ul.

De asemenea, pentru a putea lucra cu fișiere audio, vom utiliza și metoda:

⇒ `AudioClip getAudioClip(URL url)` - returnează un obiect `AudioClip` (o implementare a interfeței).

Exemplu: applet-ul următor afișează un dreptunghi roșu, dar redă în mod continuu un fișier audio:

```
import java.awt.*;
import javax.swing.*;
import java.net.*;
import java.applet.*;

public class Ap extends JApplet{
    URL adr;
    AudioClip muzica;

    public void init()
    { try {adr=new URL("http://www.ls-infomat.ro/spacemusic.au");}
      catch (Exception ex) { }
      muzica=getAudioClip(adr);
    }

    public void paint(Graphics ecr)
    { setBackground(Color.RED); }

    public void start()
    { muzica.loop(); }

    public void stop()
    { muzica.stop(); }
}
```

11.7. Exemple de applet-uri

1. Applet-ul următor afișează un text oarecare (vezi 8.10.2):

Un exemplu de applet

```
import java.awt.*;
import javax.swing.*;

public class Ap extends JApplet{
    Font f;
    public void init()
    { f=new Font("Arial",Font.BOLD, 20);
      setBackground(Color.YELLOW);
    }

    public void paint (Graphics ecr)
    { ecr.setFont(f);
      ecr.setColor(Color.BLACK);
      ecr.drawString("Un exemplu de applet",50, 150);
    }
}
```

2. Applet-ul următor afișează o imagine pe care am scris un mesaj !

```
import java.awt.*;
import javax.swing.*;
import java.net.*;

public class Ap extends JApplet{
    Image im; int w,h; URL x;
    Font f;
    public void init()
    { try
      { x=new URL("http://www.xxxxxx.ro/
                  fericire.jpg");}
      catch (Exception e) {}
      im=getImage(x);
      w=getWidth(); h=getHeight();
      f=new Font("Arial",Font.BOLD, 30);
    }

    public void paint (Graphics ecr)
    { ecr.drawImage(im,0,0,w,h,this);
      ecr.setFont(f);
      ecr.drawString("Fericire generala!!!",50,270);
    }
}
```



3. Puteti realiza un applet care rotește (ciclează) la nesfârșit un șir de caractere.

este un sir de caractere care s

Este o formă la care se apelează de multe ori atunci când se dorește ca o anumită pagină să aibă un conținut informativ! Pentru a obține acest efect am utilizat un fir de execuție. Practic, primul caracter al șirului afișat se așează pe ultima poziție, se așteaptă un interval de timp, apoi primul caracter al șirului astfel obținut se așează pe ultima poziție, se așteaptă, etc. De vreme ce avem un fir de execuție, trebuie creată o arhivă care va fi pusă pe site. Arhiva va conține fișierele **Ap.class** și **Fir.class**.

```
import java.awt.*;
import javax.swing.*;

class Fir extends Thread {
    JTextField txt;
    Fir (JTextField txt)
    { this.txt=txt;}

    public void run()
    { while (true)
      { String s=txt.getText();
        try { sleep(300); }
        catch (Exception e) {}
        s= s.substring(1)+s.substring (0,1);
        txt.setText(s);
      }
    }
}
```

```

public class Ap extends JApplet{
    JTextField txt;
    public void init()
    { Container x = getContentPane();
      x.setLayout(new BorderLayout());
      txt=new JTextField ("Acesta este un sir de caractere care se
                           rotește pe durata de viața a applet-ului.
                           Pentru a-l implementa avem nevoie de două
                           clase ");
      txt.setBackground(Color.YELLOW);
      Font f=new Font("Arial",Font.BOLD, 20);
      txt.setFont(f);
      x.add(txt,BorderLayout.NORTH);
      Fir fir=new Fir(txt);
      fir.start();
    }
}

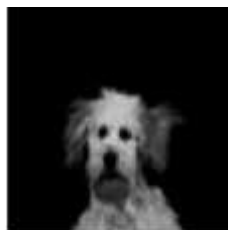
```

4. Applet-ul următor prezintă modul în care se pot realiza animații pe Internet. Mai întâi, au fost trecute pe site mai multe imagini (12). Apoi, printr-un mecanism asemănător celui de la 10.14. (revedeți), s-a realizat animația:

```

import java.awt.*;
import javax.swing.*;
import java.net.*;
class Fir extends Thread {
    Ap acesta;
    int i;
    Fir (Ap acesta)
    { this.acesta=acesta;}
    public void run()
    { while (true)
      { while (i!=11)
        { i++;
          acesta.afis(i);
          try { sleep(300); }
            catch (Exception e) {}
        }
        while (i!=0)
        { i--;
          acesta.afis(i);
          try { sleep(300); }
            catch (Exception e) {}
        }
      }
    }
}

```



```

public class Ap extends JApplet{
    Image[] im =new Image[12];
    URL[] url;
    int w,h,indice=0;
    String s=new String("http://www.xxxx.ro/Poza");
    URL t=null;

```

```

public void init()
{ for (int i=0;i<12;i++)
  { try {
      t=new URL(s+(i+1)+".jpg");
      im[i]=getImage(t);
      catch (Exception e) { }
    }
    w=getWidth(); h=getHeight();
    Fir fir=new Fir(this);
    fir.start();
  }
public void paint (Graphics ecr)
{ ecr.drawImage(im[indice],0,0,200,200,this);}
void afis(int i)
{ indice=i; repaint();}
}

```

5. **Efecte.** Applet-ul următor afișează un gradient ciclic dinamic (vezi 10.11.3). Efectul este deosebit. Analizați-!l!

```

import java.awt.*;
import javax.swing.*;
import java.awt.geom.*;

```

```

class Fir extends Thread {
  Ap acesta;
  Fir (Ap acesta) {this.acesta=acesta; }
  public void run()
  { try { sleep(50); }
    catch (Exception e) { }
    acesta.afis();}
}

```

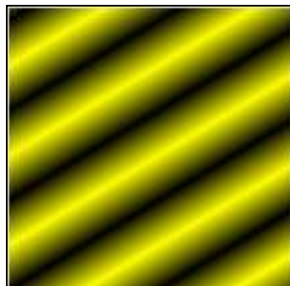
```

public class Ap extends JApplet{
  int x1,y1,x2,y2;
  public void init()
  { Fir x=new Fir(this); x.start();}
  public void paint (Graphics ecr1)
  { Graphics2D ecr=(Graphics2D) ecr1;
    GradientPaint g=new GradientPaint(x1,y1, Color.YELLOW,
                                       x2,y2, Color.BLACK, true);

    ecr.setPaint(g);
    Rectangle2D.Float drept = new Rectangle2D.Float
      (0,0, getWidth(),getHeight());

    ecr.fill(drept);
  }
  void afis()
  { x1=(int)Math.round(1+Math.ceil(Math.random()*50));
    y1=(int)Math.round(1+Math.ceil(Math.random()*50));
    x2=(int)Math.round(1+Math.ceil(Math.random()*50));
    y2=(int)Math.round(1+Math.ceil(Math.random()*50)); repaint();
  }
}

```



6. Applet-ul următor este un exemplu simplu de animație. Practic, un cerc se “plimbă” la stânga și la dreapta. Observați faptul că, de această dată, a fost necesar să rescriem metoda `update()`, care va șterge cercul înainte ca metoda `paint()` să-l redeseneze:

```
import java.awt.*;
import javax.swing.*;
import java.awt.geom.*;

class Fir extends Thread {
    Ap acesta;
    Fir (Ap acesta)
    { this.acesta=acesta; }
    public void run()
    { int i;
      while (true)
      { for (i=0;i<100;i++)
        { try { sleep(200); }
          catch (Exception e) { }
          acesta.afis(i); }
        for (;i>0;i--)
        { try { sleep(200); }
          catch (Exception e) { }
          acesta.afis(i); }
        }
    }
}

public class Ap extends JApplet{
    int x=100,y=100,r=100;

    public void init()
    { Fir f=new Fir(this);
      f.start();
    }

    public void paint (Graphics ecr)
    { ecr.fillOval(x, y, 50, 50); }

    public void update(Graphics ecr)
    { ecr.setColor(getBackground());
      ecr.fillRect(0,0,getWidth(),getHeight());
      ecr.setColor(Color.BLACK);
      paint(ecr);
    }

    void afis(int i)
    { x=100+i;
      repaint();
    }
}
```

7. Applet-ul care “plimbă” același cerc ca și cel anterior, dar care nu șterge desenul (efectul se vede în figura alăturată). Sursa s-a obținut din cea anterioară la care s-a șters metoda `update()`.

